

Poznámky k přednášce NTIN090 Úvod do složitosti a vyčíslitelnosti

Petr Kučera

30. července 2026

Obsah

I. Úvod	1
1. Motivace	2
2. Lehký úvod do teorie algoritmů	3
2.1. Začínáme pozdravem	3
2.2. Volá či nevolá program funkci <code>foo</code> ?	8
2.3. Nevýhody jazyka C pro budování teorie algoritmů	10
2.4. Bibliografické poznámky	11
3. Definice a konvence	12
3.1. Logika a logické formule	12
3.2. Množiny, relace a funkce	12
3.2.1. Množiny	12
3.2.2. Relace	13
3.2.3. Zobrazení a funkce	13
3.3. Čísla a spočetné množiny	15
3.3.1. Celá a přirozená čísla	15
3.3.2. Mohutnost množiny, spočetné a nespočetné množiny	15
3.4. Řetězce a jazyky	17
3.4.1. Řetězce	17
3.4.2. Uspořádání řetězců	17
3.4.3. Jazyky	18
3.4.4. Číslování řetězců	18
3.5. Matice a vektory	19
II. Vyčíslitelnost	20
4. Výpočetní modely	21
4.1. Turingovy stroje	21
4.1.1. Definice Turingova stroje	21
4.1.2. Varianty Turingových strojů	26
4.2. Random Access Machine	39
4.2.1. Definice	39
4.2.2. Programování RAM	47
4.2.3. Varianty RAM	48

4.3.	Ekvivalence Turingových strojů a RAM	49
4.3.1.	Převod Turingova stroje na RAM	51
4.3.2.	Převod RAM na Turingův stroj	53
4.4.	Bibliografické poznámky	55
4.5.	Cvičení	55
5.	Algoritmy	56
5.1.	Churchova-Turingova teze	56
5.2.	Kódování objektů a univerzální Turingův stroj	57
5.2.1.	Kódování Turingových strojů a Gödelovo číslo	57
5.2.2.	Kódování dalších objektů	62
5.2.3.	Univerzální Turingův stroj	62
5.3.	Algoritmicky rozhodnutelné problémy	66
5.4.	Algoritmicky vyčíslitelné funkce	67
5.5.	Univerzální funkce	68
5.6.	Bibliografické poznámky	69
5.7.	Cvičení	69
6.	Částečně rozhodnutelné jazyky a jejich vlastnosti	72
6.1.	Základní vlastnosti	72
6.1.1.	Jednoduché ekvivalentní definice	72
6.1.2.	Uzávěrové vlastnosti a Postova věta	75
6.2.	Proč nemohou všechny jazyky být částečně rozhodnutelné	78
6.3.	Nerohodnutelnost univerzálního jazyka	80
6.4.	Výčet slov jazyka	82
6.5.	Cvičení	85
7.	Převoditelnost a úplnost	87
7.1.	Problém zastavení	87
7.2.	Definice převoditelnosti	88
7.3.	Úplné problémy	94
7.4.	Riceova věta	95
7.5.	Postův korespondenční problém	97
7.5.1.	Převod problému PŘIJETÍ VSTUPU na problém MPKP	99
7.5.2.	Převod problému MPKP na problém PKP	104
7.6.	Jazyky za hranicí částečné rozhodnutelnosti	105
7.6.1.	Ekvivalence programů	105
7.6.2.	Konečnost jazyka	107
III.	Složitost	108
8.	Základní třídy problémů ve složitosti	109
8.1.	Problémy a úlohy	109

8.2.	Deterministické třídy složitosti	110
8.3.	Polynomiálně rozhodnutelné problémy	112
8.4.	Polynomiálně ověřitelné problémy	113
8.5.	Nedeterministické třídy složitosti	115
9.	Vztahy mezi třídami složitosti	118
9.1.	Vztahy mezi třídami	118
9.2.	Savičova věta	121
9.3.	Věty o hierarchii	124
9.3.1.	Deterministická prostorová hierarchie	124
9.3.2.	Deterministická časová hierarchie	127
10.	Cookova-Levinova věta	131
10.1.	Polynomiální převoditelnost a NP-úplnost	132
10.2.	Důkaz Cookovy-Levinovy věty	134
10.3.	*** Alternativní důkaz Cookovy-Levinovy věty ***	139
11.	Další NP-úplné problémy	150
11.0.1.	Splnitelnost formulí v 3-KNF (3-SAT)	150
11.0.2.	Vrcholové pokrytí v grafu	153
11.0.3.	Hamiltonovská kružnice v grafu	155
11.0.4.	Trojrozměrné párování	158
11.0.5.	Loupežníci	161
11.1.	Cvičení	163
12.	Pseudopolynomiální algoritmy a silná NP-úplnost	170
12.1.	Pseudopolynomiální algoritmus pro batoh	170
12.2.	Číselné problémy a pseudopolynomiální algoritmy	173
12.3.	Silná NP-úplnost	174
12.4.	Cvičení	175
13.	Aproximační algoritmy a schémata	176
13.1.	Aproximační algoritmy	176
13.2.	Příklad aproximačního algoritmu pro Bin Packing	177
13.3.	Úplně polynomiální aproximační schéma pro Batoh	179
13.4.	Aproximační schémata	182
13.5.	Neaproximovatelnost	184
13.6.	Cvičení	185
14.	Další třídy složitosti	187
14.1.	Třída co-NP	187
14.2.	Početní problémy — třída #P	190
14.3.	Cvičení	193

Část I.

Úvod

1. Motivace

Tento text obsahuje poznámky, které jsem si zpočátku psal jako přednášející pro sebe, ale snažil jsem se je rovnou psát i tak, aby pomohly studentům při studiu tohoto předmětu.

Přednáška by se mimo jiné měla pokusit zodpovědět následující otázky:

- (I) Co je to algoritmus?
- (II) Co všechno lze pomocí algoritmů spočítat?
- (III) Dokáží algoritmy vyřešit všechny úlohy a problémy?
- (IV) Jak poznat, že pro řešení zadané úlohy nelze sestavit žádný algoritmus?
- (V) Jaké algoritmy jsou „rychlé“ a jaké problémy jimi můžeme řešit?
- (VI) Jaký je rozdíl mezi časem a prostorem?
- (VII) Které problémy jsou lehké a které těžké? A jak je poznat?
- (VIII) Je lépe zkoušet nebo být zkoušený?
- (IX) Jak řešit problémy, pro které neznáme žádný „rychlý“ algoritmus?

Jde o otázky, které se věnují mezím toho, co je možné vyřešit pomocí algoritmů, tedy mezím nástroji, které používají ti, kdo se zabývají informatikou a tedy i programováním. Výklad začneme tou nejzákladnější otázkou, tedy co je to algoritmus.

2. Lehký úvod do teorie algoritmů

V textu se věnujeme algoritmům, což je pojem poněkud těžko uchopitelný. Než se budeme bavit o algoritmech obecně, zkusme se proto chvíli zabývat něčím méně abstraktním, a to programy v jazyce C¹.

2.1. Začínáme pozdravem

Protože se budeme dále věnovat algoritmům a programům, které je implementují, sluší se začít programem, který vypíše na obrazovku tradiční pozdrav, tedy řetězec „Hello, world“.

Výpis programu 2.1: Program `helloworld.c`

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Hello, \world\n");
    return 0;
}
```

Příklad takového programu vidíme ve výpisu 2.1, nazvěme ho `helloworld.c`. Podíváme-li se na tento program, vidíme, že tento program s jakýmkoli vstupem předaným mu na standardní vstup vypíše jako prvních dvanáct znaků svého výstupu řetězec „Hello, world“ a ihned skončí. Toto však zdaleka není jediný způsob, jak napsat program s touto funkcionalitou, uvažme program `helloworld2.c` zobrazený na výpisu 2.2.

Program `helloworld2.c` po spuštění načte celé číslo n ze standardního vstupu a poté hledá trojici čísel $x, y, z \in \mathbb{N}$, pro kterou by platilo $x^n + y^n = z^n$. Pokud je taková trojice nalezena, je vypsán řetězec „Hello, world“. K tomu ovšem dojde jen v případě, že $n = 1$ nebo 2 . Tento je ekvivalentní velké Fermatově větě, což je tvrzení, které čekalo téměř 400 let na svůj důkaz od chvíle, kdy Pierre de Fermat tuto větu vyslovil. Zamysleme se však obecně nad otázkou, zda je možné sestavit program H , který by za nás ověřil, zda

¹Volba jazyka C pro použití v tomto motivačním příkladu je víceméně náhodná a bylo by možné zvolit jakýkoli jiný vyšší programovací jazyk. Od jazyka C jsou odvozeny další jazyky, které jsou dnes používány, proto by syntaxe použitá v této sekci neměla být zcela cizí ani těm, kdo se jinak s jazykem C nesetkali.

Výpis programu 2.2: Program helloworld2.c

```
#include <stdio.h>

int exp(int i, int n)
/* Vrátí n-tou mocninu i */
{
    int moc, j;
    moc=1;
    for (j=1; j<=n; ++j) moc *= i;
    return moc;
}

int main(int argc, char *argv[]) {
    int n, total, x, y, z;
    scanf("%d", &n);
    total=3;
    while (1) {
        for (x=1; x<=total-2; ++x) {
            for (y=1; y<=total-x-1; ++y) {
                z=total-x-y;
                if (exp(x,n)+exp(y,n)==exp(z,n)) {
                    printf("Hello ,_world\n");
                    return 0;
                }
            }
        }
        ++total;
    }
}
```

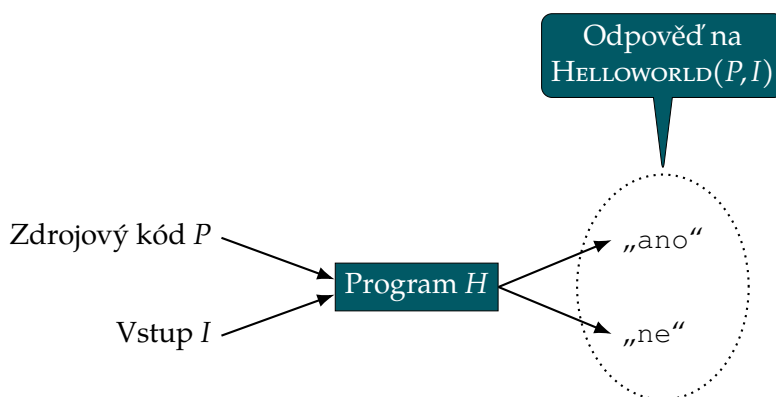
daný program P se vstupem I vypíše jako prvních dvanáct znaků svého výstupu právě „Hello, world“. Zavedme si proto problém **HELLOWORLD**.

Problém 2.1: HELLOWORLD

Instance: Zdrojový kód programu P v jazyce C a jeho vstup I .

Otázka: Je pravda, že prvních 12 znaků, které daný program vypíše, je „Hello, world“? (Nevyžadujeme zastavení.)

Ukážeme si, že žádný program v jazyce C za nás problém **HELLOWORLD** nevyřeší. Uvažujme pro chvíli (a pro spor), že máme takový program a nazvěme jej třeba H . Program H tedy očekává dva vstupy, zdrojový kód programu P v jazyce C a vstup tohoto programu I . V obou případech jde o textové soubory. Situace je znázorněna na obrázku 2.1.



Obrázek 2.1.: Vstup a výstup programu H .

Ve zbytku této podkapitoly přivedeme existenci programu H ke sporu. Základní myšlenkou je předložit programu H jeho vlastní zdrojový kód a ukázat, že program H nedokáže nic říci ani sám o sobě, natož aby to uměl obecně o všech ostatních programech. Ještě předtím však musíme program H upravit, protože tento program nevypisuje řetězec „Hello, world“ za žádné situace a navíc očekává na vstupu dva soubory, zatímco v problému **HELLOWORLD** uvažujeme pouze programy s jedním vstupním souborem. Pro zjednodušení situace budeme uvažovat pouze programy, které splňují následující dvě omezení:

- (i) Předpokládáme, že vstupní soubory jsou předávány všem uvažovaným programům pouze na standardní vstup, který čtou programy výhradně funkcí `scanf`².

²Jde o standardní funkci, jež se v jazyce C používá pro formátovaný vstup pro čtení ze standardního vstupu. Například volání `scanf("%d", &n)` použité v programu 2.2 načte ze vstupu řetězec kódující celé číslo, které dosadí do proměnné `n`.

Pokud program očekává více vstupních souborů (což je případ programu H), pak načte ze standardního vstupu nejprve jeden vstupní soubor a poté druhý vstupní soubor. Konec prvního vstupního souboru pozná program při načtení vyhrazeného znaku „EOF“. Vzhledem k tomu, že se jedná o textové soubory a že programy jsou psané v jazyce C, může roli znaku „EOF“ hrát například znak s kódem 0, tj. znak „\0“.

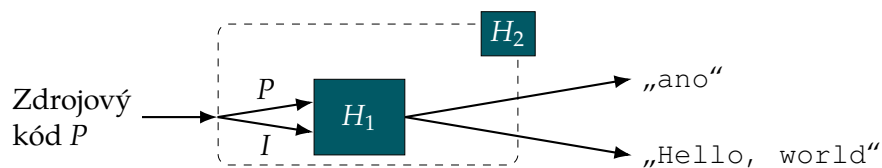
- (ii) Předpokládáme, že výstup všech programů je textový a že je zapisován na standardní výstup, a to výhradně funkcí `printf`³.

První úpravou programu H bude náhrada vypisovaného řetězce „ne“ na „Hello, world“. Program H_1 s touto funkcí získáme následující úpravou: Vypíše-li H jako první znak „n“, víme, že nakonec vypíše „ne“. Změníme tedy funkčnost funkce `printf` takovým způsobem, že v okamžiku, kdy je tato funkce poprvé zavolána a je-li prvním znakem požadovaného výpisu „n“, pak místo svého výpisu funkce `printf` vypíše řetězec „Hello, world“ a zajistí (pomocí booleovské proměnné), že od této chvíle žádné volání funkce `printf` nevykoná žádný výpis. Zkonstruujeme takto program H_1 , jehož funkčnost je naznačena na obrázku 2.2.



Obrázek 2.2.: Funkce programu H_1 .

Naším cílem je předložit programu H_1 jeho zdrojový kód a podívat se, co takto o sobě program H_1 bude schopen říci. K tomu stačí postavit zdrojový kód programu H_1 do role vstupního zdrojového kódu P , musíme však předložit programu H_1 také vstupní soubor I . Položíme tedy $I = P$ a na základě této myšlenky vytvoříme program H_2 , který bude očekávat jen jeden vstup, a to program P , který předloží programu H_1 jako oba požadované vstupy, tedy P i I . Program H_2 tak jinak pracovat tímž způsobem jako H_1 . Situace programu H_2 je naznačena na obrázku 2.3.



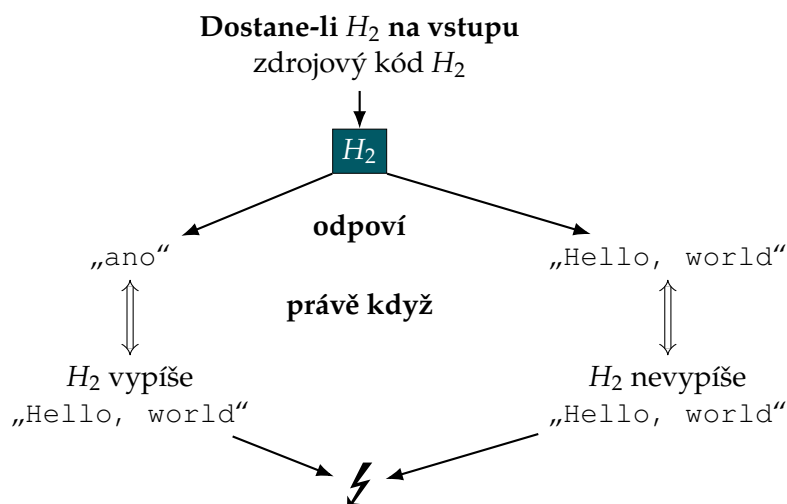
Obrázek 2.3.: Funkce programu H_2 .

Program H_2 získáme z programu H_1 následujícím postupem:

³Jde o standardní funkci jazyka C pro formátovaný výstup. Volání `printf("Hello, world")` prostě vypíše na standardní výstup řetězec „Hello, world“.

- 1: Program H_2 nejprve načte celý vstup a uloží jej v poli A , které pro tento účel alokuje v paměti (např. pomocí `malloc`).
- 2: Poté program H_2 simuluje práci H_1 , přičemž:
 - a: Ve chvíli, kdy H_1 čte vstup (pomocí `scanf`), H_2 místo čtení přistoupí do pole A . To znamená, že změníme implementaci `scanf` tak, aby místo čtení vstupu četla z pole A .
 - b: Pomocí dvou ukazatelů do pole A si H_2 pamatuje, kolik z P a I program H_1 přečetl (`scanf` čte popořadě, takže ukazatele začínají inicializované nulou a posléze je vždy při přečtení znaky ze vstupu ve funkci `scanf` program H_2 inkrementuje).

Je zřejmé, že takto zkonstruovaný program H_2 bude mít požadovanou funkci. To znamená, že pokud je mu předložen zdrojový kód programu P , pak H_2 simulací programu H_1 (potažmo H) zjistí, zda P jako prvních dvanáct znaků svého výstupu vypíše řetězec „Hello, world“. Pokud tomu tak je, vypíše program H_2 na svůj výstup řetězec „ano“, v opačném případě vypíše H_2 na svůj výstup řetězec „Hello, world“.



Obrázek 2.4.: Pokud zavoláme program H_2 předloživše mu na vstup jeho vlastní zdrojový kód, nutně dojdeme k závěru, že ani jedna z možných odpovědí H_2 není správná, a tedy program H_2 nemůže existovat.

Položme si nyní otázku, co se stane, pokud předložíme programu H_2 na vstup jeho vlastní zdrojový kód. V úvahu připadají dvě možnosti (jež jsou naznačeny na obrázku 2.4). Buď H_2 (se vstupem $P = H_2$) odpoví „ano“, k tomu může ovšem dojít jen v situaci, kdy H_2 (v roli P) ve skutečnosti jako prvních dvanáct znaků svého výstupu vypíše řetězec „Hello, world“, což ovšem není možné, buď vypíše „ano“, nebo „Hello, world“, ale nikoli obojí. Tato možnost tedy vede ke sporu. Druhou možností je, že H_2

(se vstupem $P = H_2$) odpoví „Hello, world“. To znamená, podle toho, jak jsme program H_2 konstruovali, že program H_2 (nyní v roli P , jemuž je předložen P na vstupu) ve skutečnosti nevypíše jako prvních dvanáct znaků svého výstupu „Hello, world“, a tedy vypíše „ano“. Opět tak dostáváme spor.

Jediným možným závěrem je, že program H_2 nemůžeme takto zkonstruovat, což znamená, že program H_1 a zejména ani program H nemohou existovat. Problém [HELLOWORLD](#) není řešitelný žádným programem v jazyce C. Tento fakt jsme ukázali sporem s použitím *diagonalizace*, později se dostaneme k tomu, v čem diagonalizace spočívá a z čeho pochází tento název.

2.2. Volá či nevolá program funkci `foo`?

Uvažme nyní jiný, snad i praktičtější problém, v němž se ptáme, zda daný program volá funkci s daným jménem, tomuto problému budeme říkat [VOLÁNÍ FUNKCE `foo`](#).

Problém 2.2: VOLÁNÍ FUNKCE `foo`

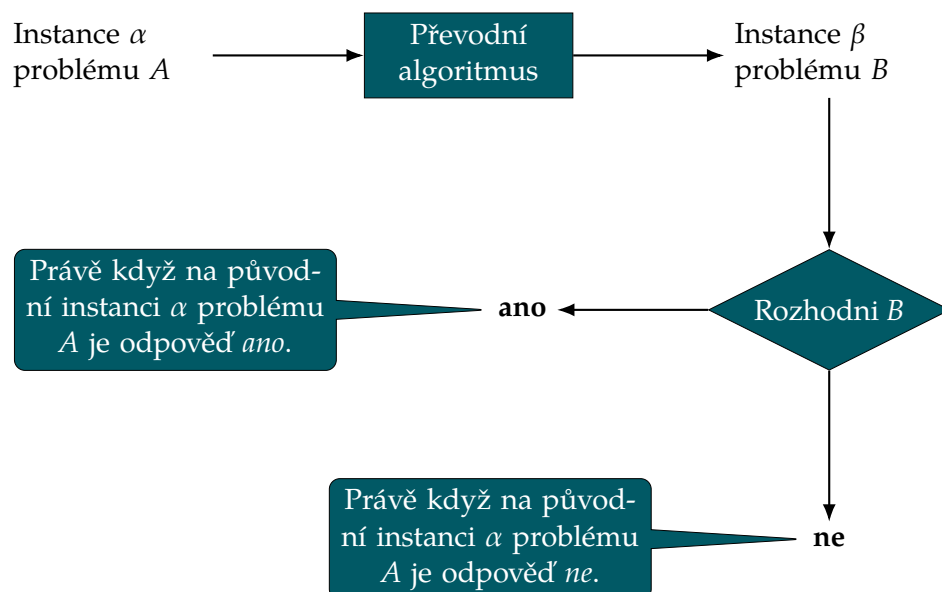
Instance: Zdrojový kód programu Q v jazyce C a jeho vstup V .

Otázka: Zavolá program Q při běhu nad vstupem V funkci jménem `foo`?

Ukážeme si, že ani tento problém není možné vyřešit žádným programem v jazyce C. Nyní však využijeme toho, že už máme jeden nerozhodnutelný problém, a to problém [HELLOWORLD](#). Ukážeme, že kdybychom byli schopni rozhodnout problém [VOLÁNÍ FUNKCE `foo`](#) programem H_3 v jazyce C, mohli bychom rozhodnout i problém [HELLOWORLD](#) (jiným) programem H v jazyce C. Protože víme, že takový program H neexistuje, ukážeme tím, že ani program H_3 rozhodující [VOLÁNÍ FUNKCE `foo`](#) nemůže existovat.

Přesněji, ukážeme, jak převést problém [HELLOWORLD](#) na problém [VOLÁNÍ FUNKCE `foo`](#). Použijeme k tomu způsob převodu jednoho problému na druhý, který je naznačen na obrázku 2.5. Uvažme problém A a problém B (každý z nich je daný popisem toho, jak vypadá *instance* problému a jakou *otázku* si klademe o takové instanci). Při převodu problému A na problém B musíme popsat *převodní algoritmus*, který přetvoří instanci α problému A na instanci β problému B . Přičemž musíme zajistit, aby platilo, že *kladná instance* problému A (tedy instance, na niž je odpověď kladná) je převedena na kladnou instanci problému B a *záporná instance* problému A (tedy instance, na niž je odpověď záporná) je převedena na zápornou instanci problému B . Potom můžeme říci, že máme-li program H_B , který rozhoduje problém B , jsme schopni zkonstruovat program H_A , který rozhoduje problém A : Program H_A se vstupem α prostě použije převodní algoritmus pro vytvoření instance β problému B , na niž pustí program H_B a vrátí touž odpověď jako tento program.

Ukážeme si nyní, jak v tomto smyslu převést problém [HELLOWORLD](#) na problém [VOLÁNÍ FUNKCE `foo`](#). Již víme, že nemůže existovat program v jazyce C (a ani v jiném programovacím jazyce), který by byl schopen rozhodnout problém [HELLOWORLD](#), dostaneme tak,



Obrázek 2.5.: Princip převodu problému A na problém B.

že ani problém **VOLÁNÍ FUNKCE foo** nelze rozhodnout programem v jazyce C.

Vyjdeme z instance problému **HELLOWORLD** a popíšeme postup, jak ji převést na instanci problému **VOLÁNÍ FUNKCE foo**. Instance obou problémů vypadají podobně — jedná se vždy o dvojici programu a vstupního souboru. Uvažme tedy dvojici P, I , která tvoří instanci problému **HELLOWORLD**, kde P je zdrojový kód programu a I je vstupní soubor. Popíšeme postup, který tuto dvojici přetvoří do jiné dvojice Q, V , kde Q je zdrojový kód programu a V je vstupní soubor, přitom tento postup zabezpečí, že P, I je kladnou instancí problému **HELLOWORLD**, právě když Q, V je kladnou instancí problému **VOLÁNÍ FUNKCE foo**. Přesněji, že:

$$\begin{array}{ll}
 P \text{ se vstupem } I \text{ vypíše} & \\
 \text{jako prvních dvanáct} & \\
 \text{znaků svého výstupu ře-} & \iff \\
 \text{tězec „Hello, world“}. & Q \text{ se vstupem } V \text{ zavolá} \\
 & \text{funkci } \mathbf{foo}.
 \end{array} \quad (2.1)$$

Při převodu P, I na Q, V budeme postupovat následujícím způsobem:

- 1: Je-li v P funkce **foo**, přejmenujeme ji i všechna její volání na dosud nepoužité jméno (*refactoring*, výsledný program nazveme P_1).
- 2: K programu P_1 přidáme funkci **foo**, funkce nic nedělá a není volána ($\rightarrow P_2$).
- 3: Upravíme program P_2 tak, aby si pamatoval prvních dvanáct znaků, které vypíše a uložil je v poli A . Jde prostě o úpravu implementace funkce `printf`, A je nové pole znaků o dvanácti prvcích, jež se v programu jinak nevyskytuje a je lokální nové implementaci funkce `printf`. ($\rightarrow P_3$).

- 4: Upravíme program P_3 tak, že pokud použije příkaz pro výstup, zkontroluje pole A , je-li v něm alespoň dvanáct znaků a na začátku obsahuje „Hello, world“. Pokud ano, zavolá funkci `foo` (tím dostaneme výsledný program Q , vstup V položíme roven I , tedy $V = I$).

Nahlédneme nyní, že skutečně platí ekvivalence (2.1).

⇒ Předpokládejme nejprve, že program P se vstupem I vypíše jako prvních dvanáct znaků svého výstupu řetězec „Hello, world“. Potom to program Q se vstupem $V = I$ ve své implementaci `printf` zjistí a zavolá funkci `foo`.

⇐ Na druhou stranu předpokládejme, že program Q se vstupem $V = I$ zavolá funkci `foo`. Z konstrukce programu Q vyplývá, že k tomuto volání může dojít jen tehdy, pokud Q ve své implementaci funkce `printf` zjistí, že pole A obsahuje řetězec „Hello, world“. To je ovšem možné jen pokud původní program P se vstupem I vypsal jako prvních dvanáct znaků svého výstupu řetězec „Hello, world“.

Z toho vyplývá, že problém **VOLÁNÍ FUNKCE `foo`** není řešitelný programem v jazyce C.

2.3. Nevýhody jazyka C pro budování teorie algoritmů

Ve zbytku tohoto textu je naším cílem popsat prostředky pro popis algoritmů. Pro tento účel bychom jistě mohli využít jazyk C s tím, pokud bychom připustili, že každý algoritmus lze implementovat v jazyce C (což nyní poznamenáváme s vědomím toho, že jsme dosud nespécifikovali formálně, co myslíme pojmem algoritmu). Jazyk C (jakož i jiný vyšší programovací jazyk) má však pro tyto účely řadu nevýhod.

- (I) Jazyk C je příliš komplikovaný. Jinými slovy, pokud bychom skutečně použít jazyk C pro teoretický popis algoritmů, museli bychom věnovat značné úsilí tomu, abychom si popsali, co myslíme jazykem C, tedy popsat jeho specifikaci. Ta je dosti složitá a nejspíš bychom narazili i na řadu nejasností a nejednoznačností.
- (II) Museli bychom definovat výpočetní model (tj. zobecněný počítač), který bude programy v jazyce C interpretovat. To proto, že k tomu, abychom skutečně byli schopni algoritmem zapsaným jako program v jazyce C něco spočítat, musíme jej přeložit a spustit na počítači. Způsob překlada a interpretace přeloženého kódu je pochopitelně podstatná pro zodpovězení otázek souvisejícím s tím, co a jak rychle je možno programy v jazyce C spočítat.
- (III) V době vzniku teorie nebyly procedurální jazyky k dispozici, proto je teorie v literatuře obvykle popisovaná tradičnějšími prostředky. Tato teorie je již dobře vybudovaná a nedává proto smysl přepisovat ji pomocí složitějších prostředků.

Místo jazyka C nebo jiného vyššího programovacího jazyka potřebujeme tedy zvolit výpočetní model, který bude dostatečně jednoduchý, aby bylo možno jej jednoduše popsat, ale i dostatečně silný, aby zachycoval to, co si představujeme pod pojmem algoritmu.

2.4. Bibliografické poznámky

Motivační příklady popsané, tj. nerozhodnutelnost problémů `HELLOWORLD` a `VOLÁNÍ FUNKCE foo`, byly převzaty z [6].

3. Definice a konvence

V této kapitole zavedeme základní pojmy a konvence použité v textu.

3.1. Logika a logické formule

Pro zápis logických formulí budeme používat běžně užívané logické spojky. Občas budeme pracovat s formulemi v konjunktivní normální formě a formulemi v disjunktivní normální formě.

Literál je buď proměnná nebo její negace, proměnnou x nazveme *pozitivním literálem* a její negaci $\neg x$ nazveme *negativním literálem*. *Klauzule* je disjunkcí literálů, která neobsahuje dva literály s touž proměnnou, například $(x \vee \bar{y} \vee \bar{z})$. Formule φ je v *konjunktivní normální formě* (KNF), pokud je konjunkcí klauzulí, například $(x \vee \bar{y} \vee \bar{z}) \wedge (\bar{x} \vee y) \wedge (y \vee z) \wedge \bar{x}$. *Term* je konjunkcí literálů, která neobsahuje dva literály s touž proměnnou, například $(x \wedge \bar{y} \wedge \bar{z})$. Formule φ je v *disjunktivní normální formě* (DNF), pokud je disjunkcí termů, například $(x \wedge \bar{y} \wedge \bar{z}) \vee (\bar{x} \wedge y) \vee (y \wedge z) \vee \bar{x}$.

Je-li φ formulí na n proměnných a $t \in \{0, 1\}^n$, pak pomocí $\varphi(t)$ označíme hodnotu, na kterou se φ vyhodnotí, přiřadíme-li proměnným hodnoty dané vektorem t . Hodnota 0 označuje lež, či *false*, zatímco hodnota 1 označuje pravdu, či *true*.

3.2. Množiny, relace a funkce

V této části zavedeme pojmy související s množinami a přirozenými čísly.

3.2.1. Množiny

Jsou-li A a B množiny, pak

$A \cup B$ označuje sjednocení množin A a B .

$A \cap B$ označuje průnik množin A a B .

$A \setminus B$ označuje rozdíl množin A a B .

$A \times B$ označuje kartézský součin množin A a B .

A^n pro $n \geq 0$ je n -tá kartézská mocnina množiny A a je definovaná jako $A = A^1$ a $A^n = A^{n-1} \times A$, pro $n > 1$.

$|A|$ označuje počet prvků množiny A , samozřejmě jen pokud je A konečná.

Jsou-li $A_1, \dots, A_n$ množiny (a $n \geq 1$) a $a_1 \in A_1, a_2 \in A_2, \dots, a_n \in A_n$ prvky těchto množin, pak uspořádanou n -tici tvořenou těmito prvky označujeme pomocí $(a_1, \dots, a_n)$. Zřejmě platí $(a_1, \dots, a_n) \in A_1 \times A_2 \times \dots \times A_n$. Pokud navíc $A = A_1 = A_2 = \dots = A_n$, pak $(a_1, \dots, a_n) \in A^n$.

Potenční množinou množiny A je množina jejích podmnožin, kterou definujeme jako

$$\mathcal{P}(A) = \{B \mid B \subseteq A\} \quad (3.1)$$

V případě konečné množiny A platí, že $|\mathcal{P}(A)| = 2^{|A|}$. Speciálně $|\mathcal{P}(A)|$ je vždy neprázdná, neboť vždy obsahuje přinejmenším prázdnou množinu.

3.2.2. Relace

Pro $n \geq 1$ definujeme n -ární relaci R mezi množinami $A_1, \dots, A_n$ jako množinu uspořádaných n -tic $R \subseteq A_1 \times \dots \times A_n$, n -ární relaci R na množině A pak definujeme jako množinu uspořádaných n -tic $R \subseteq A^n$. Relaci budeme též říkat *predikát*.

Zvláštní roli hrají *binární relace*, tedy relace arity 2. O relaci $R \subseteq A^2$ řekneme, že je

Reflexivní pokud pro každé $x \in A$ je $(x, x) \in R$.

Antireflexivní pokud pro každé $x \in A$ je $(x, x) \notin R$.

Symetrická pokud pro každé $x, y \in A$ platí, že je-li $(x, y) \in R$, pak i $(y, x) \in R$.

Antisymetrická pokud pro každé $x, y \in A$ platí, že je-li $(x, y) \in R$ a $(y, x) \in R$, pak $y = x$.

Tranzitivní pokud pro každé $x, y, z \in A$ platí, že je-li $(x, y) \in R$ a $(y, z) \in R$, pak $(x, z) \in R$.

Zvláštní typy relací, které uvažujeme, jsou například.

Ekvivalence tedy reflexivní, symetrická a tranzitivní relace.

Částečné uspořádání tedy reflexivní, antisymetrická a tranzitivní relace.

Kvaziuspořádání tedy reflexivní a tranzitivní relace.

Unární relace či predikát $R \subseteq A$ má aritu 1 a tedy jde pouze o podmnožinu množiny A .

3.2.3. Zobrazení a funkce

Zobrazením či *funkcí* z množiny A do množiny B míníme relaci $f \subseteq A \times B$ pro kterou platí, že pro každý prvek $x \in A$ existuje nejvýš jeden prvek $y \in B$, pro který platí, že $(x, y) \in f$, tuto skutečnost častěji píšeme jako $f(x) = y$. Fakt, že f je funkce z množiny A do množiny B zapíšeme pomocí

$$f : A \rightarrow B.$$

Pokud je funkce f zobrazením z množiny $A_1 \times A_2 \times \dots \times A_n$ pro nějaké $n \geq 1$ do množiny B , hovoříme o n -ární funkci (přičemž pokud je $n = 1$, jde o *unární* funkci a pokud je $n = 2$,

jde o *binární* funkci). V takovém případě píšeme hodnotu funkce f pro n -tici $a_1, \dots, a_n$ jako $f(a_1, \dots, a_n)$.

Definičním oborem (také *doménou*) funkce $f \subseteq A \times B$ je množina

$$\text{dom } f = \{x \mid (\exists y)[f(x) = y]\},$$

přičemž pro $x \in \text{dom } f$ je hodnota funkce f *definovaná*, což zapíšeme pomocí $f(x) \downarrow$, zatímco pro $x \notin \text{dom } f$ je hodnota funkce f *nedefinovaná*, což zapíšeme pomocí $f(x) \uparrow$. Fakt, že hodnota funkce $f(x)$ je definovaná pro $x \in \text{dom } f$ a současně je tato hodnota rovna $y \in B$, budeme také psát pomocí $f(x) \downarrow = y$. *Oborem hodnot* funkce f je množina

$$\text{rng } f = \{y \mid (\exists x) [f(x) \downarrow \wedge f(x) = y]\}.$$

Pokud je $f : A \rightarrow B$ pro množiny A a B a pokud platí, že $\text{dom } f = A$, pak jde o *totální* funkci, v opačném hovoříme o *částečné* funkci.

O funkci $f : A \rightarrow B$ řekneme, že je

Prostá pokud pro každé dva prvky $x_1, x_2 \in \text{dom } f$, platí, že je-li $f(x_1) = f(x_2)$, pak $x_1 = x_2$.

Na pokud $\text{rng } f = B$.

Vzájemně jednoznačné zobrazení (též *bijekce*) pokud je totální, prostá i na.

Jsou-li $f : B \rightarrow C$ a $g : A \rightarrow B$ funkce, pak složením těchto funkcí dostaneme funkci $(f \circ g) : A \rightarrow C$, která je definovaná právě pro ty prvky $x \in A$ pro něž je $x \in \text{dom } g$ a $g(x) \in \text{dom } f$, tj.

$$\text{dom}(f \circ g) = \{x \in \text{dom } g \mid g(x) \in \text{dom } f\}.$$

Pro každý prvek $x \in \text{dom}(f \circ g)$ definujeme $(f \circ g)(x) = f(g(x))$. Podobně bychom mohli zavést i složení funkcí více parametrů, přičemž pokud je f funkce arity n , pak uvažujeme n vnitřních funkcí $g_1, \dots, g_n$.

Budeme občas používat anonymní funkce, tj. nepojmenované funkce definované výrazem. K jejich zápisu budeme používat *Churchovo λ -značení*: Je-li V výraz, pak pomocí $\lambda x_1 x_2 \dots x_n V$ označíme funkci na proměnných $x_1, x_2, \dots, x_n$, která je daná výrazem V . Například $\lambda abc[a + c]$ je funkce, která očekává tři (číselné) parametry, a , b a c a hodnotou této funkce je součet prvního a třetího parametru. Všimněme si, že samotný zápis $a + c$ není dostatečný, protože nespecifikuje přesně, kolik parametrů funkce očekává ani jejich pořadí.

Je-li A množina prvků z univerza U (tedy $A \subseteq U$), pak pomocí χ_A budeme značit *charakteristickou funkci množiny* A , jde o funkci $\chi_A : U \rightarrow \{0, 1\}$, kde

$$\chi_A(x) = \begin{cases} 1 & x \in A \\ 0 & x \notin A \end{cases}$$

Speciálně, pokud A je n -ární relace (či predikát), pak můžeme také hovořit o charakteristické funkci této relace (či predikátu). Charakteristická funkce je vždy totální.

3.3. Číslo a spočetné množiny

V této části zavedeme pojmy související zejména s přirozenými čísly, připomeneme si také, co myslíme pojmem spočetné množiny.

3.3.1. Celá a přirozená čísla

Množinu celých čísel označujeme pomocí $\mathbb{Z}$, množinu přirozených čísel pak pomocí $\mathbb{N}$, přičemž nulu považujeme za přirozené číslo (tj. $0 \in \mathbb{N}$). Doplnkem množiny $A \subseteq \mathbb{N}$ míníme množinu $\bar{A} = \mathbb{N} \setminus A$. Množinu reálných čísel budeme označovat pomocí $\mathbb{R}$ a množinu racionálních čísel pomocí $\mathbb{Q}$.

3.3.2. Mohutnost množiny, spočetné a nespočetné množiny

Množiny můžeme mezi sebou porovnávat s pomocí *mohutnosti* následujícím způsobem. Jsou-li A a B množiny, pak řekneme, že

1. množina A má *shodnou nebo menší mohutnost* než množina B , pokud existuje prosté zobrazení $f : A \rightarrow B$,
2. množina A má *shodnou mohutnost* s množinou B , pokud existuje bijekce $f : A \rightarrow B$ a
3. množina A má *menší mohutnost* než množina B pokud platí bod 1, ale nikoli bod 2.

Například je-li $A \subseteq B$, pak je mohutnost množiny A shodná nebo menší než mohutnost množiny B . Řekneme, že množina A je *spočetná*, pokud je její mohutnost shodná nebo menší než mohutnost množiny přirozených čísel $\mathbb{N}$. To znamená, že každému prvku A můžeme přiřadit nějaké přirozené číslo takovým způsobem, že dvěma různým prvkům vždy přiřadíme dvě různá přirozená čísla. Například každá konečná množina je zřejmě spočetná.

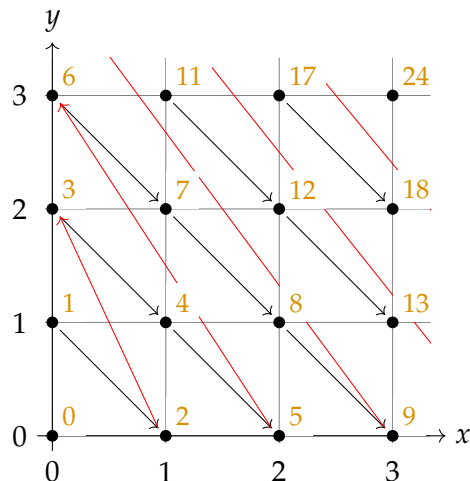
Platí, že sjednocení, průnik, rozdíl i kartézský součin spočetných množin jsou opět spočetné množiny, speciálně je-li množina A spočetná, pak i množina n -tic A^n je spočetná pro každé n .

Číslování n -tic přirozených čísel

Pokud chceme například ukázat, že $\mathbb{N}^2$ je spočetná množina, můžeme uvážit Cantorovu funkci dvojice, která je definovaná následujícím předpisem:

$$\pi_2(x, y) = \frac{(x + y)(x + y + 1)}{2} + x \quad (3.2)$$

Tato funkce je dokonce bijekcí mezi $\mathbb{N}^2$ a $\mathbb{N}$, způsob, jakým čísluje tato funkce dvojice čísel, je naznačen na obrázku 3.1.



Obrázek 3.1.: Pořadí, v jakém Cantorova párovací funkce čísluje dvojice přirozených čísel.

Z toho přímo plyne, že množina racionálních čísel $\mathbb{Q}$ a tedy i množina celých čísel $\mathbb{Z}$ jsou spočetné, protože racionální číslo odpovídá dvojici čísel přirozených — dělenci a děliteli.

Číslování dvojic přirozených čísel můžeme rozšířit na n -tice pro $n \geq 2$ indukčním způsobem:

$$\pi_n(x_1, \dots, x_n) = \begin{cases} \pi_2(x_1, x_2) & n = 2 \\ \pi_2(x_1, \pi_{n-1}(x_2, \dots, x_n)) & n > 2 \end{cases} \quad (3.3)$$

Funkce $\pi_n(x_1, \dots, x_n)$ je bijekcí mezi $\mathbb{N}^n$ a $\mathbb{N}$. Dává proto smysl zavést inverzní funkci projekce. Pomocí $\pi_{n,i}^{-1}(\cdot) : \mathbb{N} \rightarrow \mathbb{N}$, kde $n, i \in \mathbb{N}$ a $i \leq n$ označíme funkci jedné proměnné definovanou následujícím předpisem:

$$\pi_{n,i}^{-1}(\pi_n(x_1, \dots, x_n)) = x_i \quad (3.4)$$

Funkce $\pi_{n,i}^{-1}(z)$ si vyloží parametr z jako kód n -tice $x_1, \dots, x_n$ a vrátí její i -tou složku, tedy x_i .

Nespočetné množiny

Je potřeba také zmínit, že například množina reálných čísel $\mathbb{R}$ spočetná není a spočetná není ani množina $\wp(\mathbb{N})$. Podle [Cantorovy věty](#) navíc platí pro každou množinu A , že její potenční množina $\wp(A)$ má větší mohutnost než A .

3.4. Řetězce a jazyky

V této části zavedeme pojmy související s řetězcí a jazyky.

3.4.1. Řetězce

Abeceda je konečná množina znaků Σ . *Řetězec* nebo také *slovo* nad abecedou Σ je posloupnost znaků, psaná obvykle za sebou bez mezer. Například je-li $\Sigma = \{a, b, c, d\}$, pak $abc, cd, ddadb$ jsou slova nad abecedou Σ . Množinu všech řetězců nad abecedou Σ označujeme Σ^* , přičemž pokud je abeceda Σ jednoprvková, tedy například $\Sigma = \{1\}$, pak místo $\{1\}$ píšeme též 1^* . *Prázdný řetězec* označujeme ε . *Délkou slova* $w \in \Sigma^*$ rozumíme počet znaků v tomto slově a označujeme ji $|w|$. Například $|\varepsilon| = 0$, $|abc| = 3$. Je-li $w \in \Sigma^*$ řetězec a $i \in \{1, \dots, |w|\}$ index, pak $w[i]$ označuje i -tý znak řetězce w .

Množinu všech slov délky i pro $i \in \mathbb{N}$ nad abecedou Σ označíme pomocí Σ^i , platí zřejmě, že

$$\Sigma^* = \bigcup_{i \in \mathbb{N}} \Sigma^i.$$

Jsou-li $u, v \in \Sigma^*$ slova nad abecedou Σ , pak jejich *konkatenací* je slovo $w = uv$ vzniklé tak, že slovo v připojíme za slovo u . Například konkatenací slov abc a ba je slovo $abcba$. Zřejmě platí, že $w = w\varepsilon = \varepsilon w$ pro každé slovo $w \in \Sigma^*$. Je-li $w \in \Sigma^*$ slovo a $i \in \mathbb{N}$, pak w^i označuje slovo w zapsané i -krát za sebou, přitom pro $i = 0$ dostáváme $w^0 = \varepsilon$. Například $a^4 = aaaa$, $a^2b^3 = aabbbb$, $(ab)^2 = abab$. Pomocí w^R označujeme *zrcadlový obraz* slova $w \in \Sigma^*$, tedy slovo, které vznikne otočením pořadí znaků ve slově w . Například $(abcd)^R = dcba$.

3.4.2. Uspořádání řetězců

Pro porovnávání řetězců budeme používat variantu lexikografického uspořádání, kde prvním kritériem pro porovnání je délka řetězce, přičemž řetězce s touž délkou jsou porovnány jako ve slovníku. Této variantě budeme říkat *shortlex uspořádání*.

Definice 3.1 (Shortlex uspořádání) Nechť Σ je abeceda, předpokládejme navíc, že máme k dispozici relaci lineárního uspořádání na znacích abecedy (které označíme pomocí $<$). Nechť $x, y \in \Sigma^*$ jsou dva různé řetězce. Řekneme, že řetězec x je *menší v shortlex uspořádání* než y , pokud buď

- $|x| < |y|$, nebo
- $|x| = |y|$ a je-li $i = \min\{j \mid x[j] \neq y[j]\}$, pak $x[i] < y[i]$.

Tento fakt označíme pomocí $x < y$. Obvyklým způsobem definujeme další varianty tohoto značení ($\leq$ — menší nebo rovno, $>$ — větší a $\geq$ — větší nebo rovno). ◀

Například pokud $\Sigma = \{a, b\}$, kde $a < b$, pak řetězce z Σ^* v shortlex uspořádání mají pořadí: $\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, \dots$

Je potřeba si uvědomit, že shortlex uspořádání se liší od slovníkového uspořádání (tj. způsobu třídění řetězců běžně užívaného ve slovnících). Ve slovníkovém uspořádání

se obvykle kratší z řetězců x a y před porovnáním doplní mezerami zprava na délku delšího řetězce (kde mezera je znak menší než všechny ostatní). Potom teprve proběhne porovnání podle prvního lišícího se znaku. Slovníkové uspořádání není ovšem vhodné v situaci, kdy máme nekonečnou množinu řetězců. Je-li například $\Sigma = \{a, b\}$, kde $a < b$, pak v Σ^* je jen pět řetězců, jež jsou menší než ba v shortlex uspořádání. Na druhou stranu je ovšem v Σ^* nekonečně mnoho řetězců, které jsou menší než ba ve slovníkovém uspořádání (jde o prázdný řetězec a dále všechny řetězce začínající znakem a).

3.4.3. Jazyky

Množině slov $L \subseteq \Sigma^*$ nad abecedou Σ říkáme *jazyk* (nad abecedou Σ). Například $L = \{ab, cd\}$ je jazyk se dvěma řetězci, jazyk $L = \emptyset$ je prázdný a jazyk $L = \{a^i b^i \mid i \in \mathbb{N}\}$ je jazyk slov tvaru $a^i b^i$ pro $i \in \mathbb{N}$. *Doplňkem* jazyka L nad abecedou Σ myslíme jazyk $\bar{L} = \Sigma^* \setminus L$. *Konkatenací* dvou jazyků L_1 a L_2 nad abecedou Σ vznikne jazyk $L_1 \cdot L_2 = \{w_1 w_2 \mid w_1 \in L_1, w_2 \in L_2\}$. *Kleeného uzávěrem* jazyka L je jazyk

$$L^* = \{w \mid (\exists k \in \mathbb{N})(\exists w_1, \dots, w_k \in L)[w = w_1 w_2 \dots w_k]\}.$$

3.4.4. Číslování řetězců

Máme-li definované uspořádání řetězců, můžeme tyto řetězce očíslovat. Toho budeme často využívat zejména u binárních řetězců, číslování řetězců však můžeme definovat obecně. Uvažme pevnou konečnou abecedou Σ . Ke každému řetězci $w \in \Sigma^*$ definujeme jeho index

$$\text{index}(w) = |\{u \mid u < w\}|$$

Hodnota $\text{index}(w)$ je tedy definována jako počet řetězců, které jsou menší než w v shortlex uspořádání. Například $\text{index}(\varepsilon) = 0$. Z definice shortlex uspořádání plyne, že $\text{index}(w)$ je dobře definované přirozené číslo (tj. jeho hodnota je konečná). Platí dokonce, že index je bijekcí mezi Σ^* a $\mathbb{N}$.

Množina Σ^* je spočetná, neboť máme bijekci Σ^* na $\mathbb{N}$. Z toho plyne, že i libovolný jazyk $L \subseteq \Sigma^*$ je spočetný. Současně tím dostáváme vzájemně jednoznačnou korespondenci mezi množinami přirozených čísel a jazyky nad abecedou Σ . Množině přirozených čísel $A \subseteq \mathbb{N}$ odpovídá jazyk $L = \{w \mid \text{index}(w) \in A\}$. Na druhou stranu jazyku $L \subseteq \Sigma^*$ odpovídá množina kladných přirozených čísel $A = \{\text{index}(w) \mid w \in L\}$. Z toho plyne, že množina všech jazyků nad binární abecedou $\mathcal{P}(\Sigma)$ už spočetná není (protože není spočetná množina $\mathcal{P}(\mathbb{N})$). Toto platí i pro jednoprvkovou abecedu $\Sigma = \{1\}$.

Číslování binárních řetězců

Často budeme uvažovat řetězce nad binární abecedou $\{0, 1\}$, kde má číslování řetězců zajímavou interpretaci. Uvažme binární řetězec $w \in \{0, 1\}^*$ a jeho index $i = \text{index}(w)$. Potom platí, že

$$(i + 1)_B = 1w,$$

w	$1w$	$\text{index}(w) + 1$	$\text{index}(w)$
ε	1	1	0
0	10	2	1
1	11	3	2
00	100	4	3
$\vdots$	$\vdots$	$\vdots$	
001011	1001011	75	74
$\vdots$	$\vdots$	$\vdots$	

Tabulka 3.1.: Příklady binárních řetězců a jejich indexů.

kde $(i + 1)_B$ označuje binární zápis čísla $i + 1$ bez zbytečných úvodních 0 a $1w$ označuje řetězec w , na jehož začátek přidáme znak 1. V tomto případě je tedy opravdu snadné přepočítávat navzájem řetězce a jejich indexy. Příklady binárních řetězců a jejich indexů ukazuje tabulka 3.4.4.

Definice 3.2 Uvažme přirozené číslo n . Pomocí $\mathbf{w}_n$ označíme n -tý binární řetězec, tedy binární řetězec, pro který platí $\text{index}(\mathbf{w}_n) = n$. ◀

Jelikož index je bijekcí mezi $\{0, 1\}^*$ a $\mathbb{N}$, platí, že v posloupnosti $\{\mathbf{w}_n\}_{n \in \mathbb{N}}$ se každý binární řetězec vyskytuje právě jednou.

3.5. Matice a vektory

Matice budeme označovat tučnými velkými písmeny anglické abecedy (např. **A**, **B**), vektory pomocí tučných malých písmen anglické abecedy (např. **a**, **b**). Matice **A** je typu $m \times n$, má-li m řádků a n sloupců, je-li $m = n$, hovoříme o *čtvercové matici A*. Délkou vektoru **b** rozumíme počet jeho složek.

Je-li matice **A** typu $m \times n$ a jsou-li $i \in \{1, \dots, m\}$ a $j \in \{1, \dots, n\}$, pak pomocí $\mathbf{A}_{i,j}$ označíme prvek matice **A** na i -tém řádku a j -tém sloupci. Je-li **b** vektor délky n a je-li $i \in \{1, \dots, n\}$, pak pomocí $\mathbf{b}_i$ označíme prvek vektoru **b** na pozici i .

Část II.

Vyčíslitelnost

4. Výpočetní modely

V této kapitole si představíme několik různých výpočetních modelů a ukážeme si jejich ekvivalenci.

4.1. Turingovy stroje

Model Turingova stroje byl poprvé navržen Alanem Turingem v roce 1936 v článku [12]. Turingovy stroje budou naším hlavním modelem, ke kterému se uchýlíme vždy, budeme-li potřebovat napsat něco formálně. Budeme jej tedy používat zejména v definicích základních pojmů v části věnující se teorii vyčíslitelnosti (např. pojmů rozhodnutelnosti či částečné rozhodnutelnosti, převoditelnosti atd.), tak v části věnující se teorii složitosti (například v definici základních tříd složitosti, tříd P, NP a dalších). Je proto přirozené začít popisem právě tohoto modelu.

4.1.1. Definice Turingova stroje

V této kapitole zavedeme model Turingova stroje spolu se souvisejícími pojmy, jež budeme dále potřebovat.

Definice 4.1 (Turingův stroj) *(Jednopáskový deterministický) Turingův stroj M je pětice*

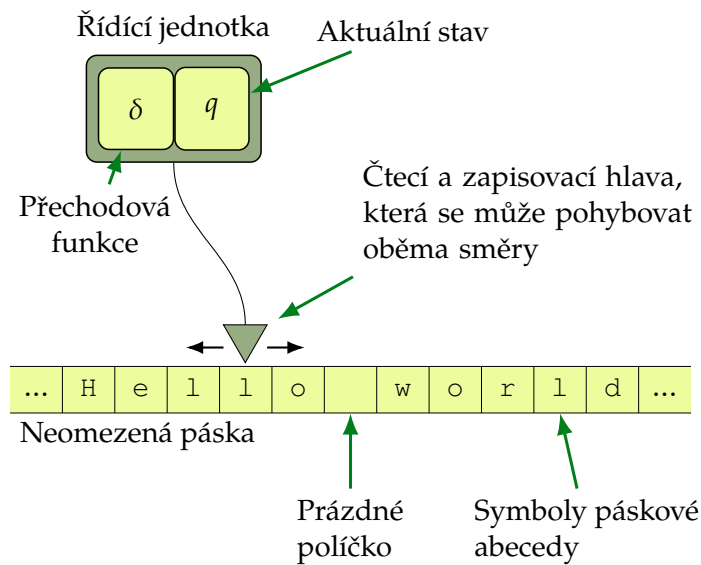
$$M = (Q, \Sigma, \delta, q_0, F),$$

kde

- Q je konečná množina stavů,
- Σ je konečná pásková abeceda, která obsahuje znak λ pro prázdné políčko,
- $\delta : Q \times \Sigma \rightarrow Q \times \Sigma \times \{R, N, L\} \cup \{\perp\}$ je přechodová funkce, kde $\perp$ označuje nedefinovaný přechod,
- $q_0 \in Q$ je počáteční stav a
- $F \subseteq Q$ je množina přijímajících stavů.

Turingův stroj (TS) sestává z *řídící jednotky, obousměrně potenciálně neomezené pásky a hlavy pro čtení a zápis, která se může pohybovat po pásce oběma směry.* ◀

Struktura Turingova stroje je zobrazena na obrázku 4.1.



Obrázek 4.1.: Struktura Turingova stroje.

Definice 4.2 (Displej, konfigurace a výpočet Turingova stroje)

Nechť $M = (Q, \Sigma, \delta, q_0, F)$ je Turingův stroj. Dvojici (q, a) stavu řídicí jednotky $q \in Q$ a čteného znaku na pásce $a \in \Sigma$, na základě níž se Turingův stroj rozhoduje, kterou instrukci použít pro další krok, říkáme *displej*. *Konfigurace* zachycuje aktuální informaci o stavu výpočtu Turingova stroje. Skládá se ze *stavu řídicí jednotky*, *slova na pásce* (od nejlevějšího do nejpravějšího neprázdného políčka) a *pozice hlavy na pásce* (v rámci slova na této pásce).

Výpočet zahajuje TS M v *počáteční konfiguraci*, tedy v počátečním stavu se vstupním slovem zapsaným na pásce a hlavou nad nejlevějším symbolem vstupního slova. Pokud se M nachází ve stavu $q \in Q$ a pod hlavou je symbol $a \in \Sigma$ (tedy (q, a) je aktuální displej Turingova stroje M), pak *krok výpočtu* probíhá následovně:

1. Je-li $\delta(q, a) = \perp$, výpočet M končí,
2. Je-li $\delta(q, a) = (q', a', Z)$, kde $q' \in Q$, $a' \in \Sigma$ a $Z \in \{L, N, R\}$, přejde M do stavu q' , zapíše na pozici hlavy symbol a' a pohne hlavou doleva (pokud $Z = L$), doprava (pokud $Z = R$), nebo hlava zůstane stát (pokud $Z = N$). ◀

Z popisu výpočtu Turingova stroje vidíme, že tabulka přechodové funkce popisuje, co se má stát v každé konfiguraci, nebo přesněji řečeno při daném displeji. Způsob programování Turingova stroje má tedy blízko k deklarativním jazykům.

Použití přechodové funkce δ rozšíříme i na konfigurace. Je-li K konfigurace Turingova stroje M , pak $\delta(K)$ označuje konfiguraci, která vznikne aplikací přechodové funkce δ na konfiguraci K , na základě displeje, který je v konfiguraci K obsažen. Hodnota $\delta(K) = \perp$,

	$q, c \rightarrow q', c', Z$
1.	INIT, $\lambda \rightarrow$ ACCEPT, λ, N
2.	INIT, $a \rightarrow$ MEM_A, a, R
3.	INIT, $b \rightarrow$ MEM_B, b, R
4.	MEM_A, $a \rightarrow$ MEM_A, a, R
5.	MEM_A, $b \rightarrow$ MEM_A, b, R
6.	MEM_A, $\lambda \rightarrow$ CHECK_A, λ, L
7.	MEM_B, $a \rightarrow$ MEM_B, a, R
8.	MEM_B, $b \rightarrow$ MEM_B, b, R
9.	MEM_B, $\lambda \rightarrow$ CHECK_B, λ, L
10.	CHECK_A, $a \rightarrow$ BACK, λ, L
11.	CHECK_B, $b \rightarrow$ BACK, λ, L
12.	BACK, $a \rightarrow$ BACK, a, L
13.	BACK, $b \rightarrow$ BACK, b, L
14.	BACK, $\lambda \rightarrow$ ERASE, λ, R
15.	ERASE, $a \rightarrow$ INIT, λ, R
16.	ERASE, $b \rightarrow$ INIT, λ, R
17.	ERASE, $\lambda \rightarrow$ ACCEPT, λ, N

Tabulka 4.1.: Přechodová funkce stroje M z příkladu 4.4.

pokud je hodnota δ nedefinovaná pro displej obsažený v konfiguraci K . O konfiguraci K , pro kterou platí, že $\delta(K) = \perp$, řekneme, že je *koncová*.

Na výpočet Turingova stroje lze také nahlížet jako na posloupnost konfigurací $K_0, \dots, K_t$, kde K_0 je počáteční konfigurace, pro každou konfiguraci K_i , $i \in \{0, \dots, t-1\}$ platí, že $K_{i+1} = \delta(K_i)$, a $\delta(K_t) = \perp$, tedy K_t je koncová konfigurace.

Definice 4.3 (Jazyk přijímaný Turingovým strojem) TS M *přijímá slovo* w , pokud výpočet M se vstupem w skončí a M se po ukončení výpočtu nachází v přijímajícím stavu. TS M *odmítá slovo* w , pokud výpočet M nad vstupem w skončí a M se po ukončení výpočtu nenachází v přijímajícím stavu. Fakt, že výpočet M nad vstupním slovem w skončí, označíme pomocí $M(w) \downarrow$ a řekneme, že výpočet *konverguje*. Fakt, že výpočet M nad vstupním slovem w nikdy neskončí, označíme pomocí $M(w) \uparrow$ a řekneme, že výpočet *diverguje*.

Množinu slov, která přijímá Turingův stroj M označíme pomocí $L(M)$, zveme ji také *jazykem přijímaným Turingovým strojem* M . Řekneme, že Turingův stroj M *rozhoduje jazyk* L , pokud $L = L(M)$ a M se zastaví nad každým vstupem. ◀

Příklad 4.4

Popišme si například Turingův stroj M , který rozhoduje jazyk

$$\text{PAL} = \{w = w^R \mid w \in \{a, b\}^*\}, \quad (4.1)$$

tj. jazyk palindromů. Algoritmus 1 popisuje práci Turingova stroje M , který rozhoduje jazyk palindromů. Jeho přechodová funkce δ je popsána v tabulce 4.1. Při konstrukci M vycházíme z toho, že prázdné slovo je palindromem, a toho, že slovo w délky $k = |w|$ je palindrom, pokud $w[1] = w[k]$ a slovo $w[2] \dots w[k-1]$ je rovněž palindrom.

Algoritmus 1: Práce TS M se vstupem w

```

1 if  $w = \varepsilon$  then accept //  $\varepsilon$  označuje prázdný řetězec
2 Zapamatuj si ve stavu první znak  $w[1]$ 
3 Přesuň hlavu na poslední políčko řetězce na pásce  $w[n]$ 
4 if  $w[1] \neq w[n]$  then reject
5 Smaž poslední znak
6 Přesuň hlavu na první políčko řetězce na pásce
7 Smaž první znak (pokud nějaký zbyl)
8 Pohni hlavou doprava
9 goto 1 // zbývá  $w[2] \dots w[n-1]$ 

```

Pásková abeceda $\Sigma = \{\lambda, a, b\}$ obsahuje ke dvěma znakům vstupní abecedy jen znak prázdného políčka. Množina stavů Q se skládá z 8 stavů, jejichž význam je možné popsat následujícím způsobem.

INIT jediný počáteční stav, odpovídá testu na řádku 1.

ACCEPT jediný přijímající stav, přechodem do tohoto stavu M přijímá vstup.

MEM_A prvním znakem je a .

MEM_B prvním znakem je b .

CHECK_A kontrola, zda posledním znakem je a .

CHECK_B kontrola, zda posledním znakem je b .

BACK návrat hlavy nad první znak slova na pásce.

ERASE smazání prvního znaku slova na pásce.

Nyní položíme $M = (Q, \Sigma, \delta, \{\text{INIT}\}, \{\text{ACCEPT}\})$.

Práci Turingova stroje M nad vstupem w lze slovy popsat takto: Pokud je vstup prázdný, je vstup přijat přechodem do stavu ACCEPT. V opačném případě si M zapamatuje přechodem do stavu MEM_A nebo MEM_B první symbol, ale zatím jej nechá beze změny. Následuje přesun pásky na konec vstupu, tedy až k prvnímu

prázdnému políčku. Na prvním prázdném políčku učiní M krok hlavou vlevo buď do stavu CHECK_A, pokud zapamatovaný znak je a , nebo do stavu CHECK_B, pokud zapamatovaný znak je b . Zde dojde ke kontrole posledního znaku (který může být současně i prvním v případě vstupu délky 1). Všimněme si, že k odmítnutí dojde ve stavu CHECK_A při čtení znaku b , neboť pro tento případ není hodnota přechodové funkce definovaná. Podobně dojde k odmítnutí ve stavu CHECK_B při čtení znaku a , neboť pro tento případ není hodnota přechodové funkce definovaná. Poslední znak je smazán a pomocí stavu BACK dojde k pohybu hlavy na začátek pásky. Zde je ve stavu ERASE ověřeno, zda ještě řetězec obsahuje první znak. Pokud ano, je smazán a hlava se pohne vpravo na předpokládaný počátek zbylého řetězce. V tomto případě dojde k zahájení další smyčky přechodem do stavu INIT. Pokud již je řetězec prázdný, přejde M ze stavu ERASE do přijímajícího stavu ACCEPT.

Poznamenejme, že PAL je bezkontextový jazyk a je pro něj možné zkonstruovat nedeterministický zásobníkový automat, který čte vstup jen jednou, zatímco jednopáskový Turingův stroj potřebuje projít vstupem w délky n až n -krát (počítáme-li průchody hlavou na konec i zpět na začátek zvlášť).

V příkladu 4.4 jsme tiše předpokládali, že vstupní slovo se skládá pouze ze znaků a a b a neobsahuje prázdná políčka. To je nutné proto, abychom poznali konec vstupu. Kdybychom povolili neomezený výskyt prázdných políček, tak by námi zkonstruovaný Turingův stroj ve skutečnosti selhal na slovech typu $aa\lambda^k a$ pro libovolné $k > 0$, tato slova by byla přijata, i když do jazyka palindromů nepatří. Ve skutečnosti by Turingův stroj nebyl schopen podobné případy rozpoznat, pokud by neznal dopředu horní odhad na hodnotu k . Pokud by totiž přečetl několik znaků λ , nemohl by vědět, zda je už na konci vstupu, nebo jen ještě nepřeskočil dostatečný počet vložených mezer. Mohli bychom sice používat prázdných políček například k oddělení jednotlivých částí vstupu, ale k tomu-to účelu můžeme použít i jiný znak. Při definici Turingova stroje se často rozlišují vstupní, pracovní a výstupní abecedy. V našem případě by tedy vstupní abeceda obsahovala pouze znaky a a b , pracovní abeceda by navíc obsahovala znak prázdného políčka λ . My jsme v definici vstupní abecedy nezaváděli, z kontextu bude vždy zřejmé, jaké znaky jsou očekávány na vstupu. Navíc přijmeme omezení dané následující úmluvou.

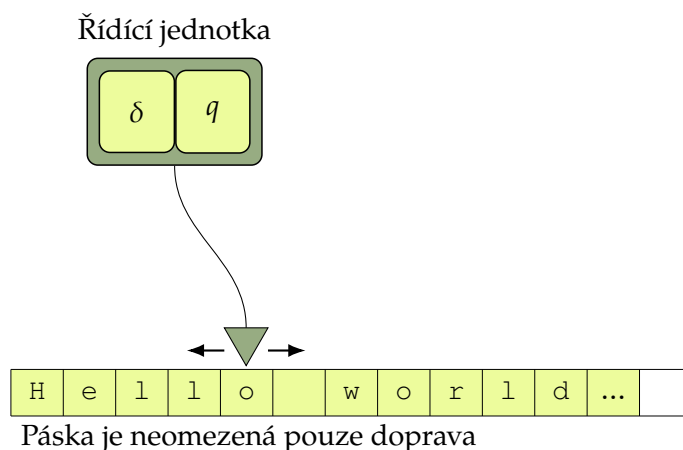
Úmluva 4.5 *Vždy předpokládáme, že vstupní řetězce neobsahují znak λ prázdného políčka.¹*

V případě jazyka přijímaného Turingovým strojem M nás zajímá stav, ve kterém Turingův stroj skončí. Každý Turingův stroj však rovněž počítá funkci z množiny řetězců do množiny řetězců.

Definice 4.6 (Turingovsky vyčíslitelné (řetězcové) funkce)

Nechť $M = (Q, \Sigma, \delta, q_0, F)$ je Turingův stroj. Pomocí f_M označíme funkci, kterou M počítá, tedy funkci $f_M : \Sigma^* \rightarrow \Sigma^*$, jejíž doména je tvořena řetězci $w \in \Sigma^*$, pro které je $M(w) \downarrow$, přičemž pro každý řetězec w z domény f_M platí, že výpočet $M(w)$ skončí s tím, že na vý-

¹Turingovy stroje v této úmluvě nezmiňujeme úmyslně, budeme toto omezení předpokládat i v případě dalších výpočetních modelů.



Obrázek 4.2.: Struktura Turingova stroje s páskou neomezenou jen doprava.

stupní pásce² je napsáno slovo $f_M(w)$. O funkci f , která je počítaná nějakým Turingovým strojem, řekneme, že je *turingovsky vyčíslitelná*. ◀

Turingovu stroji, který je použit pro přijímání slov z daného jazyka, se také někdy říká *akceptor* a Turingovu stroji, který je použit pro počítání řetězcové funkce se někdy říká *transducer*. V obou případech se jedná o též model Turingova stroje, záleží jen na tom, zajímá-li nás po ukončení výpočtu stav řídicí jednotky nebo obsah výstupní pásky.

4.1.2. Varianty Turingových strojů

Model Turingova stroje má řadu variant, které jsou ekvivalentní co do výpočetní síly. Pro nás bude základní variantou jednopáskový deterministický Turingův stroj s oboustranně potenciálně nekonečnou páskou tak, jak byl popsán v kapitole 4.1.1. Některé varianty si popíšeme v následujících podkapitolách. Zaměříme se na varianty, které budou užitečné ve zbytku tohoto textu. Nejpodrobněji se budeme věnovat vícepáskovým Turingovým strojům a nedeterministickým Turingovým strojům. K oběma modelům se později vrátíme ještě v části věnované teorii složitosti, kde zejména nedeterministické Turingovy stroje hrají důležitou roli.

Turingovy stroje s páskou neomezenou jen doprava

V první variantě, kterou si popíšeme, uvažujeme Turingovy stroje, jejichž páska je neomezená pouze doprava. To znamená, že takto omezený Turingův stroj nesmí pohnout hlavou nalevo od její počáteční pozice.

²Zatím uvažujeme jen Turingovy stroje s jednou páskou, která je současně vstupní i výstupní, nicméně definice již počítá s možností mít páska více.

Libovolný Turingův stroj lze převést na Turingův stroj, který má pásku neomezenou pouze doprava. Toto tvrzení si zformulujeme v podobě následující věty, jejíž důkaz ovšem ponecháváme čtenáři jako jednoduché cvičení.

Věta 4.7 *Ke každému Turingovu stroji M (jehož páska je neomezená v obou směrech) existuje Turingův stroj M' , jehož páska je neomezená pouze doprava a který simuluje práci M , přijímá též jazyk jako M a počítá touž funkci jako M .*

Vícepáskové Turingovy stroje

V případě více pásek je obvykle jedna páska určena jako vstupní a jedna jako výstupní, přičemž může jít i o jednu pásku, která plní obě role. Výstupní páska je navíc zajímavá jen v případě Turingova stroje, který počítá nějakou funkci. Ostatní pásy jsou pracovní. Často navíc platí omezení, že na vstupní pásku nelze zapisovat, ale lze z ní jen číst, podobně na výstupní pásku lze často pouze zapisovat a lze se na ní pohybovat jen jedním směrem ve směru zápisu, tedy doprava. Tato omezení mají význam zejména v případě, kdy se začneme zabývat omezeným prostorem a zvláště tím, kolik pracovního prostoru potřebujeme ke zpracování vstupu. K tomu se vrátíme v kapitolách věnujícím se prostorové složitosti algoritmů. Počet pásek je pro daný vícepáskový Turingův stroj pevný a nezávislý na vstupu.

My si zde ukážeme, že každý vícepáskový Turingův stroj lze simulovat na jednopáskovém Turingovu stroji. To nám přinese zjednodušení v řadě situacích, kdy bude snazší popsat vícepáskový Turingův stroj pro daný účel, než jednopáskový. Zjednoduší to například konstrukci univerzálního Turingova stroje.

Zadefinujme si nyní model vícepáskového stroje formálně.

Definice 4.8 *k -páskový deterministický Turingův stroj M , kde $k > 0$ je celočíselná konstanta, je pětice*

$$M = (Q, \Sigma, \delta, q_0, F),$$

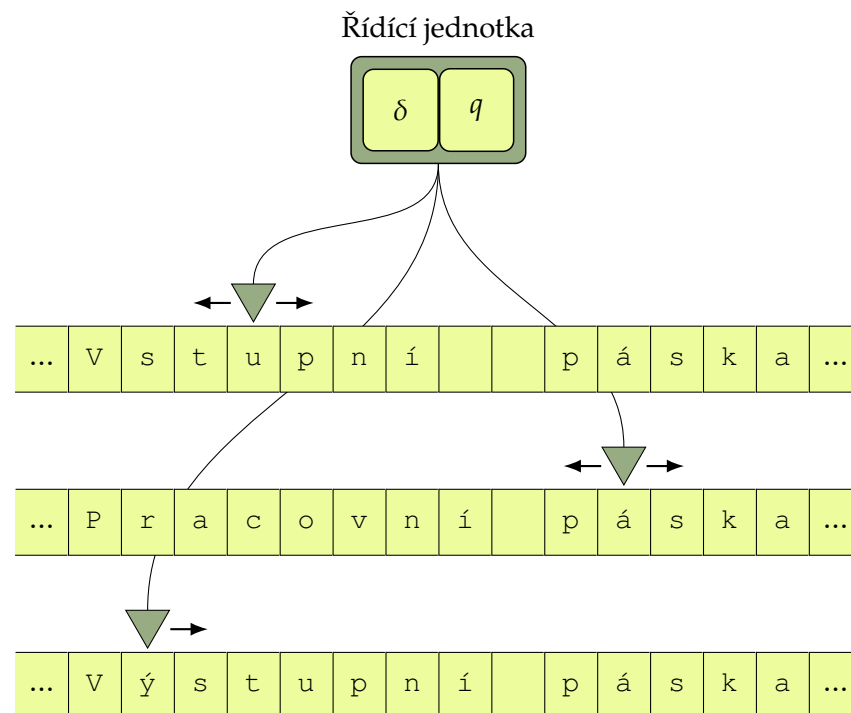
kde

- Q je konečná množina stavů,
- Σ je konečná pásková abeceda, která obsahuje znak λ pro prázdné políčko,
- $\delta : Q \times \Sigma^k \rightarrow Q \times \Sigma^k \times \{R, N, L\}^k \cup \{\perp\}$ je přechodová funkce, kde $\perp$ označuje nedefinovaný přechod,
- $q_0 \in Q$ je počáteční stav a
- $F \subseteq Q$ je množina přijímajících stavů.

Výpočet k -páskového TS probíhá podobně jako výpočet jednopáskového TS s těmito rozdíly:

- Na počátku je vstup napsán na jedné pásce, která je zvolena jako *vstupní*. Tato páska je často určena jen pro čtení a Turingův stroj na ni nezapíše. Hlava se po této pásce může pohybovat v obou směrech.
- Na konci je výstup uložen na jedné pásce, která je zvolena jako *výstupní*. Tato páska bývá často určena jen pro zápis, Turingův stroj z ní nečte a po zapsání znaku přesune hlavu doprava na následující políčko.
- Ostatní pásy jsou *pracovní*, Turingův stroj k nim může přistupovat dle potřeby.
- V každém kroku provede Turingův stroj instrukci, která je daná stavem řídicí jednotky a obsahy políček pod hlavami vstupní a pracovní pásek. Pokud odpovídající instrukce neexistuje, výpočet končí. V opačném případě zapíše na pracovní pásku symboly určené instrukcí, případně zapíše symbol na výstupní pásku, změní stav dle instrukce a vykoná pohyby hlav dané přechodovou funkcí, hlavy se pohybují nezávisle na sobě různými směry. ◀

Na obrázku 4.3 vidíme strukturu vícepáskového Turingova stroje.



Obrázek 4.3.: Struktura 3-páskového Turingova stroje s jednou vstupní, jednou pracovní a jednou výstupní páskou. Výstupní páska je určena jen pro zápis a hlava se může pohybovat jen vpravo.

	q, c_1, c_2	$\rightarrow$	q', c'_1, c'_2, Z_1, Z_2
1.	INIT, a, λ	$\rightarrow$	INIT, a, λ , R, N
2.	INIT, b, λ	$\rightarrow$	INIT, b, λ , R, N
3.	INIT, λ , λ	$\rightarrow$	COPY, λ , λ , L, N
4.	COPY, a, λ	$\rightarrow$	COPY, a, a, L, R
5.	COPY, b, λ	$\rightarrow$	COPY, b, b, L, R
6.	COPY, λ , λ	$\rightarrow$	WBACK, λ , λ , N, L
7.	WBACK, λ , a	$\rightarrow$	WBACK, λ , a, N, L
8.	WBACK, λ , b	$\rightarrow$	WBACK, λ , b, N, L
9.	WBACK, λ , λ	$\rightarrow$	CHECK, λ , λ , R, R
10.	CHECK, a, a	$\rightarrow$	CHECK, a, a, R, R
11.	CHECK, b, b	$\rightarrow$	CHECK, b, b, R, R
12.	CHECK, λ , λ	$\rightarrow$	ACCEPT, λ , λ , N, N

Tabulka 4.2.: Přechodová funkce stroje M' z příkladu 4.9.

Příklad 4.9

Uvažme opět jazyk palindromů, který je nám již známý z příkladu 4.4.

$$\text{PAL} = \{w = w^R \mid w \in \{a, b\}^*\},$$

Vzpomeňme si, že jednopáskový Turingův stroj M přijímající jazyk palindromů, který jsme popsali v příkladu 4.4 měl tu nevýhodu, že musel vstupem délky n projít až n -krát. Máme-li k dispozici dvě pásy, můžeme si na druhou pásku okopírovat zrcadlově otočený vstup a pak zkontrolovat, zda obě pásy obsahují týž řetězec. Na to nám stačí čtyři průchody vstupem (a nejspíš by stačilo i méně). Algoritmus 2 popisuje práci dvoupáskového Turingova stroje M' , který rozhoduje jazyk palindromů. Jeho první páska je vstupní, druhá pracovní. Přechodová funkce δ stroje M' je popsána v tabulce 4.2.

Algoritmus 2: Výpočet M se vstupem w

- 1 Přesuň hlavu na vstupní pásce nad poslední znak vstupu
 - 2 Přesuň hlavu na vstupní pásce zpět na první znak vstupu za současného kopírování znaků na pracovní pásku
// Pracovní páska nyní obsahuje w^R
 - 3 Přesuň hlavu na pracovní pásce nad první znak w^R
 - 4 Pohybem hlav na obou páskách současně zkontroluj, zda obsahují totéž slovo (tedy zda $w = w^R$)
-

Opět použijeme páskovou abecedu $\Sigma = \{\lambda, a, b\}$, která ke dvěma vstupním sym-

bolům obsahuje jen znak prázdného políčka. Množina stavů Q se nyní skládá jen z 5 stavů, jejichž význam lze popsat následujícím způsobem.

INIT počáteční stav, přesun hlavy na vstupní pásce na konec vstupu.

COPY kopírování zrcadlově otočeného vstupu na pracovní pásku.

WBACK přesun hlavy na pracovní pásce na začátek slova.

CHECK kontrola shody slov na vstupní a pracovní pásce.

ACCEPT jediný přijímající stav.

Nyní položíme $M' = (Q, \Sigma, \delta, \{\text{INIT}\}, \{\text{ACCEPT}\})$.

Naším dalším cílem je ukázat, že každý k -páskový Turingův stroj M má svůj ekvivalentní jednopáskový Turingův stroj M' . V tomto smyslu budeme hovořit o tom, že M' *simuluje práci Turingova stroje* M . Pojem simulace nebudeme zavádět příliš formálně, nicméně budeme jej používat v následujícím smyslu. Je-li M_1 Turingův stroj, který simuluje TS M_2 , pak předpokládáme, že na některých páskách M_1 se nějakým způsobem postupně objevuje obsah pásek stroje M_2 a že z práce M_1 lze vyčíst výpočet stroje M_2 , přičemž simulace jednoho kroku stroje M_2 může vyžadovat provedení několika kroků stroje M_1 .

Někdy budeme uvažovat situaci, kdy k jednomu TS M_3 s nějakou vlastností (například k -páskovému) zkonstruujeme jiný TS M_4 s jinou vlastností (například jednopáskový), který přijímá též jazyk nebo počítá touž funkci, v tom případě nepožadujeme, aby výpočet M_3 probíhal tímž způsobem jako výpočet M_4 . Jde o obecnější princip, než simulace, ačkoli simulace stroje M_4 strojem M_3 může být jedním způsobem, pomocí něž dosáhneme toho, aby oba stroje přijímaly též jazyk.

Věta 4.10 *Nechť $k \geq 2$ je přirozené číslo. Ke každému k -páskovému Turingovu stroji M existuje jednopáskový Turingův stroj M' , který simuluje práci M , přijímá též jazyk jako M a počítá touž funkci jako M .*

Důkaz: Podáme zde pouze ideu důkazu bez technických detailů. Předpokládejme, že $M = (Q, \Sigma, \delta, q_0, F)$ a že konstruovaný stroj $M' = (Q', \Sigma', \delta', q'_0, F')$. Při konstrukci jednopáskového Turingova stroje M' musíme vyřešit zejména následující dva problémy:

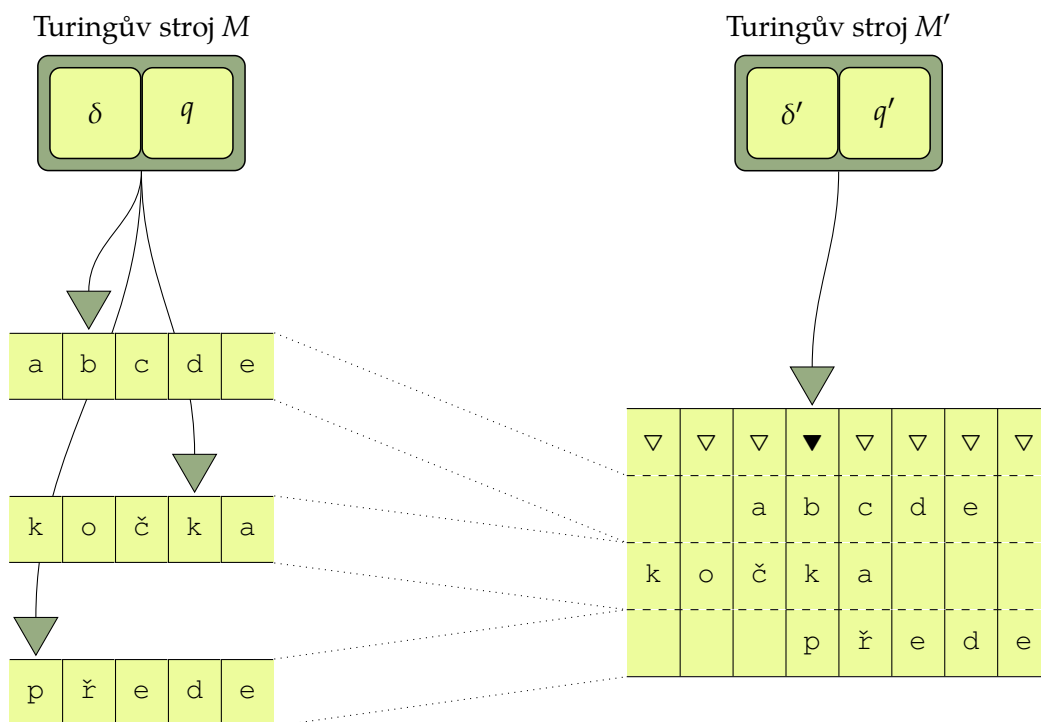
1. Jak reprezentovat k pásek na jedné pásce a
2. jak si poradit s tím, že hlavy na k páskách stroje M se pohybují nezávisle na sobě.

Reprezentaci více pásek na jedné pásce vyřešíme tím, že budeme k políček, která jsou na páskách nad sebou, reprezentovat v jednom políčku pásky stroje M' . Můžeme si to představit tak, že páska stroje M' bude mít k stop. Stopa je v tomto smyslu podobná pásce, ovšem s tím rozdílem, že máme k dispozici jen jednu hlavu pro všechny stopy.

Musíme si tedy poradit s tím, že ve stroji M máme k hlav a v stroji M' jen jednu. Při simulaci budeme zachovávat následující invariant:

Invariant I: Na začátku simulace kroku stroje M jsou pásky stroje M na pásce M' vždy zarovnané tak, aby čtené symboly byly pod sebou v jedné buňce více stopé pásky.

K tomu bude nutné posouvat obsahy stop při simulaci kroku stroje M tak, abychom dosáhli kýženého stavu. Pro posouvání budeme potřebovat značku $\blacktriangledown$ na buňku s hlavou v místě, nad kterým mají být hlavy stroje M , aby M' po posunutí stopy věděl, na jakou buňku se má s hlavou vrátit. Struktura stroje M' je naznačena na obrázku 4.4. Popišme



Obrázek 4.4.: Reprezentace 3 pásek stroje M pomocí 4 stop na jediné pásce M' . Čárkovanou čarou jsou od sebe oddělené jednotlivé stopy na pásce M' . Obsahy pásek jsou na stopách zarovnané tak, aby hlavy stroje M byly pod sebou. Značka $\blacktriangledown$ reprezentuje místo, nad kterým jsou hlavy stroje M .

si nyní konstrukci stroje M' podrobněji. Páskovou abecedu stroje M' položíme

$$\Sigma' = \Sigma \cup \left(\{ \nabla, \blacktriangledown \} \times \Sigma^k \right).$$

Jedna buňka stroje M' tedy reprezentuje k buněk stroje M , která jsou na k páskách nad sebou. Navíc abeceda umožňuje rozlišit pomocí značek $\blacktriangledown$ a ∇ zda je nebo není nad da-

nou buňkou hlava³. Stroj M' má vždy zformátovanou tu oblast pásky, která zachycuje obsah pásek M . Na začátku má tato oblast délku vstupu, v průběhu simulace se může zformátovaná oblast zvětšovat podle toho, jak M využívá další prostor na páskách. Zformátovaná oblast pásky je ohraničena prázdnými políčky a je tedy vždy poznat, kde končí a začíná (uvažujeme případ $k > 1$, pro $k = 1$ je možné rovnou nechat stroj M beze změny). Stroj M' bude při své práci postupovat následujícím způsobem (lépe řečeno přechodová funkce δ' stroje M' je zkonstruována tak, aby implementovala následující kroky):

1. V rámci inicializace proběhne přeformátování vstupu do stop. Vstup je na vstupu předán pomocí symbolů v abecedě Σ . Stroj M' si přepíše vstup tak, že nejlevější buňku vstupu obsahující $a \in \Sigma$ přepíše na $\blacktriangledown a \lambda^{k-1}$ a další buňky vstupu, které obsahují symbol $a \in \Sigma$ přepíše na $\nabla a \lambda^{k-1}$. To odpovídá tomu, že na počátku jsou pracovní pásky M prázdné, jsou tedy prázdné i odpovídající stopy. Hlava je nad nejlevější buňkou vstupu, proto je na ní zapsán symbol $\blacktriangledown$, zatímco nad dalšími buňkami je symbol ∇ . Přechodová funkce δ' Turingova stroje M' bude obsahovat instrukce, které provedou tuto inicializaci.
2. Předpokládáme, že jednotlivé instrukce z přechodové funkce δ Turingova stroje M jsou očíslované. Místo i -té instrukce $\delta(q, a_1, \dots, a_k) = (r, b_1, \dots, b_k)$, kde $q, r \in Q$ a $a_1, \dots, a_k, b_1, \dots, b_k \in \Sigma$ přidáme posloupnost instrukcí do přechodové funkce δ' , který provede simulaci této instrukce v stroji M' . Rozlišení toho, kterou posloupnost instrukcí potom M' zvolí, bude provedeno na základě displeje, δ' bude definovat přechod $\delta'(q, \blacktriangledown a_1 \dots a_k) = (q^i, \blacktriangledown a_1 \dots a_k, N)$, přičemž stavem q^i začíná posloupnost instrukcí simulující provedení i -té instrukce stroje M . Tato posloupnost se skládá z k fází. V j -té fázi pro $j = 1, \dots, k$ je odsimulována instrukce na j -té stopě (odpovídající j -té pásce stroje M). Pokud i -tá instrukce na j -té pásce stroje M prikazuje pohnout hlavou vlevo, pak se celý obsah j -té stopy posune o jedno políčko vpravo. Pokud se má pohnout hlavou na j -té pásce M vpravo, pak se celý obsah j -té stopy posune o jedno políčko vlevo. Pokud má hlava zůstat na místě, ani stopa stroje M' se neposouvá. Při posouvání stopy o políčko vpravo nebo vlevo je využita značka $\blacktriangledown$. Má-li například dojít k posunu obsahu stopy vpravo, pak Turingův stroj M' dojde na začátek pásky, a cestou doprava posouvá políčko po políčku na této stopě o jedno vpravo, zatímco ostatní stopy ponechává beze změny. Pokud poslední buňka na stopě nebyla prázdná, je nutné přiformátovat jedno políčko na pravém okraji. Posun vlevo probíhá obdobně. Po posunutí pásky M' díky značce $\blacktriangledown$ pozná, kam se má vrátit. Po provedení poslední, tedy k -té fáze přejde M' do stavu r , v němž dojde podle čtené k -tice znaků k rozhodnutí, jaká posloupnost instrukcí se použije pro odsimulování další instrukce stroje M .

3. Za každou kombinaci stavu q a k -tice znaků $a_1, \dots, a_k \in \Sigma$, pro které je $\delta(q, a_1, \dots, a_k) =$

³Je potřeba zmínit, že zde využíváme toho, že abeceda Turingova stroje může být libovolně velká a můžeme si ji tedy libovolně zvětšit, pokud zachováme to, že její velikost je pro daný Turingův stroj konstantní a není závislá na velikosti vstupu. Pokud bychom chtěli zachovat abecedu beze změny, mohli bychom reprezentovat k buněk stroje M pomocí $k + 1$ po sobě jdoucích políček.

$\perp$, přidáme do δ' přechod do počátečního stavu posloupnosti instrukcí provádějící překódování obsahu pásky tak, aby na ní zůstal jen obsah výstupní pásky stroje M . $\square$

Všimněme si, že jednopáskový Turingův stroj M' potřebuje k odsimulování provedení jedné instrukce stroje M až $\Theta(nk)$ instrukcí, kde n je délka vstupu.

Nedeterministické Turingovy stroje

Nedeterministický Turingův stroj rozšiřuje deterministický Turingův stroj o možnost činit několik rozhodnutí najednou. Základním modelem nedeterministického Turingova stroje pro nás bude stroj jednopáskový, i když budeme uvažovat i vícepáskové nedeterministické Turingovy stroje.

Definice 4.11 (Nedeterministický Turingův stroj) (*Jednopáskový*) *nedeterministický Turingův stroj (NTS)* je pětice $M = (Q, \Sigma, \delta, q_0, F)$, jejíž prvky mají též význam jako v případě deterministického Turingova stroje s tím rozdílem, že přechodová funkce má tvar

$$\delta : Q \times \Sigma \rightarrow \wp(Q \times \Sigma \times \{R, N, L\}).$$

Jinými slovy přechodová funkce danému stavu a symbolu na pásce přiřazuje několik přechodů do jiných stavů se zápisem různých symbolů a s různými pohyby. V případě, že množina možných přechodů je prázdná, není přechod definován.

- Podobně jako v případě deterministického Turingova stroje rozšíříme použití přechodové funkce i na konfigurace, tedy $\delta(K)$ označuje množinu konfigurací, která vznikne aplikací přechodové funkce na konfiguraci K na základě displeje, který je v konfiguraci K obsažen.
- *Výpočet nedeterministického Turingova stroje nad vstupem $x \in \Sigma^*$* je posloupnost konfigurací, která začíná počáteční konfigurací K_0 a pro každé dvě následující konfigurace K, K' v posloupnosti platí, že $K' \in \delta(K)$. V každém kroku si tedy nedeterministický Turingův stroj vybere, kterou z možností zvolí a tu aplikuje.
- Výpočet nedeterministického Turingova stroje je *konečný*, jde-li o konečnou posloupnost konfigurací $K_0, \dots, K_t$ a pro poslední konfiguraci K_t platí, že $\delta(K_t) = \emptyset$.
- *Výpočet nedeterministického Turingova stroje* je *přijímající*, pokud je konečný a jeho poslední konfigurace je přijímající, tj. M je na konci v přijímajícím stavu.
- Řekneme, že nedeterministický Turingův stroj M *přijme slovo x* , pokud existuje výpočet nedeterministického Turingova stroje M nad x , který je přijímající.
- Jazyk slov přijímaných nedeterministickým Turingovým strojem M označíme pomocí $L(M)$. $\blacktriangleleft$

Nedeterministický Turingův stroj se od deterministického liší jen tím, že v dané chvíli umožňuje použít víc přechodů. Náš způsob definice výpočtu nedeterministického Turingova stroje předpokládá, že nedeterministický Turingův stroj se nachází vždy v jedné konfiguraci, pokud má z této konfigurace na výběr několik možných přechodů, jeden z nich si vybere, tedy „uhodne“ jej. A pokud nějaká posloupnost těchto výběrů či „uhodnutí“ vede k přijetí, řekneme, že nedeterministický Turingův stroj daný vstup přijal. Při definici výpočtu nedeterministického Turingova stroje M však na rozdíl od deterministického Turingova stroje nepožadujeme, aby poslední konfigurace K_t v posloupnosti byla koncová, tedy aby již nebylo možno dále pokračovat aplikací přechodové funkce na konfiguraci K_t . Ponecháme na nedeterministické volbě stroje M , zda v dané konfiguraci zastaví výpočet či bude dál pokračovat. Obvykle budeme předpokládat, že přijímající konfigurace (tj. konfigurace, v nichž je M v přijímajícím stavu) koncové jsou.

Výpočet nedeterministického Turingova stroje si však lze představit i tak, že v každém kroku nedeterministický Turingův stroj použije všechny možné přechody a nachází se tak vždy ve všech dostupných konfiguracích najednou, podobně si obvykle představujeme i práci nedeterministického konečného automatu. V obou případech si také můžeme výpočet nedeterministického Turingova stroje reprezentovat pomocí stromu výpočtu.

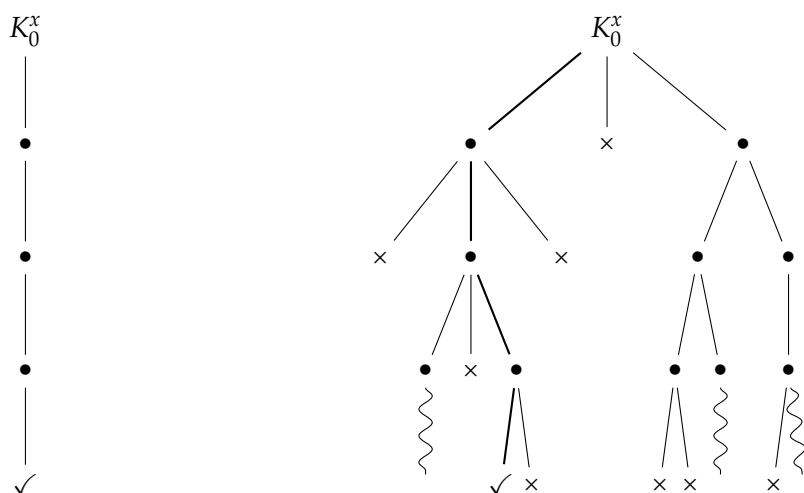
Definice 4.12 (Strom výpočtu nedeterministického Turingova stroje)

Nechť $M = (Q, \Sigma, \delta, q_0, F)$ je nedeterministický Turingův stroj. Strom výpočtu stroje M nad vstupem $x \in \Sigma^*$ je dvojice $T, \mathcal{L}$, kde

- $T = (V, E)$ je orientovaný zakořeněný strom, přičemž V a E jsou (potenciálně i nekonečné) množiny vrcholů a hran. Hrany jsou orientované od kořene směrem k listům.
- $\mathcal{L}$ je funkce, která každému vrcholu $v \in V$ přiřazuje konfiguraci stroje M a která splňuje následující podmínky:
 - Kořeni $r \in V$ stromu T přiřazuje funkce $\mathcal{L}$ počáteční konfiguraci výpočtu (tj. na pásce je zapsán vstup x , stroj M je v počátečním stavu q_0 a hlava je nad nejlevějším symbolem vstupu).
 - Pokud $(u, v) \in E$ je hrana, pak $\mathcal{L}(v) \in \delta(\mathcal{L}(u))$.
 - Pokud $u \in V$ je uzel stromu, pak pro každou konfiguraci $K \in \delta(\mathcal{L}(u))$ existuje vrchol $v \in V$, pro který platí, že $\mathcal{L}(v) = K$ a $(u, v) \in E$. ◀

Všimněme si, že jsme přesně nespecifikovali, co u nedeterministického Turingova stroje M znamená „odmítnutí“ vstupu. V případě nedeterministického Turingova stroje nás obvykle ani odmítnutí nezajímá a zajímáme se pouze o to, zda stroj přijme. Dá se ale říci, že M „odmítně“ svůj vstup, pokud je jeho strom výpočtu konečný (tj. všechny větve výpočtu jsou konečné) a žádná z nich není přijímající.

Podle naší definice je výpočet nedeterministického stroje M nad vstupem x posloupností konfigurací, což odpovídá jedné větvi stromu výpočtu. V případě, že M je deterministický Turingův stroj, strom výpočtu tvoří cestu.



Strom deterministického výpočtu

Strom nedeterministického výpočtu

Obrázek 4.5.: Nalevo je naznačen strom výpočtu deterministického Turingova stroje, tedy cesta. Napravo je naznačen strom výpočtu nedeterministického stroje. Tučně je vyznačena větev ukončená symbolem $\checkmark$ reprezentujícím přijímající konfigurací, tedy přijímající výpočet. Větve ukončené symboly $\times$ jsou ukončené, ale nikoli přijímající. Vlnovky na koncích dalších větví naznačují, že větve dále pokračují a jsou neukončené. K_0^x v kořeni obou stromů označuje počáteční konfiguraci při výpočtu nad vstupem x .

	$q, c_1, c_2, c_3 \rightarrow q', c'_1, c'_2, c'_3, Z_1, Z_2, Z_3$	Hodnoty proměnných
Kopírování vstupu s nedeterministickým ukončením		
1.	$\text{INIT}, \alpha, \lambda, \lambda \rightarrow \text{INIT}, \alpha, \alpha, \lambda, R, R, N$	$\alpha \in \{a, b\}$
2.	$\text{INIT}, \alpha, \lambda, \lambda \rightarrow \text{COPY_U}, \alpha, \lambda, \lambda, N, L, N$	$\alpha \in \{a, b\}$
3.	$\text{INIT}, \lambda, \lambda, \lambda \rightarrow \text{COPY_U}, \lambda, \lambda, \lambda, N, L, N$	
Zde začíná kontrola palindromu na druhé pásce		
4.	$\text{COPY_U}, \alpha, \beta, \lambda \rightarrow \text{COPY_U}, \alpha, \beta, \beta, N, L, R$	$\alpha \in \{a, b, \lambda\}, \beta \in \{a, b\}$
5.	$\text{COPY_U}, \alpha, \lambda, \lambda \rightarrow \text{WBACK_U}, \alpha, \lambda, \lambda, N, N, L$	$\alpha \in \{a, b, \lambda\}$
6.	$\text{WBACK_U}, \alpha, \lambda, \beta \rightarrow \text{WBACK_U}, \alpha, \lambda, \beta, N, N, L$	$\alpha \in \{a, b, \lambda\}, \beta \in \{a, b\}$
7.	$\text{WBACK_U}, \alpha, \lambda, \lambda \rightarrow \text{CHECK_U}, \alpha, \lambda, \lambda, N, R, R$	$\alpha \in \{a, b, \lambda\}$
8.	$\text{CHECK_U}, \alpha, \beta, \beta \rightarrow \text{CHECK_U}, \alpha, \lambda, \lambda, N, R, R$	$\alpha \in \{a, b, \lambda\}, \beta \in \{a, b\}$
9.	$\text{CHECK_U}, \alpha, \lambda, \lambda \rightarrow \text{PART_V}, \alpha, \lambda, \lambda, N, N, N$	$\alpha \in \{a, b, \lambda\}$
Zde začíná kopírování zbytku vstupu na druhou pásku		
10.	$\text{PART_V}, \alpha, \lambda, \lambda \rightarrow \text{PART_V}, \alpha, \alpha, \lambda, R, R, N$	$\alpha \in \{a, b\}$
11.	$\text{PART_V}, \lambda, \lambda, \lambda \rightarrow \text{COPY_V}, \lambda, \lambda, \lambda, N, L, N$	
Zde začíná kontrola palindromu na druhé pásce		
12.	$\text{COPY_V}, \alpha, \beta, \lambda \rightarrow \text{COPY_V}, \alpha, \beta, \beta, N, L, R$	$\alpha \in \{a, b, \lambda\}, \beta \in \{a, b\}$
13.	$\text{COPY_V}, \alpha, \lambda, \lambda \rightarrow \text{WBACK_V}, \alpha, \lambda, \lambda, N, N, L$	$\alpha \in \{a, b, \lambda\}$
14.	$\text{WBACK_V}, \alpha, \lambda, \beta \rightarrow \text{WBACK_V}, \alpha, \lambda, \beta, N, N, L$	$\alpha \in \{a, b, \lambda\}, \beta \in \{a, b\}$
15.	$\text{WBACK_V}, \alpha, \lambda, \lambda \rightarrow \text{CHECK_V}, \alpha, \lambda, \lambda, N, R, R$	$\alpha \in \{a, b, \lambda\}$
16.	$\text{CHECK_V}, \alpha, \beta, \beta \rightarrow \text{CHECK_V}, \alpha, \lambda, \lambda, N, R, R$	$\alpha \in \{a, b, \lambda\}, \beta \in \{a, b\}$
17.	$\text{CHECK_V}, \alpha, \lambda, \lambda \rightarrow \text{ACCEPT}, \alpha, \lambda, \lambda, N, N, N$	$\alpha \in \{a, b, \lambda\}$

Tabulka 4.3.: Přechodová funkce nedeterministického Turingova stroje N . Používáme zkrácený způsob zápisu s proměnnými α a β . Řádek využívající symboly α a β reprezentují několik instrukcí, v nichž jsou α a β nahrazeny postupně všemi hodnotami danými třetím sloupcem.

Příklad 4.13

Vzpomeňme si, že jsme již zkonstruovali dva deterministické stroje, které testují, zda slovo patří do jazyka palindromů

$$\text{PAL} = \{w = w^R \mid w \in \{a, b\}^*\},$$

jednopáskový stroj v příkladu 4.4 a dvoupáskový stroj v příkladu 4.9. Uvažme nyní jazyk

$$\text{PAL}^2 = \text{PAL} \cdot \text{PAL} = \{uv \mid u, v \in \{a, b\}^* \wedge u = u^R \wedge v = v^R\}$$

Jazyk PAL^2 tedy obsahuje řetězce, jež vzniknou konkatencí dvou palindromů za sebe.

Rozmysleme si nejprve, jak by mohl postupovat deterministický Turingův stroj M přijímající jazyk PAL^2 . Pro dané vstupní slovo $w \in \{a, b\}^*$ délky $n = |w|$ by M musel uvážit všechna místa dělení, tedy pro každé $j = 0, \dots, n$ by musel M uvážit dvojici slov $u = w[1] \dots w[j]$, $v = w[j+1] \dots w[n]$ (přičemž $u = \varepsilon$ pro $j = 0$ a $v = \varepsilon$ pro $j = n$) a otestovat, zda jde o palindromy. Test toho, zda je slovo palindrom by bylo nutné spustit až $2(n+1)$ -krát.

Popišme nyní nedeterministický Turingův stroj N , který přijímá jazyk PAL^2 . Nedeterminismus nám umožňuje „uhodnout“ správné rozdělení vstupního slova w na dvě části u a v a poté jen otestovat, zda obě části jsou palindromy. Předpokládejme pro jednoduchost, že N je třípáskový Turingův stroj. Vstup je na začátku zapsán na první pásce, která slouží jen ke čtení vstupu a N na ni nezapíše. Stroj bude postupovat dle algoritmu 3.

Algoritmus 3: Výpočet N se vstupem w

- 1 Kopíruj vstupní slovo na druhou pásku
 - 2 V nějakou chvíli se (nedeterministicky) kopírování zastaví
 - 3 Otestuj, zda slovo na druhé pásce je palindrom, pokud ne, výpočet končí odmítnutím. K tomuto testu jsou využity obě pracovní pásy a je použit postup popsáný v příkladu 4.9
 - 4 Vymaž obě pracovní pásy
 - 5 Okopíruj zbytek vstupního slova na druhou pásku
 - 6 Otestuj, zda slovo na druhé pásce je palindrom, pokud ne, výpočet končí odmítnutím
 - 7 **accept**
-

Stroj N tedy při jednom výpočtu nad vstupem w vyzkouší jednu možnost rozdělení slova w na podslova u a v . Slovo w patří do slova PAL^2 , právě když jeden z těchto výpočtů uspěje. Nedeterminismus je zde obsažen pouze v kroku 2, tedy v tom, kdy N ukončí první podslovo u .

Pásková abeceda $\Sigma = \{\lambda, a, b\}$ obsahuje ke dvěma znakům vstupní abecedy jen znak prázdného políčka. Přechodová funkce δ stroje N je popsána tabulkou 4.3.

Množina stavů Q se skládá z následujících stavů.

INIT jediný počáteční stav, stroj N v něm posune hlavu na vstupní páse doprava se současným kopírováním vstupu na druhou pásku (instrukce 1). Při tom se N v libovolný okamžik může nedeterministicky rozhodnout (v instrukci 2) kopírování vstupu ukončit a přechodem do stavu **COPY_U** zahájit kontrolu toho, zda je slovo na druhé pásce palindromem. K tomu nejpozději dojde ve chvíli, kdy dosáhne hlava na první páse konce vstupu (instrukce 3).

COPY_U zkopíruje zrcadlově otočené slovo z druhé pásy na třetí pásku. To odpovídá stavu **COPY** Turingova stroje M' popsaného v příkladu 4.9. Stav **INIT** stroje M' z uvedeného příkladu můžeme přeskočit, protože hlava na druhé pásce již je na konci slova na druhé pásce.

WBACK_U vrátí hlavu na třetí páse na začátek slova.

CHECK_U ověření toho, zda se slovo na druhé pásce shoduje se slovem na třetí pásce. Současně maže obsahy obou těchto pásek. V případě shody dojde k přechodu do stavu **PART_V**, v opačném případě dojde k ukončení práce N , a tedy i odmítnutí.

PART_V okopíruje zbytek slova z první pásky na druhou pásku.

COPY_V jako **COPY_U** pro druhou část slova.

WBACK_V jako **WBACK_U** pro druhou část slova.

CHECK_V jako **CHECK_U** pro druhou část slova. Jediným rozdílem je, že v případě shody druhé se třetí páskou dojde k přijetí.

Nyní položíme $N = (Q, \Sigma, \delta, \{\text{INIT}\}, \{\text{ACCEPT}\})$.

Nedeterministická volba je skryta pouze v instrukcích 1 a 2 přechodové funkce. Stroj N čte prochází vstupem instrukcí 1 a v každém místě má možnost nedeterministicky se rozhodnout ukončit první slovo a začít test toho, zda jde o palindrom. Ve chvíli, kdy dojde N na konec vstupu, instrukce 3 ukončí první slovo a druhé podslovo bude pak prázdné. Bloky instrukcí 4 až 9 a 12 až 17 jsou převzaty z přechodové funkce Turingova stroje z příkladu 4.9, jsou upraveny jen v tom smyslu, že používají druhou a třetí pásku, jež po sobě nechávají prázdné. Instrukce 10 a 11 kopírují druhé slovo na druhou řádku.

Zde si ukážeme, že práci nedeterministického Turingova stroje M lze simulovat na deterministickém Turingovu stroji M' . Z tohoto hlediska nepřináší nedeterminismus nic pro teorii rozhodnutelnosti. Zajímavější je však situace v teorii složitosti, kde nedeterministické Turingovy stroje hrají podstatnou roli.

Věta 4.14 Ke každému nedeterministickému Turingovu stroji M existuje jednopáskový deterministický Turingův stroj M' , který simuluje práci M a přijímá též jazyk jako M .

Důkaz: Turingův stroj M' může například implementovat průchod do šířky stromem výpočtu M . Stroj M' se vstupem x sestaví počáteční konfigurací výpočtu $M(x)$ a vloží ji do fronty. Navíc si M' inicializuje prázdný seznam S navštívených konfigurací. V každém dalším kroku pak vyzvedne konfiguraci C z fronty, vloží ji do seznamu S navštívených konfigurací a aplikuje na ni všechny možné přechody dle δ . Tím dostane nové konfigurace $C_1, \dots, C_r$, kde $r \leq |Q| \cdot |\Sigma| \cdot 3$. Pokud je kterákoli z těchto konfigurací přijímací, pak M' přijme. V opačném případě vloží M' do fronty ty z konfigurací $C_1, \dots, C_r$, které ještě nejsou na seznamu S ani ve frontě. Pokud je v tuto chvíli fronta prázdná, pak M odmítne. $\square$

Zkuste si rozmyslet, proč není možné stejně přímočaře použít i průchod do hloubky stromem výpočtů.

4.2. Random Access Machine

V této kapitole zavedeme další výpočetní model, a to RAM — *random access machine*, tedy stroj s náhodným přístupem do paměti. Jde o model, který se často (ačkoli se to ne vždy zmiňuje) používá jako základní výpočetní model při měření časové a prostorové složitosti algoritmů. Motivací k zavedení tohoto modelu byla právě snaha přiblížit se více k reálným počítačům a umožnit tak studium algoritmů a jejich implementací pro klasické počítače. Při definici a popisu dalších vlastností budeme vycházet z článku [4], v němž byl model RAM zaveden. V literatuře je ovšem možno najít řadu jiných ekvivalentních definic, které se liší zejména v použitých instrukcích a způsobu zápisu programů.

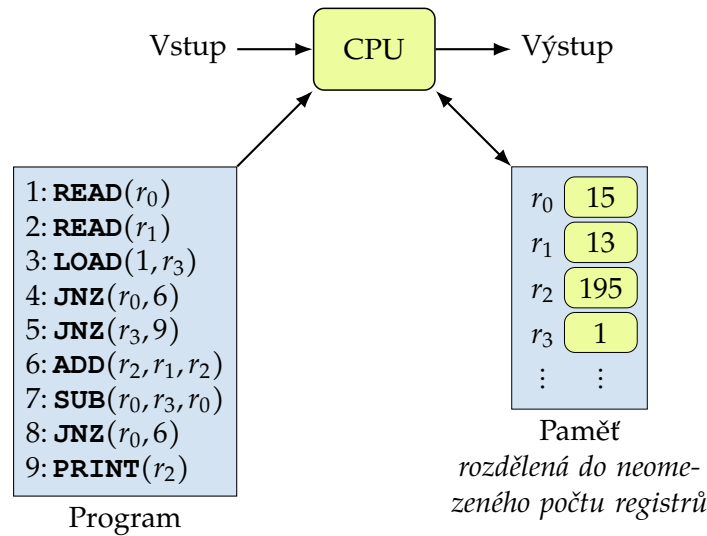
4.2.1. Definice

Podobně jako Turingovy stroje, i RAM je strojem s oddělenou pamětí pro data a pro instrukce. Struktura RAM je zobrazena na obrázku 4.6. RAM se skládá ze tří podstatných částí:

Program je konečnou posloupností instrukcí $I_0, I_1, I_2, \dots, I_\ell$, které jsou očíslované přirozenými čísly, očíslování určuje pořadí provádění těchto instrukcí a současně návěští pro příkaz podmíněného skoku. Jednotlivým prvkům posloupnosti instrukcí říkáme řádky programu.

Paměť pro data se skládá z neomezené posloupnosti registrů $r_0, r_1, r_2, \dots$, které jsou očíslované přirozenými čísly počínaje nulou, těmto číslům budeme říkat *adresy*. Obsahem registru může být libovolně velké přirozené číslo⁴.

⁴Fakt, že se omezuje na přirozená čísla, není nijak podstatný, klidně bychom mohli povolit libovolná celá čísla. Použití přirozených čísel má tu výhodu, že číslo uložené v registru je možné použít i jako ad-



Obrázek 4.6.: Struktura RAM. Program na obrázku implementuje RAM, který počítá funkci násobení.

Řídící jednotka (*central processing unit, CPU*) vykonává program. Instrukce jsou postupně vykonávány v pořadí, v jakém jsou zapsané v programu, i když některé instrukce (například instrukce podmíněného skoku) mohou toto pořadí měnit. Řídící jednotka se při své práci rovněž stará o čtení hodnot ze vstupu a zápis hodnot na výstup.

Program pro RAM se skládá z instrukcí, které jsou popsány v tabulce 4.2.1. Při jejich popisu používáme následující konvence:

- Obsah registru r_i označujeme pomocí $[r_i]$.
- V popisu instrukcí může být adresa registru určena nepřímo, tedy obsahem jiného registru, pomocí $\llbracket r_i \rrbracket$ tak označujeme obsah registru s adresou, která je určena obsahem registru r_i , tj. je-li $j = [r_i]$ obsahem registru r_i , pak $\llbracket r_i \rrbracket = [r_{[r_i]}] = [r_j]$.
- Při popisu instrukcí dále používáme symbol $\leftarrow$ ve významu přiřazení, levá strana přitom určuje registr, do kterého má být přiřazena hodnota výrazu na pravé straně. Například výraz $r_i \leftarrow C$ popisuje efekt instrukce **LOAD**(C, r_i), která do registru r_i uloží hodnotu C .

Sada instrukcí popsaná v tabulce 4.2.1 zdaleka není jediná možná. Některé varianty neumožňují obecné sčítání a odčítání, ale pouze přičtení jedničky (*increment*) a odečtení jedničky (*decrement*). Další varianty připouštějí aritmetické operace nebo nepřímou adresaci pouze s použitím zvlášť pro to vyhrazeného registru, kterému se říká akumulátor.

resu registru. V případě potřeby není velký problém reprezentovat i záporná čísla s pomocí přirozených čísel.

Instrukce	Efekt	Popis
LOAD (C, r_i)	$r_i \leftarrow C$	Přiřadí do registru r_i konstantu $C \in \mathbb{N}$.
ADD (r_i, r_j, r_k)	$r_k \leftarrow [r_i] + [r_j]$	Sečte obsahy registrů r_i a r_j a výsledek uloží do registru r_k . Adresy registrů r_i, r_j, r_k nemusí být nutně různé.
SUB (r_i, r_j, r_k)	$r_k \leftarrow \max([r_i] - [r_j], 0)$	Od obsahu registru r_i odečte obsah registru r_j a výsledek uloží do registru r_k . Adresy registrů r_i, r_j, r_k nemusí být nutně různé.
COPY ($[r_p], r_d$)	$r_d \leftarrow \llbracket r_p \rrbracket$	Do registru r_d zkopíruje obsah registru s adresou určenou obsahem registru r_p .
COPY ($r_s, [r_d]$)	$r_{[r_d]} \leftarrow [r_s]$	Do registru, jehož adresa je určena obsahem registru r_d , zkopíruje obsah registru r_s .
JNZ (r_i, I_z)	if ($[r_i] > 0$) goto z	Podmíněný skok (<i>jump if not zero</i>). Pokud je v registru r_i kladné číslo, pak program dále pokračuje instrukcí I_z , v opačném případě (tj. $[r_i] = 0$) program pokračuje instrukcí následující po právě vykonávané instrukci JNZ .
READ (r_i)	$r_i \leftarrow \text{input}$	Do registru r_i přečte další číslo na vstupu. Pokud není na vstupu další číslo, načte hodnotu 0.
PRINT (r_i)	$\text{output} \leftarrow [r_i]$	Hodnotu uloženou v registru r_i zapíše na výstup.

Tabulka 4.4.: Seznam instrukcí RAM

Na druhou stranu v některých případech se připouští obecná aritmetika, tedy ne jen sčítání a odčítání, ale například i násobení či celočíselné dělení. Prozatím se nezabýváme časovou a prostorovou složitostí, poznamenejme však již nyní, že vzhledem k tomu, že jednotlivé registry mohou obsahovat neomezeně velká čísla, je potřeba při počítání časové a prostorové složitosti algoritmu vzít do úvahy velikost reprezentace čísel uložených v použitých registrech. Tomu se budeme podrobněji věnovat v části III.

Příklad 4.15

Model RAM si ukážeme na jednoduchém příkladu stroje, který počítá funkci násobení. Program očekává na vstupu dvě čísla x_1 a x_2 , jejichž součin spočítá a vypíše na výstup. RAM R , který bude tuto funkci implementovat, bude postupovat podle velmi jednoduchého algoritmu: K hodnotě 0 postupně x_1 -krát přičte hodnotu x_2 .

Algoritmus 4: Program RAM pro výpočet součinu

```

1 READ( $r_0$ )
2 READ( $r_1$ )
   //  $r_2 \leftarrow 0$  není třeba
3 LOAD(1,  $r_3$ )                                     //  $r_3 \leftarrow 1$ 
4 JNZ( $r_0$ , 6)                                     // if [ $r_0$ ] > 0 then goto 6
5 JNZ( $r_3$ , 9)                                     // goto 9
6 ADD( $r_2$ ,  $r_1$ ,  $r_2$ )                             //  $r_2 \leftarrow [r_2] + [r_1]$ 
7 SUB( $r_0$ ,  $r_3$ ,  $r_0$ )                             //  $r_0 \leftarrow [r_0] - 1$ 
8 JNZ( $r_0$ , 6)                                     // if [ $r_0$ ] > 0 then goto 6
9 PRINT( $r_2$ )

```

V programu jsou použity čtyři registry, registry r_0 a r_1 slouží k uložení vstupních hodnot, do registru r_2 se postupně x_1 -krát přičte hodnota x_2 , zde je tedy na závěr výsledek, který je vypsán instrukcí **PRINT**. Pomocný registr r_3 je použit jen k tomu, abychom postupně mohli odčítat jedničku od hodnoty v registru r_0 (instrukce odčítání **SUB** totiž neumožňuje přímo odečíst konstantu od registru, viz dále úmluvu 4.18).

RAM R vykonává program krok po kroku, tedy tak, jak jsme zvyklí z imperativních programovacích jazyků, přičemž význam jednotlivých kroků je uveden v komentáři u každého řádku programu. Jak uvidíme, je krok 6 proveden přesně x_1 -krát (kde x_1 je první načtená hodnota uložená v r_0). Pokud je $x_1 = 0$, pak v kroku 4 neprovede skok, ale RAM bude pokračovat krokem 5, což je nepodmíněný skok na konec programu. Tento nepodmíněný skok je implementován podmíněným skokem s použitím registru r_3 , kde je uložena hodnota 1. Pokud je $x_1 > 0$ pokračuje se přičtením x_2 (v registru r_1) k 6. Poté je odečtena hodnota 1 v kroku 7. Pokud zůstane hodnota v r_0 pozitivní, dojde k opakování smyčky skokem na řádek 6. To znamená, že se krok 6 opakuje, dokud se postupným odčítáním jedničky nesníží hodnota x_1 na 0, tedy x_1 -krát.

Povšimněme si dále instrukce **COPY**, která je popsána ve dvou verzích. Tato instrukce umožňuje nepřímé adresování, bez kterého se neobejdeme, chceme-li, aby byl program schopen přistoupit k registrům s libovolně velkými adresami. Počet potřebných registrů může být pochopitelně závislý na velikosti vstupu a bez nepřímé adresace by počet registrů, k nimž může program přistoupit, byl pouze konstantní. Použití instrukce **COPY** si později ukážeme v příkladu 4.20.

Předpokládáme, že vstup je stroji RAM předán jako posloupnost čísel $x_1, x_2, \dots, x_n$. Ke svému vstupu přistupuje RAM voláním instrukce **READ**, která načte hodnotu ze vstupu (tím se posune na vstup další hodnota). Pokud tato instrukce dojde ke konci vstupu, vrací již jen hodnoty 0. RAM buď ví, kolik parametrů má načíst (například počítá-li funkci s pevným počtem parametrů), nebo očekává posloupnost čísel, jejíž délka může být variabilní. Budeme zejména uvažovat situaci, kdy tato posloupnost reprezentuje řetězec, který je ukončený 0.

Výstup zapisuje RAM pomocí instrukce **PRINT**, opět se jedná o posloupnost čísel $y_1, y_2, \dots, y_m$. Význam jednotlivých čísel na výstupu je pochopitelně dán programem, který RAM zpracovává.

Definice 4.16 (Výpočet RAM) Na začátku výpočtu dle programu $I_0, \dots, I_\ell$ mají všechny registry hodnotu 0, výpočet začíná první instrukcí I_0 a pokračuje dalšími instrukcemi v pořadí. Pro zjednodušení dalších úvah budeme předpokládat, že číslo aktuální instrukce si řídící jednotka udržuje ve zvláštním registru (*čítač instrukcí, program counter, PC*). Hodnota tohoto registru je na začátku 0 a po provedení každé instrukce se zvyšuje o jedna. V případě použití instrukce **JNZ** může dojít ke skoku na jinou instrukci, než je další v pořadí, tím dojde i k příslušné změně hodnoty *PC*. Výpočet končí v případě, kdy je hodnota čítače instrukcí *PC* vyšší, než ℓ . K tomu může dojít ve dvou případech, buď je vykonána instrukce na poslední řádce programu I_ℓ (a její součástí není skok na řádek programu s nižším číslem), nebo dojde ke skoku (příkazem **JNZ**) na řádku programu $k > \ell$. Fakt, že se výpočet RAM R nad daným vstupem $x_1, \dots, x_n$ zastaví, budeme označovat pomocí $R(x_1, \dots, x_n) \downarrow$ a budeme říkat, že výpočet *konverguje*. To, že se výpočet nezastaví, tedy že *diverguje*, budeme označovat pomocí $R(x_1, \dots, x_n) \uparrow$. ◀

Podobně jako v případě Turingových strojů, můžeme i RAM použít jednak k přijímání slov z daného jazyka, nebo k počítání hodnoty funkce. Tyto dva způsoby budeme rozlišovat zejména kvůli způsobu interpretace vstupu.

Definice 4.17 (RAM vyčíslitelné funkce) Nechť $f : \mathbb{N}^n \rightarrow \mathbb{N}$ je částečná funkce (nemusí tedy být definovaná pro všechny vstupy). Řekneme, že RAM R počítá funkci f , pokud platí:

- Pokud $f(x_1, \dots, x_n) \uparrow$, pak výpočet R se vstupem $(x_1, \dots, x_n)$ neskončí, nebo skončí s tím, že není vypsán žádný výstup pomocí **PRINT**.
- Pokud $f(x_1, \dots, x_n) \downarrow$, pak výpočet R skončí, dojde k zápisu alespoň jedné hodnoty na výstup a první hodnotou, kterou R na výstup zapíše, je hodnota $f(x_1, \dots, x_n)$

O funkci f , pro kterou existuje RAM, který ji počítá, budeme říkat, že je *RAM vyčíslitelná*. ◀

Všimněme si, že v definici 4.17 nijak neomezujeme počet provedených instrukcí **READ** ani **PRINT**. Z toho plyne, že s touto definicí každý RAM R počítá pro každé $n \in \mathbb{N}$ nějakou funkci n proměnných.

Všimněme si, že v případě Turingových strojů jsme v definici 4.6 zavedli turingovsky vyčíslitelnou funkci jako funkci řetězcovou, tedy typu $f : \Sigma^* \rightarrow \Sigma^*$. Naopak v případě RAM v definici 4.17 zavádíme RAM vyčíslitelnou funkci jako funkci nad přirozenými čísly, tedy funkci typu $f : \mathbb{N}^n \rightarrow \mathbb{N}$. Pro Turingovy stroje je přirozené pracovat s řetězci, zatímco RAM přirozeně pracuje s čísly. Na druhou stranu je potřeba si uvědomit, že tento rozdíl není příliš podstatný: Posloupnost čísel na vstupu RAM může reprezentovat i řetězec, jak si ukážeme dále v definici 4.19 a podobně i posloupnost čísel na výstupu můžeme interpretovat jako řetězec. Na druhou stranu Turingovu stroji můžeme předat na vstup posloupnost čísel zakódovaných v jednom řetězci a podobně i řetězec na výstupu může reprezentovat číslo. Tomu se budeme dále věnovat v kapitole 5.4.

Úmluva 4.18 (Zjednodušení použití některých instrukcí) Na příkladu 4.15 si můžeme všimnout, že instrukční sada, kterou jsme zvolili, je v lecčems velmi omezující, například nemůžeme rovnou napsat **SUB**($r_0, 1, r_0$) pro snížení hodnoty v registru r_0 o 1. Místo toho je potřeba využít pomocného registru r_3 , kam si nejprve uložíme konstantu 1, abychom mohli zavolat **SUB**(r_0, r_3, r_0) a tím teprve odečíst od obsahu registru r_0 hodnotu 1. Toto omezení má svůj smysl, který se ukáže teprve ve chvíli, kdy začneme jednotlivým instrukcím přiřazovat čas provedení a tím počítat časovou složitost algoritmu. Zatímco instrukce **LOAD**, tedy načtení konstanty do registru bude mít konstantní složitost, složitost operace **SUB** bude závislá na hodnotách menšence a menšitele, tedy na obsahu odpovídajících registrů. Podobně omezující je fakt, že nemáme k dispozici nepodmíněný skok, opět to lze obejít s pomocným registrem a uložením nenulové hodnoty do tohoto registru. Pro přehlednost programu v RAM by se nám navíc hodilo použití symbolických návěští.

Na základě těchto úvah si proto zavedeme následující konvenci:

- (I) Registr r_0 vyhradíme v zápisech programů pro účely následujících zjednodušení instrukcí a nebudeme jej výslovně používat.
- (II) V operacích **ADD** a **SUB** připustíme použití konstanty jako jednoho ze sčítanců, menšitele či menšence (číselnou konstantu od registru vždy odlišíme, neboť registry označujeme pomocí r_i , navíc není potřeba povolovat více konstant než jednu). Například **ADD**($r_i, 5, r_k$) je zkratkou za posloupnost **LOAD**(5, r_0), **ADD**(r_i, r_0, r_k).
- (III) Zavedeme si dodatečnou instrukci **JUMP**(I_z), která provede nepodmíněný skok na instrukci I_z . Jde o zkratku za posloupnost **LOAD**(1, r_0), **JNZ**(r_0, I_z).
- (IV) V operaci **PRINT** povolíme zápis konstanty. Například **PRINT**(5) je zkratkou za posloupnost **LOAD**(5, r_0), **PRINT**(r_0).
- (V) V zápisech programů pro RAM také umožníme používat symbolických návěští, jež budeme deklarovat pomocí návěští :. Použito v instrukci skoku pak návěští odkazuje na následující řádek.

(VI) V zápisech programů pro RAM také umožníme používat symbolické názvy registrů. V kterémkoli místě programu může být přítomna deklarace proměnné s tím, ve kterém registru bude uložena. Dále je možno používat název této proměnné místo registru. To nám umožní dát registrům význam, který bude z jejich názvu snáze pochopitelný.

Můžeme se samozřejmě ptát, proč již v tabulce 4.2.1 nemáme například k dispozici příkaz nepodmíněného skoku. Důvodem je snaha o minimalismus, snažíme se zvolit množinu instrukcí tak, aby byla co nejjednodušší a nebyly v ní nadbytečné instrukce.

Popíšeme si ještě, co znamená, že RAM R přijímá jazyk L .

Definice 4.19 Budeme uvažovat řetězce nad konečnou abecedou $\Sigma = \{\sigma_1, \dots, \sigma_p\}$. Mohli bychom sice uvažovat i nekonečnou abecedu, protože neomezujeme čísla na vstupu, nebudeme to však potřebovat. Podstatné je, že symboly abecedy indexujeme od 1. Pokud je $w = \sigma_{i_1} \sigma_{i_2} \sigma_{i_3} \dots \sigma_{i_n} \in \Sigma^*$ řetězec (který může být i prázdný), pak jej RAMu R předáme na vstup jako posloupnost čísel $i_1, \dots, i_n, 0$. Protože indexy symbolů jsou nenulové, první nula přečtená instrukcí **READ** ze vstupu označuje konec řetězce. Výpočet RAM s takto předaným vstupem budeme označovat jako $R(w)$.

- Řekneme, že RAM R přijímá slovo $w \in \Sigma^*$, pokud $R(w) \downarrow$, R použije během výpočtu alespoň jednu instrukci **PRINT** a první hodnotou zapsanou na výstup je 1.
- Řekneme, že RAM R odmítá slovo $w \in \Sigma^*$, pokud $R(w) \downarrow$ a během výpočtu buď nedojde k zápisu na výstup instrukcí **PRINT**, nebo první hodnotou zapsanou na výstup není 1.

Podobně jako v případě Turingových strojů řekneme, že RAM R přijímá (resp. odmítá) jazyk $L \subseteq \Sigma^*$, pokud R přijímá (resp. odmítá) právě slova z jazyka L . Jazyk přijímaný RAMem R označíme pomocí $L(R)$. Řekneme, že RAM R rozhoduje jazyk L , pokud přijímá L a odmítá $\bar{L}$ (tj. na všech vstupech se zastaví a buď je přijme nebo odmítne). ◀

Příklad 4.20 ukazuje RAM rozhodující jazyk palindromů. Pro srovnání si můžeme připomenout, že jsme již zkonstruovali jednopáskový deterministický Turingův stroj v příkladu 4.4 a dvoupáskový deterministický Turingův stroj v příkladu 4.9. Navíc jsme zkonstruovali dvoupáskový nedeterministický Turingův stroj přijímající jazyk PAL^2 .

Příklad 4.20

Algoritmus 5 ukazuje program RAMu R , který rozhoduje jazyk

$$\text{PAL} = \{w = w^R \mid w \in \{a, b\}^*\}.$$

Pro účely předání řetězce $w \in \{a, b\}^*$ RAMu, musíme očíslovat znaky symboly abecedy. Označíme proto $\sigma_1 = a, \sigma_2 = b$ a budeme pracovat nad abecedou $\Sigma = \{\sigma_1, \sigma_2\} = \{a, b\}$. Číslo 1 předané na vstupu tedy odpovídá znaku a a číslo 2 předané na vstupu odpovídá znaku b . Pomocí x budeme označovat vstupní řetězec.

Registry $r_1, \dots, r_7$ jsou využity pro uložení proměnných, které označujeme sym-

Algoritmus 5: RAM rozhodující jazyk palindromů

```
// Načtení vstupu
1 načti_znak: // Čtení ze vstupu
2   READ(a) //  $a \leftarrow$  znak ze vstupu
3   JNZ(a, další_znak) //  $a > 0$ , jde o další znak
4   JUMP(kontrola) //  $a = 0$ , jde o konec vstupu
5 další_znak: // Další vstupní znak
6   LOAD(8, d) //  $d \leftarrow 8$ 
7   ADD(d, n, d) //  $d \leftarrow n + d (= n + 8)$ 
8   COPY(a, [d]) //  $r_{n+8} \leftarrow a$ 
9   ADD(n, 1, n) //  $n \leftarrow n + 1$ 
10  JNZ(l, lichá) //  $l > 0$ , znak je na liché pozici
11 sudá: // Načtení znaku na sudé pozici
12  LOAD(1, l) // Příští pozice bude lichá
13  ADD(m, 1, m) // Zvýš hodnotu  $m = \lfloor \frac{n}{2} \rfloor$  o 1
14  JUMP(načti_znak) // Načti další znak
15 lichá: // Načtení znaku na liché pozici
16  LOAD(0, l) // Příští pozice bude sudá
17  JUMP(načti_znak) // Načti další znak
18 kontrola: // Začátek kontroly vstupního řetězce
19  JNZ(m, cyklus) // Více než jeden znak
20  JUMP(přijmi) // Nejvýš jeden znak, jde o palindrom
21 cyklus: // Hlavní kontrolní cyklus
22  SUB(m, 1, m) //  $m \leftarrow m - 1$ , další znak
23  LOAD(8, d) //  $d \leftarrow 8$ 
24  ADD(d, m, d) //  $d \leftarrow m + d = m + 8$ 
25  COPY([d], b) //  $b \leftarrow [d] = [r_{m+8}] = x[m + 1]$ 
26  LOAD(7, d) //  $d \leftarrow 7$ 
27  ADD(d, n, d) //  $d \leftarrow d + n = n + 7$ 
28  SUB(d, m, d) //  $d \leftarrow d - m = n + 7 - m$ 
29  COPY([d], c) //  $c \leftarrow [d] = [r_{n+7-m}] = x[n - m]$ 
30  SUB(b, c, a) //  $a \leftarrow \max(b - c, 0)$ 
31  JNZ(a, odmítni) //  $a > 0 \Leftrightarrow b - c > 0 \Leftrightarrow b > c$ , tedy  $b \neq c$ 
32  SUB(c, b, a) //  $a \leftarrow \max(c - b, 0)$ 
33  JNZ(a, odmítni) //  $a > 0 \Leftrightarrow c - b > 0 \Leftrightarrow c > b$ , tedy  $b \neq c$ 
34  JNZ(m, cyklus) //  $b = c$ , tedy  $x[m + 1] = x[n - m]$ 
35 přijmi: // Všechna porovnání uspěla
36  PRINT(1) // Přijmi zapsáním 1 na výstup
37  JUMP(konec) // Přeskoč zápis 0 na výstup (není nutné)
38 odmítni:
39  PRINT(0) // Odmítmi zapsáním 0 na výstup
40 konec:
```

bolicky. Jejich přiřazení konkrétním registrům a význam lze popsat následujícím způsobem.

n registr r_1 , délka vstupu.

m registr r_2 , po načtení vstupu obsahuje hodnotu $\lfloor \frac{n}{2} \rfloor$. Poté je použit jako index kontrolovaného znaku.

a registr r_3 , slouží k načtení dalšího znaku ze vstupu. Dále je tato proměnná použita jako pomocná při porovnávání $x[i+1]$ s $x[n-i]$.

d registr r_4 , slouží k počítání adresy pro nepřímou adresaci.

l registr r_5 , slouží k identifikaci toho, zda je aktuální hodnota n lichá.

b registr r_6 , použita pro kontrolu rovnosti dvou znaků.

c registr r_7 , použita pro kontrolu rovnosti dvou znaků.

Registry $r_8, \dots, r_{n+8}$ jsou vyhrazeny pro uložení čísel načtených ze vstupu, tedy $[r_{i+8}] = x[i]$. Hodnota n se postupně zvyšuje s každým načteným znakem až do chvíle, než je načtena hodnota 0.

Program nejprve načte celý vstupní řetězec x délky n do registrů $r_8, \dots, r_{n+7}$, během načítání vstupu si navíc program spočítá hodnotu $m = \lfloor \frac{n}{2} \rfloor$. Poté pro každý index $i = 0, \dots, m-1$ program zkontroluje, zda $[r_{4+i}] = [r_{n+3-i}]$, tedy zda $x[i+1] = x[n-i]$.

Popis jednotlivých kroků programu je uveden v komentářích.

4.2.2. Programování RAM

Abychom si dále zjednodušili situaci, uvědomme si, že programovací jazyk RAM odpovídá jednoduchému procedurálnímu jazyku podobného například Pascalu. Hlavní rozdíly proti běžnému jazyku jsou následující:

1. K dispozici jsou jen přirozená čísla.
2. Jediné přístupné aritmetické operace jsou sčítání a odčítání.
3. Procedury a funkce nelze volat rekurzivně.
4. Pole jsou jednorozměrná a s neomezenou velikostí.

Funkce jsou v programu RAM použity tím, že jejich kód je přímo vepsán do místa použití (případně s přečíslováním registrů). Funkce a procedury jsou tedy vždy *inline*. Uvažme například funkci $\text{MULT}(x_1, x_2)$, která počítá součin čísel x_1 a x_2 . Implementace této funkce může být víceméně shodná s programem v příkladu 4.15. Pokud v jiném

programu pro RAM použijeme funkci násobení tím, že napíšeme jako řádek volání

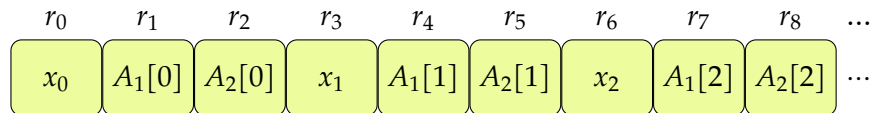
$$r_1 \leftarrow \text{MULT}(r_2, r_3),$$

je tento řádek jen zkratkou za vložení celého kódu programu pro funkci `MULT`, kde jsou vstupy vzaty z registrů r_2 a r_3 místo načtení ze vstupu a kde je výsledná hodnota uložena do registru r_1 místo zapsání na výstup. Absence rekurze není příliš omezující, neboť ji vždy můžeme nahradit pomocí cyklu **while**, který je možné v RAM zapsat pomocí podmíněného skoku.

Zastavme se chvíli u proměnných a polí. Paměť RAM je tvořena jedním neomezeným polem. Předpokládejme, že program stroje RAM R používá skalární proměnné (tedy jednoduché proměnné pro čísla) $x_0, \dots, x_s$ a pole $A_1, \dots, A_p$, která jsou indexována přirozenými čísly, tedy počínaje nulou, a jejichž délka není omezena. Potom všechna tato data reprezentujeme v jednorozměrné paměti RAM, následujícím způsobem:

- Prvek $A_i[j]$, kde $i \in \{1, \dots, p\}$, $j \in \mathbb{N}$, umístíme do registru $r_{i+(p+1)j}$.
- Prvky pole A_i , $i = 1, \dots, p$ jsou tedy v registrech $r_i, r_{i+p+1}, r_{i+2(p+1)}, \dots$.
- Proměnnou x_i , kde $i \in \{0, \dots, s\}$ umístíme do registru $r_{i+(p+1)}$.
- Skalární proměnné jsou tedy postupně v registrech $r_0, r_{p+1}, r_{2(p+1)}, r_{3(p+1)}, \dots, r_{s(p+1)}$.

Připomeňme si, že registr r_0 má zvláštní význam pro zjednodušení programu RAM dané úmluvou 4.18. Tento zvláštní význam má tedy při použití zjednodušeného zápisu instrukcí i proměnná x_0 . Pole a skalární proměnné jsou tedy vzájemně propleteny jako zip. V paměti je nejprve uložena první skalární proměnná, poté první prvky polí, druhá skalární proměnná, druhé prvky polí, třetí skalární proměnná, třetí prvky polí atd. Tuto situaci ilustruje pro případ dvou polí obrázek 4.7. Uvědomme si, že není možné prostě zapsat do paměti pole za sebe, neboť jejich délka není omezena.



Obrázek 4.7.: Způsob alokace proměnných a polí v programu RAM, kde jsou použity skalární proměnné $x_0, \dots, x_s$ a dvě pole A_1 a A_2 .

4.2.3. Varianty RAM

V této podkapitole zmíníme dvě varianty modelu RAM, které je možné najít v literatuře — RASP a PRAM.

RASP

Stroj RAM ani Turingovy stroje nejsou stroji Von Neumannovy architektury, neboť mají oddělený paměťový prostor pro program a data. Tím se i RAM odlišuje od běžného počítače, byť se mu jinak snaží blížit⁵. Stroje RASP odstraňují tento nedostatek. Zkratka RASP znamená *random access stored program*, název tedy naznačuje, že program je uložen v hlavní paměti spolu s dalšími daty. Program má navíc možnost zasahovat i do paměti se svým kódem, protože ten je součástí přístupného paměťového prostoru.

Popíšeme si stručně verzi RASP, která byla popsána v článku [4]. Model RASP, který v tomto článku autoři popsali, má k dispozici dva zvláštní registry — *akumulátor* (*accumulator*, AC) a *čítač instrukcí* (*instruction counter*, IC). Na počátku jsou hodnoty v obou těchto registrech nulové. Kromě toho má RASP, stejně jako stroj RAM, neomezený počet registrů $r_0, r_1, r_2, \dots$. Každá instrukce RASP je uložena v paměti ve dvou po sobě následujících registrech, kde v prvním registru je uveden *operační kód* instrukce a ve druhém registru je uvedena hodnota parametru instrukce, každá instrukce má tedy právě jeden parametr. Na začátku je program zapsán v po sobě jdoucích registrech, přičemž první instrukce zabírá první dva registry r_0, r_1 . Program je ukončen instrukcí **HALT**. Veškerá aritmetika a další operace probíhají s pomocí akumulátoru. Seznam instrukcí RASP i s jejich operačními kódy je uveden v tabulce 4.2.3.

Všimněme si, že instrukční sada v tabulce 4.2.3 neobsahuje instrukce **COPY** pro nepřímou adresaci. Nepřímou adresaci je možné provést úpravou programu za běhu tak, že program upraví hodnotu parametru příslušné instrukce (například **STORE** nebo **ADD**). Jinak jsou ovšem programy pro RASP podobné těm pro RAM a vždy je možno program pro RASP upravit na program pro RAM a naopak.

PRAM

Paralelní RAM (PRAM) je obvyklým modelem používaným při studiu paralelních algoritmů. Od běžného stroje RAM se liší tím, že má neomezený počet procesorů, které vykonávají též program. Paměť je tvořena jednou neomezenou posloupností registrů a je sdílená pro všechny procesory. Při přístupu do paměti je tedy potřeba nějakým způsobem řešit kolize. Zde si vystačíme s tímto jednoduchým popisem, neboť paralelnímu programování se tento text nevěnuje.

4.3. Ekvivalence Turingových strojů a RAM

V této kapitole si ukážeme, že Turingovy stroje a RAM mají shodnou výpočetní sílu. To znamená, že ke každému Turingovu stroji M lze sestrojít RAM R , který „dělá totéž“, a naopak. Každý směr převodu popíšeme zvlášť. Zejména nás přitom zajímají jazyky přijímané Turingovým strojem a RAMem a funkce vyčíslitelné Turingovým strojem a RAMem.

⁵Z praktického hlediska není ovšem tento rozdíl příliš podstatný, neboť běžně nepíšeme programy, jež by potřebovaly upravovat svůj kód.

Instrukce	Op. kód	Efekt	Popis
LOAD (j)	1	$AC \leftarrow j$ $IC \leftarrow [IC] + 2$	Přiřadí do akumulátoru konstantu $j \in \mathbb{N}$.
ADD (j)	2	$AC \leftarrow [AC] + [r_j]$ $IC \leftarrow [IC] + 2$	Přičte k akumulátoru hodnotu z registru r_j .
SUB (j)	3	$AC \leftarrow \max([AC] - [r_j], 0)$ $IC \leftarrow [IC] + 2$	Od akumulátoru odečte obsah registru r_j .
STORE (j)	4	$r_j \leftarrow [AC]$	Do registru r_j vloží obsah akumulátoru.
JNZ (j)	5	if ($[AC] > 0$) then $IC \leftarrow j$ else $IC \leftarrow [IC] + 2$	Pokud je hodnota v akumulátoru kladná, pokračuje se instrukcí v registru r_j , jinak se pokračuje následující instrukcí.
READ (j)	6	$r_j \leftarrow \text{input}$	Do registru r_j přečte další číslo na vstupu. Pokud není na vstupu další číslo, načte hodnotu 0.
PRINT (j)	7	$\text{output} \leftarrow [r_j]$	Hodnotu uloženou v registru r_j zapíše na výstup.
HALT	0, 8 až ∞	stop	Ukončení programu, operačním kódem je libovolné číslo, které není kódem jiné instrukce (tedy 0 nebo alespoň 8.)

Tabulka 4.5.: Seznam instrukcí RASP

Věta 4.21 (Ekvivalence Turingových strojů a RAM) *Nechť $L \subseteq \Sigma^*$ je jazyk a $f : \Sigma^* \rightarrow \Sigma^*$ je řetězcová funkce. Potom platí:*

1. *Jazyk L je přijímaný nějakým Turingovým strojem M , právě když je přijímaný nějakým RAM R .*
2. *Funkce f je turingovsky vyčíslitelná, právě když je RAM vyčíslitelná.*

Viz definice 4.3 a 4.19.

Viz definice 4.6 a 4.17.

Důkaz obou ekvivalencí provedeme ve dvou krocích, nejprve si ukážeme, jak převést Turingův stroj na RAM, který přijímá týž jazyk a počítá touž funkci. Poté si ukážeme druhý směr, tedy převod RAM na Turingův stroj, který přijímá týž jazyk a počítá touž funkci.

4.3.1. Převod Turingova stroje na RAM

Uvažme Turingův stroj $M = (Q, \Sigma, \delta, q_0, F)$. Ukážeme, jak zkonstruovat RAM R , který přijímá jazyk $L(M)$ a počítá funkci f_M .

Pro jednoduchost budeme předpokládat, že M má pásku neomezenou jen doprava, z věty 4.7 víme, že tento předpoklad není nijak omezující. Budeme dále předpokládat, že stavy i symboly páskové abecedy jsou očíslované, přesněji,

- $Q = \{q_0, q_1, \dots, q_r\}$ pro nějaké $r \geq 0$, kde q_0 je počáteční stav a
- $\Sigma = \{\sigma_0, \sigma_1, \sigma_2, \dots, \sigma_s\}$ pro nějaké $s \geq 1$, kde $\sigma_0 = \lambda$ je znak prázdného políčka.

Připomeňme si, že hodnota 0 na vstupu RAMu označuje konec vstupního řetězce, což je konzistentní s tím, že σ_0 označuje znak prázdného políčka. Předpokládáme dále, že buňky na pásce M jsou očíslované přirozenými čísly, přičemž nejlevější políčko má index 0.

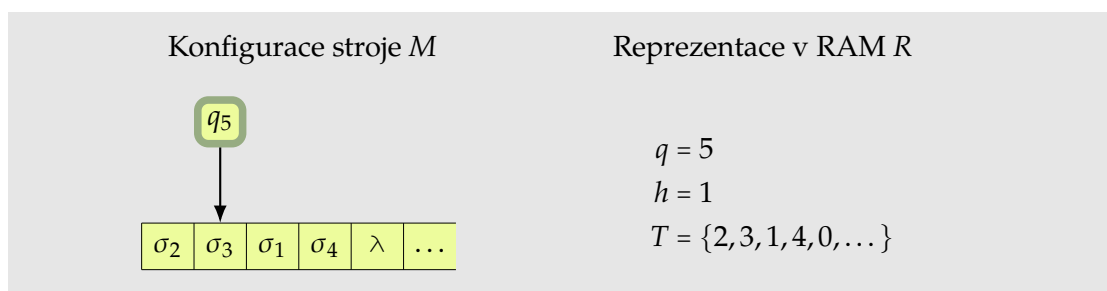
Konstruovaný RAM R bude v paměti reprezentovat aktuální konfiguraci, a tedy i displej stroje M . K tomu R použije následující proměnné a datové struktury (připomeňme si, že způsob uložení polí a proměnných v paměti RAM jsme popsali v kapitole 4.2.2).

Viz definice 4.2

- Proměnná q označuje index aktuálního stavu stroje M , tj. hodnota $q = i$ reprezentuje fakt, že M je ve stavu q_i .
- Proměnná h označuje index buňky, nad kterou je hlava M .
- Pole T reprezentuje obsah pásky M . Symbol pod hlavou M je tedy na pozici $T[h]$.

Příklad 4.22

Níže je ukázán příklad konfigurace a její reprezentace v RAM M .



Práci RAM R můžeme rozdělit do tří fází. V první fázi načte vstup do pole T , ve druhé fázi provádí samotnou simulaci stroje M a ve třetí fázi zpracuje výsledek této simulace.

Načtení vstupu: Zatímco Turingův stroj M očekává, že má vstup zapsán na pásce, RAM R oproti tomu čte vstup pomocí instrukcí **READ**. RAM R tedy nejprve načte celý vstupní řetězec a uloží jej do pole T na pozice $T[0], \dots, T[n-1]$, kde n označuje počet načtených nenulových indexů znaků před hodnotou 0, která označuje konec vstupního řetězce. Hodnoty proměnných h a q jsou na počátku 0, což je dáno počátečním obsahem registrů R .

Provedení kroku: Provedení jednoho kroku je provedeno pomocí sady podmíněných příkazů, z nichž každý implementuje jeden řádek tabulky přechodové funkce δ Turingova stroje M . Instrukce $\delta(q_i, \sigma_j) = (q_k, \sigma_l, R)$ je v programu R implementována podmíněným příkazem

```

1 if  $q = i$  and  $T[h] = j$  then
2    $q \leftarrow k$ 
3    $T[h] \leftarrow l$ 
4    $h \leftarrow h + 1$ 

```

V případě pohybu N zůstává hodnota h beze změny a v případě pohybu L je na řádku 4 použit rozdíl $h \leftarrow h - 1$, přičemž není nutné před odečtením jedničky kontrolovat, zda $h = 0$, protože předpokládáme, že M nepohne hlavou nalevo od nejlevější buňky pásy.

Pokud některý z těchto podmíněných příkazů uspěje, pokračuje R simulací dalšího kroku. V opačném případě cyklus vykonávání kroků končí a pokračuje se další fází, tedy zapsáním výstupu.

Příklad 4.23

Uvažme Turingův stroj M s následující přechodovou funkcí.

	$q, c \rightarrow q', c', Z$
1.	$q_0, \sigma_2 \rightarrow q_3, \sigma_1, R$
2.	$q_3, \sigma_1 \rightarrow q_2, \sigma_0, L$
3.	$q_2, \sigma_0 \rightarrow q_0, \sigma_2, N$

RAM R tuto přechodovou funkci implementuje následující posloupností podmíněných příkazů.

```

1 if  $q = 0$  and  $T[h] = 2$  then
2    $q \leftarrow 3$ 
3    $T[h] \leftarrow 1$ 
4    $h \leftarrow h + 1$ 
5 else if  $q = 3$  and  $T[h] = 1$  then
6    $q \leftarrow 2$ 
7    $T[h] \leftarrow 0$ 
8    $h \leftarrow h - 1$ 
9 else if  $q = 2$  and  $T[h] = 0$  then
10   $q \leftarrow 0$ 
11   $T[h] \leftarrow 2$ 
12 else
13  Konec simulace

```

Zpracování výsledku simulace: Pokud nás zajímá, zda Turingův stroj M přijal svůj vstup, tedy zda vstupní slovo patří do jazyka $L(M)$, pak je na výstup zapsána hodnota 1 za podmínky, že stav uložený v proměnné q je přijímající. Pokud nás zajímá hodnota funkce f_M vyčíslované Turingovým strojem M , je na výstup zapsáno slovo na pásce aktuálně uložené v poli T . Buď R zapíše na výstup obsah T až do první prázdné buňky, nebo zapíše ten kus pole T , k němuž se M během výpočtu dostal, to je dáno maximální hodnotou h během výpočtu.

4.3.2. Převod RAM na Turingův stroj

Popišme si nyní, jak převést RAM R na Turingův stroj M , který počítá touž funkci a přijímá též jazyk. Turingův stroj M zkonstruujeme jako 4-páskový.

Význam pásek je následující:

1. **Vstupní páska.** Posloupnost čísel, která má dostat R na vstup. Jsou zakódovaná binárně a oddělená znakem $\#$. Z této pásky M jen čte.
2. **Výstupní páska.** Sem zapisuje M čísla, která R zapisuje na výstup. Jsou zakódovaná binárně a oddělená znakem $\#$. Na tuto pásku M jen zapisuje.

3. **Paměť RAM.** Obsah paměti stroje R . Formát této pásky popíšeme níže.
4. **Pomocná páska.** Pro výpočty součtu, rozdílu, nepřímých adres, posunu části paměťové pásky a podobně.

Předpokládáme, že vstupem R mohou být libovolná přirozená čísla, proto předpokládáme, že Turingův stroj M dostane na vstup tato čísla zapsaná binárně. To je ovšem technická záležitost, bylo by samozřejmě možné Turingovu stroji rovnou dát seznam znaků, které jsou těmito čísly reprezentovány, pokud nás zajímá náležení řetězce do jazyka. Podobně přistupujeme i k výstupu Turingova stroje M , který má být opět v podobě binárních zápisů čísel, jež by vypsál RAM R .

Je třeba si podrobněji popsat reprezentaci obsahu paměti RAM na 3. pásce. Stačí si pamatovat obsahy využitých registrů, tedy těch, do nichž byla uložena nějaká hodnota v průběhu práce R . Ostatní registry obsahují 0. Obsahy registrů si zapíšeme za sebe v rostoucím pořadí podle čísel registrů⁶ Za každý registr r_i přidáme dvojici tvořenou indexem i a číslem $[r_i]$ uloženým v registru r_i . Obě čísla jsou zapsaná binárně a oddělená znakem $|$. Jednotlivé dvojice jsou pak odděleny znakem $\#$. To znamená, že jsou-li aktuálně využité registry $r_{i_1}, r_{i_2}, \dots, r_{i_m}$, kde $i_1 < i_2 < \dots < i_m$, pak je na pásce reprezentující paměť RAM R řetězec

$$(i_1)_B | ([r_{i_1}])_B \# (i_2)_B | ([r_{i_2}])_B \# \dots \# (i_m)_B | ([r_{i_m}])_B.$$

Předpokládejme, že RAM R se řídí programem složeným z instrukcí $I_1, \dots, I_\ell$, kde $\ell \in \mathbb{N}$. Přechodová funkce M zabezpečí vykonání těchto instrukcí v daném pořadí. Za tím účelem bude součástí stavu M hodnota čítače instrukcí (*program counter*, PC), což je číslo z rozmezí $1, \dots, \ell$. Uvědomme si, že ℓ je konstanta nezávislá na vstupu a je tedy možné, aby hodnota PC byla uložena ve stavu. Každá instrukce bude nahrazena posloupností instrukcí Turingova stroje. Přičemž posloupnost instrukcí implementující instrukci I_j pro $j \in \{1, \dots, \ell\}$ bude končit ve stavu, který bude počátečním pro posloupnost implementující následující instrukci v programu R . Obvykle po I_j následuje I_{j+1} kromě případu instrukce **JNZ**, která může změnit hodnotu PC na hodnotu danou parametrem. Posloupnost s číslem $\ell + 1$, tedy za koncem programu, pak bude provádět zakončení práce programu.

Implementace každé jednotlivé instrukce RAM na Turingovu stroji je při dané reprezentaci paměti RAM celkem přímočará a daná přímo významem instrukcí v tabulce 4.2.1. Při implementaci těchto instrukcí je podstatné zejména to, že Turingův stroj může manipulovat se seznamem dvojic reprezentujících data v registrech na 3. pásce. To znamená, že pro daný index i může najít blok odpovídající registru r_i , přečíst a upravit hodnotu v něm uloženou. Navíc Turingův stroj dokáže poznat, zda daný index i nemá dosud svůj záznam v paměti a je tedy potřeba jej přidat. Aritmetické operace nad binárními čísly lze opět snadno implementovat na Turingovu stroji (zejména je-li k dispozici pomocná 4. páska).

⁶Není sice nutné, aby čísla registrů byla uspořádaná, ale trochu to zjednodušuje hledání registru s daným indexem v seznamu.

4.4. Bibliografické poznámky

4.5. Cvičení

1. Ukažte větu [4.7](#).

5. Algoritmy

V této kapitole nejprve vyslovíme Churchovu-Turingovu tezi, která spojuje intuitivní pojem algoritmu s matematickým pojmem Turingova stroje. Zavedeme také číslování Turingových strojů a popíšeme univerzální Turingův stroj. Ten se nám bude dále hodit v našich úvahách o algoritmicky řešitelných problémech a algoritmicky vyčíslitelných funkcí, což jsou pojmy, které zavedeme v závěru kapitoly.

5.1. Churchova-Turingova teze

Pod pojmem algoritmu si obvykle intuitivně představujeme konečnou posloupnost jednoduchých instrukcí (příkazů, pokynů), která vede k řešení zadané úlohy. V tomto smyslu se dá za algoritmus považovat vlastně jakýkoli postup či návod, od Euklidova algoritmu přes popis cesty ke kamarádovi domů po návod na použití fotoaparátu či kuchařský recept. Kromě posloupnosti instrukcí potřebujeme však pochopitelně i prostředek, na kterém budeme tuto posloupnost vykonávat, ať už je to naše hlava, nohy, fotoaparát a ruce či my a kuchyňské vybavení.

Nás budou zajímat algoritmy, které pracují s matematickými objekty (číslly, řetězci, funkcemi apod.). Chceme-li formalizovat pojem algoritmu v matematice, potřebujeme model výpočetního prostředku, který by jednak byl dostatečně obecný, aby obsáhl naši intuitivní představu algoritmu, a jednak dostatečně jednoduchý, aby se s ním dobře manipulovalo. Dosud jsme si popsali tři takové modely — Turingovy stroje, RAM a částečně rekurzivní funkce. Ukázali jsme si také, že všechny tři tyto modely jsou stejně silné, je tedy možno v nich implementovat výpočet týchž funkcí, přijímání téže množiny jazyků. Potažmo je v těchto modelech tedy možno vyřešit touž třídu problémů a úloh. Kromě toho však existuje i celá řada dalších stejně silných výpočetních modelů — programy ve vyšších programovacích jazycích jako je C, Pascal, Basic, Java (i když zde je třeba mít vždy na paměti i počítač, na kterém jsou tyto programy interpretovány), programy ve funkcionálních jazycích jako je λ -kalkulus (který pochází od Alonza Churcha, 1936 a který tvoří teoretický základ funkcionálních jazyků), Haskell, Lisp, mezi dalšími můžeme zmínit třeba skript pro sed.

Kterýkoli z těchto prostředků bychom si mohli vybrat a vybudovat tutéž teorii, protože všechny jsou co do výpočetní síly ekvivalentní. V roce 1936 právě Church, Turing a Kleene ukázali, že Turingovy stroje a λ -kalkulus jsou stejně silné prostředky jako o něco starší částečně rekurzivní funkce. Zformulovali současně tezi, které se říká Churchova-Turingova, a podle ní právě Turingovy stroje a jim ekvivalentní prostředky zachycují intuitivní pojem algoritmu.

Teze 5.1: Churchova-Turingova (1936)

Ke každému algoritmu v intuitivním smyslu existuje Turingův stroj, který jej implementuje.

Churchova-Turingova teze se snaží spojit intuitivní představu pojmu algoritmu s přesnou matematickou definicí Turingova stroje. Spojuje tak dva velmi rozdílné světy a ze své podstaty nejde o matematické tvrzení, jež by bylo možno dokázat, tezi pouze přijmout nebo odmítnout. My tuto tezi pro další výklad přijmeme, neboť je podpořena řadou ekvivalentních modelů a dá se v tomto smyslu říci, že jako reálný výpočetní prostředek nic lepšího než Turingovy stroje nemáme.

Churchovu-Turingovu tezi budeme dále často využívat v obou směrech. Často budeme popisovat algoritmy v intuitivním smyslu, přičemž na základě Churchovy-Turingovy teze budeme předpokládat, že bychom tento algoritmus mohli implementovat na Turingovu stroji (RAMu, ČRF, či v jiném výpočetním modelu). Na druhou stranu ve chvílích, kdy budeme potřebovat říci něco skutečně formálně, budeme hovořit o Turingových strojích, při tom ovšem budeme mít na paměti, že současně můžeme dané výsledky vztáhnout i na jiné výpočetní modely, potažmo na algoritmy obecně.

5.2. Kódování objektů a univerzální Turingův stroj

Abychom mohli k Turingovým strojům (a tedy i k algoritmům) přistupovat uniformním způsobem bude se nám hodit to, že každý Turingův stroj lze zakódovat pomocí řetězce a přiřadit mu přirozené číslo. Toto kódování se nám bude hodit i při konstrukci univerzálního Turingova stroje, který se nám bude též při dalších úvahách o Turingových strojích velmi hodit. Například si tímto způsobem můžeme zodpovědět otázku, kolik je vlastně Turingových strojů.

5.2.1. Kódování Turingových strojů a Gödelovo číslo

V této kapitole zavedeme číslování Turingových strojů. Je třeba předeslat, že způsob kódování Turingových strojů a způsob přiřazení čísla, které zde popíšeme, je jen jedním z mnoha možných. Konkrétní způsob kódování není pro další úvahy o algoritmech a Turingových strojích nijak podstatný. Potřebujeme si nějaký způsob zvolit, abychom jej mohli dále využívat, ale jakýkoli jiný způsob kódování s podobnými vlastnostmi by bylo možno využít stejně dobře. Podstatnou vlastností kódu, který popíšeme, je jeho efektivita. To znamená, že kód je možno dobře zpracovávat na Turingovu stroji (potažmo na jiném výpočetním modelu).

Naším dalším cílem je také každému Turingovu stroji přiřadit jeho Gödelovo číslo. Postup tohoto přiřazení je rozložen do tří kroků.

1. Turingův stroj nejprve reprezentujeme řetězcem v abecedě

$$\Gamma = \{0, 1, L, N, R, |, \#, ;\}. \quad (5.1)$$

2. Poté každý znak abecedy Γ přepíšeme pomocí tří bitů, tedy znaků 0 a 1.
3. Nakonec využijeme toho, že binární řetězce lze očíslovat přirozenými čísly, jak jsme si ukázali v kapitole 3.4.4.

Omezení kladená na kódované Turingovy stroje

Pro účely popisu kódování budeme uvažovat jen Turingovy stroje splňující jisté omezující předpoklady, jež jim však neubírají na výpočetní síle. Přesněji, budeme uvažovat, že Turingův stroj $M = (Q, \Sigma, \delta, q_0, F)$, který kódujeme, splňuje následující omezení:

- (i) Vstupní abeceda M je binární abeceda $\{0, 1\}$. To znamená, že stroji M jsou na vstupu předávány pouze binární řetězce.
- (ii) $F = \{q_1\}$, tedy M má jediný přijímající stav, který označíme pomocí $q_1 \in Q$.

Zdůrazněme ihned, že páskovou abecedu stroje M nijak neomezujeme, jediné, co vyžadujeme, aby spolu se symboly 0 a 1 vstupní abecedy obsahovala i symbol λ prázdného políčka.

Vstupní abecedu M omezujeme proto, že vstupní abeceda univerzálního Turingova stroje (který budeme dále konstruovat) musí obsahovat symboly vstupní abecedy simulovaného stroje. Mohli bychom zvolit libovolně velkou abecedu. Binární abeceda se pro naše účely hodí proto, že je sice dostatečně malá, aby se s ní dobře pracovalo, ale je současně dostatečně velká, abychom pomocí ní mohli zakódovat řetězec v libovolné abecedě (jak uvidíme dále). Mohli bychom ve skutečnosti uvažovat i jednoprvkovou vstupní abecedu, ale pak by popis kódování byl zbytečně komplikovaný, protože všechno bychom museli rovnou zakódovat do jediného přirozeného čísla, což učiníme až ve třetím kroku. Na druhou stranu tříprvková abeceda nepřináší proti binární nic podstatného.

Předpoklad, že kódovaný stroj má pouze jeden přijímající stav, nemá na výpočetní sílu Turingova stroje žádný vliv. Libovolný TS M lze totiž triviálně převést na ekvivalentní TS M' , který má jediný přijímající stav, z něž nevedou již žádné instrukce. Pokud má totiž M více přijímajících stavů, nebo z jeho přijímajících stavů vedou nějaké instrukce, přidáme k M nový stav q_1 a instrukce, které ve chvíli ukončeného výpočtu v některém z přijímajících stavů zabezpečí přechod do stavu q_1 . Pokud naopak TS M nemá žádný přijímající stav, pak ničemu nevádí, pokud k němu přidáme nový stav, který bude přijímající, ale nebude do něj možné přejít pomocí žádné instrukce.

Kódování Turingových strojů řetězci v abecedě Γ

V této podkapitole popíšeme kódování Turingových strojů pomocí řetězců v abecedě $\Gamma = \{0, 1, L, N, R, |, \#, ;\}$. Za podmínek, které klademe na Turingův stroj, stačí k jeho popisu vhodným způsobem zakódovat přechodovou funkci, neboť veškeré další informace o stroji lze již ze zápisu přechodové funkce vyčíst. Dosud jsme uvažovali přechodovou

funkci zapsanou v tabulce. Kód Turingova stroje dostaneme prostě tak, že zapíšeme jednotlivé řádky tabulky přechodové funkce za sebe. Pro zápis jedné instrukce, tedy jednoho řádku tabulky, využijeme první šestici znaků z abecedy Γ , znak $\#$ posléze použijeme k oddělení jednotlivých instrukcí. Znak $;$ nebude v kódu použit a je možné jej využít například k oddělení kódu stroje od jeho vstupu na vstupní pásce univerzálního TS.

Uvažme tedy TS $M = (Q, \Sigma, \delta, q_0, F)$, kde $F = \{q_1\}$. Předpokládáme, že stavy a symboly páskové abecedy jsou očíslované, tedy $Q = \{q_0, q_1, \dots, q_r\}$ pro nějaké $r \geq 1$ a podobně $\Sigma = \{X_0, X_1, X_2, \dots, X_s\}$ pro nějaké $s \geq 2$. Dále předpokládáme, že q_0 označuje počáteční stav a q_1 jediný přijímající stav. A konečně předpokládáme, že X_0 označuje symbol 0 , X_1 označuje symbol 1 a X_2 označuje symbol prázdného políčka λ . Pokud $s > 2$, pak $X_3, \dots, X_s$ označují další symboly páskové abecedy stroje M . Instrukci $\delta(q_i, X_j) = (q_k, X_l, Z)$, kde $Z \in \{L, N, R\}$ označuje pohyb hlavy, zakódujeme řetězcem:

$$(i)_B | (j)_B | (k)_B | (l)_B | Z \quad (5.2)$$

Pomocí $(n)_B$ označujeme binární zápis čísla n . V kódu Turingova stroje přitom připouštíme libovolný počet nul na začátku tohoto zápisu. Nechť C_1 až C_n označují kódy instrukcí M v abecedě Γ , potom kód stroje M vznikne jejich konkatencí s použitím oddělovače $\#$. Tedy kódem M v abecedě Γ je řetězec:

$$C_1 \# C_2 \# \dots \# C_{n-1} \# C_n$$

Příklad 5.2

Uvažme například Turingův stroj M , který přijímá jazyk palindromů PAL a který jsme zkonstruovali v příkladu 4.4. Pro tento stroj platí, že q_1 je jediným přijímajícím stavem. Vstupní abeceda M je dvoupřvková, byť symboly použité v příkladu 4.4 jsou a a b . My místo toho využijeme 0 (místo a) a 1 (místo b). To znamená, že budeme uvažovat stroj, který přijímá jazyk palindromů nad abecedou $\{0, 1\}$.

$$\text{PAL}_b = \{w = w^R \mid w \in \{0, 1\}^*\}, \quad (5.3)$$

Kódování jednotlivých řádků tabulky přechodové funkce je uvedeno v tabulce 5.1.

Tabulka 5.1.: Kódy řádků přechodové tabulky stroje M .

	q, c	$\rightarrow$	q', c', Z	Kód instrukce
1.	q_0, λ	$\rightarrow$	q_1, λ, N	$0 10 1 10 N$
2.	$q_0, 0$	$\rightarrow$	$q_2, 0, R$	$0 0 10 0 R$
3.	$q_0, 1$	$\rightarrow$	$q_3, 0, R$	$0 1 11 0 R$
4.	$q_2, 0$	$\rightarrow$	$q_2, 0, R$	$10 0 10 0 R$
5.	$q_2, 1$	$\rightarrow$	$q_2, 1, R$	$10 1 10 1 R$

6.	$q_2, \lambda \rightarrow q_3, \lambda, L$	10 10 11 10 L
7.	$q_3, 0 \rightarrow q_3, 0, R$	11 0 11 0 R
8.	$q_3, 1 \rightarrow q_3, 1, R$	11 1 11 1 R
9.	$q_3, \lambda \rightarrow q_5, \lambda, L$	11 10 101 10 L
10.	$q_4, 0 \rightarrow q_6, \lambda, L$	100 0 110 10 L
11.	$q_5, 1 \rightarrow q_6, \lambda, L$	101 1 110 10 L
12.	$q_6, 0 \rightarrow q_6, 0, L$	110 0 110 0 L
13.	$q_6, 1 \rightarrow q_6, 1, L$	110 1 110 1 L
14.	$q_6, \lambda \rightarrow q_7, \lambda, R$	110 10 111 10 R
15.	$q_7, 0 \rightarrow q_0, \lambda, R$	111 0 0 10 R
16.	$q_7, 1 \rightarrow q_0, \lambda, R$	111 1 0 10 R
17.	$q_7, \lambda \rightarrow q_1, \lambda, N$	111 10 1 10 N

Spojením kódů jednotlivých řádků s použitím oddělovacího znaku # dostaneme kód Turingova stroje M :

```
0|10|1|10|N#0|0|10|0|R#0|1|11|0|R#10|0|10|0|R#10|1|10|1|R#
10|10|11|10|L#11|0|11|0|R#11|1|11|1|R#11|10|101|10|L#
100|0|110|10|L#101|1|110|10|L#110|0|110|0|L#110|1|110|1|L#
110|10|111|10|R#111|0|0|10|R#111|1|0|10|R#111|10|1|10|N
```

Převod řetězce v abecedě Γ do binárního řetězce

Dalším krokem kódování Turingova stroje bude převod řetězce v abecedě Γ do binárního řetězce. Každý znak abecedy Γ zapíšeme v binární abecedě pomocí tří bitů podle tabulky 5.2.

Γ	kód	Γ	kód
0	000	R	100
1	001		101
L	010	#	110
N	011	;	111

Tabulka 5.2.: Převod znaků abecedy Γ do binární abecedy $\{0, 1\}$.

Náhradou znaků v řetězci v abecedě Γ dostaneme jeho přepis pomocí binárního řetězce.

Příklad 5.3

Navazme na příklad 5.2, v němž jsme sestrojili řetězec v abecedě Γ , který kóduje Turingův stroj M rozhodující jazyk palindromů PAL_b . Přepisovat zde celý kód do binárního řetězce by asi nemělo příliš smysl, ale podívejme se alespoň na jednu instrukci, například

$$q_3, \lambda \rightarrow q_5, \lambda, L.$$

Této instrukci odpovídá kód v abecedě Γ

$$11|10|101|10|L,$$

který je podle tabulky 5.2 převeden do binárního řetězce následujícím způsobem (na horním řádku je řetězec v abecedě Γ , na spodním řádku je řetězec v binární abecedě, který mu odpovídá, „zlomky“ naznačují, jaký znak daná trojice bitů kóduje):

$$\begin{array}{cccccccccccccccc} 1 & 1 & | & 1 & 0 & | & 1 & 0 & 1 & | & 1 & 0 & | & L \\ 001 & 001 & 101 & 001 & 000 & 101 & 001 & 000 & 001 & 101 & 001 & 000 & 101 & 010 \end{array}$$

Všimněme si, že na pořadí, v jakém očíslovujeme stavy a znaky páskové abecedy či v jakém zapíšeme instrukce, nijak nezáleží. Z toho plyne, že každý TS M může mít mnoho různých kódů, které jsou zcela ekvivalentní. Ve skutečnosti pokud si uvědomíme, že stavům (kromě q_0 a q_1) či symbolům páskové abecedy (kromě $0, 1, \lambda$) můžeme přiřadit libovolná přirozená čísla a ne jen čísla z množiny $\{0, 1, \dots, |Q| - 1\}$ v případě stavů či $\{0, 1, 2, \dots, |\Sigma| - 1\}$ v případě páskové abecedy, získáme dokonce nekonečně mnoho řetězců, které kódují též TS. To je naprosto v pořádku, podobně když v programu v jazyce C použijeme různé názvy proměnných či funkcí, dostaneme různé zdrojové kódy, třebaže na programu samotném jsme nic nezměnili.

Zřejmě ne každý binární řetězec kóduje nějaký Turingův stroj (počet bitů v syntakticky správném řetězci musí například být dělitelný třemi). Abychom se vyhnuli technickým obtížím s tím spojeným, přijmeme následující úmluvu.

Úmluva 5.4 *Binární řetězce, které nejsou syntakticky správnými kódy Turingova stroje, reprezentují Turingův stroj s prázdnou přechodovou funkcí.*

Turingův stroj s prázdnou přechodovou funkcí nutně zamítne každý vstup hned v prvním kroku, neboť počáteční stav je různý od přijímajícího. Poznat, zda daný řetězec je syntakticky správným kódem Turingova stroje, lze přitom celkem snadno, neboť tvar kódování je dobře popsáný.

Pro označení řetězce kódujícího Turingův stroj si zavedeme následující značení.

Definice 5.5 Nechť M je Turingův stroj, pomocí $\langle M \rangle$ označíme binární řetězec, který kóduje M . ◀

Vzhledem k tomu, že kód Turingova stroje není jednoznačný, označuje $\langle M \rangle$ nějaký jeho kód, zkonstruovaný výše uvedeným způsobem pro konkrétní očíslování stavů, znaků páskové abecedy, pořadí řádků v přechodové tabulce, atd.

Gödelovo číslo Turingova stroje

V kapitole 3.4.4 jsme zavedli číslování řetězců. Toho můžeme nyní využít k tomu, abychom přiřadili čísla Turingovým strojům.

Definice 5.6 (Gödelovo číslo Turingova stroje) Nechť M je Turingův stroj a $\langle M \rangle$ je jeho kód, pak index $\langle M \rangle$ je Gödelovým číslem Turingova stroje M . ◀

Vzhledem k tomu, že kód Turingova stroje M není jednoznačný, ani Gödelovo číslo není jednoznačné a dokonce M má nekonečně mnoho Gödelových čísel.

Protože jsme přiřadili prázdný stroj i řetězcům, které nejsou syntakticky správným kódem žádného TS, dostali jsme tak očíslování všech Turingových strojů a naopak každému přirozenému číslu jsme přiřadili nějaký Turingův stroj (těm číslům, jimž odpovídající řetězec neodpovídá syntakticky správnému kódu TS, jsme přiřadili prázdný TS). Jako důsledek očíslování Turingových strojů dostáváme jejich podstatnou vlastnost.

Důsledek 5.7 *Všech Turingových strojů je spočetně mnoho, neboť jsme schopni je očíslovat přirozenými čísly. Na základě teze 5.1 můžeme tedy říci, že i algoritmů je spočetně mnoho.*

5.2.2. Kódování dalších objektů

Podobně jako jsme popsali kódování Turingových strojů pomocí binárních řetězců, lze takto kódovat i další objekty, například čísla, RAM, logickou formuli, grafy apod. A navíc lze takto kódovat i n -tice těchto objektů. Pro nás nebude dále konkrétní způsob kódování podstatný. Podstatné jen je, aby zvolené kódování bylo vždy efektivní, to znamená, že je s ním možno algoritmicky pracovat, tedy že je s ním schopen Turingův stroj pracovat. To znamená, že je možné vždy z kódu efektivně (ve smyslu algoritmicky) získat všechny potřebné informace o daném objektu. Na základě těchto úvah si zavedeme následující značení.

Definice 5.8 Pomocí $\langle X \rangle$ budeme označovat binární řetězec kódující objekt X . Pomocí $\langle X_1, \dots, X_n \rangle$ označíme binární řetězec kódující n -tici objektů $X_1, \dots, X_n$. ◀

Například pomocí $\langle M \rangle$ označujeme kód Turingova stroje M (takové kódování jsme zavedli v kapitole 5.2.1), pomocí $\langle M, x \rangle$ budeme označovat kód dvojice, kde M je Turingův stroj a x je řetězec (ne nutně binární).

5.2.3. Univerzální Turingův stroj

V této kapitole si popíšeme univerzální Turingův stroj, který si označíme jako $\mathcal{U}$. Vstupem univerzálního Turingova stroje je řetězec $\langle M, x \rangle$, kde M je Turingův stroj a x je binární řetězec. Univerzální Turingův stroj simuluje práci Turingova stroje M nad vstupem x . Výsledek této simulace je určen výsledkem výpočtu M nad x . Přesněji řečeno, $\mathcal{U}(\langle M, x \rangle) \downarrow$, právě když $M(x) \downarrow$ a $\mathcal{U}(\langle M, x \rangle)$ přijme, právě když $M(x)$ přijme. Po ukončení výpočtu $\mathcal{U}(\langle M, x \rangle)$ navíc jeho páska v nějaké podobě reprezentuje obsah pásky po ukončení výpočtu $M(x)$.

Ve výsledku tedy chceme, aby univerzální Turingův stroj $\mathcal{U}$ přijímal **univerzální jazyk** definovaný následujícím způsobem:

$$L_u = \{\langle M, x \rangle \mid x \in L(M)\} \quad (5.4)$$

Současně ovšem univerzální Turingův stroj bude implementovat univerzální funkci.

Je dobré si uvědomit, že univerzální jazyk $L_u = L(\mathcal{U}) = \{\langle M, x \rangle \mid x \in L(M)\}$ je formalizací problému PŘIJETÍ VSTUPU, který definujeme následujícím způsobem.

Problém 5.9: PŘIJETÍ VSTUPU

Instance: Turingův stroj M a řetězec x .

Otázka: Je výpočet Turingova stroje M se vstupem x přijímající?

Univerzální Turingův stroj popíšeme jako třípáskový, neboť je to technicky jednodušší, než popisovat jednopáskový univerzální Turingův stroj. Na základě věty 4.10 víme, že třípáskový stroj lze vždy převést na jednopáskový. Tak dostaneme jednopáskový univerzální Turingův stroj, který je potom v případě potřeby schopen simulovat i sám sebe.

Jako pracovní abecedu $\mathcal{U}$ použijeme abecedu Γ definovanou v (5.1). Pro zjednodušení popisu budeme navíc přistupovat ke kódu $\langle M \rangle$ (tedy první složce dvojice $\langle M, x \rangle$) jako k řetězci v abecedě Γ , i když je třeba mít na paměti, že přečtení jednoho znaku z abecedy Γ potom znamená přečtení tří po sobě jdoucích bitů v binárním řetězci $\langle M \rangle$.

1. páska obsahuje vstup $\mathcal{U}$, tedy kód $\langle M, x \rangle$.

$\langle M, x \rangle$

Na **2. pásce** je uložen obsah pracovní pásky M . Symboly X_i jsou zapsány jako $(i)_B$ v blocích téže délky oddělených $|$.

... | 010 | 001 | 100 | 000 | 010 | 011 | ...

3. páska obsahuje číslo aktuálního stavu q_i stroje M .

10011 ($= (i)_B$)

Obrázek 5.1.: Rozdělení funkcí pásek univerzálního Turingova stroje.

Rozdělení funkcí pásek UTS je následující (viz též obrázek 5.1):

- 1. Vstupní páska.** Na vstupní pásce je na počátku uveden vstup tvořený kódem dvojice $\langle M, x \rangle$, kde M je simulovaný stroj a x je jeho vstup. Pro jednoduchost můžeme

předpokládat, že na pásce je nejprve zapsán kód $\langle M \rangle$ následovaný oddělovačem, za nímž následuje přímo zapsaný řetězec x . Jako oddělovač můžeme použít například znak „;“ z abecedy Γ zapsaný trojicí bitů 111. Podstatné je jen to, že $\mathcal{U}$ může číst přímo jak kód $\langle M \rangle$, tak jeho vstup x . Za tohoto předpokladu můžeme předpokládat, že vstupní páska je jen pro čtení a $\mathcal{U}$ ji během výpočtu neupravuje.

2. **Pracovní páska M .** Tato páska je v průběhu výpočtu využita k reprezentaci obsahu (jediné) pásky M . Připomeňme si, že páskovou abecedu stroje M jsme nijak neomezovali, její znaky jsou proto kódovány na této pásce tímž způsobem, jakým jsou zapsány v kódu $\langle M \rangle$, tedy znak X_j je zapsán jako $(j)_B$. Navíc jsou však kódy znaků zleva zarovnány nulami tak, aby všechny bloky reprezentující znaky páskové abecedy M měly touž délku, přičemž jednotlivé bloky jsou mezi sebou odděleny znakem $|$. Polohu hlavy stroje M si UTS pamatuje polohou hlavy na této pracovní pásce.
3. **Stavová páska M .** Na této pásce je uložen stav, v němž se aktuálně stroj M nachází. Stav q_i s číslem i je zakódován jako $(i)_B$, tedy jako číslo i zapsané binárně. Jde o totéž číslo, které lze vyčíst z přechodové funkce zakódované v $\langle M \rangle$.

Postup výpočtu $\mathcal{U}(\langle M, x \rangle)$ je naznačen v algoritmu 6. Některé kroky tohoto algoritmu si nyní popíšeme podrobněji.

Krok 1 Zde se kontroluje, zda má $\langle M \rangle$ správný tvar popsáný v kapitole 5.2.1. To znamená, že se skládá ze správně zapsaných kódů jednotlivých instrukcí oddělených znakem $\#$, kde každá instrukce má tvar popsáný v (5.2). Na to stačí jeden průchod kódem $\langle M \rangle$ a není třeba další páska. Součástí syntaktické kontroly by mohl být i test toho, zda se v kódu nenacházejí dvě instrukce s týmž displejem, tj. test toho, zda je stroj na vstupu deterministický. Tento test však pro práci $\mathcal{U}$ není podstatný, protože pokud jsou v kódu $\langle M \rangle$ dvě instrukce s týmž displejem, použije se ta, kterou najde $\mathcal{U}$ jako první.

Krok 2 Pokud syntaktická kontrola selhala, je M podle úmluvy 5.4 považován za Turingův stroj s prázdnou přechodovou funkcí a odmítnutí je zde tedy na místě.

Krok 3 Délka bloku obsahující binární zápis čísla znaku na 2. pásce musí být dostatečně velká, aby do něj bylo možno zapsat kterýkoli znak, jenž se vyskytuje v kódu $\langle M \rangle$. Délku tohoto bloku si označíme pomocí b , její určení provede $\mathcal{U}$ následujícím způsobem: Před čtením bloku v kódu instrukce, který kóduje znak páskové abecedy (tj. 2. a 4. blok v pětici popisující instrukci, viz (5.2)) se $\mathcal{U}$ vrátí na začátek 2. pásky. Poté se čtením bloku píše na 2. pásku znaky 0. Na konci po přečtení celého kódu $\langle M \rangle$ zůstane na 2. pásce nejdelší posloupnost znaků 0. Tím bude dána délka bloku na pracovní pásce. Mezi každé dva bloky je vždy vložen znak oddělovače $|$. Vždy, když během simulace potřebuje UTS přiformátovat další blok délky b na 2. pásku, formátuje ho na délku totožnou se sousedícím blokem. To může $\mathcal{U}$ učinit tak, že si bude označovat naposledy přepsané políčko v předchozím bloku. Přitom si může $\mathcal{U}$ pamatovat označenou číslici ve stavu a psát místo ní na chvíli například

Algoritmus 6: Výpočet univerzálního Turingova stroje $\mathcal{U}$.

Vstup: Kód simulovaného stroje $\langle M \rangle$, kde $M = (Q, \Sigma, \delta, q_0, \{q_1\})$ a vstupní řetězec x , obojí zapsané na vstupní pásce. Druhá a třetí páska jsou prázdné.

Výstup: Výpočet $\mathcal{U}$ se zastaví, právě když výpočet $M(x)$ je konečný. Výpočet $\mathcal{U}$ je přijímající, právě když je přijímající výpočet $M(x)$. Po ukončení výpočtu obsahuje 2. páska slovo, které zůstalo na pásce M po ukončení výpočtu (zakódované způsobem popsáním při popisu 2. pásky $\mathcal{U}$).

// Inicializace

1 **if** $\langle M \rangle$ nekóduje syntakticky správně Turingův stroj **then**

2 **reject**

3 Urči délku b nejdelšího bloku pro zápis znaku na druhé pásce.

4 Překóduj vstup x na 2. pásku, tedy pracovní pásku M .

5 Na 3. zapiš znak 0, tedy binární zápis čísla počátečního stavu q_0 stroje M .

6 Vrať hlavu na začátky všech tří pásek.

// Hlavní cyklus simulace

7 **while** $\langle M \rangle$ obsahuje instrukci $\delta(q_i, X_j) = (q_k, X_l, Z)$, kde (q_i, X_j) je displej M **do**

8 Přepiš 3. pásku řetězcem $(k)_B$, kódujícím nový stav q_k .

9 Přepiš číslo znaku X_j v bloku pod hlavou na 2. pásce číslem znaku X_l .

10 **if** $Z \in \{L, R\}$ **then**

11 Přesuň hlavu na 2. pásce na sousední blok (v odpovídajícím směru).

12 **if** v odpovídajícím směru není žádný sousední blok (páska je prázdná) **then**

13 Přidej blok kódující $X_2 = \lambda$, tedy $0^{b-2}10$.

// Zakončení a úklid

14 **if** na 3. pásce je číslo stavu q_1 **then**

15 **accept**

16 **else**

17 **reject**

znak #. Zapsanou značku si pak $\mathcal{U}$ posouvá spolu s přiřisováním znaku 0 do nově vytvářeného bloku.

Krok 4 Znak 0 je zapsán jako 0^b a znak 1 jako $0^{b-1}1$, kde b označuje délku bloku na 2. pásce (viz bližší popis kroku 3). Připomeňme si, že v kódování abecedy popsáném v sekci 5.2.1 je $X_0 = 0$ a $X_1 = 1$, proto jsou kódy těchto dvou znaků při dané velikosti bloku b už pevně dané. Pokud je vstup x prázdný, pak je na 2. pásku zapsán jediný blok $0^{b-2}10$ kódující znak prázdného políčka $X_2 = \lambda$.

Krok 7 V tomto kroku je v kódu $\langle M \rangle$ hledána instrukce, jejíž displej je shodný s aktuálním stavem a čteným znakem M . Tedy hledá se instrukce $\delta(q_i, X_j) = (q_k, X_l, Z)$, kde q_i je stav s číslem zapsaným na 3. pásce $\mathcal{U}$ a X_j je znak kódovaný v bloku 2. pásky, který se nachází pod hlavou $\mathcal{U}$. K tomu stačí jednou projít řetězec $\langle M \rangle$.

Krok 9 Binární řetězec $(l)_B$ kódující znak X_l zapisuje $\mathcal{U}$ odprava od konce příslušného bloku 2. pásky, přičemž jej zleva doplní nulami do délky b (tedy až k předchozímu znaku $|$).

Krok 13 Pokud by se při pohybu přesunul M nad prázdné nenaformátované políčko λ , je třeba je nahradit kódem prázdného políčka. Protože je pevně určeno $X_2 = \lambda$, je kódem prázdného políčka řetězec $0^{b-2}10$.

Z konstrukce UTS je zřejmé, že splňuje požadavky na něj kladené, zejména tedy platí, že $L(\mathcal{U}) = L_u = \{\langle M, x \rangle \mid x \in L(M)\}$. Z toho vyplývá, že univerzální jazyk je částečně rozhodnutelný.

Věta 5.10 Univerzální jazyk $L(\mathcal{U}) = L_u = \{\langle M, x \rangle \mid x \in L(M)\}$ je částečně rozhodnutelný.

5.3. Algoritmicky rozhodnutelné problémy

Nyní jsme připraveni upřesnit to, co míníme algoritmicky řešitelným problémem. Rozhodovací problém P je pro nás daný popisem instance I problému a otázkou, kterou si o dané instanci klademe. My si rozhodovací problém formalizujeme jako jazyk L_P , který obsahuje slova kódující kladné instance tohoto problému, tedy instance, u nichž je na otázku kladenou v daném problému kladná odpověď. Otázka kladená v problému P je tedy převedena na otázku, zda dané slovo patří do jazyka L_P , reprezentuje tedy instanci problému P takovou, že otázka problému P má pro tuto instanci kladnou odpověď.

Příklad 5.11: Jazyk problému **HELLOWORLD**

Problém 2.1

Vzpomeňme si například na problém **HELLOWORLD**. Instancí tohoto problému je dvojice souborů — soubor P se zdrojovým kódem v jazyce C a vstupní soubor I . V problému **HELLOWORLD** se ptáme, zda prvních dvanáct znaků, jež program P se vstupem I vypíše, tvoří řetězec „Hello, world“. Jazyk L_{HW} , který tomuto problému

odpovídá, je potom

$$L_{HW} = \left\{ \langle P, I \rangle \mid \begin{array}{l} P \text{ je program v jazyce } C \text{ a } I \text{ je vstupní soubor, pro které platí,} \\ \text{že prvních dvanáct znaků, jež } P \text{ se vstupem } I \text{ vypíše, tvoří} \\ \text{řetězec „Hello, world“}. \end{array} \right\}$$

Na základě [Churchovy-Turingovy teze](#) můžeme nyní definovat pojem rozhodnutelného (a také částečně rozhodnutelného) jazyka (potažmo problému) následujícím způsobem.

Teze 5.1

Definice 5.12 (Rozhodnutelné a částečně rozhodnutelné jazyky) Nechť L je jazyk nad abecedou Σ . Řekneme, že jazyk L je

- *částečně rozhodnutelný* (též *rekurzivně spočetný*), pokud existuje Turingův stroj M , který jej přijímá, tedy $L = L(M)$ a že je
- *rozhodnutelný* (též *rekurzivní*), pokud existuje Turingův stroj M , který jej přijímá, tedy $L = L(M)$ a navíc se výpočet M zastaví pro každé vstupní slovo $x \in \Sigma^*$.

Třidu rozhodnutelných jazyků budeme označovat DEC (z *decidable*), třídu částečně rozhodnutelných jazyků budeme označovat PD (z *partially decidable*) a třídu doplňků částečně rozhodnutelných jazyků budeme označovat co-PD = $\{L \mid \bar{L} \in PD\}$. ◀

Očíslování Turingových strojů, jež jsme zavedli v kapitole 5.2.1, můžeme nyní použít i k očíslování částečně rozhodnutelných jazyků. Z toho plyne, třída PD je spočetná množina. Na druhou stranu třída všech jazyků nad abecedou Σ , tedy $\wp(\Sigma^*)$ je nespočetná dle [Cantorovy věty](#), což platí už pro jednoprvkovou abecedu $\Sigma = \{1\}$. Z toho plyne, že ve skutečnosti většina jazyků není částečně rozhodnutelná, tím spíše to platí o jazycích rozhodnutelných.

Viz též kapitoly 3.3.2 a 3.4.4

5.4. Algoritmicky vyčíslitelné funkce

Dalším důležitým pojmem pro nás budou funkce, jejichž hodnotu lze vyčíslit algoritmicky. Budeme převážně uvažovat řetězcové funkce. Na základě [Churchovy-Turingovy teze](#) můžeme pojem algoritmicky vyčíslitelné (řetězcové) funkce definovat následujícím způsobem.

Teze 5.1

Definice 5.13 (Algoritmicky vyčíslitelná funkce) Nechť Σ je konečná abeceda a $f : \Sigma^* \rightarrow \Sigma^*$ je (částečná) funkce. Řekneme, že f je *algoritmicky vyčíslitelná*, pokud je tato funkce turingovsky vyčíslitelná (ve smyslu definice 4.6). ◀

Je potřeba zmínit, že z principu nejsou všechny algoritmicky vyčíslitelné funkce totální, tedy definované pro všechny vstupy. To proto, že Turingovy stroje se nemusí zastavit pro všechny vstupy. Přesněji, je-li $f : \Sigma^* \rightarrow \Sigma^*$ algoritmicky vyčíslitelná funkce, jež je vyčíslovaná Turingovým strojem M , pak

$$\text{dom } f = \{x \in \Sigma^* \mid M(x) \downarrow\}. \quad (5.5)$$

Funkce f je tedy totální (čili definovaná pro všechny vstupy), má-li M tu vlastnost, že jeho výpočet se zastaví pro každý vstup x . Totální algoritmicky vyčíslitelné funkce pro nás budou dále podstatné například při definici převoditelnosti.

Funkce, které zobrazují řetězce na řetězce, jsou velmi obecné. Uvážíme-li kódování objektů zavedené v definici 5.8, dají se i funkce jiných typů převést na řetězcové funkce. Dále budeme uvažovat zejména funkce více parametrů a aritmetické funkce s číselnými parametry.

Úmluva 5.14 (Funkce více parametrů a různých typů) *Aritmetické funkci $f : \mathbb{N}^n \rightarrow \mathbb{N}$ takto odpovídá funkce $f' : \Sigma^* \rightarrow \Sigma^*$, pro niž platí, že pro libovolnou n -tici vstupních hodnot $x_1, \dots, x_n \in \mathbb{N}$ je $f'(\langle x_1, \dots, x_n \rangle) \simeq \langle f(x_1, \dots, x_n) \rangle$. Například funkci $f(x_1, x_2) = x^2 + y^2$ odpovídá řetězcová funkce $f'(\langle x_1, x_2 \rangle) = \langle x^2 + y^2 \rangle$. Dále budeme volně používat jako parametrů i hodnot funkcí řetězce či čísla podle potřeby.*

Podobně můžeme použít kódování objektů i při použití parametrů a hodnot funkcí jiných typů.

Teze 5.1 Poznámenejme, že na základě úmluvy 5.14 a Churchovy-Turingovy teze splývá pojem algoritmicky vyčíslitelné funkce s pojmy RAM-vyčíslitelné funkce a částečně rekurzivní funkce.

Definice 4.17 a ??

Očíslování Turingových strojů, jež jsme zavedli v kapitole 5.2.1, můžeme nyní použít i k očíslování algoritmicky vyčíslitelných funkcí. Na základě úmluvy 5.14 můžeme rozšířit číslování i na funkce více parametrů.

5.5. Univerzální funkce

Podobně jako máme k dispozici univerzální jazyk, který v sobě kóduje všechny částečně rozhodnutelné jazyky, máme na základě univerzálního Turingova stroje k dispozici i univerzální funkci, která reprezentuje ostatní algoritmicky vyčíslitelné funkce.

Věta 5.15 (O univerzální funkci) *Definujme univerzální funkci pro algoritmicky vyčíslitelné funkce jako funkci Ψ , která pro každou dvojici $\langle M, x \rangle$, kde M je Turingův stroj vyčíslovací funkci f_M a x je řetězec, splňuje*

$$\Psi(\langle M \rangle, x) \simeq f_M(x). \quad (5.6)$$

Potom funkce Ψ je algoritmicky vyčíslitelná.

Viz sekce 5.2.3 **Důkaz:** Univerzální Turingův stroj $\mathcal{U}$ je schopen simulovat jiné Turingovy stroje se zadaným vstupem a je jisté možné jej upravit tak, aby funkce jím vyčíslovaná byla právě univerzální funkce Ψ . $\square$

5.6. Bibliografické poznámky

5.7. Cvičení

Konstrukce Turingových strojů

V následujících cvičeních můžete pro konstrukci Turingových strojů použít model k -páskového stroje, pokud se vám to hodí. Pod „Turingovým strojem, který rozhoduje jazyk L “, míníme Turingův stroj M , který se vždy zastaví a $L = L(M)$. „Sestrojte“ znamená včetně instrukcí. „Popište“ znamená popište práci odpovídajícího Turingova stroje bez rozepisování instrukcí.

1. Sestrojte Turingův stroj, který rozhoduje jazyk palindromů (pomocí w^R označujeme zrcadlově obrácené slovo w , tj. napsané pozpátku):

$$L = \{ww^R \mid w \in \{0,1\}^*\}$$

2. Sestrojte Turingův stroj, který rozhoduje jazyk

$$L = \{a^n b^n c^n \mid n \geq 0\}.$$

3. Sestrojte Turingův stroj, který počítá funkci sčítání $f(x, y) = x + y$.
4. Popište, jak by pracovaly Turingovy stroje počítající funkce $f(x, y) = x \cdot y$ (násobení), $f(x, y) = x \operatorname{div} y$ (celočíslné dělení), $f(x, y) = x \bmod y$ (zbytek po celočíselném dělení).
5. Popište, jak by pracoval Turingův stroj rozhodující jazyk

$$L = \{a^i b^j c^k \mid i = j \text{ nebo } i = k\}.$$

6. Popište, jak by pracoval Turingův stroj rozhodující jazyk

$$L = \{a^i b^j c^k \mid i = j \text{ nebo } i = k \text{ nebo } j = k\}.$$

7. Popište, jak by pracoval Turingův stroj, který převádí číslo x zakódované unárně, tj. řetězcem 1^x , na číslo x zakódované binárně, tj. $(x)_B$. Rozmyslete si i převod opačným směrem.
8. Popište Turingův stroj, který přijímá jazyk

$$L = \{x \in \{0,1\}^* \mid x \text{ je binární reprezentací prvočísla}\}.$$

9. Popište, jak by pracoval Turingův stroj M , který ignoruje svůj vstup a během své práce postupně na výstupní pásku zapisuje seznam prvočísel (zakódovaných binárně, oddělených například pomocí #), tj. na výstupní pásce se postupně objevuje seznam 2, 3, 5, 7, 11, 13,

Varianty a omezení modelu Turingova stroje

10. Ukažte, jak lze libovolný Turingův stroj M převést na stroj M' , který pracuje stejně a má pouze binární abecedu, tj. vstupní abecedu $\{0, 1\}$, v pracovní je navíc jen znak $\wedge$ prázdného políčka.
11. Ukažte, jak lze libovolný Turingův stroj M rozšířit o práci se zářázkami. Modifikovaný stroj M' by měl mít v abecedě dva nové znaky, znak pro levou zářáčku $\triangleright$ a znak pro pravou zářáčku $\triangleleft$, při své práci potom udržuje invariant, že levá zářáčka $\triangleright$ je na nejlevější pozici, kam se dostala hlava stroje M' , zatímco pravá zářáčka $\triangleleft$ je na nejpravější pozici, na kterou se dostala během výpočtu hlava stroje M' , jinak stroj M' pracuje stejně jako M . (tj. zářázky v M' ohraničují prostor na pásce skutečně využitý strojem M' při výpočtu nad daným vstupem.)
12. Ukažte, jak lze libovolný Turingův stroj M s jednou obousměrně potenciálně nekonečnou páskou převést na Turingův stroj M' , který má pouze jednu jednosměrně (doprava) potenciálně nekonečnou pásku. V tomto modelu předpokládáme, že na začátku pásky je znak levé zářázky $\triangleright$, za ním následuje vstup. Hlava M' je na začátku na nejlevějším symbolu vstupu. Zářážka během výpočtu pomáhá M' poznat, kde je začátek pásky a z kterého políčka již nesmí učinit krok vlevo.
13. Ukažte, jak lze libovolný Turingův stroj M s $k \geq 1$ páskami převést na jednopáskový Turingův stroj M' . Stačí rozmyslet si princip úpravy M na M' . Jde tedy o náznak důkazu věty 4.10.
14. Ukažte, jak lze libovolný (jednopáskový) Turingův stroj M převést na Turingův stroj M' , který v každém kroku provádí jen dvě ze tří možných akcí (tj. každá instrukce buď změni stav a pozici hlavy, změni stav a písmeno na pásce, nebo změni písmeno na pásce a pozici hlavy, ale neučiní všechny tyto akce najednou).
15. Ukažte, jak lze jednopáskový Turingův stroj M převést na Turingův stroj M' , který ve svých instrukcích vždy pohne hlavou, tj. v žádné instrukci nezůstane hlava stát na místě.
16. Rozmyslete si, jakou třídu jazyků přijímají jednopáskové Turingovy stroje, které nesmí pohybovat hlavou vpravo (tj. hlava může zůstat stát nebo se pohnout vlevo).
17. Rozmyslete si, jakou třídu jazyků přijímají jednopáskové Turingovy stroje, které nesmí pohybovat hlavou vlevo (tj. hlava může zůstat stát nebo se pohnout vpravo).
18. Uvažte Turingův stroj s jednou páskou, která je potenciálně nekonečná jen v jednom směru (doprava). Uvažme, že Turingovu stroji povolíme jen dva typy pohybu hlavou: R a RESET. Při pohybu R se hlava pohne o jedno políčko doprava, při RESET se hlava přesune na začátek pásky. Ukažte, že takto omezený Turingův stroj je ekvivalentní s jednopáskovým Turingovým strojem.

19. UkaŹte, Źe Turingův stroj, který můŹe na kaŹdé políčko pásky zapsat nejvřjš jednu (číst ho můŹe vícekrát), je ekvivalentní s jednopáskovým Turingovým strojem.
20. UkaŹte, Źe pokud jednopáskovému Turingovu stroji zakáŹeme přepisovat políčka vstupu, pak takto omezené stroje přijímají právě regulární jazyky.
21. Stroj s více zásobníky definujeme podobně jako Turingův stroj, s tím rozdílem, Źe k páskám přistupuje jako k zásobníkům, tj. jedná se o rozšíření zásobníkového automatu o více zásobníků. V jednom kroku můŹe stroj přidat symboly na zásobníky, odebrat je ze zásobníky, nebo je nechat netknuté, na kaŹdém zásobníku pracuje nezávisle. V kaŹdém kroku se rozhoduje na základě svého stavu a znaků na vrcholech svých zásobníků. Instrukce v případě tří zásobníků můŹe vypadat například takto

$$\delta(q, a, b, c) = (q', PUSH\ a', POP, NOP),$$

tato instrukce v situaci, kdy řídící jednotka je ve stavu q a na vrcholech zásobníků jsou symboly a , b a c , přikazuje změnit stav řídící jednotky na q' , na první zásobník připsat a' , z druhého zásobníku odstranit vrchol zásobníku a třetí zásobník nechat netknutý. Symbol prázdného zásobníku $\triangleleft$ na vrcholu zásobníku znamená, Źe je prázdný a operace POP jej nechá netknutý, tento symbol nesmí být použit v operaci $PUSH$. Na začátku je vstup v prvním zásobníku.

UkaŹte, Źe tento model je ekvivalentní s modelem Turingova stroje (tj. mezi oběma modely lze převádět oběma směry), připustíme-li více než jeden zásobník.

Rozhodnutelné a částečně rozhodnutelné jazyky

22. UkaŹte, Źe rozhodnutelné jazyky jsou uzavřeny na operace sjednocení, průnik, doplněk, konkatenace (jsou-li L_1 a L_2 jazyky, pak jejich konkatenací je jazyk $L = L_1 \circ L_2 = \{xy \mid x \in L_1 \wedge y \in L_2\}$) a hvězdička (je-li L jazyk, pak jeho uzávěrem na hvězdičku je jazyk $L^* = \{x_1x_2 \dots x_k \mid (k \geq 0) \wedge \bigwedge_{i=1}^k (x_i \in L)\}$).
23. UkaŹte, Źe částečně rozhodnutelné jazyky jsou uzavřeny na operace sjednocení, průnik, konkatenace a hvězdička, zkuste si rozmyslet, jak je to s uzavřeností na doplněk.

6. Částečně rozhodnutelné jazyky a jejich vlastnosti

V této kapitole probereme některé vlastnosti částečně rozhodnutelných jazyků (a tedy i problémů). Podíváme se na některé ekvivalentní charakterizace částečně rozhodnutelných a rozhodnutelných jazyků, na uzávěrové vlastnosti těchto tříd. Ukážeme si navíc, že univerzální jazyk není algoritmicky rozhodnutelný.

6.1. Základní vlastnosti

V této kapitole prozkoumáme některé základní vlastnosti částečně rozhodnutelných jazyků.

6.1.1. Jednoduché ekvivalentní definice

Definice 5.12 Částečně rozhodnutelné jazyky jsme definovali jako jazyky přijímané Turingovými stroji. Mohli bychom je však ekvivalentně definovat pomocí zastavení Turingova stroje, jako domény algoritmicky vyčíslitelných funkcí, nebo pomocí existenční kvantifikace a rozhodnutelného jazyka.

Věta 6.1 (Ekvivalentní definice částečně rozhodnutelných jazyků) *Nechť L je jazyk nad abecedou Σ . Pak následující tvrzení jsou ekvivalentní:*

- (i) L je částečně rozhodnutelný.
- (ii) Existuje Turingův stroj M splňující

$$L = \{x \in \Sigma^* \mid M(x) \downarrow\}. \quad (6.1)$$

- (iii) Existuje algoritmicky vyčíslitelná funkce f splňující

$$L = \text{dom } f. \quad (6.2)$$

- (iv) Existuje rozhodnutelný jazyk B splňující

$$L = \{x \in \Sigma^* \mid (\exists y \in \Sigma^*)[\langle x, y \rangle \in B]\}. \quad (6.3)$$

Důkaz: Důkaz si rozdělíme na jednotlivé implikace.

(i) $\Rightarrow$ (ii) Je-li L částečně rozhodnutelný jazyk, pak $L = L(M)$ pro nějaký Turingův stroj M . Na základě tohoto stroje zkonstruujeme Turingův stroj M' , který se vstupem $x \in \Sigma^*$ pracuje následujícím způsobem:

Algoritmus 7: Výpočet M' se vstupem x

```

1 Pustř  $M(x)$ .
2 if  $M$  přijal vstup  $x$  then
3   | accept
4 else
5   | pokračuj neukončeným výpočtem (zacykli se)
```

Je zřejmé, že $x \in L = L(M)$, právě když $M'(x) \downarrow$, M' tedy splňuje (6.1).

(ii) $\Rightarrow$ (i) Nechť M je Turingův stroj, který splňuje (6.1). Sestrojíme Turingův stroj M' , který se vstupem $x \in \Sigma^*$ pracuje následujícím způsobem:

Algoritmus 8: Výpočet M' se vstupem x

```

1 Pustř  $M(x)$ .
2 accept
```

Je zřejmé, že potom M' přijímá právě řetězce z L , a tedy $L = L(M')$.

(ii) $\Leftrightarrow$ (iii) Tato ekvivalence plyne přímo z definice algoritmicky vyčíslitelné funkce a z (5.5).

Definice 5.13

(i) $\Rightarrow$ (iv) Předpokládejme, že L je částečně rozhodnutelný. Nechť M je Turingův stroj, který přijímá L , tedy $L = L(M)$. Definujme jazyk B následujícím způsobem:

$$B = \{ \langle x, y \rangle \mid (x \in \Sigma^*), (y \in \mathbb{N}) \text{ a } M \text{ přijme } x \text{ v nejvýše } y \text{ krocích} \}$$

Jazyk B je rozhodnutelný, neboť k rozhodnutí toho, zda daná dvojice $\langle x, y \rangle$ patří do B stačí simulovat běh $M(x)$ po y kroků. Pokud do té doby M přijme, pak $\langle x, y \rangle \in B$. Pokud se do té doby M nezastaví, nebo pokud odmítne, pak $\langle x, y \rangle \notin B$. Přitom platí, že $x \in L$, právě když $M(x)$ přijme. A pokud y je počet kroků tohoto přijímajícího výpočtu, pak $\langle x, y \rangle \in B$. Jazyk B tedy splňuje (6.3).

(iv) $\Rightarrow$ (i) Předpokládejme, že B je jazyk, který splňuje (6.3). Zkonstruujme Turingův stroj M , který se vstupem x pracuje podle následujícího algoritmu:

Algoritmus 9: Výpočet M se vstupem x

```

1 forall  $y \in \Sigma^*$  do
2   | if  $\langle x, y \rangle \in B$  then
3     | | accept
```

Algoritmus generuje v cyklu všechny řetězce y ze Σ^* v **shortlex uspořádání**. Pro každý takový řetězec y algoritmus otestuje v kroku 2, zda právě on není svědkem toho, že $x \in L$. $\square$

Důkaz toho, že (iv) implikuje (i) ve větě 6.1 lze snadno upravit i pro situaci, kdy jazyk B je jen částečně rozhodnutelný. To znamená, že nezáleží na tom, kolik existenčně kvantifikovaných proměnných použijeme v (6.3). Zformulujme toto tvrzení přesněji.

Věta 6.2 *Nechť B je částečně rozhodnutelný jazyk, potom i jazyk*

$$L = \{x \in \Sigma^* \mid (\exists y \in \Sigma^*)[\langle x, y \rangle \in B]\} \quad (6.4)$$

je částečně rozhodnutelný.

Důkaz: Je-li B částečně rozhodnutelný jazyk, pak podle bodu (iv) věty 6.1 existuje rozhodnutelný jazyk A , který splňuje, že

$$B = \{u \in \Sigma^* \mid (\exists v \in \Sigma^*)[\langle u, v \rangle \in A]\}.$$

Dohromady s (6.4) dostáváme, že

$$L = \{x \in \Sigma^* \mid (\exists u \in \Sigma^*)(\exists v \in \Sigma^*)[\langle \langle x, u \rangle, v \rangle \in A]\}.$$

Položme jazyk $A' = \{\langle x, u, v \rangle \mid \langle \langle x, u \rangle, v \rangle \in A\}$. Z rozhodnutelnosti jazyka A plyne, že i jazyk A' je rozhodnutelný. Navíc platí, že

$$L = \{x \in \Sigma^* \mid (\exists u \in \Sigma^*)(\exists v \in \Sigma^*)[\langle x, u, v \rangle \in A']\}.$$

Pokud nyní budeme uvažovat $y = \langle u, v \rangle$, pak

$$L = \{x \in \Sigma^* \mid (\exists y \in \Sigma^*)[y = \langle u, v \rangle \text{ a } \langle x, u, v \rangle \in A']\}. \quad (6.5)$$

Podmínka v (6.5) je rozhodnutelná a tedy podle bodu (iv) věty 6.1 je jazyk L částečně rozhodnutelný. $\square$

Dá se tedy říci, že částečně rozhodnutelné jazyky jsou uzavřené na existenční kvantifikaci v podmínce, která daný jazyk definuje.

Příklad 6.3: Částečná rozhodnutelnost neprázdnosti jazyka

Uvažme problém, kde se ptáme o daném Turingovu stroji M , zda přijímá alespoň jeden vstup, tedy zda jazyk $L(M)$ přijímaný strojem M je neprázdný. Tento problém je formalizován pomocí jazyka následujícím způsobem.

$$\text{NONEMPTY} = \{\langle M \rangle \mid L(M) \neq \emptyset\}.$$

Tuto definici můžeme zapsat i jako

$$\text{NONEMPTY} = \{\langle M \rangle \mid (\exists x \in \Sigma^*)[x \in L(M)]\},$$

S pomocí univerzálního jazyka $L_u = \{\langle M, x \rangle \mid x \in L(M)\}$ můžeme dále psát

$$\text{NONEMPTY} = \{\langle M \rangle \mid (\exists x \in \Sigma^*)[\langle M, x \rangle \in L_u]\}.$$

Jazyk L_u je částečně rozhodnutelný, jsa přijímaný **univerzálním Turingovým strojem**. Na základě věty 6.2 je tedy i jazyk NONEMPTY částečně rozhodnutelný. Později v příkladu 7.10 si ukážeme, že tento problém není rozhodnutelný.

6.1.2. Uzávěrové vlastnosti a Postova věta

V této kapitole se budeme věnovat jednoduchým uzávěrovým vlastnostem rozhodnutelných a částečně rozhodnutelných jazyků. Jde zejména o uzavřenost na běžné množinové operace.

Věta 6.4 *Uvažme jazyky L_1, L_2 nad abecedou Σ .*

- *Jsou-li L_1 a L_2 rozhodnutelné jazyky, potom $L_1 \cup L_2$, $L_1 \cap L_2$, $L_1 \cdot L_2$ a L_1^* jsou rozhodnutelné jazyky.*
- *Jsou-li L_1 a L_2 částečně rozhodnutelné jazyky, potom $L_1 \cup L_2$, $L_1 \cap L_2$, $L_1 \cdot L_2$ a L_1^* jsou částečně rozhodnutelné jazyky.*

Důkaz: Tvrzení ukážeme pro sjednocení, přičemž průnik, konkatenaci $L_1 \cdot L_2$ a Kleeneho uzávěr L_1^* ponecháme jako cvičení. Předpokládejme, že $L_1 = L(M_1)$ a $L_2 = L(M_2)$ pro dva Turingovy stroje M_1 a M_2 . Popíšeme konstrukci Turingova stroje M , který přijímá jazyk $L_1 \cup L_2$. Idea práce Turingova stroje M je velmi jednoduchá, se vstupem x bude postupovat podle následujícího algoritmu.

Algoritmus 10: Výpočet Turingova stroje M se vstupem x , kde $L(M) = L_1 \cup L_2$

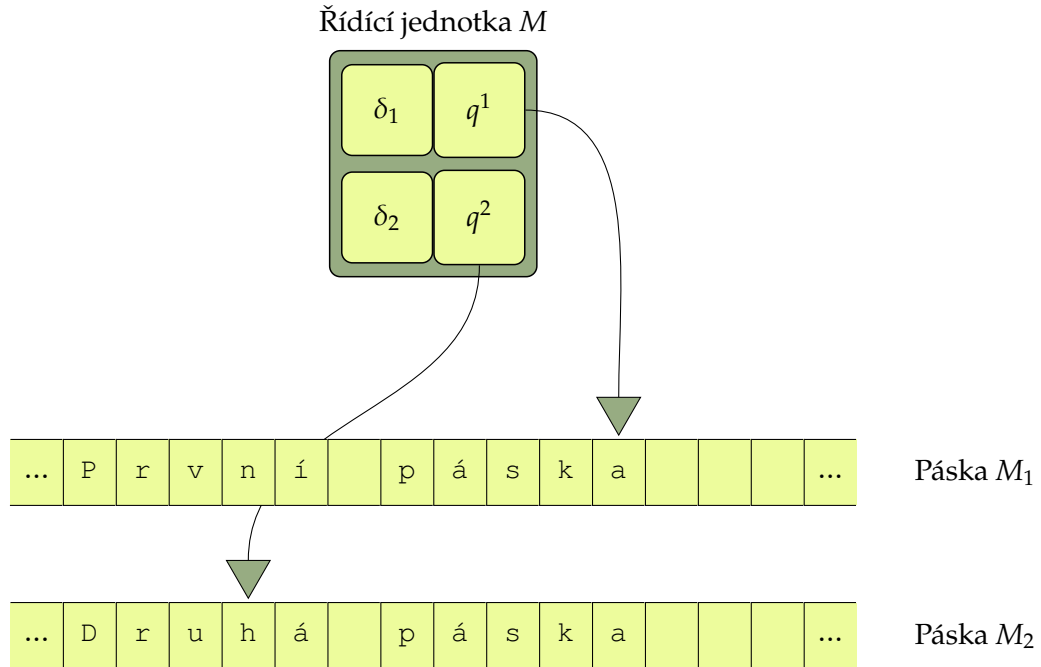
```

1 Pust'  $M_1(x)$  a  $M_2(x)$  paralelně
2 if jeden z výpočtů  $M_1(x)$  a  $M_2(x)$  skončí přijetím then
3   | accept
4 if oba výpočty  $M_1(x)$  a  $M_2(x)$  skončí odmítnutím then
5   | reject
```

Z existence Turingova stroje M přímo plyne, že $L_1 \cup L_2$ je částečně rozhodnutelný jazyk. Je zřejmé, že pokud M_1 a M_2 jsou Turingovy stroje, jejichž výpočet je pro každý vstup konečný, pak má tuto vlastnost i stroj M . To znamená, že pokud L_1 a L_2 jsou rozhodnutelné jazyky, pak je jazyk $L_1 \cup L_2$ rovněž rozhodnutelný.

Zbývá popsat, jakým způsobem bude Turingův stroj M simulovat paralelní běh dvou Turingových strojů M_1 a M_2 . Podle předpokladu jsou M_1 i M_2 jednopáskové Turingovy stroje. Stroj M popíšeme jako dvoupáskový s tím, že na základě věty 4.10 jej můžeme převést na jednopáskový. Na každé páse stroje M bude probíhat výpočet jednoho z strojů M_1 a M_2 . Přitom využíváme toho, že hlavy na páskách se pohybují nezávisle. Stav řídící

jednotky M bude složen ze stavů strojů M_1 a M_2 a přechodová funkce M bude složena z přechodových funkcí M_1 a M_2 . Řídící jednotka M tak v sobě vlastně obsahuje řídící jednotky obou strojů M_1 i M_2 .



Obrázek 6.1.: Struktura Turingova stroje M , v němž paralelně běží výpočty Turingových strojů M_1 a M_2 . Stav řídící jednotky M se skládá ze stavu q^1 Turingova stroje M_1 a q^2 Turingova stroje M_2 . Přechodová funkce Turingova stroje M vznikne kombinací přechodové funkce δ_1 Turingova stroje M_1 a přechodové funkce δ_2 Turingova stroje M_2 . První páska odpovídá pásce stroje M_1 a hlava na ní je řízena stavem q^1 a přechodovou funkcí δ_1 . Podobně druhá páska odpovídá pásce stroje M_2 a hlava na ní je řízena stavem q^2 a přechodovou funkcí δ_2 .

Pro úplnost si popíšeme konstrukci M i detailně, čtenář, který si dovede konstrukci M představit již na základě předešlého popisu, může zbytek důkazu směle přeskočit. Předpokládejme, že $M_1 = (Q_1, \Sigma_1, \delta_1, q_0^1, F_1)$ a $M_2 = (Q_2, \Sigma_2, \delta_2, q_0^2, F_2)$, na základě nich zkonstruujeme $M = (Q, \Sigma, \delta, q_0, F)$, kde

- $Q = (Q_1 \times Q_2) \cup \{q_0, q_{back}\},$
- $\Sigma = \Sigma_1 \cup \Sigma_2$ a
- $F = (Q_1 \times F_2) \cup (F_1 \times Q_2).$

Přechodová funkce δ je definovaná následujícím způsobem. Nejprve přidáme přechody, které okopírují vstup z první pásky i na druhou pásku.

$$\begin{aligned} (\forall a \in \Sigma \setminus \{\lambda\}) \quad & \delta(q_0, a, \lambda) = (q_0, a, a, R, R) \\ & \delta(q_0, \lambda, \lambda) = (q_{back}, \lambda, \lambda, L, L) \\ (\forall a \in \Sigma \setminus \{\lambda\}) \quad & \delta(q_{back}, a, a) = (q_{back}, a, a, L, L) \\ & \delta(q_{back}, \lambda, \lambda) = ([q_0^1, q_0^2], \lambda, \lambda, R, R) \end{aligned}$$

Dále definujeme hodnoty přechodové funkce, které odpovídají přechodovým funkcím strojů M_1 a M_2 . Pro každou dvojici stavů $q^1 \in Q$ a $q^2 \in Q_2$ a dvojici znaků $a, b \in \Sigma$ definujeme hodnotu $\delta([q^1, q^2], a, b)$ jedním z následujících způsobů:

1. Je-li $\delta_1(q^1, a) = (r^1, c, Z_1)$ a $\delta_2(q^2, b) = (r^2, d, Z_2)$, kde $r^1 \in Q_1$, $r^2 \in Q_2$, $c, d \in \Sigma$ a $Z_1, Z_2 \in \{L, N, R\}$, pak definujeme

$$\delta([q^1, q^2], a, b) = ([r^1, r^2], c, d, Z_1, Z_2).$$

2. Je-li $\delta_1(q^1, a) = (r^1, c, Z_1)$ a $\delta_2(q^2, b) = \perp$, kde $r^1 \in Q_1$, $c \in \Sigma$ a $Z_1 \in \{L, N, R\}$, a pokud navíc platí, že $q^2 \notin F_2$, pak definujeme

$$\delta([q^1, q^2], a, b) = ([r^1, q^2], c, b, Z_1, N).$$

Pokud platí, že $q^2 \in F_2$ (je to přijímající stav), pak ponecháme $\delta([q^1, q^2], a, b) = \perp$ (výpočet končí přijetím).

3. Je-li $\delta_1(q^1, a) = \perp$ a $\delta_2(q^2, b) = (r^2, d, Z_2)$, kde $r^2 \in Q_2$, $d \in \Sigma$ a $Z_2 \in \{L, N, R\}$, a pokud navíc platí, že $q^1 \notin F_1$, pak definujeme

$$\delta([q^1, q^2], a, b) = ([q^1, r^2], a, d, N, Z_2).$$

Pokud platí, že $q^1 \in F_1$ (je to přijímající stav), pak ponecháme $\delta([q^1, q^2], a, b) = \perp$ (výpočet končí přijetím).

4. V ostatních případech je $\delta([q^1, q^2], a, b) = \perp$, výpočet tedy končí. □

Způsob důkazu, kterým jsme ukazovali ve větě 6.4, že sjednocením dvou částečně rozhodnutelných jazyků vznikne opět částečně rozhodnutelný jazyk, využijeme i při důkazu Postovy věty. Tato věta dává do souvislosti rozhodnutelné jazyky, částečně rozhodnutelné jazyky a jejich doplňky.

Věta 6.5 (Postova věta) *Jazyk $L \subseteq \Sigma^*$ je rozhodnutelný, právě když jazyky L i $\bar{L}$ jsou částečně rozhodnutelné.*

Důkaz: Předpokládejme nejprve, že jazyk L je rozhodnutelný. Není těžké nahlédnout, že doplněk jazyka L , tedy jazyk $\bar{L}$, je také rozhodnutelný. Stačí uvážit, že je-li M Turingův stroj, který rozhoduje L (tj. $L = L(M)$ a $M(x) \downarrow$ pro každý vstup $x \in \Sigma^*$), pak Turingův

Definice 5.12

stroj M' , který vznikne z M tak, že zaměníme význam přijímajícího a nepřijímajícího stavu, tj. znegujeme jeho odpověď, bude rozhodovat jazyk $\bar{L}$. Z definice rozhodnutelného jazyka přímo plyne, že oba jazyky L a $\bar{L}$, jsouce rozhodnutelné, jsou i částečně rozhodnutelnými jazyky.

Předpokládejme nyní, že jazyky L a $\bar{L}$ jsou oba částečně rozhodnutelné a ukažme, že za tohoto předpokladu je jazyk L ve skutečnosti rozhodnutelný. Podle předpokladu existují Turingovy stroje M a M' , pro které platí, že $L = L(M)$ a $\bar{L} = L(M')$. Turingův stroj N , který bude rozhodovat L bude pracovat podle následujícího algoritmu.

Algoritmus 11: Výpočet N se vstupem x

```
1 Pust'  $M(x)$  a  $M'(x)$  paralelně, dokud jeden z nich nepřijme.  
2 if výpočet  $M(x)$  skončil přijetím then  
3   | accept  
4 else if výpočet  $M'(x)$  skončil přijetím then  
5   | reject
```

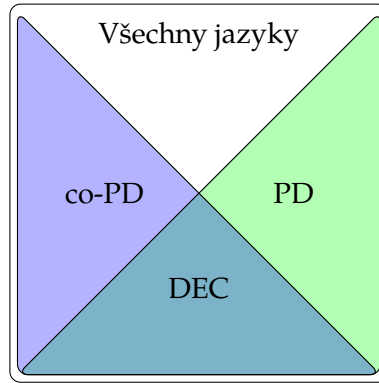
Uvědomme si, že pro každý vstup $x \in \Sigma^*$ platí, že $x \in L(M)$ nebo $x \in L(M')$. To proto, že $L(M) \cup L(M') = L \cup \bar{L} = \Sigma^*$. Z toho důvodu simulace v kroku 1 skončí s tím, že jeden ze strojů M nebo M' přijme vstup x . Pokud přijal Turingův stroj M , pak jistě $x \in L$, pokud naopak přijal Turingův stroj M' , pak $x \in \bar{L}$ a tedy $x \notin L$. Platí tedy, že $L = L(N)$ a navíc $N(x) \downarrow$ pro každý vstup x . Paralelní běh strojů M a M' lze zabezpečit tímž způsobem jako v důkazu věty 6.4. $\square$

Připomeňme si, že částečně rozhodnutelné jazyky jsou dále uzavřeny na existenční kvantifikaci ve smyslu věty 6.2. Toto však nelze říci o všeobecném kvantifikátoru, pomocí něž lze naopak charakterizovat doplňky částečně rozhodnutelných jazyků. Uvidíme dále, že univerzální jazyk L_u definovaný v (5.4) je sice částečně rozhodnutelný, ale nikoli rozhodnutelný. Připomeňme si značení zavedené v definici 5.12, kde jsme pomocí DEC označili třídu rozhodnutelných jazyků, pomocí PD třídu částečně rozhodnutelných jazyků a pomocí co-PD třídu jejich doplňků. Platí tedy, že $DEC \subsetneq PD$, $DEC \subsetneq co-PD$ a podle Postovy věty navíc $DEC = PD \cap co-PD$. Tato situace je naznačena na obrázku 6.2.

6.2. Proč nemohou všechny jazyky být částečně rozhodnutelné

Zamysleme se nyní nad tím, zda mohou být všechny jazyky nad nějakou abecedou Σ alespoň částečně rozhodnutelné. Ukážeme si, že nikoli, důvod je přitom jednoduchý, všech jazyků již nad jednoprvkovou abecedou není spočetně mnoho, zatímco těch částečně rozhodnutelných je jen spočetně mnoho. K popisu konkrétního jazyka, který není částečně rozhodnutelný, použijeme důkaz diagonalizací.

Diagonalizační argument si ukážeme při důkazu Cantorovy věty, která říká, že potenční množina $\mathcal{P}(A)$ množiny A nemůže mít shodnou mohutnost s množinou A .



Obrázek 6.2.: Vztahy mezi třídami rozhodnutelných jazyků DEC, částečně rozhodnutelných jazyků PD a doplňků částečně rozhodnutelných jazyků co-PD.

Věta 6.6 *Nechť A je množina a $\wp(A)$ označuje její potenční množinu. Potom $\wp(A)$ má větší mohutnost než množina A .*

Důkaz: Jistě platí, že mohutnost množiny $\wp(A)$ je shodná nebo větší než mohutnost množiny A , neboť zobrazení $g : A \rightarrow \wp(A)$ definované pro $a \in A$ jako $g(a) = \{a\}$ je jistě prosté. Ukážeme, že mohutnost množiny $\wp(A)$ nemůže být shodná s mohutností množiny A . K tomu je třeba ukázat, že neexistuje bijekce mezi množinami $\wp(A)$ a A . Nechť $f : A \rightarrow \wp(A)$ je libovolné prosté zobrazení množiny A do $\wp(A)$. Ukážeme, že existuje množina $B \subseteq A$ taková, že pro žádný prvek a neplatí $f(a) = B$ a tedy že f nemůže být bijekcí. Definujme množinu B jako

$$B = \{a \in A \mid a \notin f(a)\}. \quad (6.6)$$

Kdyby pro nějaký prvek a platilo, že $f(a) = B$, pak dostáváme, že

$$a \in f(a) \Leftrightarrow a \in B \Leftrightarrow a \notin f(a), \quad (6.7)$$

kde první ekvivalence platí díky tomu, že $f(a) = B$ a druhá ekvivalence platí dle definice B v (6.6). Platilo by tedy, že a patří do $f(a)$, právě když tam nepatří, což není pochopitelně možné, a takový prvek tedy neexistuje.

Dostáváme tedy, že pro každé prosté zobrazení $f : A \rightarrow \wp(A)$ platí, že existuje prvek $B \in \wp(A)$, který není obrazem žádného prvku $a \in A$, a tedy f nemůže být bijekce. Protože neexistuje bijekce mezi množinou A a množinou $\wp(A)$, platí tedy, že mohutnost $\wp(A)$ je větší než mohutnost A . $\square$

V důkazu věty 6.6 jsme použili diagonalizační argument, který spočíval v tom, že jsme při definici jazyka B použili ve výrazu $a \notin f(a)$ prvek a na obou stranách relace. Díváme se tak na diagonální prvky relace $b \in f(a)$. Jako jednoduchý důsledek tohoto tvrzení dostáváme, že množina jazyků nad konečnou abecedou není spočetná.

Důsledek 6.7 *Nechť Σ je neprázdná konečná abeceda a necht' $\mathcal{L} = \wp(\Sigma^*)$ je množina jazyků nad abecedou Σ . Potom $\mathcal{L}$ není spočetná množina.*

Důkaz: Z kapitoly 3.4.4 víme, že funkce index je bijekcí mezi Σ^* a $\mathbb{N}$. Množina řetězců Σ^* má tedy shodnou mohutnost s $\mathbb{N}$. Podle věty 6.6 platí, že mohutnost $\mathcal{L} = \wp(\Sigma^*)$ je větší než mohutnost $\mathbb{N}$ a množina $\mathcal{L}$ tedy není spočetná. $\square$

Uvědomme si, že třída částečně rozhodnutelných jazyků PD je spočetná, neboť každému částečně rozhodnutelnému jazyku můžeme přiřadit Gödelovo číslo Turingova stroje, který jej přijímá. Můžeme tedy psát $PD = \{L(M_e) \mid e \in \mathbb{N}\}$. Z toho plyne, že musí existovat jazyky, které nejsou částečně rozhodnutelné. Ve skutečnosti se dá říci, že většina jazyků nemůže být z principu ani částečně rozhodnutelná, natož rozhodnutelná.

6.3. Nerozhodnutelnost univerzálního jazyka

Nyní máme již dost nástrojů k tomu ukázat si nerozhodnutelnost univerzálního jazyka

$$L_u = \{\langle M, x \rangle \mid x \in L(M)\},$$

Univerzální jazyk L_u byl definován vzorcem (5.4) v podkapitole 5.2.3, kde byl zaveden i univerzální Turingův stroj $\mathcal{U}$.

Definice 5.12

který je formalizací problému [PŘIJETÍ VSTUPU](#). K důkazu tohoto faktu použijeme diagonalizační argument. Omezme se pro jednoduchost na Turingovy stroje, jejichž vstupní abeceda je binární a na jazyky, které jsou definované nad binární abecedou $\{0, 1\}$. Víme, že binární řetězce můžeme očíslovat přirozenými čísly pomocí číslování zavedeného v podkapitole 3.4.4, přesněji w_i označuje binární řetězec s číslem $i \in \mathbb{N}$. V podkapitole 5.2.1 jsme zavedli číslování Turingových strojů pomocí Gödelových čísel. Pomocí M_i označíme označili Turingův stroj s Gödelovým číslem $i \in \mathbb{N}$. Uvědomme si, že podle definice částečně rozhodnutelného jazyka jde právě o jazyky přijímané Turingovými stroji, číslování Turingových strojů lze tedy přenést i na částečně rozhodnutelné jazyky — i -tý částečně rozhodnutelný jazyk je ten, který je přijímán Turingovým strojem M_i , tedy $L(M_i)$.

Na základě těchto úvah si můžeme charakteristickou funkci univerzálního jazyka reprezentovat jako matici $\mathbf{A}$, v níž řádky i sloupce jsou indexovány přirozenými čísly. Řádek s indexem $i \in \mathbb{N}$ odpovídá jazyku $L(M_i)$ přijímanému Turingovu stroji s Gödelovým číslem i . Sloupec s indexem $j \in \mathbb{N}$ pak odpovídá j -tému binárnímu slovu w_j (omezíme-li se na Turingovy stroje s binární vstupní abecedou). Hodnota prvku $\mathbf{A}_{i,j}$ je pak určena podle toho, zda $w_j \in L(M_i)$, přesněji

$$\mathbf{A}_{i,j} = \begin{cases} 1 & w_j \in L(M_i) \\ 0 & w_j \notin L(M_i). \end{cases} \quad (6.8)$$

Řádek s indexem i tedy určuje charakteristickou funkci jazyka $L(M_i)$, přičemž každý částečně rozhodnutelný jazyk má v matici $\mathbf{A}$ svůj řádek (dokonce nekonečně mnoho řádků).

Chceme-li zkonstruovat jazyk, který není částečně rozhodnutelný, stačí zkonstruovat nekonečný vektor $\mathbf{b}$, který není roven žádnému řádku v matici $\mathbf{A}$. Uvážíme-li tento

vektor jako charakteristickou funkci jazyka, půjde o jazyk, který není částečně rozhodnutelný (jinak by měl svůj řádek v matici $\mathbf{A}$). Diagonalizace nám nabízí jednoduchý způsob, jak takovýto vektor najít. Hodnotu $\mathbf{b}_i$ pro index $i = 1, \dots, n$ určíme tak, aby se lišila od prvku $\mathbf{A}_{i,i}$, přesněji pro dané $i \in \mathbb{N}$ definujeme hodnotu vektoru $\mathbf{b}$ na indexu i jako

$$\mathbf{b}_i = 1 - \mathbf{A}_{i,i}.$$

Pro takto definovaný vektor $\mathbf{b}$ jistě platí, že se od každého řádku i sloupce matice $\mathbf{A}$ liší v diagonálním prvku. Tato situace je naznačena na obrázku 6.3.

		binární řetězce						
		w_0	w_1	w_2	...	w_i	...	w_j
částečně rozhodnutelné jazyky	$L(M_0)$	0	1	0	...	0	...	1
	$L(M_1)$	1	1	0	...	1	...	0
	$L(M_2)$	1	1	0	...	0	...	1
	...							
	$L(M_i)$	0	0	1	...	1	...	0
	DIAG	1	0	1	...	0	...	1

Obrázek 6.3.: Matice $\mathbf{A}$ použitá při definici diagonálního jazyka DIAG.

Připomeňme si, že řetězec w_i je kódem Turingova stroje s Gödelovým číslem i , tedy pro každé $i \in \mathbb{N}$ platí $w_i = \langle M_i \rangle$. Vektoru $\mathbf{b}$ tak odpovídá *Diagonální jazyk* DIAG daný doplnkem diagonály matice $\mathbf{A}$:

$$\text{DIAG} = \{ \langle M \rangle \mid \langle M \rangle \notin L(M) \}$$

Diagonále matice $\mathbf{A}$ pak odpovídá doplněk diagonálního jazyka. Připomeňme si, že podle naší konvence každý řetězec kóduje nějaký Turingův stroj, tedy doplněk diagonálního jazyka je dán následujícím předpisem:

Konvence 5.4

$$\overline{\text{DIAG}} = \{ \langle M \rangle \mid \langle M \rangle \in L(M) \}. \quad (6.9)$$

Do DIAG tedy patří ty kódy Turingových strojů, jež nepřijímají vlastní kód. Vektor $\mathbf{b}$ určuje charakteristickou funkci jazyka DIAG, neboť pro $i \in \mathbb{N}$ platí, že

$$\mathbf{w}_i = \langle M_i \rangle \in \text{DIAG} \Leftrightarrow \mathbf{w}_i \notin L(M_i) \Leftrightarrow \mathbf{A}_{i,i} = 0 \Leftrightarrow \mathbf{b}_i = 1. \quad (6.10)$$

Z definice $\mathbf{b}$ plyne, že tento vektor není shodný s žádným řádkem matice $\mathbf{A}$, a tedy jazyk DIAG není částečně rozhodnutelný. Formální důkaz tohoto faktu lze zapsat velmi jednoduše.

Věta 6.8 *Jazyk DIAG není částečně rozhodnutelný. Jazyk $\overline{\text{DIAG}}$ není rozhodnutelný, je však částečně rozhodnutelný.*

Důkaz: Předpokládejme sporem, že DIAG je částečně rozhodnutelný jazyk. Existuje tedy Turingův stroj M , který tento jazyk přijímá, čili $L(M) = \text{DIAG}$. Potom ovšem

$$\langle M \rangle \in L(M) \Leftrightarrow \langle M \rangle \in \text{DIAG} \Leftrightarrow \langle M \rangle \notin L(M), \quad (6.11)$$

kde první ekvivalence plyne z faktu $L(M) = \text{DIAG}$ a druhá ekvivalence plyne z definice diagonálního jazyka v (6.3). Tím dostáváme $\langle M \rangle \in L(M) \Leftrightarrow \langle M \rangle \notin L(M)$, což je pochopitelně spor. Jazyk DIAG tedy nemůže být částečně rozhodnutelný. Nerozhodnutelnost jazyka $\overline{\text{DIAG}}$ plyne z Postovy věty, jeho částečná rozhodnutelnost pak z existence univerzálního Turingova stroje, neboť $\langle M \rangle \in \overline{\text{DIAG}}$, právě když $\langle M \rangle \in L(M)$, tedy právě když $\mathcal{U}(\langle M \rangle, \langle M \rangle)$ přijme. $\square$

Věta 6.5

S pomocí diagonálního jazyka již snadno ukážeme, že univerzální jazyk je nerozhodnutelný.

Věta 6.9 *Univerzální jazyk L_u je algoritmicky nerozhodnutelný.*

Důkaz: Pro každý Turingův stroj M platí, že

$$\langle M \rangle \in \overline{\text{DIAG}} \Leftrightarrow \langle M \rangle \in L(M) \Leftrightarrow \langle M, \langle M \rangle \rangle \in L_u, \quad (6.12)$$

kde první ekvivalence plyne z definice diagonálního jazyka v (6.9) a druhá ekvivalence z definice univerzálního jazyka v (5.4). Z toho plyne, že kdyby byl jazyk L_u rozhodnutelný, byl by rozhodnutelný i jazyk $\overline{\text{DIAG}}$, neboť k rozhodnutí $\overline{\text{DIAG}}$ by stačilo zavolat algoritmus, který rozhoduje L_u se vstupem $\langle M, \langle M \rangle \rangle$. Podle věty 6.8 však $\overline{\text{DIAG}}$ rozhodnutelný není, a tedy ani jazyk L_u není rozhodnutelný. $\square$

Uvědomme si, že z Postovy věty (Věta 6.5) a toho, že L_u je částečně rozhodnutelný jazyk (Věta 5.10), který není rozhodnutelný (Věta 6.9), plyne, že doplněk univerzálního jazyka $\overline{L_u}$ není částečně rozhodnutelný jazyk.

6.4. Výčet slov jazyka

V této kapitole se podíváme na částečně rozhodnutelné jazyky z pohledu enumerace. Enumerací jazyka L zde myslíme to, že jsme algoritmicky schopni vypsát všechna slova z tohoto jazyka. Tuto vlastnost jazyka budeme formalizovat pomocí zvláštního Turingova stroje, kterému budeme říkat enumerátor.

Definice 6.10 (Enumerátor) Nechť $L \subseteq \Sigma^*$ je jazyk nad abecedou Σ . Turingův stroj E nazveme *enumerátorem* pro jazyk L , pokud splňuje následující vlastnosti:

1. Turingův stroj E ignoruje svůj vstup.
2. Během své práce vypisuje E na zvláštní výstupní pásku řetězce z jazyka L ,
3. Každý řetězec $w \in L$ je někdy vypsan Turingovým strojem E . ◀

Pokud je L nekonečný jazyk, pak z principu své činnosti enumerátor E nikdy svou činnost neukončí. Ani pokud je L konečný jazyk, nemusí se E zastavit — buď dokola vypisuje již vypsaná slova z jazyka L , nebo po vypsání posledního slova z L vstoupí do nekonečného cyklu bez vypisování dalších slov. Enumerátor E může vypsat některá slova z jazyka L vícekrát, není ovšem těžké změnit jakýkoli enumerátor takovým způsobem, aby každé slovo vypsal nejvýš jednou. Stačí E změnit tak, aby se vždy před vypsáním slova w z L podíval, zda již bylo slovo w vypsáno či nikoli. Pokud již slovo w vypsáno bylo, E jej již podruhé vypisovat nemusí.

Příklad 6.11

Popišme například enumerátor pro jazyk palindromů

$$\text{PAL} = \{w = w^R \mid w \in \{a, b\}^*\}, \quad (6.13)$$

Připomeňme si, že v příkladu 4.4 jsme zkonstruovali Turingův stroj M , který rozhoduje jazyk PAL. Nyní zkonstruujeme s pomocí tohoto Turingova stroje jednoduchý enumerátor pro tento jazyk.

Algoritmus 12: Algoritmus enumerátoru E pro jazyk PAL

```

1 forall  $w \in \Sigma^*$  do                                     //  $\forall$  shortlex uspořádání.
2   | if  $w = w^R$  then
3   |   | print( $w$ )

```

Viz též (4.1)
v příkladu 4.4.

Charakterizaci částečně rozhodnutelných jazyků pomocí enumerátorů zachycuje následující věta.

Věta 6.12 (Vyčíslitelnost částečně rozhodnutelných jazyků) Jazyk L je částečně rozhodnutelný, právě když existuje enumerátor E pro L .

Důkaz: Předpokládejme nejprve, že L je částečně rozhodnutelný jazyk. Dle bodu (iv) ve větě 6.1 existuje tedy rozhodnutelný jazyk B splňující

$$L = \{x \in \Sigma^* \mid (\exists y \in \Sigma^*)[\langle x, y \rangle \in B]\}.$$

Na základě toho můžeme sestavit pro jazyk L enumerátor E , který bude pracovat dle následujícího algoritmu:

Algoritmus 13: Algoritmus enumerátoru E pro jazyk L

Vstup: Vstup je ignorován

Výstup: Vypisuje postupně prvky jazyka L

```
1 forall  $w \in \Sigma^*$  do                                     // v shortlex uspořádání.
2   | if  $w$  kóduje dvojici  $\langle x, y \rangle \in B$  then
3   |   | print ( $x$ )
```

Předpokládejme na druhou stranu, že E je enumerátor pro jazyk L a popišme Turingův stroj M , který bude přijímat L , tedy $L = L(M)$. Tento stroj bude pracovat dle následujícího algoritmu:

Algoritmus 14: Algoritmus Turingova stroje M , přijímajícího L

Vstup: $x \in \Sigma^*$

Výstup: Algoritmus přijme, právě když $x \in L$.

```
1 Pusť enumerátor  $E$  a průběžně čti jeho výstupní pásku
2 if  $E$  vypsál  $x$  then
3   | accept          // Současně je pochopitelně zastavena činnost  $E$ 
```

Dle definice enumerátoru $M(x)$ přijme, právě když řetězec x je někdy enumerátorem vypsán, tedy právě když $x \in L$. Jinými slovy $L(M) = L$. $\square$

V případě rozhodnutelného jazyka L jsme schopni navíc sestavit enumerátor, který vypisuje slova tohoto jazyka systematicky.

Věta 6.13 (Vyčíslitelnost rozhodnutelných jazyků) *Jazyk $L \subseteq \Sigma^*$ je rozhodnutelný, právě když existuje enumerátor E , který vypisuje slova L v pořadí daném shortlex uspořádáním.*

Definice 3.1

Důkaz: Předpokládejme nejprve, že jazyk L je rozhodnutelný. Potom následující algoritmus popisuje práci enumerátoru E , který vypisuje slova L v pořadí daném shortlex uspořádáním:

Algoritmus 15: Algoritmus enumerátoru E pro jazyk L

Vstup: Žádný.

Výstup: Slova z L v pořadí daném shortlex uspořádáním.

```
1 forall  $w \in \Sigma^*$  do                                     // v shortlex uspořádání.
2   | if  $w \in L$  then
3   |   | print ( $x$ )
```

Předpokládejme na druhou stranu, že E je enumerátor, který vypisuje slova v pořadí daném shortlex uspořádáním. Pokud je jazyk L konečný, pak je jistě rozhodnutelný (dokonce konečným automatem), nadále budeme tedy předpokládat, že L je nekonečný jazyk. Nyní můžeme popsat algoritmus Turingova stroje M , který rozhoduje L , tj. přijímá L a navíc se jeho výpočet zastaví s každým vstupem.

Algoritmus 16: Algoritmus Turingova stroje M rozhodujícího L

Vstup: Řetězec $x \in \Sigma^*$

Výstup: M přijme, pokud $x \in L$, jinak odmítne.

```
1 Pust' enumerátor  $E$  a průběžně čti slova, která vypisuje na svou výstupní pásku
2 if  $E$  vypsál  $x$  then
3   | accept
4 else if  $E$  vypsál slovo  $y > x$  then
5   | reject
```

Z vlastností **shortlex uspořádání** vyplývá, že všechny řetězce y , které jsou delší než x , jsou větší než y v **shortlex uspořádání** (tedy $y > x$). Vzhledem k tomu, že abeceda Σ je konečná, L obsahuje jen konečně mnoho řetězců, které jsou v **shortlex uspořádání** menší než x . Protože jazyk L je sám nekonečný, musí obsahovat řetězec $y > x$. Řetězec y je někdy vypsán enumerátorem E . Připomeňme si, že E vypisuje řetězce z L v pořadí daném **shortlex uspořádáním**, mohou nastat tedy následující dva případy.

- (i) Buď $x \in L$, v tom případě musel E vypsát řetězec x před řetězcem y a řetězec x je přijat v kroku 3 dříve, než je vůbec vypsán jakýkoli řetězec $y > x$.
- (ii) Nebo $x \notin L$, v tom případě E řetězec x vůbec nevypíše. Vypíše ovšem někdy řetězec $y \in L$, který je větší než x v **shortlex uspořádání**. V tom okamžiku je už ovšem jasné, že enumerátor E nikdy x nevypíše a v kroku 5 může tedy Turingův stroj M odmítnout.

Turingův stroj M tedy skutečně rozhoduje jazyk L . □

6.5. Cvičení

Diagonalizace a nerozhodnutelné problémy

1. Pomocí diagonalizačního argumentu ukažte, že následující jazyky nejsou rekurzivně spočetné:

$$\begin{aligned} L_1 &= \{w_i \mid w_{2i} \notin L(M_i)\} \\ L_2 &= \{w_i \mid w_i \notin L(M_{2i})\} \end{aligned}$$

Nápověda: Při důkazu si nevystačíte s diagonálou matice, pomocí níž je definován jazyk DIAG, hledejte jinou nekonečnou množinu prvků této matice, která by mohla posloužit podobně jako diagonála v případě DIAG. U jazyka L_2 je třeba využít vlastností kódování, které jsme použili pro Turingovy stroje, rozmyslete si, že ke každému TS M existuje (v našem kódování) ekvivalentní TS, který má sudé Gödelovo číslo

2. Definujme problém použití stavu následovně:

Problém 6.14: Použití stavu TS

Instance: Kód Turingova stroje (zapsaný binárně) x , vstupní řetězec $y \in \{0,1\}^*$ a číslo stavu q .

Otázka: Použije Turingův stroj M_x při výpočtu nad y stav q ?

Ukažte, že problém Použití stavu TS je algoritmicky nerozhodnutelný (tj. odpovídající jazyk není rekurzivní).

7. Převoditelnost a úplnost

Z kapitoly 6.3 víme, že jazyk univerzálního stroje, čili univerzální jazyk L_u je algoritmic-
ky nerozhodnutelný. Důkaz tohoto faktu jsme provedli diagonalizací. Naším cílem je
nyní využít nerozhodnutelnosti univerzálního jazyka k důkazům nerozhodnutelnosti
dalších jazyků a problémů. Postup důkazu pomocí převoditelnosti si nejprve ukážeme
na problému zastavení.

7.1. Problém zastavení

Uvažme následující problém.

Problém 7.1: ZASTAVENÍ

Instance: Turingův stroj M a řetězec $x \in \Sigma^*$.

Otázka: Platí $M(x) \downarrow$, čili zastaví se výpočet Turingova stroje M nad
vstupem x ?

Naším cílem je ukázat, že tento problém je algoritmic-
ky nerozhodnutelný. Mohli bychom
postupovat obdobně jako při důkazu nerozhodnutelnosti univerzálního jazyka, a tedy
problému **PŘIJETÍ VSTUPU**, v kapitole 6.3. Upravit diagonalizační argument použitý v dů-
kazu věty 6.9 by nebylo příliš složité. My však budeme postupovat jinak. Využijeme
toho, že problém **ZASTAVENÍ** se od problému **PŘIJETÍ VSTUPU** příliš neliší. Skutečně jediný
rozdíl je, zda nás zajímá přijetí či zastavení, ovšem kdybychom definovali přijetí vstu-
pu Turingovým strojem s pomocí zastavení a nikoli přijímajícího stavu, tento rozdíl by
zmizel. Úvaha, jež nás vede k důkazu nerozhodnutelnosti problému **ZASTAVENÍ** je násle-
dující: Kdyby byl problém **ZASTAVENÍ** algoritmic-
ky rozhodnutelný, znamenalo by to, že
problém **PŘIJETÍ VSTUPU** by rovněž musel být rozhodnutelný. Víme ovšem, že tak tomu
není. Problém **ZASTAVENÍ** formalizujeme jako jazyk

Problém 5.9

$$\text{HALT} = \{ \langle M, x \rangle \mid M(x) \downarrow \}.$$

Věta 7.2 *Jazyk HALT je algoritmic-
ky nerozhodnutelný.*

Důkaz: Předpokládejme pro spor, že HALT je algoritmic-
ky rozhodnutelný jazyk, má-
me tedy Turingův H , který se vždy zastaví a $\text{HALT} = L(H)$. Na základě tohoto Turingova
stroje nyní sestavíme Turingův stroj V , který bude rozhodovat univerzální jazyk L_u :

Algoritmus 17: Výpočet V se vstupem $\langle M, x \rangle$

- 1 Uprav M na Turingův stroj M' , který se místo odmítnutí vždy zacyklí
 - 2 Pusť $H(\langle M', x \rangle)$
-

Turingův stroj M' se od M liší jen velmi málo. Je-li $M = (Q, \Sigma, \delta, q_0, F)$, pak $M' = (Q, \Sigma, \delta', q_0, F)$, kde pro $q \in Q$ a $a \in \Sigma$ definujeme následujícím způsobem:

$$\delta'(q, a) = \begin{cases} \delta(q, a) & \delta(q, a) \neq \perp \text{ nebo } q \in F \\ (q, a, N) & \text{jinak, tj. } \delta(q, a) = \perp \text{ a } q \notin F \end{cases}$$

Jde tedy opravdu jen o to, že v situaci, kdy by M odmítl tedy skončil výpočet ve stavu, který není přijímající, přejde M' místo toho do nekonečné smyčky. Uvažme nyní, že

$$\langle M, x \rangle \in L_u \Leftrightarrow x \in L(M) \Leftrightarrow x \in L(M') \Leftrightarrow M'(x) \downarrow \Leftrightarrow \langle M', x \rangle \in \text{HALT}. \quad (7.1)$$

Vskutku, pokud $\langle M, x \rangle \in L_u$, potom $M(x)$ přijme, tedy i $M'(x)$ přijme a tím i $M'(x) \downarrow$. Pokud na druhou stranu $\langle M, x \rangle \notin L_u$, potom buď $M(x) \uparrow$ nebo $M(x)$ odmítne, v obou případech ovšem $M'(x) \uparrow$. Volání $H(\langle M', x \rangle)$ přijme, právě když $\langle M', x \rangle \in \text{HALT} = L(H)$, a tedy právě když $\langle M, x \rangle \in L_u$ (dle (7.1)). Turingův stroj V tedy přijímá jazyk L_u a vždy se zastaví. To je ovšem v rozporu s nerozhodnutelností L_u , již jsme ukázali ve větě 6.9. $\square$

7.2. Definice převoditelnosti

Převoditelnost nám nabízí obecný nástroj, který umožňuje s pomocí jednoho nerozhodnutelného problému ukázat nerozhodnutelnost jiného problému.

Definice 7.3 (m -převoditelnost) Necht A a B jsou dva jazyky nad abecedou Σ . Řekneme, že jazyk A je m -převoditelný na jazyk B , pokud existuje totální algoritmicky vyčíslitelná funkce $f : \Sigma^* \rightarrow \Sigma^*$, pro kterou platí, že

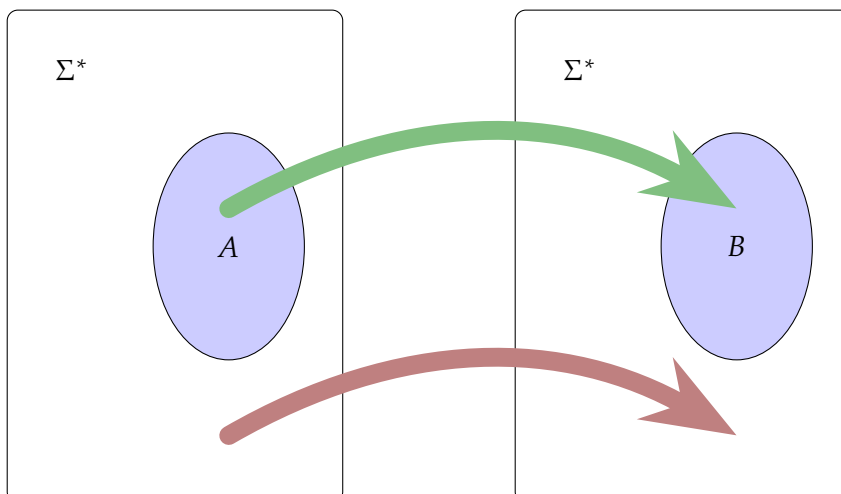
$$(\forall x \in \Sigma^*) [x \in A \Leftrightarrow f(x) \in B]. \quad (7.2)$$

Tento fakt označíme pomocí $A \leq_m B$. $\blacktriangleleft$

Funkce f , která převádí jazyk A na jazyk B tedy zobrazuje prvky z A na prvky B a prvky mimo A na prvky mimo B . Toto je naznačeno na obrázku 7.1.

Poznámka 7.4 (1-převoditelnost) Znak m v pojmu m -převoditelnosti se dá považovat za zkratku many to one reducibility, nebo mapping reducibility. První z těchto výkladů pochází z toho, že funkce f není nutně prostá, může tedy dojít k tomu, že se několik řetězců zobrazí funkcí f na jeden řetězec. Poznamenejme, že kromě m -převoditelnosti se často zavádí i 1-převoditelnost, jež se od m -převoditelnosti liší tím, že navíc vyžaduje, aby funkce f byla prostá (tedy one to one). Pro naše účely však 1-převoditelnost nepřináší nic zajímavého a vystačíme si s m -převoditelností.

Poznámka 7.5 (Turingovská převoditelnost) Pomocí převoditelnosti se mimo jiné snažíme zachytit následující intuici:



Obrázek 7.1.: Princip převodu problému A na problém B pomocí funkce f . Prvky z A jsou mapovány na prvky z B , zatímco prvky mimo A jsou mapovány na prvky mimo B .

Je-li problém A převoditelný na problém B a B je algoritmicky řešitelný, pak je algoritmicky řešitelný i problém A .

Uvědomme si ovšem, že m -převoditelnost je mnohem restriktivnější. Z faktu, že $A \leq_m B$ víme, že pokud máme k dispozici algoritmus rozhodující B , pak rozhodnutí, zda $x \in A$ dokážeme učinit výpočtem hodnoty $f(x)$ (kde f je funkce převádějící A na B) a dotazem, zda $f(x) \in B$, přičemž odpověď na tento dotaz dává přímo odpověď na původní otázku, zda $x \in A$. Naopak, pokud chceme ukázat $A \leq_m B$, nestačí popsat algoritmus rozhodující A , který může volat rozhodovací algoritmus pro B libovolně. Jsme opět omezeni tím, že je třeba popsat algoritmicky vyčíslitelnou funkci f tak, aby pro rozhodnutí, zda $x \in A$, stačilo rozhodnout, zda $f(x) \in B$.

Výše zmíněné intuici odpovídá turingovská převoditelnost, kterou si (ne zcela formálně) zavedeme. Řekneme, že jazyk A je turingovsky převoditelný na jazyk B , pokud lze popsat algoritmus rozhodující A s tím, že může během výpočtu pokládat dotazy na to, zda $y \in B$ pro libovolný počet řetězců y . Tento fakt označíme pomocí $A \leq_T B$.

Zřejmě platí, že pokud je $A \leq_m B$, pak i $A \leq_T B$, ale naopak to platit nemusí. Například $\overline{\text{HALT}} \leq_T \text{HALT}$, ovšem $\overline{\text{HALT}} \not\leq_m \text{HALT}$, neboť HALT je částečně rozhodnutelný jazyk, zatímco $\overline{\text{HALT}}$ nikoli. Pomocí m -převoditelnosti dokážeme tedy rozlišit mezi jazykem HALT a jeho doplňkem $\overline{\text{HALT}}$, pomocí turingovské převoditelnosti nikoli. I proto dále budeme využívat m -převoditelnost.

Připomeňme si, že v kapitole 2.2 jsme si ukázali převod problému [HELLOWORLD](#) na problém [VOLÁNÍ FUNKCE foo](#), přičemž konstrukce splňovala požadavky definice 7.3, ukázali jsme tedy, že problém [HELLOWORLD](#) je m -převoditelný na problém [VOLÁNÍ FUNKCE foo](#).

V důkazu věty 7.2 jsme ve skutečnosti ukázali, že jazyk L_u je převoditelný na jazyk HALT. Zformulujme si zde nyní toto tvrzení jako důsledek.

Důsledek 7.6 Platí, že $L_u \leq_m \text{HALT}$.

Důkaz: Připomeňme si, že v důkazu věty 7.2 jsme popsali konstrukci Turingova stroje M' z Turingova stroje M , kde M' se od M lišil jen v tom, že na každém vstupu x , jež by M odmítl, výpočet M' vůbec neskončil. Jinak byl M' shodný s M . Protože úprava M na M' je velmi jednoduchá a bylo by lze ji provést algoritmicky, je funkce $f : \Sigma^* \rightarrow \Sigma^*$ definovaná jako $f(\langle M, x \rangle) = \langle M', x \rangle$ jistě algoritmicky vyčíslitelná. Dle ekvivalence (7.1) navíc dostáváme, že

$$\langle M, x \rangle \in L_u \Leftrightarrow f(\langle M, x \rangle) = \langle M', x \rangle \in \text{HALT}. \quad (7.3)$$

Funkce f je tedy převodní funkcí, jež ukazuje, že $L_u \leq_m \text{HALT}$. $\square$

Diagonální jazyk
DIAG byl zaveden
v (6.3) a jeho
doplňěk $\overline{\text{DIAG}}$ byl
zaveden v (6.9).

I při důkazu nerozhodnutelnosti univerzálního jazyka ve větě 6.9 jsme využili převoditelnosti. Při důkazu této věty jsme totiž vlastně ukázali, že $\text{DIAG} \leq_m L_u$. Platí však i opačný směr, tedy $L_u \leq_m \overline{\text{DIAG}}$.

Věta 7.7 Platí, že

- (i) $\overline{\text{DIAG}} \leq_m L_u$ a současně
- (ii) $L_u \leq_m \overline{\text{DIAG}}$.

Důkaz: Připomeňme, že dle (6.9) je $\overline{\text{DIAG}} = \{\langle M \rangle \mid \langle M \rangle \in L(M)\}$.

- (i) Pro každý Turingův stroj M platí, že

$$\langle M \rangle \in \text{DIAG} \Leftrightarrow \langle M \rangle \in L(M) \Leftrightarrow \langle M, \langle M \rangle \rangle \in L_u,$$

kde první ekvivalence platí dle (6.9) a druhá dle definice univerzálního jazyka v (5.4). Funkce f definovaná jako $f(\langle M \rangle) = \langle M, \langle M \rangle \rangle$ je jistě totální algoritmicky vyčíslitelná a ukazuje, že $\overline{\text{DIAG}} \leq_m L_u$.

- (ii) Tento směr je o něco obtížnější, musíme popsat totální algoritmicky vyčíslitelnou funkci f , jež splňuje, že $f(\langle M, x \rangle) = \langle M' \rangle$, kde

$$x \in L(M) \Leftrightarrow \langle M' \rangle \in L(M'). \quad (7.4)$$

Popíšeme, jak bude probíhat výpočet M' nad daným vstupem.

Algoritmus 18: Algoritmus výpočtu Turingova stroje M' nad vstupem y

```

1 Smaž vstup  $y$ 
2 Zapiš na pásku slovo  $x$ 
3 Proveď výpočet Turingova stroje  $M$  nad vstupem  $x$ 
4 if  $M$  přijal then
5   | accept
6 else
7   | reject
```

Všimněme si, že Turingův stroj M' ignoruje zcela svůj vstup y . Místo toho je výpočet $M'(y)$ ekvivalentní výpočtu $M(x)$. Funkce f zkonstruuje kód stroje M' na základě kódu stroje M a řetězce x . Tento postup je celkem přímočarý, jedná se vlastně jen o úpravu přechodové funkce M tak, aby před samotným výpočtem M došlo k přepsání vstupu řetězcem x . Funkce f je tedy jistě algoritmicky vyčíslitelná a také je jistě totální. Všimněme si, že

$$L(M') = \begin{cases} \Sigma^* & \text{pokud } x \in L(M) \\ \emptyset & \text{jinak, tedy pokud } x \notin L(M). \end{cases}$$

Platí tedy, že

$$\begin{aligned} \langle M, x \rangle \in L_u &\Rightarrow x \in L(M) \Rightarrow L(M') = \Sigma^* \Rightarrow \langle M' \rangle \in L(M') \Rightarrow \langle M' \rangle \in \overline{\text{DIAG}} \\ \langle M, x \rangle \notin L_u &\Rightarrow x \notin L(M) \Rightarrow L(M') = \emptyset \Rightarrow \langle M' \rangle \notin L(M') \Rightarrow \langle M' \rangle \notin \overline{\text{DIAG}} \end{aligned}$$

Dohromady dostáváme, že

$$\langle M, x \rangle \in L_u \Leftrightarrow f(\langle M, x \rangle) = \langle M' \rangle \in \overline{\text{DIAG}},$$

a tedy $L_u \leq_m \overline{\text{DIAG}}$. □

Podívejme se nyní na některé základní vlastnosti m -převoditelnosti. Ukážeme si, že jako relace je m -převoditelnost reflexivní a tranzitivní, jde tedy o kvaziuspořádání.

Lemma 7.8 *Relace $\leq_m$ je reflexivní a tranzitivní.*

Důkaz: Reflexivita vyplývá z toho, že funkce identity, tj. funkce $f(x) = x$ je totální algoritmicky vyčíslitelná funkce. Díky tomu platí $A \leq_m A$ pro každý jazyk A .

Pro důkaz tranzitivity uvažme tři jazyky A, B, C a předpokládejme, že $A \leq_m B$ a $B \leq_m C$. Ukážeme, že potom $A \leq_m C$. Je-li $A \leq_m B$ na základě funkce f a $B \leq_m C$ na základě funkce g , pak platí pro každý řetězec $x \in \Sigma^*$

$$x \in A \Leftrightarrow f(x) \in B \Leftrightarrow g(f(x)) \in C.$$

To znamená, že definujeme-li funkci h jako složení funkce g s funkcí f , tedy $h(x) = g(f(x))$ pro každý řetězec x , pak tato funkce je jistě totální algoritmicky vyčíslitelná funkce, neboť f a g jsou obě totální algoritmicky vyčíslitelné funkce. Platí tedy, že $A \leq_m C$ na základě funkce h . □

Nás zajímá převoditelnost zejména jako nástroj pro důkazy nerozhodnutelnosti problémů a jazyků. Intuitivně pokud $A \leq_m B$, znamená to, že jsme schopni rozhodnout jazyk A pomocí B . Tuto intuici formalizujeme v následujícím tvrzení.

Věta 7.9 *Předpokládejme, že A a B jsou jazyky, pro něž platí, že $A \leq_m B$. Potom platí, že*

- (i) *je-li B rozhodnutelný jazyk, potom je rozhodnutelný jazyk i A a*

(ii) je-li B částečně rozhodnutelný jazyk, potom je částečně rozhodnutelný jazyk A .

Důkaz: Označme převodní funkci ukazující, že $A \leq_m B$, jako f a uvažme následující algoritmus:

Algoritmus 19: Algoritmus rozhodující A pomocí B

Vstup: Řetězec x

```

1 Spočti  $y = f(x)$ 
2 if  $y \in B$  then
3   | accept
4 else
5   | reject
```

Vzhledem k tomu, že f je totální algoritmicky vyčíslitelná funkce, první krok lze vždy provést algoritmicky. Proveditelnost algoritmu tedy závisí na kroku 2.

- (i) Pokud je jazyk B rozhodnutelný, existuje Turingův stroj, který se vždy zastaví a přijímá jazyk B . To znamená, že uvedený algoritmus lze implementovat takovým způsobem, aby se vždy zastavil a rozhodoval A . Jazyk A je tedy rozhodnutelný.
- (ii) Pokud je jazyk B částečně rozhodnutelný, existuje Turingův stroj, který přijímá jazyk B . To znamená, že uvedený algoritmus lze implementovat takovým způsobem, aby přijímal A , ale nemusí se zastavit na vstupech x , pro které platí $x \notin A$, a tedy $y = f(x) \notin B$. Jazyk A je tedy částečně rozhodnutelný. $\square$

Algoritmus v důkazu věty 7.9 tedy pracuje dle schématu, jež je naznačeno na obrázku 7.2.

My budeme převoditelnost používat v opačném směru. Uvažme jazyky A a B , pro něž platí, že $A \leq_m B$. Z věty 7.9 vyplývá, že není-li A (částečně) rozhodnutelný jazyk, pak ani B není (částečně) rozhodnutelný. Víme například, že $\overline{\text{DIAG}}$, L_u ani HALT nejsou rozhodnutelné jazyky, byť jde o jazyky částečně rozhodnutelné. To znamená, že pokud pro nějaký jazyk A platí například $L_u \leq_m A$, pak A není rozhodnutelný a pokud o jiném jazyku B platí, že $\text{DIAG} \leq_m B$, pak B není částečně rozhodnutelný (dle věty 6.8 není totiž DIAG částečně rozhodnutelný). To je také způsob, který dále zvolíme pro dokazování toho, že nějaký problém není rozhodnutelný.

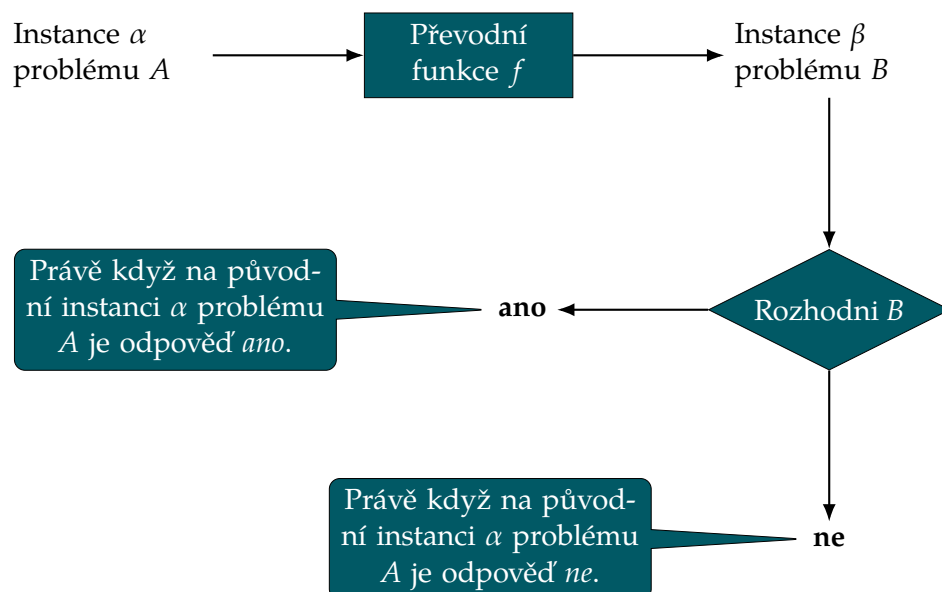
Věta 6.8, věta 6.9
a věta 7.2

Příklad 7.10: Nerozhodnutelnost neprázdnosti jazyka

Připomeňme si jazyk

$$\text{NONEMPTY} = \{\langle M \rangle \mid L(M) \neq \emptyset\}$$

zavedený v příkladu 6.3. Ukážeme, že $L_u \leq_m \text{NONEMPTY}$. Z toho tedy plyne, že NONEMPTY není rozhodnutelný jazyk. Musíme popsat totální částečně rekurzivní funkci $f : \Sigma^* \rightarrow \Sigma^*$, pro kterou platí ekvivalence (7.2), tj. pro každou dvojici



Obrázek 7.2.: Princip převodu problému A na problém B pomocí funkce f .

Turingova stroje M a řetězce x platí, že

$$\langle M, x \rangle \in L_u \Leftrightarrow f(\langle M, x \rangle) \in \text{NONEMPTY}. \quad (7.5)$$

Popíšeme, jak bude probíhat výpočet Turingova stroje $M' = f(\langle M, x \rangle)$ se vstupem $y \in \Sigma^*$.

Algoritmus 20: Výpočet stroje M' se vstupem y

- 1 Vymaž pásku (tj. y)
 - 2 Zapiš na pásku řetězec x
 - 3 Pokračuj výpočtem $M(x)$
// Zastavení, přijetí, či odmítnutí se řídí výsledkem $M(x)$.
-

Přechodová funkce M' se tedy od přechodové instrukce M liší jen tím, že jsou v ní navíc instrukce pro provedení prvních dvou kroků algoritmu, tedy nahrazení vstupu y řetězcem x . Přidání těchto instrukcí je úkolem funkce f , která z dvojice $\langle M, x \rangle$ vytvoří kód $\langle M' \rangle$. Je potřeba si uvědomit, že funkce f sama nesimuluje výpočet $M(x)$, a její výpočet není tedy závislý na tom, zda se výpočet $M(x)$ zastaví. Podívejme se nyní na jazyk přijímaný strojem M' . Všimněme si, že výpočet $M'(y)$ na vstupu y vůbec nezávisí. Pokud je $x \in L(M)$, pak $M'(y)$ přijme každý řetězec y , pokud naopak $x \notin L(M)$, pak $M'(y)$ žádný řetězec y nepřijme. Toto lze shrnout

následujícím výrazem:

$$L(M') = \begin{cases} \Sigma^* & x \in L(M) \\ \emptyset & x \notin L(M) \end{cases}$$

Z toho již snadno dostaneme platnost ekvivalence (7.5), neboť

$$\begin{aligned} \langle M, x \rangle \in L_u &\Rightarrow x \in L(M) \Rightarrow L(M') = \Sigma^* \Rightarrow f(\langle M, x \rangle) = \langle M' \rangle \in \text{NONEMPTY} \\ \langle M, x \rangle \notin L_u &\Rightarrow x \notin L(M) \Rightarrow L(M') = \emptyset \Rightarrow f(\langle M, x \rangle) = \langle M' \rangle \notin \text{NONEMPTY} \end{aligned}$$

Z toho tedy vyplývá, že $L_u \leq_m \text{NONEMPTY}$, a jazyk NONEMPTY je tedy nerozhodnutelný. V příkladu 6.3 jsme viděli, že NONEMPTY je částečně rozhodnutelný jazyk a z Postovy věty tedy plyne, že $\overline{\text{NONEMPTY}}$ není částečně rozhodnutelný jazyk.

Věta 6.5

7.3. Úplné problémy

Jak jsme již zmínili, dle lemmatu 7.8 je m -převoditelnost kvaziuspořádáním (tedy tranzitivní a reflexivní relací). Zřejmě ne všechny jazyky jsou na sebe vzájemně převoditelné (například $\overline{\text{HALT}}$ není převoditelný na HALT). Proto se může zdát i trochu překvapivé, že existují jazyky, na které jsou převoditelné všechny ostatní částečně rozhodnutelné jazyky. Takovým jazykem je například univerzální jazyk L_u nebo problém zastavení HALT . Budeme říkat, že takové problémy jsou m -úplné.

Definice 7.11 (m -úplnost) Řekneme, že jazyk $A \subseteq \Sigma^*$ je m -úplný, pokud je

1. A částečně rozhodnutelný jazyk a
2. pro každý částečně rozhodnutelný jazyk B platí, že $B \leq_m A$. ◀

Řada z nerozhodnutelných problémů, s nimiž jsme se setkali, je m -úplná.

Věta 7.12 Jazyky L_u , HALT a $\overline{\text{DIAG}}$ jsou m -úplné.

Důkaz: Víme již, že tyto jazyky jsou částečně rozhodnutelné díky existenci univerzálního Turingova stroje. Nechť A je libovolný částečně rozhodnutelný jazyk, který je přijímaný Turingovým strojem M_A . Pak platí, že $x \in A$, právě když $\langle M_A, x \rangle \in L_u$. Funkce f definovaná jako $f(x) = \langle M_A, x \rangle$ je jistě algoritmicky vyčíslitelná a ukazuje tedy, že $A \leq_m L_u$. Jazyk L_u je tedy m -úplný. Z věty 7.7 plyne, že $\overline{\text{DIAG}}$ je m -úplný, podobně m -úplnost HALT vyplývá z důsledku 7.6. ◻

Jako další příklad m -úplného problému nám poslouží otázka neprázdnosti jazyka přijímaného daným Turingovým strojem:

Příklad 7.13: Úplnost neprázdnoti jazyka

Vzpomeňme si, že v příkladu 7.10 jsme ukázali, že $L_u \leq_m \text{NONEMPTY}$ a v příkladu 6.3 jsme ukázali, že je jazyk NONEMPTY částečně rozhodnutelný. Dohromady dostáváme, že je tento jazyk m -úplný.

7.4. Riceova věta

V této kapitole zformulujeme a ukážeme Riceovu větu. Tato věta říká, že otázka, zda jazyk přijímaný daným Turingovým strojem (algoritmem) splňuje danou vlastnost, je rozhodnutelná, jen pokud odpověď na tuto otázku je buď triviálně ano (je tedy splněna každým jazykem), nebo triviálně ne (není tedy splněna žádným jazykem). V tomto kontextu můžeme mluvit stejně dobře o jazyku přijímaném Turingovým strojem (algoritmem) jako o funkci vyčíslované Turingovým strojem. Pro úplnost si postupně zformulujeme Riceovu větu jak ve verzi pro jazyky, tak ve verzi pro funkce.

Začneme verzí pro jazyky.

Viz definice Turingovskyy vyčísitelné funkce v definici 4.6

Věta 7.14 (Riceova věta pro jazyky) *Nechť $\mathcal{C}$ je třída částečně rozhodnutelných jazyků. Potom jazyk*

$$L_{\mathcal{C}} = \{ \langle M \rangle \mid L(M) \in \mathcal{C} \}$$

je rozhodnutelný, právě když je třída $\mathcal{C}$ triviální, tj. buď je prázdná, nebo obsahuje všechny částečně rozhodnutelné jazyky.

Důkaz: Budeme předpokládat, že třída $\mathcal{C}$ obsahuje jazyky nad abecedou Σ , kde Σ je současně abeceda použitá pro kódování Turingových strojů a jiných objektů. Je-li třída $\mathcal{C}$ prázdná, pak $L_{\mathcal{C}} = \emptyset$, pokud naopak třída $\mathcal{C}$ obsahuje všechny částečně rozhodnutelné jazyky, pak $L_{\mathcal{C}} = \Sigma^*$. V obou případech je jistě $L_{\mathcal{C}}$ rozhodnutelný jazyk.

Předpokládejme nyní, že $\mathcal{C}$ je netriviální třída částečně rozhodnutelných jazyků, tedy je neprázdná, ale neobsahuje ani všechny částečně rozhodnutelné jazyky. Ukážeme, že v tom případě platí

$$L_u \leq_m L_{\mathcal{C}} \quad \text{nebo} \quad (7.6)$$

$$L_u \leq_m \overline{L_{\mathcal{C}}}. \quad (7.7)$$

Například $\Sigma = \{0, 1\}$ nebo $\Sigma = \Gamma$, použijeme-li kódování popsané v kapitole 5.2.1. Připomeňme si též, že dle konvence 5.4 každý řetězec odpovídá nějakému Turingovu stroji.

V obou případech dostaneme, že $L_{\mathcal{C}}$ není rozhodnutelný jazyk. To, zda ukážeme převod (7.6) nebo (7.7), závisí na tom, kam patří prázdný jazyk. Předpokládejme nejprve, že prázdný jazyk do třídy $\mathcal{C}$ nepatří, tedy $\emptyset \notin \mathcal{C}$. V tom případě ukážeme, že platí (7.6), tedy $L_u \leq_m L_{\mathcal{C}}$. Nechť L je navíc libovolný částečně rozhodnutelný jazyk z třídy $\mathcal{C}$. Speciálně $L \neq \emptyset$. Předpokládejme navíc, že N je Turingův stroj, který přijímá L , tj. $L = L(N)$. Převodní funkce f se vstupem $\langle M, x \rangle$ vrátí kód Turingova stroje M' , který se vstupem y pracuje následujícím způsobem:

Algoritmus 21: Výpočet stroje M' , kde $\langle M' \rangle = f(\langle M, x \rangle)$ se vstupem y

```
1 Pustř  $M(x)$ 
2 if  $M(x)$  odmítl then
3   reject
4 Pustř  $N(y)$ 
5 if  $N(y)$  přijal then
6   accept
7 else
8   reject
```

Kód Turingova stroje M' vznikne složením kódů Turingových strojů M a N a vstupu x . Funkce f je tedy jistě algoritmicky vyčíslitelná a navíc je i definovaná pro každý vstup. Stačí ukázat, že $\langle M, x \rangle \in L_u \Leftrightarrow \langle M' \rangle \in L_C$, tím dostaneme, že $L_u \leq_m L_C$.

Předpokládejme nejprve, že $\langle M, x \rangle \in L_u$, tedy že $x \in L(M)$. V tomto případě výpočet $M(x)$ skončí a přijme, tedy dojde ke spuštění $N(y)$ pro každý vstup y . To znamená, že $L(M') = L(N) = L \in C$.

Předpokládejme nyní, že $\langle M, x \rangle \notin L_u$, tedy $x \notin L(M)$. V tomto případě buď výpočet $M(x)$ vůbec neskončí, nebo nezávisle na vstupu y odmítne. Turingův stroj M' tedy rozhodně žádný vstup y nepřijme, a proto $L(M') = \emptyset$, podle předpokladu tedy $L(M') \notin C$.

Případ, kdy prázdný jazyk patří do třídy C , je symetrický. Stačí zaměnit role C doplňku $\bar{C}$, dostaneme tak převod $L_u \leq_m \bar{L}_C$. $\square$

Podobně můžeme zformulovat Riceovu větu i pro funkce.

Věta 7.15 (Riceova věta pro funkce) *Nechť C je třída algoritmicky vyčíslitelných funkcí. Potom jazyk*

$$L_C = \{ \langle M \rangle \mid f_M \in C \}$$

je rozhodnutelný, právě když je třída C triviální, tj. buď je prázdná, nebo obsahuje všechny algoritmicky vyčíslitelné funkce.

Důkaz: Důkaz je zcela analogický důkazu verze Riceovy věty pro jazyky, tedy věty 7.14. Budeme předpokládat, že funkce v třídě C jsou typu $\Sigma^* \rightarrow \Sigma^*$, kde Σ je současně abeceda použitá pro kódování Turingových strojů a jiných objektů.

Pokud je třída C triviální, pak zřejmě buď $L_C = \emptyset$, nebo $L_C = \Sigma^*$. V obou případech jde o rozhodnutelný jazyk. Nechť g_0 označuje funkci, která není definovaná pro žádný vstup, tedy pro každý řetězec $y \in \Sigma^*$ platí $g_0(y) \uparrow$. Předpokládejme bez újmy na obecnosti, že $g_0 \notin C$ (v opačném případě zaměníme role C a doplňku $\bar{C}$). Ukážeme, že potom $L_u \leq_m L_C$, z čehož plyne, že L_C je nerozhodnutelný jazyk. Nechť g_1 je libovolná funkce, která naopak patří do C . Funkce f , která převádí L_u na L_C se vstupem $\langle M, x \rangle$ vrátí kód Turingova stroje M' , který vyčísluje funkci $f_{M'}$, jež pro vstup $y \in \Sigma^*$ splňuje

$$f_{M'}(y) = \begin{cases} g_1(y) & x \in L(M) \\ \uparrow & x \notin L(M) \end{cases}$$

Uvědomme si, že kód Turingova stroje M' lze zkonstruovat jen se znalostí kódu M , vstupu x a kódu Turingova stroje, který vyčísluje funkci g_1 . Zejména pak platí, že f je algoritmicky vyčíslitelná funkce.

Nechť nyní $\langle M, x \rangle \in L_u$, tedy $x \in L(M)$. Potom $f(\langle M, x \rangle) = \langle M' \rangle$, kde $f_{M'} \simeq g_1 \in \mathcal{C}$. Na druhou stranu pokud $\langle M, x \rangle \notin L_u$, tedy $x \notin L(M)$, pak funkce $f_{M'}$ není definovaná pro žádný vstup $y \in \Sigma^*$, a tedy $f_{M'} \simeq g_0 \notin \mathcal{C}$. Dohromady dostáváme, že $\langle M, x \rangle \in L_u$, právě když $f(\langle M, x \rangle) \in L_C$. Z toho plyne, že $L_u \leq_m L_C$ a L_C tedy není algoritmicky rozhodnutelný jazyk. $\square$

Jak jsme zmínili v úvodu, Riceova věta ukazuje o určitém typu problémů, že nemožno být algoritmicky rozhodnutelné. Jde o problémy, kde vstupem je kód Turingova stroje (nebo obecněji program v jakémkoli programovacím jazyku, který je s Turingovými stroji ekvivalentní) a kde se ptáme, zda funkce nebo jazyk určená tímto Turingovým strojem splňuje danou netriviální vlastnost. Nejde tedy o otázky, odpověď na něž je závislá na konkrétním kódu (programu), ale naopak závisí na funkci daného kódu (programu). Například nerozhodnutelnost diagonálního jazyka $\text{DIAG} = \{\langle M \rangle \mid \langle M \rangle \notin L(M)\}$ zavedeného předpisem (6.3) nevyplývá z Riceovy věty, protože to, zda $\langle M \rangle \in L(M)$ podstatně závisí na konkrétním kódu $\langle M \rangle$. Nerozhodnutelnost řady jiných jazyků však z Riceovy věty plyne.

Důsledek 7.16 *Následující jazyky jsou nerozhodnutelné*

$$\begin{aligned} \text{NONEMPTY} &= \{\langle M \rangle \mid L(M) \neq \emptyset\} \\ \text{ALL} &= \{\langle M \rangle \mid L(M) = \Sigma^*\} \\ \text{Tot} &= \{\langle M \rangle \mid (\forall y \in \Sigma^*)[f_M(y) \downarrow]\} \\ \text{Fin} &= \{\langle M \rangle \mid L(M) \text{ je konečný jazyk}\} \\ \text{Inf} &= \{\langle M \rangle \mid L(M) \text{ je nekonečný jazyk}\} \\ \text{Cof} &= \{\langle M \rangle \mid \overline{L(M)} \text{ je konečný jazyk}\} \\ \text{Regular} &= \{\langle M \rangle \mid L(M) \text{ je regulární jazyk}\} \end{aligned}$$

Důkaz: Plyne přímo z věty 7.14. $\square$

U všech těchto jazyků je vlastnost, která nás u daného Turingova stroje zajímá, vlastností jím vyčíslované funkce či jím přijímaného jazyka, jde tedy o funkční vlastnost. Všimněme si také, že Riceova věta nevyklučuje částečnou rozhodnutelnost, například jazyk NONEMPTY je částečně rozhodnutelný, jak jsme si ukázali v příkladu 6.3.

7.5. Postův korespondenční problém

Problémy, o nichž jsme si dosud ukázali, že jsou nerozhodnutelné, nějakým způsobem souvisely s Turingovými stroji (potažmo algoritmy a programy obecně) a jejich výpočty. V této kapitole si ukážeme nerozhodnutelnost problému, jehož definice se na Turingův stroj nijak neodkazuje a pracuje pouze s řetězci.

Instancí **POSTOVA KORESPONDENČNÍHO PROBLÉMU** je množina dvojic řetězců, které si můžeme představit jako dominové kostky

$$\begin{array}{|c|} \hline t_i \\ \hline b_i \\ \hline \end{array},$$

kde $t_i, b_i \in \Sigma^*$ pro $i = 1, \dots, k$ a pevnou abecedu Σ . Od každého typu je k dispozici neomezené množství kostek, otázkou je, zda je možno vybrat párovací posloupnost kostek. *Párovací posloupností* je zde konečná neprázdná posloupnost $i_1, \dots, i_l \in \{1, \dots, k\}$, pro kterou platí, že srovnáme-li vybrané kostky vedle sebe, nahoře i dole dostaneme týž řetězec. Jinými slovy platí $t_{i_1} t_{i_2} \dots t_{i_l} = b_{i_1} b_{i_2} \dots b_{i_l}$. Formálně definujeme **POSTŮV KORESPONDENČNÍ PROBLÉM** následujícím způsobem.

Problém 7.17: POSTŮV KORESPONDENČNÍ PROBLÉM (PKP)

Instance: Množina „dominových kostek“ P :

$$P = \left\{ \begin{array}{|c|} \hline t_1 \\ \hline b_1 \\ \hline \end{array}, \begin{array}{|c|} \hline t_2 \\ \hline b_2 \\ \hline \end{array}, \dots, \begin{array}{|c|} \hline t_k \\ \hline b_k \\ \hline \end{array} \right\}$$

kde $t_1, \dots, t_k, b_1, \dots, b_k \in \Sigma^*$ jsou řetězce.

Otázka: Existuje *párovací posloupnost* indexů $i_1, i_2, \dots, i_l \in \{1, \dots, k\}$, kde $l \geq 1$ a $t_{i_1} t_{i_2} \dots t_{i_l} = b_{i_1} b_{i_2} \dots b_{i_l}$?

Příklad 7.18

Uvažme například množinu P , která obsahuje kostky

Jel	e	n	ovi	p	ivo	nel	j
J	el	eno	vi	piv	o	n	ej

Z této množiny není těžké vybrat následující párovací posloupnost:

Jel	e	n	ovi	p	ivo	nel	e	j
J	el	eno	vi	piv	o	n	el	ej

Všimněme si, že kostka

$$\begin{array}{|c|} \hline e \\ \hline e1 \\ \hline \end{array}$$

je v této posloupnosti použita dvakrát.

Naším cílem je nyní ukázat, že **POSTŮV KORESPONDENČNÍ PROBLÉM** je algoritmicky neřešitelný. To ukážeme tím, že popíšeme převod z problému **PŘIJETÍ VSTUPU** (čili z univerzálního jazyka L_u) na **PKP**. Budeme postupovat ve dvou krocích, jako mezikrok použijeme trochu zjednodušenou verzi problému **PKP**, kde máme pevně danou první kostkou, kterou musí začínat hledaná párovací posloupnost.

Problém 7.19: MODIFIKOVANÝ POSTŮV KORESPONDENČNÍ PROBLÉM (MPKP)

Instance: Shodná s instancí **POSTOVA KORESPONDENČNÍHO PROBLÉMU**.

Otázka: Existuje párovací posloupnost, která začíná kostkou

$$\begin{array}{|c|} \hline t_1 \\ \hline b_1 \\ \hline \end{array} ?$$

Nejprve ukážeme, že problém **PŘIJETÍ VSTUPU** (univerzální jazyk L_u) je převoditelný na problém **MPKP**, poté ukážeme, jak převést problém **MPKP** na problém **PKP**. Problém 5.9

7.5.1. Převod problému **PŘIJETÍ VSTUPU** na problém **MPKP**

Připomeňme si, že instancí problému **PŘIJETÍ VSTUPU** je dvojice tvořená kódem Turingova stroje $\langle M \rangle$ a jeho vstupu $x \in \Sigma^*$, kde Σ je vstupní abeceda Turingova stroje M . Musíme popsat algoritmus, který převede tuto instanci na instanci problému **MPKP**, tedy množinu dominových kostek

$$P = \left\{ \begin{array}{|c|} \hline t_1 \\ \hline b_1 \\ \hline \end{array}, \begin{array}{|c|} \hline t_2 \\ \hline b_2 \\ \hline \end{array}, \dots, \begin{array}{|c|} \hline t_k \\ \hline b_k \\ \hline \end{array} \right\}$$

pro kterou platí následující ekvivalence:

$$x \in L(M) \Leftrightarrow \text{z } P \text{ lze vybrat párovací posloupnost, jež začíná kostkou } \begin{array}{|c|} \hline t_1 \\ \hline b_1 \\ \hline \end{array} \quad (7.8)$$

Označme si Turingův stroj $M = (Q, \Sigma, \delta, q_0, F)$ a předpokládejme vstup $x = x_1x_2 \dots x_n$ délky n . Zjednodušíme si poněkud situaci tím, že budeme o Turingovu stroji M předpokládat, že má jediný přijímající stav q_1 , tedy $F = \{q_1\}$. Navíc předpokládáme, že přejde-li M do stavu q_1 , jeho výpočet skončí, tedy předpokládáme, že $\delta(q_1, a) = \perp$ pro každý znak $a \in \Sigma$.

Definice 4.2 Podle definice výpočtu Turingova stroje platí, že $x \in L(M)$ právě když existuje posloupnost konfigurací $K_0, \dots, K_t$ Turingova stroje M , kde

- K_0 je počáteční konfigurací Turingova stroje M při výpočtu nad vstupem x ,
- K_t je přijímající konfigurací Turingova stroje M a
- pro každý index $i = 1, \dots, t$ platí, že konfigurace K_i vznikne z konfigurace K_{i-1} aplikací přechodové funkce δ .

Naším cílem bude vytvořit k Turingovu stroji M a vstupu x takovou sadu kostek, pro kterou bude platit, že párovací posloupnost odpovídá právě posloupnosti konfigurací určující přijímající výpočet. Konfiguraci stroje M zapíšeme jako posloupnost symbolů na pásce, přičemž před čtený symbol zapíšeme symbol reprezentující stav, v němž se Turingův stroj nachází. Pro zápis řetězců v instanci **MPKP** tedy budeme používat abecedu, která odpovídá sjednocení Q a Σ . Kromě toho budeme používat symbol $\#$ pro ohrazení zápisu jedné konfigurace. Například řetězec

$\# \lambda a a q_7 b b \#$

popisuje konfiguraci, kde na pásce je zapsáno slovo $aabb$, Turingův stroj se nachází ve stavu q_7 a čte první ze dvou symbolů b . Součástí zápisu je v tomto příkladu i jedno prázdné políčko před prvním políčkem řetězce. Konfigurace vždy zachycuje konečný kus pásky, který je dost velký, aby zahrnul všechny neprázdné znaky, může ovšem zahrnovat i prázdná políčka v okolí.

Myšlenka konstrukce je následující. Spodní řada bude o jednu konfiguraci napřed. První kostka bude dole obsahovat počáteční konfiguraci a nahoře jen oddělovač $\#$. Další kostky pak budou implementovat přechodovou funkci δ , kde horní řetězec bude odpovídat situaci před aplikací přechodové funkce a spodní řetězec situaci po aplikaci přechodové funkce. Po ukončení výpočtu dojde v případě přijetí k zarovnání horní a spodní řady řetězců tak, aby ve výsledku byly řetězce nahoře i dole v párovací posloupnosti shodné. Popíšeme nyní dominové kostky, které budou zkonstruovány pro Turingův stroj $M = (Q, \Sigma, \delta, q_0, F = \{q_1\})$ a vstup $x = x_1 x_2 \dots x_n$. Tyto kostky budou tvořit výslednou množinu P .

- (I) První kostka vynutí to, že první konfigurací v párovací posloupnosti je počáteční konfigurace K_0 pro vstup $x = x_1 x_2 \dots x_n$.

$$\begin{array}{|c|} \hline t_1 \\ \hline b_1 \\ \hline \end{array} = \begin{array}{|c|} \hline \# \\ \hline \# \lambda q_0 x_1 x_2 \dots x_n \# \\ \hline \end{array}$$

Pokud by byl vstup x prázdný, pak vložíme za q_0 znak prázdného políčka. Všimněme si dvou znaků prázdných políček vložených před symbol stavu q_0 . Ty jsou zde proto, aby bylo možno hned v prvním možno pohnout ze stavu q_0 hlavou doleva. Dva jsou proto, aby i po pohybu hlavou doleva v prvním kroku, před hlavou stále ještě jeden znak prázdného políčka zbýval.

- (II) Pro každý znak $a \in \Sigma$ vložíme do P kostku, která provede okopírování znaku do následující konfigurace v místech, která nemají být změněna. Jde o znaky na pásce, které nejsou pod hlavou Turingova stroje.

a
.....
a

- (III) Pro každou dvojici znaků $a, b \in \Sigma$ a dvojici stavů $q, r \in Q$, pro které platí $\delta(q, a) = (r, b, N)$, vložíme do P kostku

qa
.....
rb

- (IV) Pro každou dvojici znaků $a, b \in \Sigma$ a dvojici stavů $q, r \in Q$, pro které platí $\delta(q, a) = (r, b, R)$, vložíme do P kostku

qa
.....
br

- (V) Pro každou trojici znaků $a, b, c \in \Sigma$ a dvojici stavů $q, r \in Q$, pro které platí $\delta(q, a) = (r, b, L)$, vložíme do P kostku

cqa
.....
rcb

- (VI) Dále přidáme do P čtyři kostky, které umožní zakončit konfiguraci.

$\#$	$\#$	$\#$	$\#$
.....			
$\#$	$\#\lambda$	$\lambda\#$	$\lambda\#\lambda$

První kostka pouze zakončuje konfiguraci nahoře i dole. Druhá a třetí kostka umožňují rozšířit konfiguraci o prázdné políčko nalevo nebo napravo. Čtvrtá kostka pak umožňuje přidat prázdné políčko nalevo i napravo.

- (VII) V případě, že Turingův stroj M přejde do přijímajícího stavu q_1 , můžeme použít následujících kostek k postupnému vymazání znaků z pásy. Pro každý znak $a \in \Sigma$

vložíme do P kostky

aq_1	q_1a
q_1	q_1

(VIII) Na úplný závěr je potřeba odstranit i stav q_1 z konfigurace a provést zarovnání znaků $\#$ kostkou

$q_1\#\#$
$\#$

Ukážeme si nyní na příkladu, jakým způsobem odpovídá přijímající výpočet Turingova stroje M nad vstupem x párovací posloupnosti pro množinu P , která se skládá z kostek typů (I) až (VIII).

Příklad 7.20

Uvažme Turingův stroj $M = (Q, \Sigma, \delta, q_0, \{q_1\})$, který jsme popsali v příkladu 4.4. Připomeňme si, že se jednalo o Turingův stroj, který rozhoduje jazyk palindromů $PAL = \{w = w^R \mid w \in \{a, b\}^*\}$. Rozmysleme si, jak by vypadala párovací sekvence v sadě kostek P vytvořené k dvojici Turingova stroje M a vstupu $x = „a“$. První kostkou je nutně kostka typu (I), která obsahuje počáteční konfiguraci výpočtu:

$\#$
$\#\lambda\lambda q_0 a \#$

Vzhledem k tomu, že dle tabulky 4.1 je $\delta(q_0, a) = (q_2, a, R)$, jediná kostka, která

přichází v úvahu pro párování znaku odpovídajícího stavu q_0 , je kostka

q_0a
aq_2

typu (IV). Před jejím použitím je nutné spárovat znaky prázdných políček dvěma

kostkami

λ
λ

typu (II). Potřebujeme dále přidat prázdné políčko na konec kon-

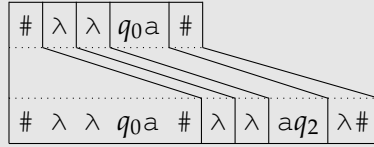
figurace a uzavřít konfiguraci kostkou

$\#$
$\lambda\#$

typu (VI). Volíme zde kostku s při-

dáním prázdného políčka doprava, abychom měli za znakem stavu q_2 ještě čtené

prázdné políčko. Situace nyní vypadá takto:



Potřebujeme tedy zarovnat řetězec $\lambda a q_2 \lambda \#$. Dle přechodové funkce M je $\delta(q_2, \lambda) =$

(q_4, λ, L) . Musíme tedy použít kostku

$a q_2 \lambda$
$q_4 a \lambda$

 typu (V). Tuto kostku doplníme kost-

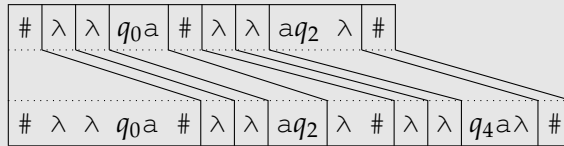
kami

λ
λ

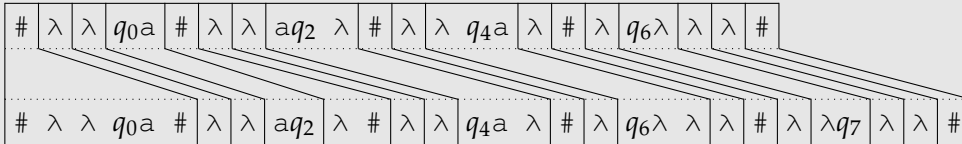
 typu (II) a

$\#$
$\#$

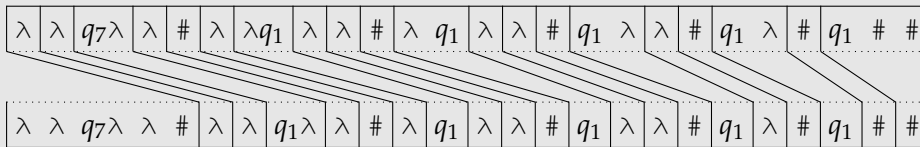
 typu (VI). Dostaneme následující situaci:



Takto pokračujeme dál použitím instrukcí $\delta(q_4, a) = (q_6, \lambda, L)$ a $\delta(q_6, \lambda) = (q_7, \lambda, R)$, s použitím příslušných kostek se dostaneme do následující situace:



Nyní přidáme kostky odpovídající provedení instrukce $\delta(q_7, \lambda) = (q_1, \lambda, N)$. Ve spodní řadě nyní přebývá řetězec $\lambda \lambda q_1 \lambda \lambda \#$. K jeho spárování použijeme čtyři kostky typu (VII) spolu s kostkami typu (II) a (VI). Párovací posloupnost ukončíme kostkou (VIII):



Tím jsme dokončili konstrukci párovací posloupnosti.

Na příkladu 7.20 si můžeme všimnout, že volba kostek, které simulují výpočet je da-

ná přechodovou funkcí, jež nám nedává na výběr. Můžeme sice v konfiguraci přidávat prázdná políčka na okrajích, ale jinak je následující konfigurace v řadě daná přechodovou funkcí. Navíc si můžeme všimnout, že od začátku je řetězec na spodní straně delší, než řetězec na horní straně kostek. Jediné kostky, které mohou zkrátit horní řetězec jsou přitom typu (VII) a (VIII), tedy kostky, které vyžadují přítomnost přijímajícího stavu. Spárování je tedy skutečně možné jen v případě, kdy je výpočet přijímající. Z těchto úvah plyne i korektnost celého převodu. Formální důkaz platnosti ekvivalence (7.8) zde však popisovat nebudeme, protože by byl technicky náročný a nepřinesl by již žádné nové myšlenky.

7.5.2. Převod problému MPKP na problém PKP

Zbývá ukázat, že problém MPKP je m -převoditelný na problém PKP. Instance obou problémů vypadají podobně, avšak v MPKP máme danou první kostku, kterou musí začínat každá párovací posloupnost. Mějme tedy množinu kostek P , musíme popsat, jak zkonstruovat množinu kostek P' , pro kterou by platila následující ekvivalence:

$$\begin{array}{c} \boxed{\begin{array}{c} t_1 \\ \hline b_1 \end{array}} \end{array} \text{ právě } \quad (7.9)$$

Z P lze vybrat párovací posloupnost, jež začíná kostkou

když z P' lze vybrat párovací posloupnost.

Trik je v tom, že vhodnou úpravou kostek v P vynutíme, že každá párovací posloupnost

v P' musí začínat variantou kostky $\boxed{\begin{array}{c} t_1 \\ \hline b_1 \end{array}}$. Abecedu Σ rozšíříme o dva nové znaky $*$ a $\diamond$.

Pro účely konstrukce si zavedeme následující značení. Je-li $u = u_1 u_2 \dots u_n \in \Sigma^*$ řetězec, označíme

$$\begin{aligned} *u &= *u_1 * u_2 * \dots * u_n \\ u* &= u_1 * u_2 * \dots * u_n * \\ *u* &= *u_1 * u_2 * \dots * u_n * . \end{aligned}$$

Nyní definujeme P' na základě P následujícím způsobem.

$$P' = \left\{ \boxed{\begin{array}{c} *t_i \\ \hline b_i* \end{array}} \mid \boxed{\begin{array}{c} t_i \\ \hline b_i \end{array}} \in P \right\} \cup \left\{ \boxed{\begin{array}{c} *t_1 \\ \hline *b_1* \end{array}}, \boxed{\begin{array}{c} *\diamond \\ \hline \diamond \end{array}} \right\} \quad (7.10)$$

Máme-li párovací posloupnost $i_1, i_2, \dots, i_l$ v instanci P , kde $i_1 = 1$, jednoduše z ní vy-

tvoříme posloupnost pro P' — tvoří ji kostky $\boxed{\begin{array}{c} *t_1 \\ \hline *b_1* \end{array}}, \boxed{\begin{array}{c} *t_{i_2} \\ \hline b_{i_2}* \end{array}}, \dots, \boxed{\begin{array}{c} *t_{i_l} \\ \hline b_{i_l}* \end{array}}, \boxed{\begin{array}{c} *\diamond \\ \hline \diamond \end{array}}.$

Na druhou stranu předpokládejme, že $i_1, i_2, \dots, i_l$ je párovací posloupnost v instanci

P' . První kostkou v této posloupnosti musí být $\begin{array}{|c|} \hline *t_1 \\ \hline *b_1* \\ \hline \end{array}$, to je totiž jediná kostka, kde

spodní řetězec začíná hvězdičkou. Přitom horní řetězec vždy začíná hvězdičkou, tedy horní i spodní řetězec v párovací posloupnosti musí hvězdičkou začínat. Naopak po-

sledním znakem je jistě $\begin{array}{|c|} \hline * \diamond \\ \hline \diamond \\ \hline \end{array}$, který dorovná přebývajících hvězdičku na spodní straně.

Není těžké nahlédnout, že odstraníme-li z posloupnosti výskyty $\begin{array}{|c|} \hline * \diamond \\ \hline \diamond \\ \hline \end{array}$ a nahradíme-li ostatní kostky v párovací posloupnosti jejich obrazy v P (tedy odstraníme vložené hvězdičky z obou řetězců v každé kostce), dostaneme párovací posloupnost pro P , kte-

rá začíná kostkou $\begin{array}{|c|} \hline t_1 \\ \hline b_1 \\ \hline \end{array}$.

Dohromady dostáváme, že ekvivalence (7.9) je splněna a tím jsme dokončili převod $\text{PŘIJETÍ VSTUPU} \leq_m \text{MPKP} \leq_m \text{PKP}$. Problém PKP je tedy algoritmicky nerozhodnutelný.

7.6. Jazyky za hranicí částečné rozhodnutelnosti

V této kapitole si ukážeme několik příkladů jazyků, pro které platí, že ani ony ani jejich doplňky nejsou částečně rozhodnutelné.

7.6.1. Ekvivalence programů

Uvažme nejprve problém, v němž se ptáme, zda dva dané Turingovy stroje přijímají týž jazyk.

Problém 7.21: EKVIVALENCE PROGRAMŮ

Instance: Dva Turingovy stroje M_1 a M_2 dané svými kódy $\langle M_1 \rangle$ a $\langle M_2 \rangle$.

Otázka: Platí $L(M_1) = L(M_2)$?

Tento problém můžeme zapsat ve formě jazyka EQ:

$$\text{EQ} = \{ \langle M_1, M_2 \rangle \mid L(M_1) = L(M_2) \} \quad (7.11)$$

Ukážeme si, že EQ ani jeho doplněk $\overline{\text{EQ}}$ nejsou částečně rozhodnutelné jazyky. Rozmysleme si nejprve, že tento jazyk není rozhodnutelný. Uvažme jazyk

$$\text{NONEMPTY} = \{\langle M \rangle \mid L(M) \neq \emptyset\},$$

který jsme zavedli v příkladu 6.3, kde jsme ukázali, že tento jazyk je částečně rozhodnutelný. V příkladu 7.10 jsme dále ukázali, že tento jazyk není rozhodnutelný, což přímo vyplývá i z Riceovy věty, jak jsme již zmínili v důsledku 7.16. Z Postovy věty dále vyplývá, že doplněk tohoto jazyka

$$\text{EMPTY} = \overline{\text{NONEMPTY}} = \{\langle M \rangle \mid L(M) = \emptyset\}$$

není částečně rozhodnutelný. Není těžké nahlédnout, že $\text{EMPTY} \leq_m \text{EQ}$. Převod je velmi jednoduchý: Uvažme Turingův stroj N , pro který platí, že $L(N) = \emptyset$ — může se například jednat o Turingův stroj s přechodovou funkcí, která není definovaná pro žádný displej, takový stroj všechny vstupy odmítne. Potom pro každý Turingův stroj M platí

$$\langle M \rangle \in \text{EMPTY} \Leftrightarrow L(M) = \emptyset \Leftrightarrow L(M) = L(N) \Leftrightarrow \langle M, N \rangle \in \text{EQ}.$$

Z toho plyne, že funkce $f(\langle M \rangle) = \langle M, N \rangle$ je algoritmicky vyčíslitelnou funkcí, jež převádí EMPTY na EQ. Dle bodu (ii) věty 7.9 tedy nejenže EQ není rozhodnutelný jazyk, ale není navíc ani částečně rozhodnutelný.

Podobným způsobem ukážeme, že ani $\overline{\text{EQ}}$ není částečně rozhodnutelný jazyk. Ukážeme, že univerzální jazyk L_u (viz (5.4)) je převoditelný na EQ. Z toho dostaneme, že $\overline{L_u} \leq_m \overline{\text{EQ}}$, a tedy EQ není částečně rozhodnutelný. Připomeňme si, že

Zavedeno v (5.4)

$$L_u = \{\langle M, x \rangle \mid x \in L(M)\}$$

K danému Turingovu stroji M a vstupu x sestrojíme Turingův stroj N , který nad vstupem y pracuje dle následujícího algoritmu.

Algoritmus 22: Výpočet Turingova stroje N nad vstupem y

```

1 if  $y = x$  then
2   | accept
3 else
4   | Pust  $M(y)$ 
5   | if výpočet  $M(y)$  skončil přijetím then
6   |   | accept
7   |   | else
8   |   | reject
```

Jediný vstup, na kterém se může jazyk $L(N)$ lišit od $L(M)$ je x . Přesněji, platí $L(N) = L(M) \cup \{x\}$. Z toho plyne, že $L(N) = L(M)$ právě když $x \in L(M)$. Pokud definujeme funkci $f(\langle M, x \rangle) = \langle M, N \rangle$, pak f je algoritmicky vyčíslitelná funkce, která převádí univerzální jazyk L_u na EQ, a tedy i $\overline{L_u}$ na $\overline{\text{EQ}}$. Jazyk $\overline{\text{EQ}}$ tedy není částečně rozhodnutelný.

7.6.2. Konečnost jazyka

Uvažme nyní problém, v němž se ptáme, zda je jazyk přijímaný daným Turingovým strojem konečný.

Problém 7.22: KONEČNOST JAZYKA

Instance: Turingův stroj M daný svým kódem $\langle M \rangle$

Otázka: Je $L(M)$ konečný jazyk?

Tento problém a jeho doplněk formalizujeme jazyky

$$\text{Fin} = \{\langle M \rangle \mid L(M) \text{ je konečný jazyk}\} \quad (7.12)$$

$$\text{Inf} = \overline{\text{Fin}} = \{\langle M \rangle \mid L(M) \text{ je nekonečný jazyk}\} \quad (7.13)$$

V důsledku 7.16 jsme již nahlédli, že nerozhodnutelnost obou těchto jazyků plyne z Riceovy věty. Definujeme-li třídy jazyků

Věta 7.14

$$\begin{aligned} \mathcal{FIN} &= \{L \mid L \text{ je konečný jazyk}\} \\ \mathcal{INF} = \overline{\mathcal{FIN}} &= \{L \mid L \text{ je nekonečný jazyk}\}, \end{aligned}$$

pak (ve smyslu znění Riceovy věty) $\text{Fin} = L_{\mathcal{FIN}}$ a $\text{Inf} = L_{\mathcal{INF}}$. Vzhledem k tomu, že prázdný jazyk $\emptyset \in \mathcal{FIN}$, převod v důkazu Riceovy věty v sekci 7.4 ukazuje, že $L_u \leq_m \text{Inf}$. Ukážme, že platí i $L_u \leq_m \text{Fin}$, z toho plyne, že ani Inf ani Fin nejsou částečně rozhodnutelné jazyky.

Nechť M je Turingův stroj a x je jeho vstup. Definujme na základě M a x Turingův stroj N , jehož práce nad vstupem y se řídí následujícím algoritmem.

Algoritmus 23: Práce Turingova stroje N nad vstupem y

```

1 Simuluj práci  $M$  nad vstupem  $x$  po  $|y|$  kroků
2 if  $M(x)$  v daném limitu přijal then
3   | reject
4 else
5   | accept
```

Pokud $x \in L(M)$ a délka výpočtu $M(x)$ je t , pak $L(N) = \{y \in \Sigma^* \mid |y| < t\}$, proto $L(N) \in \mathcal{FIN}$ a $\langle N \rangle \in \text{Fin}$. Na druhou stranu pokud $x \notin L(M)$, pak $L(N) = \Sigma^* \in \mathcal{INF}$, a tedy $\langle N \rangle \in \text{Inf}$. Dohromady dostáváme, že

$$\langle M, x \rangle \in L_u \Leftrightarrow x \in L(M) \Leftrightarrow \langle N \rangle \in \text{Fin}.$$

Funkce $f(\langle M, x \rangle) = \langle N \rangle$ je tedy algoritmicky vyčíslitelnou funkcí, jež převádí L_u na Fin .

Část III.

Složitost

8. Základní třídy problémů ve složitosti

V této kapitole zavedeme základní třídy složitosti a podíváme se na jejich základní vlastnosti.

8.1. Problémy a úlohy

Budeme obvykle rozlišovat rozhodovací problém a úlohu.

Rozhodovací problém V rozhodovacím problému se ptáme, zda daný vstup má danou vlastnost. V problému souvislosti grafu se například ptáme, zda daný graf G je souvislý. Na otázku kladenou v definici takového problému tedy očekáváme odpověď typu ano/ne. Vstupu budeme říkat *instance problému*, přičemž *pozitivní instance* jsou ty, pro které je odpověď „ano“, zatímco *negativní instance* jsou ty, pro které je odpověď „ne“. Například instancí problému souvislosti grafu je neorientovaný graf, pozitivními instancemi jsou souvislé grafy a negativními instancemi jsou grafy, jež mají alespoň dvě komponenty. Rozhodovací problém formalizujeme jako jazyk slov, která kódují pozitivních instancí. Například jazyk

$$\text{CONN} = \{\langle G \rangle \mid G \text{ je souvislý graf}\},$$

formalizuje problém souvislosti grafu. Rozhodnutí, zda daný graf G je souvislý je potom ekvivalentní tomu, zda $\langle G \rangle \in \text{CONN}$. Poznamenejme, že instancí problému $w \in \text{CONN}$ je libovolný řetězec $w \in \Sigma^*$, tedy i řetězec, který nekóduje graf. Předpokládáme ovšem, že poznat, zda daný řetězec kóduje graf, je jednoduché. Můžeme si proto dovolit předpokládat, že instancí problému souvislosti grafu formalizovaného jazykem CONN je skutečně graf. Tohoto zjednodušení se budeme dopouštět velmi často.

Úloha V případě úlohy hledáme k danému vstupu x vhodné y , které splňuje danou vlastnost. Například v úloze hledání cesty v orientovaném grafu předpokládáme na vstupu orientovaný graf G a dva vrcholy s a t , tato trojice tedy tvoří *instanci* této úlohy. Očekávaným výstupem je potom seznam vrcholů na cestě z s do t . Pokud žádná taková cesta neexistuje, pak výstupem může být například prázdný seznam vrcholů. Na tomto příkladu vidíme, že výstup y nemusí být určen jednoznačně (cest z s do t může být více) a nemusí ani existovat. Formálně definujeme úlohu jako binární relaci $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$. Úlohou je potom najít k danému řetězci x řetězec y tak, aby $(x, y) \in R$, nebo oznámit, že takový řetězec y neexistuje.

Například formalizací úlohy nalezení cesty v orientovaném grafu je relace

$$\text{PATH} = \{(\langle G, s, t \rangle, \langle y_1, \dots, y_k \rangle) \mid y_1, \dots, y_k \text{ tvoří cestu v orientovaném grafu } G \text{ z } s \text{ do } t\}.$$

Úloze se též říká vyhledávací problém (*search problem*).

Optimalizační úloha je úloha, kde navíc požadujeme, aby nalezený výstup y byl optimální vzhledem k nějaké míře. Může jít například o hledání maximálního toku v síti. Tento typ úloh zavedeme později v kapitole věnující se aproximačním algoritmům.

Většinou se budeme zabývat rozhodovacími problémy, které budeme identifikovat s jazyky (tj. pojmy problém a jazyk budeme používat jako synonyma). V některých místech budeme však též používat úlohy, pro které budeme používat i pojem relace. Pro účely aproximace pak budeme uvažovat i optimalizační úlohy.

8.2. Deterministické třídy složitosti

Nyní již můžeme zavést základní třídy rozhodovacích problémů, které jsou rozhodnutelné v daném čase a prostoru. Pro účel definice použijeme výpočetní model Turingova stroje. Řekněme si nejprve, co znamená, že Turingův stroj M pracuje v omezeném čase nebo prostoru.

Definice 8.1 Nechť $f : \mathbb{N} \rightarrow \mathbb{N}$ je totální funkce a M je Turingův stroj.

- Řekneme, že M pracuje v čase $f(n)$, pokud výpočet M nad libovolným vstupem $x \in \Sigma^*$ délky $|x| = n$ skončí po provedení nejvýš $f(n)$ kroků.
- Řekneme, že M pracuje v prostoru $f(n)$, pokud výpočet M nad libovolným vstupem $x \in \Sigma^*$ délky $|x| = n$ skončí a během výpočtu využije M nejvýše $f(n)$ buněk pracovní pásky. ◀

S pomocí těchto pojmů nyní definujeme dvě základní třídy složitosti.

Definice 8.2 Nechť $f : \mathbb{N} \rightarrow \mathbb{N}$ je totální funkce, pak definujeme následující třídy problémů:

$\text{TIME}(f(n))$ je třídou problémů rozhodnutelných v čase $O(f(n))$. Přesněji, jazyk $L \subseteq \Sigma^*$ patří do třídy $\text{TIME}(f(n))$, právě když existuje Turingův stroj M , který rozhoduje jazyk L a který pracuje v čase $O(f(n))$.

$\text{SPACE}(f(n))$ je třídou problémů rozhodnutelných v prostoru $O(f(n))$. Přesněji, jazyk $L \subseteq \Sigma^*$ patří do třídy $\text{SPACE}(f(n))$, právě když existuje Turingův stroj M , který rozhoduje jazyk L a který pracuje v prostoru $O(f(n))$. ◀

Poznámka 8.3 Poznamenejme, že v literatuře je možné nalézt drobné odlišnosti v definicích tříd $\text{TIME}(f(n))$ a $\text{SPACE}(f(n))$.

- V případě $\text{TIME}(f(n))$ se často vyžaduje, aby Turingův stroj M pracoval v čase $f(n)$, nepovoluje se tedy násobek konstantou jako v naší definici, kde připouštíme čas $O(f(n))$. V případě $\text{SPACE}(f(n))$ je častá podobná úprava definice. Je potřeba zmínit, že v případě Turingova stroje toto není podstatný rozdíl, neboť za určitých ne příliš omezujících předpokladů je možné každý Turingův stroj M zrychlit konstantním faktorem, či provést kompresi prostoru na pásce konstantním faktorem.¹ Je potřeba ovšem zdůraznit, že tato možnost je specifická pro Turingův stroj, kde připouštíme libovolně velkou abecedu. Je proto jednodušší přímo povolit čas nebo prostor $O(f(n))$ v definici tříd $\text{TIME}(f(n))$ a $\text{SPACE}(f(n))$.
- V případě třídy $\text{SPACE}(f(n))$ se často nevyžaduje zastavení Turingova stroje M pro všechny vstupy. Opět platí, že pokud funkce $f(n)$ je v jistém smyslu hezká (přesněji je prostorově konstruovatelná, což je pojem, který budeme definovat později), pak je možné detekovat zacyklení M nad daným vstupem bez nárůstu požadavků na prostor. Dává proto smysl přímo předpokládat, že Turingův stroj M se zastaví nad každým vstupem. Těto vlastnosti budeme dále využívat, protože zjednodušuje argumentaci o daném Turingovu stroji.

Zavedme si některé podstatné třídy problémů:

$$P = \bigcup_{k=0}^{\infty} \text{TIME}(n^k) \quad (8.1)$$

$$\text{EXPTIME} = \bigcup_{k=0}^{\infty} \text{TIME}(2^{n^k}) \quad (8.2)$$

$$\text{PSPACE} = \bigcup_{k=0}^{\infty} \text{SPACE}(n^k) \quad (8.3)$$

$$L = \text{SPACE}(\log_2 n) \quad (8.4)$$

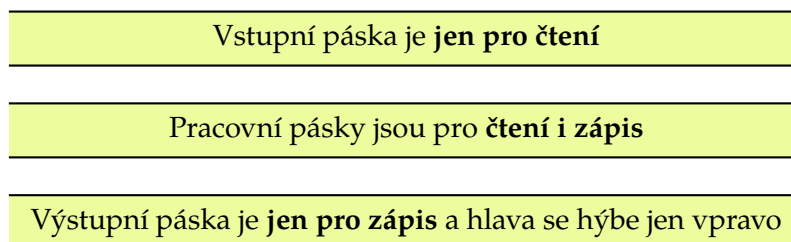
Poznámka 8.4 Pověsimněme si třídu L , kde omezení na využitý prostor je menší než samotná velikost vstupu. Abychom mohli vůbec měřit prostor menší než lineární, je třeba oddělit vstupní a pracovní pásku. V případech, kdy Turingův stroj počítá funkci, je navíc vhodné uvažovat i oddělenou výstupní pásku. Obecně tedy uvažujeme k -páskový Turingův stroj $M = (Q, \Sigma, \delta, q_0, F)$, kde

- První páska je vstupní, tedy je na ní na začátku napsán vstup a Turingův stroj na této pásce nepřepisuje symboly. Hlavou však může pohybovat libovolně. Tato páska se nepočítá do využitého prostoru.
- Pásky 2 až $k - 1$ jsou pracovní. Na začátku výpočtu jsou prázdné a Turingův stroj je může používat libovolným způsobem. Čtení, zápis a pohyb po této pásce nejsou nijak omezeny.

¹https://en.wikipedia.org/wiki/Linear_speedup_theorem

- *k-tá páska je výstupní, sem Turingův stroj zapisuje svůj výstup. Po této pásce může Turingův stroj pohybovat hlavou jen doprava a může na ni jen zapisovat, číst z ní nemůže. Obsah této pásky se nepočítá do využitého prostoru.*

Do prostoru počítáme jen počet buněk, které Turingův stroj využije na pracovních páskách, tedy počítáme prostor využití navíc ke vstupu a výstupu. To odpovídá představě programu, který čte svůj vstup ze standardního vstupu, výstup zapisuje na standardní výstup. Do prostoru počítáme jen to, kolik paměti využije tento program, nikoli to, kolik znaků musí přečíst ze vstupu nebo zapsat na výstup.



Obrázek 8.1.: Struktura pásek Turingova stroje, u nějž měříme prostor jen na pracovních páskách. Naznačen je případ s jednou pracovní páskou.

8.3. Polynomiálně rozhodnutelné problémy

Teze 5.1

Zastavme se chvíli u třídy P , kterou jsme zavedli v (8.1). Rádi bychom řekli, že jde o třídu rozhodovacích problémů, pro které existuje polynomiální algoritmus, který daný problém rozhoduje. Ovšem definice říká, že jde o třídu problémů rozhodnutelných nějakým deterministickým Turingovým strojem v polynomiálním čase. Přijmeme-li Churchovu-Turingovu tezi, pak pojmy „problém rozhodnutelný Turingovým strojem“ a „algoritmicky rozhodnutelný problém“ splývají. To ale ještě nutně neznamená, že splynou i pojmy „problém rozhodnutelný Turingovým strojem v polynomiálním čase“ a „problém rozhodnutelný nějakým algoritmem v polynomiálním čase“. Na druhou stranu v sekci 4.2 jsme zavedli výpočetní model RAM a ukázali jsme si, že tento je ekvivalentní s Turingovým strojem v tom smyslu, že problémy rozhodnutelné Turingovými stroji jsou právě ty, které jsou rozhodnutelné na RAMu. Navíc projdeme-li důkaz této ekvivalence popsany v sekci 4.3, zjistíme, že jsme ukázali ve skutečnosti něco silnějšího. K Turingovu stroji M můžeme sestrojit RAM R , který přijímá též jazyk a pracuje až na konstantní faktor stejně rychle jako M . Naopak k RAMu R můžeme sestrojit Turingův stroj M , který přijímá též jazyk a navíc pracuje jen s polynomiálním zpomalením oproti R^2 . To znamená, že pokud bychom třídu P zavedli pomocí RAMu místo Turingových strojů, dostali bychom touž třídu jazyků.

²Je ovšem pravda, že jsme nijak formálně nezavedli měření času na RAMu, zde prozatím ponecháváme prostor čtenářově intuici.

Při zavádění třídy P tedy vycházíme z předpokladu, který autoři [11] formulují jako tezi o invarianci (*Invariance Thesis*).

Teze 8.5: Teze o invarianci (1984)

Existuje standardní třída výpočetních modelů, která mimo jiné obsahuje všechny varianty Turingových strojů, všechny varianty strojů RAM a RASP, kde počítáme aritmetické operace s logaritmickou cenou, stejně jako všechny varianty RAM a RASP, kde je cena všech instrukcí shodná a využíváme jen standardní aritmetické instrukce (sčítání, odčítání, přičtení jedničky, odečtení jedničky a test na nulu). Stroje v této standardní třídě mohou simulovat sebe navzájem s polynomiálním zpomalením a s konstantním nárůstem prostoru.

Teze 8.5 tedy říká, že za rozumných předpokladů na uvažované výpočetní modely můžeme brát definici třídy P jako nezávislou na použitém výpočetním modelu. Dává proto smysl hovořit o polynomiálním algoritmu bez bližší specifikace modelu.

S definicí třídy P souvisí i další teze, pojmenovaná po Alanu Cobhamovi a Jacku Edmondsovi.

Teze 8.6: Cobhamova-Edmondsova teze (1965)

Třída P odpovídá třídě problémů, které lze prakticky řešit na počítači.

Tato teze má však své limity, pokud složitost algoritmu bude například n^{500} , nelze o něm říci, že by byl prakticky použitelný, byť jde o polynomiální algoritmus. Na druhou stranu je třeba zmínit, že obvyklé praktické problémy, které jsou řešitelné v polynomiálním čase, mají složitost danou polynomem s malým stupněm, takže pokud už problém do třídy P patří, obvykle je i prakticky řešitelný.

8.4. Polynomiálně ověřitelné problémy

Bohužel zdaleka ne všechny problémy jsou rozhodnutelné v polynomiálním čase a nelze to říci ani o problémech praktických. O řadě problémů, které jsou celkem přirozené, však můžeme říci, že jsou polynomiálně ověřitelné. Uvažme následující problém.

Problém 8.7: PROBLÉM OBCHODNÍHO CESTUJÍCÍHO

Instance: Je dáno n měst $c_1, \dots, c_n$, pro každou dvojici měst c_i, c_j , $i \neq j$, $i, j \in \{1, \dots, n\}$ je určena její vzdálenost $d(c_i, c_j) \in \mathbb{N}$ a je zadáno číslo $D \in \mathbb{N}$.

Otázka: Existuje pořadí měst, v němž je má obchodní cestující projet, aby najetá vzdálenost byla nejvýš D ? Přesněji, existuje permutace $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$, pro kterou platí, že

$$\sum_{i=1, \dots, n-1} d(c_{\pi(i)}, c_{\pi(i+1)}) + d(c_{\pi(n)}, c_{\pi(1)}) \leq D?$$

PROBLÉM OBCHODNÍHO CESTUJÍCÍHO je přirozeně optimalizační problém, protože ve skutečnosti je přirozené ptát se po co nejkratší cestě přes všechna města, uvedená verze je rozhodovací verzí a vypadá o něco jednodušeji, stačí jen najít pořadí měst, při kterém obchodní cestující najezdí nejvýš vzdálenost D . Nejen, že neumíme tento problém rozhodnout v polynomiálním čase, ale dokonce se řada vědců domnívá, že to ani nelze (byť ani to zatím neumíme ukázat). Na druhou stranu, pokud dostane obchodní cestující kandidáta na řešení, tedy dodá-li mu někdo konkrétní pořadí měst, není pro něj problém ověřit, zda dané pořadí splňuje požadovanou podmínku. K tomu stačí posčítat vzdálenosti hran v daném pořadí a porovnat součet s hodnotou D . Jde tedy o polynomiálně ověřitelný problém. Formálně definujeme tyto problémy s využitím pojmu polynomiálního verifikátoru.

Definice 8.8 Řekneme, že rozhodovací problém (čili jazyk) $A \subseteq \Sigma^*$ je *polynomiálně ověřitelný*, pokud existuje algoritmus $V(x, y)$, který pracuje v polynomiálním čase vzhledem k $|x|$ a pro který platí, že

$$A = \{x \mid (\exists y \in \Sigma^*)[V(x, y) \text{ přijme}]\}.$$

Algoritmu V budeme říkat *polynomiální verifikátor* (*polynomial verifier*). Řetězci y , pro který platí, že $V(x, y)$ přijme, říkáme *certifikát* pro řetězec x . ◀

Povšimněme si několika vlastností pojmů definovaných v definici 8.8.

- Vstup polynomiálního verifikátoru V je tvořen dvojicí řetězců x a y , přitom ale časovou složitost V měříme jen vzhledem k délce řetězce x , nikoli také vzhledem k délce řetězce y . Toto je trochu neobvyklé, protože obvykle měříme složitost algoritmu vzhledem k délce celého vstupu.
- Certifikát y pro řetězec x dosvědčuje, že $x \in A$. Uvážíme-li, že algoritmus $V(x, y)$ pracuje v čase polynomiálním vzhledem k $|x|$, nutně musí platit, že počet znaků y , které stihne při své práci $V(x, y)$ přečíst, je opět jen polynomiální v $|x|$. To znamená, že pro účely certifikátu nemá smysl uvažovat delší než polynomiálně dlouhé certifikáty. Hovoříme také o *polynomiálním certifikátu*.

- Konečně si povšimněme podobnosti s částečně rozhodnutelnými jazyky, zvláště pak s bodem (iv) věty 6.1. Podle tohoto bodu můžeme říci, že jazyk A je částečně rozhodnutelný, právě když existuje rozhodnutelný jazyk B , pro který platí, že

$$A = \{x \mid (\exists y \in \Sigma^*)[\langle x, y \rangle \in B]\}.$$

Můžeme tedy říci, že algoritmus rozhodující B zde hraje roli verifikátoru, který však nemusí pracovat v polynomiálním čase a řetězec y zde pak hraje roli certifikátu, který dokazuje, že řetězec x patří do jazyka A .

Nyní již můžeme definovat třídu NP.

Definice 8.9 (Třída NP) NP definujeme jako třídu polynomiálně ověřitelných jazyků. ◀

Třidu NP lze definovat i jiným způsobem, jako třídu jazyků, jež jsou přijímány nějakým nedeterministickým Turingovým strojem v polynomiálním čase. Odtud též pochází zkratka NP, tedy *nondeterministically polynomial*. Tuto definici si ukážeme v sekci 8.5.

8.5. Nedeterministické třídy složitosti

V sekci 4 jsme zavedli pojem nedeterministického Turingova stroje (NTS). Ve větě 4.14 jsme si ukázali, že každý nedeterministický Turingův stroj M lze převést na jednopáskový deterministický Turingův stroj M' , který přijímá též jazyk jako M . Podíváme se nyní na nedeterministický Turingův stroj z pohledu složitosti. Na rozdíl od deterministického Turingova stroje, přechodová funkce nedeterministického Turingova stroje nabízí pro každý displej hned množinu možných přechodů. Výpočet NTS $M = (Q, \Sigma, \delta, q_0, F)$ jsme definovali jako posloupnost konfigurací, která začíná v počáteční konfiguraci a každá další v posloupnosti pak odpovídá nějakému přechodu dle přechodové funkce. Výpočet je přijímající, pokud končí přijímající konfigurací (tj. konfigurací v níž je M v přijímajícím stavu) a navíc z poslední konfigurace není již možný žádný další přechod (tj. množina přechodů daných přechodovou funkcí δ je prázdná, výpočet v této konfiguraci končí). Vstup $x \in \Sigma^*$ je přijat strojem M , pokud existuje přijímající výpočet počínající v konfiguraci, kde na pásce je zapsán vstup x .

Připomenuvše si základní pojmy, můžeme přistoupit k definici základních nedeterministických tříd složitosti. Začneme popisem toho, co znamená když řekneme, že nedeterministický Turingův stroj M pracuje v omezeném čase nebo prostoru.

Definice 8.10 Nechť $f : \mathbb{N} \rightarrow \mathbb{N}$ je totální funkce a M je nedeterministický Turingův stroj.

- Řekneme, že M pracuje v čase $f(n)$, pokud každý výpočet M nad libovolným vstupem $x \in \Sigma^*$ délky $|x| = n$ skončí po provedení nejvýš $f(n)$ kroků.
- Řekneme, že M pracuje v prostoru $f(n)$, pokud každý výpočet M nad libovolným vstupem $x \in \Sigma^*$ délky $|x| = n$ skončí a během výpočtu využije nejvýše $f(n)$ buněk pracovní pásky. ◀

S pomocí těchto pojmů nyní definujeme dvě základní nedeterministické třídy složitosti.

Definice 8.11 Necht $f : \mathbb{N} \rightarrow \mathbb{N}$ je totální funkce, pak definujeme následující třídy problémů:

$\text{NTIME}(f(n))$ je třídou problémů přijímaných nedeterministickými Turingovými stroji v čase $O(f(n))$. Přesněji, jazyk $L \subseteq \Sigma^*$ patří do třídy $\text{NTIME}(f(n))$, právě když existuje nedeterministický Turingův stroj M , který přijímá jazyk L a který pracuje v čase $O(f(n))$.

$\text{NSPACE}(f(n))$ je třídou problémů přijímaných nedeterministickými Turingovými stroji v prostoru $O(f(n))$. Přesněji, jazyk $L \subseteq \Sigma^*$ patří do třídy $\text{NSPACE}(f(n))$, právě když existuje nedeterministický Turingův stroj M , který přijímá jazyk L a který pracuje v prostoru $O(f(n))$. ◀

Poznámka 8.12 Poznamenejme, že v literatuře se lze setkat i s definicemi, které se od té naší mírně liší. Často se podmínka počtu kroků a využitého prostoru vztahuje jen na přijímající výpočty. O těch nepřijímajících se potom nehovoří. Což dává dobrý smysl, neboť je potřeba si uvědomit, že v případě NTS nás zajímají pouze přijímající výpočty, nepřijímající nás nezajímají. Nám se bude hodit, že omezení kladená na čas nebo prostor se vztahují i na nepřijímající výpočty, jde však pouze o technický předpoklad, který může v některých místech naše úvahy zjednodušit.

Nedeterminismus nám dovoluje zachytit existenční kvantifikaci. Třidu NP lze alternativně definovat následujícím způsobem.

Věta 8.13 Platí, že

$$\text{NP} = \bigcup_{k=0}^{\infty} \text{NTIME}(n^k). \quad (8.5)$$

Důkaz: Uvažme nejprve jazyk $L \in \text{NP}$. Dle definice 8.9 existuje polynomiální verifikátor $V(x, y)$ pro jazyk L . Popišme nyní nedeterministický Turingův stroj M , který přijímá L v polynomiálním čase.

Algoritmus 24: Výpočet M se vstupem x

```

1 Simuluj verifikátor  $V$ 
2 Nedeterministicky zvol znaky  $y$ , když je  $V$  potřebuje přechíst
3 if  $V$  přijme then
4   | accept
5 else
6   | reject
```

Stroj M tedy pracuje stejně jako verifikátor V jen s tím rozdílem, že v okamžiku, kdy by V chtěl přechíst další znak certifikátu y , M zvolí tento znak nedeterministicky, tedy svůj výpočet rozvětví do dvou větví, v jedné je dalším znakem 0 a v druhé je dalším znakem 1. Protože V pracuje v polynomiálním čase, platí to i pro stroj M . Zřejmě také

platí, že certifikáty y si vzájemně odpovídají s výpočty stroje M . Tedy, $M(x)$ přijme, právě když existuje přijímající výpočet M nad x , což je právě když $V(x, y)$ přijme pro nějaký certifikát y .

Uvažme naopak jazyk $L \in \bigcup_{k=0}^{\infty} \text{NTIME}(n^k)$. Jazyk L je přijímán nějakým NTS M , který pracuje v polynomiálním čase $p(|x|)$. Pro daný řetězec x platí, že je přijat strojem M , právě když existuje přijímající výpočet $M(x)$. Jde o posloupnost konfigurací $K_0^x, \dots, K_t^x$, kde $t \leq p(|x|)$ a k zápisu každé konfigurace stačí řetězec délky $p(|x|)$. Jako certifikát y dosvědčující, že $x \in L$ můžeme tedy použít přímo tento výpočet. Verifikátor $V(x, y)$ pro jazyk L pak bude jen ověřovat, jestli y kóduje přijímající výpočet $M(x)$ délky nejvýš $p(|x|)$. Toto ověření již lze provést deterministicky v polynomiálním čase vzhledem k $|x|$. $\square$

Na závěr této sekce zavedme dvě další nedeterministické prostorové třídy.

$$\text{NPSPACE} = \bigcup_{k=0}^{\infty} \text{NSPACE}(n^k) \quad (8.6)$$

$$\text{NL} = \text{NSPACE}(\log_2 n) \quad (8.7)$$

9. Vztahy mezi třídami složitosti

V této kapitole si ukážeme některé známé vztahy mezi třídami složitosti, jež jsme zavedli v kapitole 8. Začneme těmi nejzákladnějšími vztahy, dále si ukážeme Savičovu větu a věty o deterministické časové a prostorové hierarchii.

9.1. Vztahy mezi třídami

V této sekci si ukážeme některá základní tvrzení o vztahu mezi třídami složitosti, které jsme zavedli v předchozích kapitolách. Začneme těmi nejzákladnějšími vztahy, které povětšinou plynou z definic.

Věta 9.1 *Nechť $f : \mathbb{N} \rightarrow \mathbb{N}$ je totální funkce, pak*

$$\text{TIME}(f(n)) \subseteq \text{NTIME}(f(n)) \subseteq \text{SPACE}(f(n)) \subseteq \text{NSPACE}(f(n)).$$

Důkaz: První a třetí inkluze vyplývají triviálně z definic, neboť deterministický Turingův stroj je zvláštním případem stroje nedeterministického.

Zbývá ukázat prostřední inkluzi, tedy $\text{NTIME}(f(n)) \subseteq \text{SPACE}(f(n))$. Nechť $L \in \text{NTIME}(f(n))$ a M je nedeterministický Turingův stroj, který pracuje v čase $O(f(n))$ a $L(M) = L$. Popíšeme deterministický Turingův stroj M' , který přijímá jazyk L (tedy $L(M') = L$) a který pracuje v prostoru $O(f(n))$.

Pro důkaz bychom mohli zkusit použít průchod do šířky stromem výpočtu stejně jako ve větě 4.14. Ovšem velikost fronty by v tom případě mohla být exponenciální v $f(n)$, protože $f(n)$ omezuje hloubku tohoto stromu. Druhý nápad je použít průchod do hloubky (DFS) stromem výpočtu. To už vypadá lépe, protože DFS si musí v zásobníku pamatovat jen aktuální větev stromu výpočtu, přičemž délka každé větve je $O(f(n))$. Stačí si tedy rozmyslet, že na zásobníku si za každou konfiguraci na cestě dané aktuální větví stačí pamatovat jen informaci konstantní velikosti. Jednotlivý přechod změny konfiguraci jen málo — změni stav, jedno políčko na pásce a případně posune hlavu doleva nebo doprava. Při přechodu do další konfigurace si tedy stačí pamatovat jen předchozí stav, předchozí znak pod hlavou a také symbol R , N nebo L , který určuje, kterým směrem (pokud vůbec) se pohnula hlava. To nám vždy umožní vrátit se zpět k předchozí konfiguraci. Takto tedy můžeme projít celý strom výpočtu $M(x)$. Prostor potřebný strojem M' je $O(f(n))$. Potřebujeme $O(f(n))$ prostoru pro zásobník a $O(f(n))$ prostoru pro uložení aktuální konfigurace. Je potřeba si uvědomit, že konstanta schovaná v O notaci závisí na stroji M , protože velikost informace uložené na zásobník i počet bitů na uložení stavu a jednotlivých políček pásky v aktuální konfiguraci závisí na počtu stavů a velikosti abecedy. Z hlediska M' ale jde o konstantu, protože nezávisí na velikosti vstupu.

Pokud při průchodu M' narazí na přijímající konfiguraci, přijme, v opačném případě po průchodu celým stromem výpočtu odmítne. $\square$

Věta 9.1 ukazuje, že můžeme přejít od nedeterministického času k deterministickému prostoru. Uvažme nyní NTS M , který pracuje v prostoru $f(n)$. Budeme chtít zkonstruovat DTS M' , který simuluje M . Při simulaci nad vstupem x nemůžeme vyloučit situaci, kdy je nutné projít všechny konfigurace, do kterých se může M při výpočtu nad x dostat. Odhadněme tedy nejprve, kolik takových konfigurací může být.

Lemma 9.2 *Nechť M je nedeterministický TS, který pracuje v prostoru $f(n)$, kde $f(n) \geq \log_2 n$. Potom existuje konstanta c_M , pro kterou platí, že pro libovolný vstup x délky n je počet různých konfigurací M v nichž se může NTS M nacházet při výpočtu nad vstupem x nejvýš $2^{c_M f(n)}$.*

Důkaz: Nechť $M = (Q, \Sigma, \delta, q_0, F)$. Budeme předpokládat, že M má strukturu popsanou v poznámce 8.4 tak, abychom mohli připustit prostor menší než lineární. M má tedy tři pásy — vstupní jen pro čtení, pracovní pásku pro čtení i zápis a výstupní pásku jen pro zápis, na níž se hlava pohybuje jen doprava. Mohli bychom uvažovat i obecnější případ, kdy M by měl k pracovních pásek, ale je to zbytečné. Redukce počtu pásek popsaná ve větě 4.10 zvětší prostor jen na násobek daný konstantou k , což bychom mohli zahrnout do konstanty c_M .

Konfigurace se skládá ze slova na pásce, poloh hlavy na vstupní a pracovní pásce a stavu, v němž se stroj M nachází. Délka vstupu je n a délka slova na pásce je nejvýš $f(n)$. Počet různých poloh hlavy v rámci vstupu je n (uvažujeme-li pro jednoduchost, že první a poslední znak vstupu je označen nějakou značkou). Počet různých poloh hlavy na pracovní pásce je nejvýš $f(n)$ a počet stavů je $|Q|$. Označíme-li C počet různých konfigurací, v nichž se M může nacházet při výpočtu nad vstupem x délky n , pak dostaneme

$$\begin{aligned} C &\leq |\Sigma|^{f(n)} \cdot n \cdot f(n) \cdot |Q| = 2^{f(n) \cdot \log_2 |\Sigma|} \cdot 2^{\log_2 n} \cdot 2^{\log_2 f(n)} \cdot 2^{\log_2 |Q|} = \\ &= 2^{f(n) \log_2 |\Sigma| + \log_2 n + \log_2 f(n) + \log_2 |Q|} \leq 2^{f(n) \log_2 |\Sigma| + f(n) + f(n) + \log_2 |Q|} \leq \\ &\leq 2^{f(n) \cdot (\log_2 |\Sigma| + 2 + \log_2 |Q|)}, \end{aligned}$$

kde druhá nerovnost platí díky tomu, že $f(n) \geq \log_2 n$ a $f(n) \geq \log_2 f(n)$, poslední nerovnost pak platí díky tomu, že $f(n) \geq \log_2 n \geq 1$ pro $n \geq 2$. Stačí tedy zvolit hodnotu konstanty $c_M = \log_2 |\Sigma| + \log_2 |Q| + 2$. $\square$

Uvážíme-li tedy Turingův stroj M , který pracuje v prostoru $f(n) \geq \log_2 n$ a vstup x délky n , pak provede-li v nějakém výpočtu M více než $2^{c_M f(n)}$ kroků, nutně se musejí zopakovat konfigurace. Je-li M deterministický, pak to znamená, že $M(x)$ se zacyklí. I v případě, kdy M je nedeterministický TS, nemá smysl uvažovat situaci, kdy se ve výpočtu zopakuje konfigurace (celou část výpočtu mezi dvěma výskyty téže konfigurace lze jistě zapomenout). Dá se tedy předpokládat, že M by mělo být lze nahradit deterministickým strojem M' , který pracuje v čase $2^{c \cdot f(n)}$ pro nějakou konstantu c , která závisí na M . V kapitole 4 jsme zavedli pojem stromu výpočtu. Deterministický stroj M' by tedy mohl procházet stromem výpočtu M nad vstupem x do hloubky nebo do šířky. To by

Definice 4.12

skutečně šlo provést, stačilo by vždy průchod větve ukončit v konfiguraci, která již byla zpracována. My však stroj M' popíšeme s pomocí grafu výpočtu. Graf výpočtu M nad vstupem x se od stromu výpočtu liší tím, že vrcholy odpovídající téže konfiguraci sloučíme do jednoho vrcholu. Tento pojem se nám bude hodit i dále při důkazu Savičovy věty.

Definice 9.3 (Graf konfigurací nedeterministického Turingova stroje) Nechť $M = (Q, \Sigma, \delta, q_0, F)$ je nedeterministický Turingův stroj. *Graf konfigurací* $G_{M,x}$ stroje M nad vstupem x (též *konfigurační graf*) definujeme jako orientovaný graf, jehož vrcholy $G_{M,x}$ jsou tvořeny možnými konfiguracemi výpočtu M nad vstupem x . Jsou-li K_1 a K_2 dvě konfigurace a tedy vrcholy $G_{M,x}$, pak (K_1, K_2) je hranou $G_{M,x}$, právě když z K_1 lze do K_2 přejít s pomocí přechodové funkce δ . ◀

Předpokládejme, že nedeterministický TS M pracuje v prostoru $f(n)$, kde $f(n) \geq \log_2 n$. Z lemmatu 9.2 plyne, že je-li $G_{M,x} = (V, E)$ grafem výpočtu nedeterministického stroje M nad vstupem x délky n , pak $|V| \leq 2^{c_M f(n)}$ a $|E| \leq 2^{2c_M f(n)}$ pro nějakou konstantu c_M , která závisí na Turingovu stroji M , ale nikoli na vstupu x . Výpočet stroje M nad vstupem x je posloupností konfigurací, která odpovídá tahu v grafu $G_{M,x}$, který začíná v počáteční konfiguraci. Pokud se ve výpočtu neopakuje konfigurace, pak tento odpovídá cestě v grafu $G_{M,x}$. Můžeme tedy říci, že $M(x)$ přijme, právě když v $G_{M,x}$ existuje cesta z počáteční konfigurace $M(x)$ do nějaké přijímající konfigurace. S tímto pozorováním již není těžké popsat deterministický stroj M' , který bude simulovat M . Toho využijeme k důkazu následujícího tvrzení.

Věta 9.4 *Nechť $f(n)$ je funkce, pro kterou platí $f(n) \geq \log_2 n$. Pro každý jazyk L platí, že*

$$L \in \text{NSPACE}(f(n)) \Rightarrow (\exists c_L \in \mathbb{N}) \left[L \in \text{TIME}(2^{c_L f(n)}) \right].$$

Důkaz: Uvažme $L \in \text{NSPACE}(f(n))$, potom existuje NTS $M = (Q, \Sigma, \delta, q_0, F)$, který přijímá L , tedy $L(M) = L$, a který pracuje v prostoru $O(f(n))$. Uvažme deterministický TS M' , který průchodem grafu $G_{M,x} = (V, E)$ (do hloubky nebo do šířky) hledá cestu z počáteční konfigurace K_0^x se slovem x na vstupní pásce do nějaké přijímající konfigurace. Pokud takovou cestu najde, $M'(x)$ přijme, jinak odmítne. Platí, že $L(M') = L(M) = L$, navíc M' pracuje v polynomiálním čase vzhledem k $|V| \leq 2^{c_M f(n)}$ (dle lemmatu 9.2), tedy v čase $2^{c_L f(n)}$ pro nějakou konstantu c_L , která závisí na stroji M a implementaci průchodu do hloubky nebo do šířky.

Zbývá rozmyslet jeden technický detail. Stroj M' nezná funkci $f(n)$ a nemůže si spočítat její hodnotu, protože o funkci $f(n)$ nepředpokládáme, že je algoritmicky vyčíslitelná. To znamená, že M' nemůže nejprve zkonstruovat graf $G_{M,x}$, neboť neví, jak dlouhý může být řetězec kódující konfiguraci. Pro danou konfiguraci K je ovšem dle přechodové funkce δ dáno, které konfigurace jsou sousední, dokonce těch sousedních konfigurací je jen konstantní počet. Stroj M' si tedy může graf $G_{M,x}$ generovat za běhu. Začne v počáteční konfiguraci, která je daná vstupem a další objevené konfigurace bude ukládat na pásku. Ve chvíli, kdy vygeneruje konfiguraci, jež sousedí s aktuálně navštěvovanou konfigurací dle δ , porovná je se seznamem objevených konfigurací. Z předpokladu je zaručeno, že seznam objevených konfigurací nemůže být delší než $2^{c_M f(n)}$. Práce M' končí

ve chvíli, kdy objeví přijímající konfiguraci (pak přijme), nebo kdy již není možné vygenerovat další konfiguraci (všichni sousedi konfigurací v seznamu jsou již v seznamu) a všechny konfigurace jsou již zpracované algoritmem průchodu do hloubky (nebo do šířky). $\square$

Je dobré si uvědomit, že z věty 9.4 nevyplývá, že $\text{NSPACE}(f(n)) \subseteq \text{TIME}(2^{cf(n)})$ pro nějakou konstantu c , protože konstanta c_L v tvrzení věty závisí na konkrétním jazyku L a pro každý jazyk může být jiná. Na druhou stranu můžeme zformulovat následující důsledek.

Důsledek 9.5 *Je-li $f(n)$ funkce, pro kterou platí $f(n) \geq \log_2 n$ a je-li $g(n)$ funkce, pro kterou platí $f(n) = o(g(n))$, pak*

$$\text{NSPACE}(f(n)) \subseteq \text{TIME}(2^{g(n)}).$$

Důkaz: Uvažme jazyk $L \in \text{NSPACE}(f(n))$. Dle věty 9.4 existuje konstanta $c_L \in \mathbb{N}$, pro kterou platí, že $L \in \text{TIME}(2^{c_L f(n)})$. Z předpokladu $f(n) = o(g(n))$ existuje $n_0 \in \mathbb{N}$ takové, že pro $n \geq n_0$ platí $c_L f(n) \leq g(n)$. Z toho plyne, že $\text{TIME}(2^{c_L f(n)}) \subseteq \text{TIME}(2^{g(n)})$ a tedy $L \in \text{TIME}(2^{g(n)})$. $\square$

Věty 9.1 a 9.4 nám umožňují ukázat některé vztahy mezi třídami jež jsme si dříve zavedli.

Věta 9.6 *Platí následující inkluze*

$$L \subseteq NL \subseteq P \subseteq NP \subseteq PSPACE \subseteq NPSPACE \subseteq EXPTIME.$$

Důkaz: Inkluze $L \subseteq NL$ a $P \subseteq NP \subseteq PSPACE \subseteq NPSPACE$ plynou přímo z věty 9.1. Uvažme jazyk $L \in NL = \text{NSPACE}(\log_2 n)$, dle věty 9.4 existuje konstanta c_L , pro kterou platí $L \in \text{TIME}(2^{c_L \log_2 n}) = \text{TIME}(n^{c_L}) \subseteq P$, tedy $NL \subseteq P$. Uvažme jazyk $L \in NPSPACE$, tedy $L \in \text{NSPACE}(n^k)$ pro nějakou konstantu $k \in \mathbb{N}$. Podle důsledku 9.5 platí, že $\text{NSPACE}(n^k) \subseteq \text{TIME}(2^{n^{k+1}}) \subseteq EXPTIME$. $\square$

Z věty 9.7, kterou si ukážeme v kapitole 9.2 plyne, že $PSPACE = NPSPACE$, proto se také obvykle v literatuře setkáme s třídou $PSPACE$ a nikoli s třídou $NPSPACE$. U zbývajících inkluzí není známo, zda jsou ostré, ovšem některé z nich ostré jsou, neboť z vět o hierarchii, které si ukážeme v kapitole 9.3 plyne, že $L \subsetneq PSPACE$ a $P \subsetneq EXPTIME$.

9.2. Savičova věta

V této kapitole si ukážeme Savičovu větu, která ukazuje, že nedeterministický Turingův stroj M lze převést na deterministický Turingův stroj M' , který přijímá též jazyk a využívá jen kvadraticky více prostoru. Důkaz Savičovy věty v této kapitole je převzat z [10].

Věta 9.7 (Savičova věta) *Pro každou funkci $f(n) \geq \log_2 n$ platí, že*

$$\text{NSPACE}(f(n)) \subseteq \text{SPACE}(f^2(n)).$$

Důkaz: K důkazu Savičovy věty využijeme graf konfigurací zavedený v definici 9.3 podobně jako v důkazu věty 9.4. Uvažme jazyk $L \in \text{NSPACE}(f(n))$, existuje tedy nedeterministický Turingův stroj $M = (Q, \Sigma, \delta, q_0, F)$, který přijímá jazyk $L = L(M)$ a který pracuje v prostoru $O(f(n))$. Popíšeme deterministický Turingův stroj M' , který přijímá jazyk L v prostoru $O(f^2(n))$, tedy $L \in \text{SPACE}(f^2(n))$.

Uvažme vstup $x \in \Sigma^*$. Víme, že $x \in L$, právě když existuje cesta v grafu konfigurací $G_{M,x}$ z počáteční konfigurace K_0^x se vstupem x do nějaké přijímající konfigurace. Pro jednoduchost budeme předpokládat, že M má jedinou přijímající konfiguraci K_F (tj. před přechodem do jediného přijímajícího stavu q_1 stroj M vymaže obsah pásky a vrátí se na nejlevější pozici využitého prostoru). Potom stačí ověřit, zda existuje v grafu $G_{M,x} = (V, E)$ cesta z konfigurace K_0^x do konfigurace K_F . V důkazu věty 9.4 nám stačilo použít průchod grafem do šířky či do hloubky k nalezení této cesty. S tím si zde ovšem nevystačíme, neboť průchod do šířky (BFS) i do hloubky (DFS) vyžadují ke své práci prostor lineární ve velikosti grafu $G_{M,x}$. Dle lematu 9.2 platí, že $|V| \leq 2^{c_M f(n)}$, kde $n = |x|$, pro vhodnou konstantu c_M závislou jen na stroji M a nikoli na vstupu x . Ovšem z důkazu tohoto odhadu také plyne, že byť jde o horní odhad, existují Turingovy stroje, kde tento odhad je těsný. Jinými slovy graf $G_{M,x}$ je příliš velký na to, aby si jej stroj M' mohl zkonstruovat a uložit na páse, navíc není možné použít průchod do šířky ani do hloubky tímto grafem k ověření existence cesty z K_0^x do K_F .

Budeme tedy postupovat jinak. Předpokládejme na chvíli, že známe hodnotu $t = c_M f(n)$ (byť o funkci f nepředpokládáme ani to, že by byla algoritmicky vyčíslitelná, s čímž si budeme muset v nějakou chvíli poradit). To znamená, že $|V| \leq 2^t$, a tedy délka nejdelší cesty mezi libovolnými dvěma vrcholy $G_{M,x}$ je nejvýš 2^t . To nabízí následující postup, který nám umožní ušetřit paměť: Cesta z K_0^x do K_F délky nejvýš 2^t existuje právě tehdy, když existuje „prostřední“ konfigurace K , pro kterou platí, že v $G_{M,x}$ existují cesty z K_0^x do K a z K do K_F poloviční délky, tedy nejvýš 2^{t-1} . Tyto dvě podmínky můžeme ověřit rekurzivně, přičemž můžeme využít toho, že pro oba testy (cesty z K_0^x do K a cesty z K do K_F) můžeme použít touž paměť. Rekurse se zastaví ve chvíli, kdy $t = 0$, neboť potom je již ověření existence cesty snadné, máme-li k dispozici přechodovou funkci δ nedeterministického Turingova stroje M . Stroj M' samozřejmě nemůže dopředu vědět, kterou konfiguraci K zvolit jako prostřední, bude tedy postupně zkoušet všechny vrcholy $G_{M,x}$. Pokud známe hodnotu t , stačí zkoušet všechny řetězce dané délky, jež mohou kódovat konfiguraci. Takto dostaneme rekurzivní algoritmus, kde hloubka rekurse je $O(f(n))$ a navíc v každé instanci rekurzivně volané funkce je využita paměť $O(f(n))$, tedy dohromady bude využita paměť $O(f^2(n))$. Na závěr si musíme však poradit s tím, že M' nezná hodnotu $t = c_M f(n)$ a nemůžeme ani předpokládat, že si ji může spočítat, protože o funkci $f(n)$ nepředpokládáme, že by byla algoritmicky vyčíslitelná.

Základem algoritmu je funkce `Dosažitelná()` (funkce 25), která ověřuje dosažitelnost v grafu. Korektnost této funkce byla zdůvodněna v předchozím textu. Je potřeba si uvědomit, že M' zná přechodovou funkci M , a tedy může ověřit, zda $(K_1, K_2) \in E$. Stačí

Funkce 25: $\text{DOSAŽITELNÁ}(K_1, K_2, t)$

Vstup: Konfigurace K_1 a K_2 , limit t

Výstup: true , pokud v $G_{M,x}$ existuje cesta z K_1 do K_2 délky nejvýš 2^t , jinak false

```
1 if  $t = 0$  then
2   if  $K_1 = K_2$  or  $(K_1, K_2) \in E$  then
3     return true
4   else
5     return false
6 forall  $K \in V$  do
7   if  $\text{Dosažitelná}(K_1, K, t-1)$  and  $\text{Dosažitelná}(K, K_2, t-1)$  then
8     return true
9 return false
```

tedy, aby M' zavolal $\text{DOSAŽITELNÁ}(K_0^x, K_F, t_0)$ pro vhodnou počáteční hodnotu t_0 . Cyklus na řádce 6 pak implementujeme postupným generováním binárních řetězců délky t_0 .

Ideální by jistě bylo rovnou použít hodnotu $t_0 = c_M f(n)$ a volat tedy

$$\text{Dosažitelná}(K_0^x, K_F, c_M f(n)).$$

To bychom mohli udělat, pokud bychom předpokládali, že funkce $f(n)$ je tzv. *prostorově konstruovatelná*, což je pojem, který si zavedeme později v definici 9.9. Často se Savičova věta uvádí s tímto předpokladem, který není příliš omezující, neboť běžné funkce, které používáme k omezení prostorových nároků prostorově konstruovatelné jsou.

My jsme ovšem tento předpoklad ve znění věty neuvedli a ukážeme si nyní, jak se bez něj obejít. Bez prostorové konstruovatelnosti nemůžeme předpokládat, že M' může vyčíslit hodnotu funkce $f(n)$. Místo toho necháme stroj M' postupně zkoušet různé odhady na hodnotu funkce $f(n)$ v podobě $k = 1, 2, 3, \dots$. Hodnota k tedy určuje, kolik buněk na pracovní pásce umožňujeme v danou chvíli stroji M použít. S danou hodnotou k tedy provedeme volání $\text{Dosažitelná}(K_0^x, K_F, c_M k)$. Třetí parametr $c_M k$ odpovídá tomu, že při aktuálním odhadu počtu buněk k může M vykonat nejvýše $2^{c_M k}$ kroků než zopakuje konfiguraci. Pokud toto volání uspěje, bylo ověřeno, že cesta z K_0^x do K_F v grafu $G_{M,x}$ existuje. V opačném případě je možné, že selhání nebylo způsobeno neexistencí cesty, ale tím, že náš odhad počtu buněk k nebyl dostatečný a je jej třeba zvýšit. Musíme tedy zjistit, zda stroj M využije buňku navíc, zvýšíme-li hodnotu k . K tomu je potřeba ověřit, zda existuje konfigurace M , v níž je hlava M nad buňkou s pořadovým číslem $k+1$ a která je dosažitelná z počáteční konfigurace. K tomu stačí projít všechny takové konfigurace K a voláním funkce $\text{Dosažitelná}(K_0^x, K, c_M k)$ ověřit jejich dosažitelnost (stačí uvažovat $c_M k$ kroků, protože víme, že přijímající konfigurace s tímto omezením dosažitelná není, konfigurace K , která je tedy navíc nemění odhad počtu kroků potřebný k dosažení K). Pokud některé z těchto volání uspěje, znamená to, že dává smysl ještě zvýšit hodnotu k . V opačném případě není M schopen buňku navíc využít, což zname-

Algoritmus 26: Práce M' se vstupem x

```
1 forall  $k = 1, 2, 3, \dots$  do
2   if Dosažitelná( $K_0^x, K_F, c_M k$ ) then
3     accept
4   forall konfigurace  $K$  s hlavou nad  $(k+1)$ . políčkem pracovní pásky do
5     if Dosažitelná( $K_0^x, K, c_M k$ ) then
6       continue
7   reject
```

ná, že ani neexistuje cesta z K_0^x do K_F a M' může odmítnout. Tento postup je popsán v algoritmu 26.

Maximální hodnotou k , již musí algoritmus 26 vzít do úvahy, je hodnota $k = f(n)$. Prostorové nároky jedné instance funkce Dosažitelná jsou tedy $O(f(n))$. Hloubka rekurze každého volání je daná třetím parametrem a je tedy také $O(f(n))$. Dohromady dostáváme, že prostorové nároky algoritmu 26 jsou $O(f^2(n))$. Jelikož se jedná o deterministický algoritmus a platí $L = L(M')$, dostáváme, že $L \in \text{SPACE}(f^2(n))$. $\square$

Ze Savičovy věty plyne následující důsledek, který jsme již zmiňovali.

Důsledek 9.8 $\text{NPSPACE} = \text{PSPACE}$.

V literatuře se ve znění Savičovy věty často předpokládá, že funkce f je prostorově konstruovatelná, případně vyčíslitelná v prostoru $f(n)$. Za tohoto předpokladu je možné, aby si M' na začátku spočítal hodnotu $f(n)$, protože konstanta c_M je závislá na M , můžeme předpokládat, že ji M' zná a tedy může rovnou určit ten správný limit t_0 pro volání funkce Dosažitelná, což zjednoduší algoritmus 26. Pojem prostorové konstruovatelnosti si zavedeme v sekci 9.3, neboť jej budeme potřebovat ve větě o prostorové hierarchii.

9.3. Věty o hierarchii

Intuitivně bychom řekli, že pokud nějakému výpočetnímu stroji umožníme využít více prostředků pro výpočet, měl by být schopen spočítat více. V případě Turingových strojů pracujeme se dvěma prostředky, velikostí paměti a časem, po který necháme stroj běžet, od toho se odvíjí třídy $\text{TIME}(f(n))$ a $\text{SPACE}(f(n))$. V této kapitole si ukážeme, že zmíněná intuice je platná, tedy alespoň za určitých předpokladů, které jsou však velmi realistické. Důkazy v této kapitole jsou převzaty z [10].

9.3.1. Deterministická prostorová hierarchie

Nejprve se budeme věnovat prostoru. Uvažme funkci $f(n)$ a funkci $g(n) = o(f(n))$. Naším cílem je ukázat, že $\text{SPACE}(g(n)) \subsetneq \text{SPACE}(f(n))$. K tomu potřebujeme ukázat, že

existuje jazyk L , pro který existuje Turingův stroj M , který přijímá L , tedy $L = L(M)$ v prostoru $O(f(n))$, ale neexistuje žádný Turingův stroj M' , který by přijímal L v prostoru $O(g(n))$. Abychom však byli schopni uvedené tvrzení dokázat, budeme se muset omezit na funkce $f(n)$, které splňují následující předpoklad: K výpočtu hodnoty $f(n)$ postačuje prostor, který je úměrný velikosti výstupu. Přesněji tento pojem definujeme následujícím způsobem.

Definice 9.9 Funkci $f : \mathbb{N} \rightarrow \mathbb{N}$, kde $f(n) \geq \log_2 n$, nazveme **prostorově konstruovatelnou**, je-li funkce, která zobrazuje 1^n na binární reprezentaci $f(n)$ vyčíslitelná v prostoru $O(f(n))$. ◀

Všechny běžné funkce, které používáme k měření složitosti jsou prostorově konstruovatelné. Jde například o polynomy, exponenciály, funkce využívající logaritmy jako $\log_2 n$, $n \log_2 n$ a další. Omezíme-li se na prostorově konstruovatelné funkce, neztrácíme tedy příliš. Je-li funkce prostorově konstruovatelná, znamená to, že při výpočtu můžeme funkci využít k vymezení nebo alokaci prostoru. Je-li dán vstup x s $|x| = n$, můžeme na začátku výpočtu určit hodnotu $f(n)$ a následně vyznačit na pásce prostor této velikosti. Pro tento způsob využití je také pojem prostorové konstruovatelnosti navržen. Je-li totiž hodnota n dána délkou vstupního řetězce, dává smysl počítat hodnotu funkce $f(n)$ se vstupem v tomto tvaru, definice vyžaduje řetězec délky n na vstupu složený ze samých jedniček, který z x vytvoříme jednoduše náhradou každého znaku znakem 1. Toto omezení je v definici proto, abychom zdůraznili, že hodnota funkce nezávisí na obsahu řetězce, jenom na jeho délce. Zmiňme, že v literatuře se definice prostorové konstruovatelnosti vyskytuje v různých podobách, někdy se vyžaduje, aby existoval Turingův stroj, který při výpočtu nad vstupem x využije přesně $f(n)$ buněk pracovní pásky, ale existují i jiné varianty. V případě Turingových strojů lze obvykle ukázat ekvivalenci mezi různými definicemi, definice 9.9 nabízí vcelku jednoduché použití, je podle ní například vcelku přímočaré ukázat, že polynomy a další funkce jsou prostorově konstruovatelné.

Nyní již můžeme zformulovat větu o deterministické prostorové hierarchii.

Věta 9.10 (Věta o deterministické prostorové hierarchii) *Pro každou prostorově konstruovatelnou funkci $f : \mathbb{N} \rightarrow \mathbb{N}$ existuje jazyk A , který je rozhodnutelný v prostoru $O(f(n))$, nikoli však v prostoru $o(f(n))$.*

Důkaz: Popíšeme Turingův stroj N , pro který bude platit, že pracuje v prostoru $O(f(n))$, ale neexistuje žádný deterministický Turingův stroj, který by přijímal též jazyk $A = L(N)$ v prostoru $o(f(n))$. Při popisu algoritmu stroje N využijeme techniky diagonalizace. Pro každý deterministický Turingův stroj M , který pracuje v prostoru $o(f(n))$ musí platit, že A se od $L(M)$ liší alespoň na jednom vstupu. Přirozeným kandidátem na vstup, v němž se mají tyto dva jazyky lišit, je kód stroje M , tedy $\langle M \rangle$, tento krok využívá zmíněnou diagonalizaci. Nestačí však uvažovat jediný řetězec, na kterém se budou A a $L(M)$ lišit, protože nám jde o asymptotickou složitost daného algoritmu. Pokud by platilo že, A se od $L(M)$ liší právě jen na vstupu $\langle M \rangle$, pak jednoduchou úpravou, která by ošetřila tuto výjimku, bychom dostali Turingův stroj, který přijímá A a pracuje v témž prostoru jako $\langle M \rangle$. Je proto nutné uvažovat řetězce různých délek. My budeme uvažovat řetězce tvaru $\langle M \rangle 10^*$. Aby mohl stroj N rozhodnout, má-li přijmout vstup tvaru $\langle M \rangle 10^*$, potřebuje

Algoritmus 27: Práce stroje N se vstupem x

Vstup: Vstupní řetězec $x \in \Sigma^*$.

- 1 **if** $x \neq \langle M \rangle 10^*$ *pro nějaký TS M* **then**
- 2 **reject**
- 3 Na základě prostorové konstruovatelnosti urči hodnotu funkce $f(|x|)$
- 4 Vyznač $f(|x|)$ buněk na pracovní pásce, pokud v kterémkoli z dalších kroků hlava N opustí vymezený prostor, **reject**
- 5 Simuluj výpočet stroje $M(x)$ a počítej kroky simulace, pokud je odsimulováno více než $2^{f(n)}$ kroků, **reject**
- 6 **if** M přijal **then**
- 7 **reject**
- 8 **else**
- 9 **accept**

na tomto vstupu odsimulovat stroj M a rozhodnout opačně. Jsou-li prostorové nároky stroje M určeny funkcí $g(n) = o(f(n))$, pak k simulaci je potřeba $c_M g(n)$ buněk pracovní pásy, kde c_M je konstanta závislá na stroji M . Vzhledem k tomu, že N může využít prostor $O(f(n))$, má N dost prostoru k dokončení simulace na dost dlouhých vstupech, tedy i na nějakém vstupu tvaru $\langle M \rangle 10^*$, ovšem pouze za předpokladu, že výpočet M nad vstupem x skončí, tedy $M(x) \downarrow$. Pokud $M(x) \uparrow$, může stroj N využít toho, že počet konfigurací, v nichž se výpočet $M(x)$ může nacházet je dle lemmatu 9.2 omezený a N tedy podle počtu provedených kroků může poznat, zda se výpočet $M(x)$ zacyklil.

Algoritmus 27 popisuje práci stroje N se vstupem x . Položme $A = L(N)$. Není těžké nahlédnout, že N pracuje v prostoru $O(f(n))$, výpočet hodnoty $f(n)$, kde $n = |x|$, v kroku 4 lze provést v prostoru $O(f(n))$, neboť funkce $f(n)$ je dle předpokladu prostorově konstruovatelná. Všechny další kroky probíhají ve vyznačeném prostoru $f(|x|)$ buněk. Počítadlo kroků se do daného prostoru vejde také, uvážíme-li, že abeceda N může být větší než binární (počítadlo může být na jiné stopě pásy, než vstup a simulace). Celkově tedy dostáváme, že A je rozhodnutelný v prostoru $O(f(n))$.

Algoritmus 6 Předpokládejme nyní sporem, že A je rozhodnutelný v prostoru $o(f(n))$. Existuje tedy deterministický Turingův stroj M , který rozhoduje jazyk A v prostoru $g(n)$, kde $g(n) = o(f(n))$. Simulace výpočtu $M(x)$ probíhá podobně jako simulace [univerzálním Turingovým strojem](#). K simulaci tedy N potřebuje $c_M g(n)$ buněk pracovní pásy, kde c_M je konstanta závislá na stroji M . Přesněji c_M určíme na základě lemmatu 9.2. Stroji N stačí c_M buněk k zakódování jednoho políčka pásy M . Protože $g(n) = o(f(n))$, existuje přirozené číslo n_0 , pro které platí, že $c_M g(n_0) \leq f(n)$. Uvažme nyní vstup $x = \langle M \rangle 10^{n_0}$. Simulace $M(x)$ strojem N doběhne až do konce, neboť na ni stačí vyznačený prostor. Počet kroků je též menší než $2^{f(n)}$, jelikož dle předpokladu je výpočet $M(x)$ konečný, a tedy nepoužije žádnou konfiguraci dvakrát. Na základě podmíněného příkazu na řádce 6 tak dostáváme, že $x \in L(M)$ právě když $x \notin L(N)$. Z toho plyne, že $L(M) \neq A$, a to je spor s předpokladem. Platí tedy, že neexistuje Turingův stroj, který by A rozhodl v prostoru

$o(f(n))$.

□

Z věty 9.10 vyplývají následující důsledky.

Důsledek 9.11

- (i) Jsou-li $f_1, f_2 : \mathbb{N} \rightarrow \mathbb{N}$ funkce, pro které platí, že $f_1(n) \in o(f_2(n))$ a f_2 je prostorově konstruovatelná, potom

$$\text{SPACE}(f_1(n)) \subsetneq \text{SPACE}(f_2(n)).$$

- (ii) Pro každá dvě reálná čísla $0 \leq \epsilon_1 < \epsilon_2$ platí, že

$$\text{SPACE}(n^{\epsilon_1}) \subsetneq \text{SPACE}(n^{\epsilon_2}).$$

- (iii) Platí $\text{NL} \subsetneq \text{PSPACE} \subsetneq \text{EXPSPACE} = \bigcup_{k \in \mathbb{N}} \text{SPACE}(2^{n^k})$.

Důkaz: Tvrzení (i) vyplývá přímo z věty 9.10. Uvažme nyní dvě reálná čísla $0 \leq \epsilon_1 < \epsilon_2$. Platí jistě $n^{\epsilon_1} = o(n^{\epsilon_2})$, musíme však ověřit podmínku prostorové konstruovatelnosti. Ne-ní těžké nahlédnout, že je-li ϵ_2 ve skutečnosti celé číslo, pak n^{ϵ_2} je prostorově konstruovatelná funkce. Trochu obtížnější, byť stále možné, je ukázat, že n^{ϵ_2} je prostorově konstruovatelná funkce i v případě, kdy ϵ_2 je racionální číslo. Je-li ϵ_2 iracionální číslo, pak z hustoty racionálních čísel existuje racionální číslo ϵ'_2 , pro které platí $\epsilon_1 < \epsilon'_2 < \epsilon_2$. Protože ϵ'_2 je číslo racionální, plyne z předchozích úvah, že $\text{SPACE}(n^{\epsilon_1}) \subsetneq \text{SPACE}(n^{\epsilon'_2}) \subseteq \text{SPACE}(n^{\epsilon_2})$. Dostáváme tedy tvrzení (ii).

Připomeňme si, že $\text{NL} = \text{NSPACE}(\log_2 n)$, dle Savičovy věty platí $\text{NSPACE}(\log n) \subseteq \text{SPACE}((\log_2 n)^2)$, dle věty 9.10 dostáváme tedy $\text{NL} \subseteq \text{SPACE}((\log n)^2) \subsetneq \text{SPACE}(n) \subsetneq \text{PSPACE}$. Z definice třídy PSPACE dostáváme, že $\text{PSPACE} \subseteq \text{SPACE}(2^n)$, a tedy z věty 9.10 dostáváme $\text{PSPACE} \subsetneq \text{SPACE}(2^{n^2})$. Dohromady tedy dostáváme tvrzení (iii). □

Věta 9.7

Větu 9.10 jsme ukazovali pro prostorové třídy, jež jsme zavedli pro výpočetní model jednopáskového deterministického Turingova stroje. Je dobré si však uvědomit, že volba modelu v tomto případě nehraje podstatnou roli. Větu 9.10 by bylo možné ukázat i v případě, kdy bychom uvažovali více páskové Turingovy stroje nebo RAM. Větu 9.10 je možné ukázat i pro nedeterministické třídy prostorové složitosti, v tomto případě je situace složitější, neboť nedeterministické Turingovy stroje nenabízejí jednoduchý způsob pro negaci odpovědi.

9.3.2. Deterministická časová hierarchie

Analogii věty 9.10 můžeme ukázat i v případě časových tříd složitosti, i když tvrzení, které v tomto případě ukážeme, je o něco slabší. Budeme postupovat velmi podobně jako v kapitole 9.3.1. Omezíme se na funkce, které jsou časově konstruovatelné, což je pojem analogický prostorové konstruovatelnosti, avšak nyní omezujeme čas výpočtu, nikoli prostor.

Definice 9.12 Funkci $f : \mathbb{N} \rightarrow \mathbb{N}$, kde $f(n) = \Omega(n \log_2 n)$, nazveme **časově konstruovatelnou**, je-li funkce, která zobrazuje 1^n na binární reprezentaci $f(n)$ vyčíslitelná v čase $O(f(n))$. ◀

I v tomto případě platí, že všechny běžné funkce, které používáme k měření složitosti, jsou časově konstruovatelné. Jde například o polynomy (vyššího stupně než 1), exponenciály, funkce využívající logaritmy jako $n \log_2 n$, ale i funkce s odmocninami jako $\lceil n \sqrt{n} \rceil$ a další. Na rozdíl od prostorové konstruovatelnosti, v případě časové konstruovatelnosti vyžadujeme, aby platilo, že $f(n) = \Omega(n \log_2 n)$. Tento silnější požadavek je zde proto, že výstupem je binární zápis hodnoty funkce, zatímco vstupní hodnota n je zapsána v podobě 1^n . Na jednopáskovém Turingovu stroji potřebujeme k převodu řetězce 1^n do binárního zápisu čísla n čas $\Omega(n \log_2 n)$. Proto nepovažujeme funkci $f(n) = n$ za časově konstruovatelnou. Jak uvidíme, není tento předpoklad omezující. Požadovat binární zápis hodnoty funkce přitom dává dobrý smysl, který plyne ze způsobu použití časové konstruovatelnosti. Je-li funkce $f(n)$ časově konstruovatelná, můžeme totiž na začátku algoritmu určit hodnotu $f(n)$ a inicializovat binární čítač kroků. Poté můžeme počítat kroky a výpočet zastavit při dosažení $f(n)$ kroků. Pro operace s čítačem (například přesun, zvýšení či snížení hodnoty o 1) nám stačí $O(\log_2 f(n))$ kroků právě díky tomu, že hodnota $f(n)$ je zapsaná binárně. Tento způsob použití využijeme při důkazu věty o deterministické časové hierarchii.

Je třeba zmínit, že definice 9.12 není jedinou možností, jak pojem časové konstruovatelnosti zavést. V literatuře najdeme různé způsoby, které také uvažují různá omezení na funkci $f(n)$. Definice 9.12 má výhodu v tom, že je jednoduchá a slouží přesně účelu, ke kterému chceme pojem časové konstruovatelnosti využít.

Nyní již můžeme zformulovat větu o deterministické časové hierarchii.

Věta 9.13 (Věta o deterministické časové hierarchii) *Pro každou časově konstruovatelnou funkci $f : \mathbb{N} \rightarrow \mathbb{N}$ existuje jazyk A , který je rozhodnutelný v čase $O(f(n))$, nikoli však v čase $o(f(n)/\log_2 f(n))$.*

Důkaz: Postup důkazu je analogický důkazu věty 9.10 o deterministické prostorové hierarchii. Sestrojíme Turingův stroj N , pro který bude platit, že pracuje v čase $O(f(n))$, ale neexistuje žádný deterministický Turingův stroj, který by přijímal týž jazyk $A = L(N)$ v čase $o(f(n)/\log_2 f(n))$. Struktura stroje N je podobná stroji popsanému v rámci důkazu věty 9.10. Stroj N opět předpokládá vstupní řetězec x ve tvaru $x = \langle M \rangle 10^*$ pro nějaký Turingův stroj M . Jádro práce stroje N pak spočívá v simulaci výpočtu $M(x)$ s tím, že pokud výpočet doběhne v čase $f(n)/\log_2 f(n)$, zneguje N jeho odpověď. Stroj N tedy provádí simulaci a navíc počítá kroky, toto je v případě omezeného času složitější, než v případě omezeného prostoru. V případě času si stroj N nevystačí s konstantním počtem kroků na simulaci jednoho kroku výpočtu $M(x)$ a současně aktualizaci počítadla kroků, proto je nutné požadovat logaritmický odstup uvažovaných funkcí.

Algoritmus 28 popisuje práci stroje N se vstupem x . Položme $A = L(N)$. Kroky 1 až 3 lze provést v čase $O(f(n))$, kde $n = |x|$. Rozmysleme si nejprve, jak bude probíhat krok 4, tedy simulace výpočtu $M(x)$ spolu s dekrementací čítače po každém kroku. Pro simulaci kroku výpočtu M , musí N znát aktuální stav stroje M , symbol pod hlavou stroje M a musí

Algoritmus 28: Práce stroje N se vstupem x

Vstup: Vstupní řetězec $x \in \Sigma^*$.

```
1 if  $x \neq \langle M \rangle 10^*$  pro nějaký TS  $M$  then
2   | reject
3 Na základě časové konstruovatelnosti urči hodnotu funkce  $f(|x|)$  a inicializuj
   binární čítač hodnotou  $f(|x|)/\log_2 f(|x|)$ . Před provedením každé instrukce
   potřebné pro simulaci  $M(x)$  proved' snížení hodnoty čítače o 1, pokud hodnota
   čítače dosáhne hodnoty 0, reject
4 Simuluj výpočet  $M(x)$ 
5 if  $M$  přijal then
6   | reject
7 else
8   | accept
```

mít přístup k přechodové funkci stroje M zapsané v kódu $\langle M \rangle$. Tyto tři údaje potřebuje mít N na pásce blízko sebe, protože pokud by byl například stav zapsaný na druhém konci pásky než je $\langle M \rangle$, znamenalo by to, že pro určení následující instrukce potřebuje N projít celou svou páskou mnohokrát.

Připomeňme si, že při konstrukci [univerzálního Turingova stroje](#) v kapitole 5.2.3 jsme každé části výše uvedených dat vyhradili zvláštní pásku. Nyní ovšem popisujeme N jako jednopáskový Turingův stroj, není možné tedy mít stav zapsaný na jiné pásce než $\langle M \rangle$ a než obsah pracovní pásky M . Toto jsme při popisu univerzálního Turingova stroje vyřešili použitím generického převodu k -páskového Turingova stroje na 1-páskový popsaného v důkazu věty 4.10. Nicméně tento postup není pro nás dostačující, protože použitím postupu z důkazu věty 4.10 by provedení jednoho kroku simulace M zabralo čas úměrný velikosti využitého prostoru stroje M , tedy až $f(n)/\log_2 f(n)$. Tím bychom ovšem nedosáhli požadovaného výsledku.

Musíme proto postupovat jinak. Pásku N si rozdělíme na tři stopy (podobně jako v důkazu věty 4.10). Na první stopě bude zakódována pracovní páska M . Na druhé stopě bude zapsán aktuální stav výpočtu $M(x)$ a přechodová funkce zakódovaná pomocí $\langle M \rangle$. Na třetí stopě bude zapsána aktuální hodnota čítače kroků. Podstatné je, že obsahy stop budou zarovnané. Poloha hlavy na pásce odpovídá poloze hlavy na pásce M , tedy na první stopě. Dále je zachován následující invariant: Na začátku simulace jednoho kroku M obsah druhé stopy vždy začíná pod hlavou.

Odmysleme si zatím práci s čítačem a popíšme kroky simulace. Během inicializace dojde k zakódování vstupu x ve vhodném tvaru na první stopě, na druhou stopu je překopírován kód $\langle M \rangle$ a stav číslo 0 (tedy číslo počátečního stavu). Na inicializaci stačí čas $O(n)$. Dále v cyklu probíhá simulace kroků výpočtu $M(x)$. Při simulaci jednoho kroku nejprve N najde v kódu $\langle M \rangle$ vhodnou instrukci a poté ji provede. Pokud instrukce zahrnuje pohyb hlavou, dojde k posunu kódu $\langle M \rangle$ tak, aby začínal pod hlavou na začátku simulace dalšího kroku. Dohromady lze simulaci jednoho kroku výpočtu M provést pomocí d kroků N , kde d je konstanta závislá na stroji M ($\langle M \rangle$ je sice součástí vstupu, ale

v další argumentaci budeme uvažovat fixní M , tedy d je možné považovat za konstantu).

Zbývá rozmyslet si, jak bude probíhat práce s čítačem. Hodnota čítače bude zapsána na třetí stopě pásky s tím, že bude začínat vždy pod hlavou. To znamená, že po provedení každého kroku stroje N bude v případě nutnosti posunut obsah čítače. Při tom bude provedena i jeho dekrementace. K posunu a dekrementaci čítače postačuje $O(\log_2 f(n))$ kroků, neboť tolik bitů čítač zabírá. Dle kroku 3 dostáváme, že N vykoná v následujících krocích $f(n)/\log_2 f(n)$ instrukcí, po provedení každé z nich provede dekrementaci a posun čítače, tedy dohromady provede $O(f(n))$ instrukcí. Kroky 1 až 3 lze provést v čase $O(f(n))$, celkově tedy N pracuje v čase $O(f(n))$.

Předpokládejme nyní sporem, že M je Turingův stroj, který přijímá jazyk A v čase $g(n) = o(f(n)/\log_2 f(n))$. Z popisu simulace plyne, že simulaci výpočtu M se vstupem x provede N v čase $dg(n)\log_2 f(n)$, kde d je konstanta závislá na stroji M , přesněji na jeho kódu $\langle M \rangle$. Protože $g(n) = o(f(n)/\log_2 f(n))$, existuje $n_0 \in \mathbb{N}$, takové, že $dg(n) \leq f(n)/\log_2 f(n)$ pro každé $n \geq n_0$. Uvažme nyní vstup $x = \langle M \rangle 10^{n_0}$, pro který platí, že $n = |x| > n_0$. Stroj N tedy simulaci $M(x)$ dokončí v rámci $dg(n)$ simulovaných kroků než hodnota čítače dosáhne 0. Tedy výpočet N se dostane k provedení kroku 5. Z toho plyne, že $x \in L(M)$ právě když $x \notin L(N) = A$. Tedy $A \neq L(M)$, což je spor s předpokladem. Žádný takový stroj M tedy neexistuje. $\square$

Z věty 9.13 vyplývají následující důsledky.

Důsledek 9.14

- (i) Jsou-li $f_1, f_2 : \mathbb{N} \rightarrow \mathbb{N}$ funkce, pro které platí, že $f_1(n) \in o(f_2(n)/\log_2 f_2(n))$ a f_2 je časově konstruovatelná, potom

$$\text{TIME}(f_1(n)) \subsetneq \text{TIME}(f_2(n)).$$

- (ii) Pro každá dvě reálná čísla $1 \leq \epsilon_1 < \epsilon_2$,

$$\text{TIME}(n^{\epsilon_1}) \subsetneq \text{TIME}(n^{\epsilon_2}).$$

- (iii) $P \subsetneq \text{EXPTIME}$.

Důkaz: Tvrzení (i) plyne přímo z věty 9.13. Podobně jako v případě důsledku 9.11 není těžké nahlédnout, že je-li ϵ_2 přirozené číslo větší než 1, pak n^{ϵ_2} je časově konstruovatelná funkce. Trochu obtížněji je možné ukázat, že n^{ϵ_2} je časově konstruovatelná funkce i pokud $\epsilon_2 > 1$ je racionální číslo. Dále můžeme využít hustoty racionálních čísel a toho, že $n^{\epsilon_1} = o(n^{\epsilon_2}/\log_2 n)$ k důkazu (ii) (podobně jako při důkazu (ii) v důkazu důsledku 9.11). Z definice třídy P dostáváme, že $P \subseteq \text{TIME}(2^n)$. Zřejmě platí, že $2^n = o(2^{n^2}/n)$, a tedy z věty 9.13 dostáváme, že $P \subsetneq \text{EXPTIME}$, tedy (iii). $\square$

Podobně jako v případě prostoru, větu 9.10 jsme ukazovali pro časové třídy, jež jsme zavedli pro výpočetní model jednopáskového deterministického Turingova stroje. Analogickou větu lze ukázat i pro k -páskové Turingovy stroje. Pro nedeterministické časové třídy dokonce stačí slabší předpoklad $f_1(n+1) = o(f_2(n))$, aby platila ostrá inkluze $\text{NTIME}(f_1(n)) \subsetneq \text{NTIME}(f_2(n))$. V případě RAM stačí předpoklad $f_1(n) = o(f_2(n))$.

10. Cookova-Levinova věta

V kapitole 8 jsme zavedli třídu P problémů, které lze vyřešit deterministickým algoritmem v polynomiálním čase. V kapitole 8.3 jsme si rozmysleli, že je řada důvodů, proč třídu P považovat za třídu problémů, které umíme efektivně řešit počítačem. Naproti tomu jsme zavedli třídu NP problémů, které jsou polynomiálně ověřitelné. Také můžeme říct, že jde o problémy přijímané nedeterministickými Turingovými stroji v polynomiálním čase. Nabízí se otázka, zda jsou tyto třídy shodné. Tedy, zda platí, že pokud umíme problém ověřovat v polynomiálním čase, umíme jej v polynomiálním čase i rozhodovat. Tato otázka se dá stručně formulovat takto: Platí $P = NP$? Odpověď na tuto otázku neznáme. To je zajímavé i ve srovnání s prostorem, kde podle Savičovy věty víme, že $PSPACE = NPSPACE$.

Věta 9.7.

V případě času se naopak většina vědců domnívá, že $P \neq NP$. Ačkoli tuto nerovnost ukázat neumíme, existuje řada problémů, které patří do třídy NP, ale které v polynomiálním čase v tuto chvíli řešit neumíme. Nejznámějším problémem tohoto typu je bezesporu problém splnitelnosti formule v konjunktivní formě.

Problém 10.1: SPLNITELNOST FORMULE V KNF (SAT)

Instance: Formule φ na n proměnných v konjunktivní normální formě.

Otázka: Existuje ohodnocení proměnných $v \in \{0, 1\}^n$, pro které platí $\varphi(v) = 1$? Jinými slovy, existuje ohodnocení, které formuli φ splní?

Problém SAT odpovídá jazyku

$$\text{SAT} = \{\langle \varphi \rangle \mid \varphi \text{ je KNF na } n \text{ proměnných a } (\exists v \in \{0, 1\}^n)[\varphi(v) = 1]\}$$

Není těžké nahlédnout, že tento problém patří do třídy NP. Patří ovšem i do třídy P? To sice nevíme, můžeme ale ukázat následující tvrzení, které říká, že rozhodnutí toho, zda SAT patří do třídy P rovná se rozhodnutí toho, zda P je rovno NP.

Věta 10.2 (Cookova-Levinova) SAT patří do P, právě když $P = NP$.

Tento výsledek je připisován nezávisle Stephenovi A. Cookovi [3] a Leonidu A. Levinovi [9]. K důkazu této věty musíme mimo jiné ukázat, že pokud SAT lze rozhodnout v polynomiálním čase, pak lze v polynomiálním čase rozhodnout i jakýkoli jiný problém z třídy NP. K tomu použijeme polynomiální převoditelnost, na základě níž budeme definovat pojem NP-úplnosti. Nahlédneme, že pokud nějaký NP-úplný problém patří do

třídy P, potom to platí pro každý problém z třídy NP. Důkaz věty 10.2 pak bude spočívat v důkazu NP-úplnosti problému SAT.

10.1. Polynomiální převoditelnost a NP-úplnost

S principem převoditelnosti stejně jako s úplností jsme se již setkali v kapitole 7. Tehdy nás zajímala převoditelnost, která zachovávala algoritmickou řešitelnost a k tomu nám stačilo, že převádějící funkce byla algoritmicky vyčíslitelná. Nyní chceme, aby převádějící funkce zachovávala polynomiální řešitelnost, a proto od ní budeme požadovat, aby byla vyčíslitelná v polynomiálním čase.

Definice 10.3 Nechť A a B jsou dva jazyky nad abecedou Σ . Řekneme, že jazyk A je *polynomiálně převoditelný* na jazyk B , pokud existuje funkce $f : \Sigma^* \rightarrow \Sigma^*$, která je vyčíslitelná v polynomiálním čase¹ a pro kterou platí, že

$$(\forall x \in \Sigma^*) [x \in A \Leftrightarrow f(x) \in B].$$

Tento fakt označíme pomocí $A \leq_m^p B$, ◀

V literatuře se kromě značení $A \leq_m^p B$ též objevuje značení $A \propto B$, námi zvolené značení však lépe naznačuje, že polynomiální převoditelnost je m -převoditelnost s přidaným požadavkem vyčíslitelnosti převádějící funkce v polynomiálním čase. Pokud je $A \leq_m^p B$, znamená to, že rozhodnutí, zda $x \in A$ lze provést tak, že nejprve určíme $y = f(x)$ a poté položíme dotaz zda $y \in B$. Pokud je tedy jazyk B rozhodnutelný v polynomiálním čase, pak totéž platí i o A . Na druhou stranu, pokud neznáme žádný polynomiální algoritmus rozhodující A , pak neznáme ani polynomiální algoritmus rozhodující B . Pro tyto úvahy samozřejmě potřebujeme to, že hodnotu převádějící funkce $f(x)$ lze určit v polynomiálním čase. Polynomiální převoditelnost má řadu vlastností společných s m -převoditelností.

Lemma 10.4 (Vlastnosti polynomiální převoditelnosti)

1. Relace $\leq_m^p$ je reflexivní a tranzitivní.
2. Nechť A a B jsou jazyky, pro něž platí $A \leq_m^p B$. Pokud je $B \in P$, pak i $A \in P$.
3. Nechť A a B jsou jazyky, pro něž platí $A \leq_m^p B$. Pokud je $B \in NP$, pak i $A \in NP$.

Důkaz: 1. Reflexivita plyne z toho, že identita je funkce vyčíslitelná v polynomiálním čase. Tranzitivita plyne z toho, že složením dvou polynomů vznikne opět polynom, přesněji, je-li f funkce převádějící jazyk A na jazyk B , tedy $A \leq_m^p B$ s použitím f , a g je funkce převádějící jazyk B na jazyk C , tedy $B \leq_m^p C$ s použitím g , pak $g \circ f$ převádí A na C , jsou-li f i g vyčíslitelné v polynomiálním čase, lze totéž říci i o $g \circ f$, tedy $A \leq_m^p C$ s použitím $g \circ f$.

¹To znamená, že existuje Turingův stroj M , který pracuje v polynomiálním čase a počítá funkci f . Viz též definice 5.13 a 4.6.

2. Je-li $B \in P$, pak existuje Turingův stroj M , který přijímá B v polynomiálním čase. Je-li f funkce, která převádí A na B , a je-li f vyčíslitelná v polynomiálním čase, pak Turingův stroj M' , který pro vstup x spočítá $f(x)$ a poté pustí M k rozhodnutí, zda $f(x) \in B$, přijímá A v polynomiálním čase.
3. Platí z téhož důvodu jako předchozí bod, protože tytéž argumenty lze použít i pro nedeterministický Turingův stroj. $\square$

Poznámka 10.5 V poznámce 7.5 jsme navíc k m -převoditelnosti neformálně zavedli turingovskou převoditelnost. I tu můžeme omezit polynomiálním časem a takovému převodu se říká Cookův či cookovský, zatímco relaci $\leq_m^p$ se říká Karpův či karpovský převod. Přesněji tedy řekneme, že problém A je cookovsky převoditelný na problém B , což označíme pomocí $A \leq_T^p B$, pokud existuje algoritmus, který rozhoduje problém A , přičemž může pokládat dotazy černé skříňce, či orákulu, která umí řešit problém B , a tento algoritmus pracuje v polynomiálním čase, počítáme-li dobu zodpovězení dotazů černou skříňkou (orákulem) jako konstantní. Toto tedy odpovídá následující intuici: „Pokud existuje polynomiální algoritmus rozhodující problém B , pak existuje i polynomiální algoritmus rozhodující problém A .“ Podobně jako v případě m -převoditelnosti a turingovské převoditelnosti i zde platí, že je-li $A \leq_m^p B$, platí také $A \leq_T^p B$, ale opačná implikace platit nemusí. Triviálním příkladem je opět jazyk DIAG neboť zřejmě platí, že $\text{DIAG} \leq_T^p \overline{\text{DIAG}}$, ale jistě neplatí $\text{DIAG} \leq_m^p \overline{\text{DIAG}}$, protože to by muselo platit i $\text{DIAG} \leq_m \overline{\text{DIAG}}$, víme přitom, že to není možné, protože DIAG není částečně rozhodnutelný jazyk, zatímco $\overline{\text{DIAG}}$ ano. Až se dostaneme k třídě co-NP , všimneme si, že pokud se co-NP nerovná NP , pak mají tuto vlastnost všechny NP -úplné úlohy.

Stejně jako v případě m -převoditelnosti, i zde budou předmětem našeho zájmu ty nejtěžší problémy, přičemž nás nejvíce zajímají problémy z třídy NP .

Definice 10.6 Jazyk A je NP -těžký, pokud pro každý jazyk $B \in \text{NP}$ platí, že $B \leq_m^p A$. O jazyku A řekneme, že je NP -úplný, pokud je NP -těžký a navíc patří do třídy NP . $\blacktriangleleft$

Pojem úplnosti můžeme definovat pro libovolnou třídu, nejen pro třídu NP , v literatuře se mnoho prostoru věnuje například PSPACE -úplným problémům. Pokud bychom ale definovali pojem P -úplnosti s pomocí polynomiální převoditelnosti, zjistili bychom, že pak by byly P -úplné všechny netriviální úlohy v P . Za triviální považujeme problém bez pozitivních instancí, který odpovídá prázdnému jazyku, a problém bez negativních instancí, který odpovídá jazyku všech řetězců. Všechny netriviální problémy v P jsou na sebe totiž vzájemně polynomiálně převoditelné. Pro studium struktury třídy P se proto obvykle používá převoditelnost s logaritmickým meziprostorem, která omezuje pomocný prostor algoritmu pro výpočet funkce.

Pokud by se ukázalo, že nějaký NP -úplný problém patří do P , platilo by $P = \text{NP}$. Protože se domníváme, že tato rovnost neplatí, předpokládáme také, že je-li problém NP -úplný, pak nespíš nepatří do P . Přinejmenším, dosud nikdo nepřišel s algoritmem, který by rozhodoval jakýkoli NP -úplný problém v polynomiálním čase. Naše šance na nalezení polynomiálního algoritmu řešící tento problém jsou tedy nespíš velmi malé a dává nám to dobrý důvod napřít naše úsilí jiným směrem. Na druhou stranu se dá

ukázat, že je-li $P \subsetneq NP$, pak existují i jazyky, které sice nejsou NP-úplné, ale patří do $NP \setminus P$, a skutečně existují i praktické problémy, u kterých byly zatím veškeré snahy o nalezení polynomiálního algoritmu marné, ale na druhou stranu nejsme zatím schopni dokázat či vyvrátit jejich NP-úplnost².

Chceme-li o nějakém problému A ukázat, že je NP-úplný, můžeme samozřejmě postupovat podle definice a popsat, jak libovolný problém z NP polynomiálně převést na A . Takový důkaz je sice poněkud obtížný, díky tranzitivitě nám však stačí jej provést jen pro jeden problém. Máme-li už nějaký NP-úplný problém, můžeme dále využít tranzitivity polynomiální převoditelnosti a následujícího lemmatu.

Lemma 10.7 *Nechť A a B jsou jazyky patřící do třídy NP . Platí-li $A \leq_m^P B$ a je-li A NP-úplný, pak i B je NP-úplný.*

Důkaz: Tvrzení plyne přímo z tranzitivity relace $\leq_m^P$ zaručené lemmatem 10.4 a definice NP-úplného problému. $\square$

V kapitole 10.2 ukážeme, že problém splnitelnosti formule v KNF, tedy problém SAT je NP-úplný. Samotná Cookova-Levinova věta pak plyne z následující vlastnosti NP-úplných problémů.

Věta 10.8 *Patří-li nějaký NP-úplný problém do třídy P , pak $NP = P$.*

Důkaz: Uvažme problém $B \in P$, který je NP-úplný a libovolný problém $A \in NP$. Podle definice NP-úplnosti je $A \leq_m^P B$ a podle lemmatu 10.4 je $A \in P$. Tedy $NP \subseteq P$, protože současně $P \subseteq NP$, je $P = NP$. $\square$

10.2. Důkaz Cookovy-Levinovy věty

V této kapitole si ukážeme, že problém SAT je NP-úplný. Důkaz těžkosti bude vycházet z definice 10.6 NP-úplného problému. Jako důsledek NP-úplnosti problému SAT pak dostaneme Cookovu-Levinovu větu 10.2. Poznamenejme, že Cook v důkazu této věty ve skutečnosti používal Cookovu převoditelnost (viz poznámka 10.5), pojem polynomiální převoditelnosti, který používáme my, pochází od Richarda M. Karpa z [8]. Důkaz v této kapitole byl převzat z [10].

Věta 10.9 *Problém SAT je NP-úplný.*

Důkaz: Nahlédněme nejprve, že problém SAT patří do třídy NP . Polynomiální verifikátor $V(\varphi, y)$ dostane na vstupu jednak formuli φ v KNF, jednak certifikát y , který interpretuje jako ohodnocení proměnných. Verifikátor vyhodnotí φ s ohodnocením y a přijme, právě když je výsledná hodnota 1 (true).

Zbývá ukázat, že problém SAT je NP-těžký. Budeme postupovat podle definice 10.6. Uvažme jazyk $A \subseteq \Sigma^*$ z třídy NP a ukažme, že $A \leq_m^P SAT$. K tomu potřebujeme popsat

²Jde např. o rozhodnutí, zda je daná formule φ v konjunktivně normální formě ekvivalentní dané formuli ψ v disjunktivně normální formě.

funkci f vyčíslitelnou v polynomiálním čase, pro niž platí

$$(\forall w \in \Sigma^*)[w \in A \iff f(w) \in \text{SAT}].$$

Popíšeme polynomiální algoritmus, který funkci f vyčísluje. Vstupem algoritmu bude řetězec $w \in \Sigma^*$ a výstupem tohoto algoritmu bude formule φ v KNF, pro kterou bude platit

$$(\forall w \in \Sigma^*)[w \in A \iff \varphi \in \text{SAT}].$$

Při popisu algoritmu si však nemůžeme vystačit jen s řetězcem w . Je potřeba využít toho co víme o jazyku A . Tento jazyk patří do třídy NP, což znamená dle věty 8.13, že existuje jednopáskový nedeterministický Turingův stroj $M = (Q, \Sigma, \delta, q_0, F)$, který přijímá jazyk A v polynomiálním čase. Popis stroje M nebude součástí vstupu algoritmu vyčíslujícího funkci f , ale bude součástí konstrukce formule φ na základě w .

Idea konstrukce je nyní následující. Slovo w patří do A , právě M přijme w přijme. Podle definice 4.11 to je právě když existuje přijímající výpočet M se vstupem w v podobě posloupnosti konfigurací

$$C_0^w \xrightarrow{\delta} C_1 \xrightarrow{\delta} C_2 \xrightarrow{\delta} \dots \xrightarrow{\delta} C_t,$$

jejíž délka je polynomiální v w a kde C_t je přijímající konfigurace. Řekněme, že M pracuje v čase n^k pro nějaké $k \in \mathbb{N}$. Pak stačí uvažovat posloupnosti délky $t \leq n^k$. Formule φ budeme konstruovat tak, aby její modely odpovídaly právě takovým posloupnostem. Formulí φ definujeme na výrokových proměnných, každou konfiguraci tedy musíme reprezentovat pomocí množiny výrokových proměnných. Jednotlivé části formule pak budou deklarovat, jaké podmínky musí tyto proměnné splňovat, aby platilo, že reprezentují přijímající posloupnost konfigurací.

- Ohodnocení lze interpretovat jako posloupnost konfigurací.
- První konfigurace je počáteční konfigurací stroje M se vstupem w zapsaným na pásce.
- Každá konfigurace v posloupnosti následuje po předchozí přechodovou funkcí.
- Posloupnost je přijímající.

K zajištění poslední podmínky nám bude stačit, že jedna z konfigurací v posloupnosti je přijímající. To je jednodušší s ohledem na to, že n^k je pouze horní odhad na t , délky přijímajících posloupností se mohou lišit pro různé vstupy téže délky n , ale při konstrukci formule φ ani nevíme, jestli je přijímající výpočet nad w existuje, tím méně, jak je dlouhý, máme jen horní odhad délky v podobě n^k .

Z technických důvodů budeme předpokládat, že M pracuje v čase $n^k - 1$ a v prostoru $n^k - 3$ pro nějaké k a že nepohne hlavou nalevo od její počáteční pozice. Navíc budeme předpokládat, že M má jediný přijímající stav q_1 , tedy $M = (Q, \Sigma, \delta, q_0, \{q_1\})$ pro množinu stavů Q , páskovou abecedu Σ , přechodovou funkci δ , počáteční stav q_0 a přijímající stav q_1 . O stavu q_1 budeme dále předpokládat, že přechodem do tohoto stavu M přijme, i kdyby ještě z tohoto stavu vedly další přechody. Protože výpočty M mohou být různě

n^k	#	q_0	w_1	w_2	$\cdots$		w_n	λ	$\cdots$		λ	#	C_0^w	
	#												#	C_1
	#												#	C_2
	$\vdots$												$\vdots$	
	#	b	i	g	q_3	λ	e	r	λ	λ	λ	#	C_i	
	#	b	i	g	g	q_4	e	r	λ	λ	λ	#	C_{i+1}	
	$\vdots$												$\vdots$	
	#												#	C_{n^k}
n^k														

Obrázek 10.1.: Příklad tabla reprezentujícího posloupnost konfigurací $C_0^w \xrightarrow{\delta} C_1 \xrightarrow{\delta} C_2 \xrightarrow{\delta} \dots \xrightarrow{\delta} C_{n^k}$, která odpovídá výpočtu M nad vstupem w . Přejít z C_i do C_{i+1} odpovídá přechodu $(q_4, g, R) \in \delta(q_3, \lambda)$.

dlouhé pro různé vstupy, což se nám příliš nehodí, budeme také předpokládat, že M obsahuje přechod $(q_1, a, N) \in \delta(q_1, a)$ pro každý znak abecedy $a \in \Sigma$. Pokud M přejde do stavu q_1 , pak přijme, což znamená, že nás už nezajímá, co se děje dál, ale protože se nám hodí, aby M ještě stále měl co dělat, než vyprší čas $n^k - 1$, dali jsme mu možnost zůstat ve stavu q_1 beze změny konfigurace.

Výpočet stroje M se vstupem w délky $n = |w|$ si budeme reprezentovat tabulkou, které budeme říkat *tablo*. Příklad takového tabla je na obrázku 10.1. Jde o matici T o rozměrech $n^k \times n^k$. Jednotlivé řádky T reprezentují konfigurace a T tak reprezentuje posloupnost konfigurací délky n^k . První a poslední buňka každého řádku obsahuje speciální symbol $\#$. Většina ostatních buněk na řádku obsahuje nějaký symbol abecedy Σ . Takto je reprezentován obsah pásky M v dané konfiguraci. Buňka nalevo od čteného symbolu obsahuje symbol aktuálního stavu z množiny Q . Buňky T tedy obsahují symboly z abecedy

$$S = \Sigma \cup Q \cup \{\#\}.$$

Pomocí T_i budeme označovat i -tý řádek T a pomocí $T[i, j]$ budeme označovat buňku T na i -tém řádku a v j -tém sloupci pro $i, j = 1, \dots, n^k$. Abychom mohli říci, že jde o tablo, které reprezentuje výpočet, musí navíc platit, že T_i reprezentuje počáteční konfiguraci a pro $i = 1, \dots, n^k - 1$ platí, že je možné z konfigurace na řádku T_i přejít přechodovou funkcí δ do konfigurace na řádku T_{i+1} . Tablo nazveme *přijímající*, pokud reprezentuje přijímající výpočet, což znamená, že v nějaké buňce je symbol přijímajícího stavu q_1 .

Zbývá popsat formuli φ , jejíž modely budou reprezentovat přijímající tabla. Pro každou buňku tabla $T[i, j]$, $i, j = 1, \dots, n^k$ a každý symbol $s \in S$ zavedeme výrokovou proměnnou $x_{i,j,s}$, která reprezentuje fakt, že v buňce $T[i, j]$ je symbol s . Formule φ se bude skládat ze čtyř podformulí, které dále popíšeme.

$$\varphi = \varphi_{\text{cell}} \wedge \varphi_{\text{start}} \wedge \varphi_{\text{move}} \wedge \varphi_{\text{accept}}$$

Formule φ_{cell} reprezentuje fakt, že každé buňce je přiřazen právě jeden symbol.

$$\varphi_{\text{cell}} = \bigwedge_{i=1}^{n^k} \bigwedge_{j=1}^{n^k} \left[\left(\bigvee_{s \in S} x_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s,t \in S \\ s \neq t}} \neg x_{i,j,s} \vee \neg x_{i,j,t} \right) \right]$$

Klauzule $\bigwedge_{s \in S} x_{i,j,s}$ reprezentuje fakt, že alespoň jeden symbol v buňce $T[i, j]$ být musí, zatímco klauzule $\neg x_{i,j,s} \vee \neg x_{i,j,t}$ zabraňuje tomu, aby symboly s a t byly v buňce $T[i, j]$ přítomny současně, konjunkce těchto klauzulí přes všechny dvojice symbolů $s, t \in S$ tedy zaručuje, že v buňce $T[i, j]$ bude nejvýš jeden ze symbolů z S .

Formule φ_{start} zajišťuje, aby první řádek tabla reprezentoval počáteční konfiguraci pro vstup w . Toto je také jediná část formule, k jejíž konstrukci je použit vstup w . Víme přesně, jak má počáteční konfigurace vypadat, jde tedy jen o konjunkci literálů odpovídajícím symbolům na prvním řádku.

$$\varphi_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \dots \wedge x_{1,n+2,w_n} \wedge x_{1,n+3,\lambda} \wedge \dots \wedge x_{1,n^k-1,\lambda} \wedge x_{1,n^k,\#}$$

Formule φ_{accept} zajišťuje to, že tablo T je přijímající, jde jen o to, aby kdekoli v tabulce byl přítomen symbol jediného přijímajícího stavu q_1 .

$$\varphi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_1}$$

Zbývá popsat formuli φ_{move} , která má pro každý řádek T_i , $i = 1, \dots, n^k - 1$ platilo, že do následujícího řádku T_{i+1} lze přejít přechodovou funkcí δ . Využijeme toho, že změna řádku daná přechodovou funkcí je ve skutečnosti velmi lokální. Symbol stavu je vždy umístěn před symbolem pod hlavou, na řádku T_{i+1} může být posunut o jedno políčko doprava nebo doleva, kromě toho se může lišit obsah políčka, které je na řádku T_i za stavem (tedy pod hlavou). Celý efekt instrukce se tedy vejde do podmatice velikosti 2×3 , kterému budeme říkat *okno*. Pokud bychom věděli, na kterém místě na řádku T_i je umístěn symbol stavu, stačilo by se zaměřit na okno, v němž je symbol stavu nahoře uprostřed, zbytek řádku T_{i+1} se musí shodovat s řádkem T_{i+1} . My ovšem nemůžeme vědět, kde je na daném řádku hlava, ta může být kdekoli, tedy musíme mít přehled o všech oknech v tablu T . Řekneme, že okno je *přípustné*, pokud se může vyskytnout jako součást nějakého přechodu daným přechodovou funkcí δ .

Uvažme velmi jednoduchý stroj M , jehož přechodová funkce má pouze tyto přechody:

$$\begin{aligned} \delta(q_2, a) &= \{(q_4, c, L), (q_3, b, R)\} \\ \delta(q_4, b) &= \{(q_2, a, L)\} \end{aligned}$$

Podle této funkce lze vykonat následující přechod:

#	λ	λ	b	q_2	a	c	λ	λ	λ	#
#	λ	λ	q_4	b	c	c	λ	λ	λ	#

Každé z následujících oken dané tímto přechodem je tedy přípustné.

#	λ	λ								
#	λ	λ								
			λ	λ	b					
			λ	λ	q_4					
			λ	b	q_2					
			λ	q_4	b					
			b	q_2	a					
			q_4	b	c					
			q_2	a	c					
			b	c	c					
a	c	λ								
c	c	λ								
			c	λ	λ					
			c	λ	λ					
			λ	λ	λ					
			λ	λ	λ					
			λ	λ	#					
			λ	λ	#					

Známe-li přechodovou funkci, je možné ověřit, zda dané okno je přípustné. Následující okna jsou například přípustná podle výše uvedené jednoduché přechodové funkce.

#	q_2	a								
#	b	q_3								
	λ	λ	q_4							
	λ	q_2	λ							
	λ	b	c							
	λ	b	q_4							
	a	b	a							
	q_3	b	a							
	b	c	c							
	b	c	c							

Při ověření přípustnosti těchto oken jde jen o to, zda je možné přidat vlevo nebo vpravo jeden až dva sloupce tak, aby se v nich ukázal přechod dle δ . U následujících oken to nelze provést, a tedy přípustná nejsou.

#	q_2	a								
#	q_2	a								
	q_3	λ	q_4							
	λ	q_2	λ							
	λ	b	c							
	λ	c	b							
	b	a	a							
	q_2	b	a							
	a	a	q_4							
	q_2	a	a							

Na základě přechodové funkce δ můžeme tedy zkonstruovat množinu všech přípustných oken W , jejíž velikost je nejvýš $|S|^6$ a nezávisí tedy na velikosti vstupu w , jen na popisu Turingova stroje M . Z hlediska algoritmu vyčíslovací $f(w)$ je tedy velikost W konstantní.

Podstatnou vlastností přípustných oken je to, že pokud první řádek tablu reprezentuje počáteční konfiguraci a všechna okna v tablu jsou přípustná, pak každý řádek T reprezentuje konfiguraci a pro každou konfiguraci $T_i, i = 1, \dots, n^k - 1$ platí, že T_i i T_{i+1} jsou konfigurace z T_i lze do T_{i+1} přejít pomocí přechodové funkce δ . Toto tvrzení ukážeme indukcí dle i . Podle předpokladu reprezentuje T_1 počáteční konfiguraci. Předpokládejme nyní, že T_i reprezentuje konfiguraci a ukažme, že potom T_{i+1} reprezentuje konfiguraci, do které lze z T_i přejít podle přechodové funkce δ . Uvážíme postupně všechny buňky na řádku T_i a to, jak se přenesou či změní při přechodu na řádek T_{i+1} za předpokladu, že všechna okna v T jsou přípustná.

- Symboly # na okrajích jsou přeneseny na další řádek, protože to je jediná možnost pro přípustné okno.

- Je-li symbol $a \in \Sigma$ na pozici T_i , která nesousedí se symbolem stavu, pak opět bude a překopírován na další řádku beze změny, stačí uvážit, že okno, která má buňku a nahoře uprostřed musí být přípustné, a tedy okopírovat střední symbol dolů.
- Zbývají tři buňky, které jsme neuvážili, stav a buňky okolo něj. Uvažme okno, v němž je buňka nahoře uprostřed. Pak toto okno zahrnuje celý možný efekt instrukce. Protože je toto okno přípustné, tak i změna na další řádek v tomto okně musí odpovídat nějaké instrukci v přechodové funkci δ .

S touto vlastností tedy stačí popsat formuli φ_{move} tak, aby vynutila, že každé okno v T je přípustné. Připomeňme, že W označuje množinu přípustných oken a že každé okno je šestice symbolů $s_1, \dots, s_6$, kde s_1, s_2, s_3 jsou symboly na prvním řádku a s_4, s_5, s_6 jsou symboly na druhém řádku. Definujeme nejprve formuli $\text{legal}_{i,j}$, která reprezentuje fakt, že okno s levým horním rohem na buňce $T[i, j]$ je přípustné.

$$\text{legal}_{i,j} = \bigvee_{(s_1, \dots, s_6) \in W} x_{i,j,s_1} \wedge x_{i,j+1,s_1} \wedge x_{i,j+2,s_1} \wedge x_{i+1,j,s_1} \wedge x_{i+1,j+1,s_1} \wedge x_{i+1,j+2,s_1}$$

Formule $\text{legal}_{i,j}$ sice není v KNF, ale její velikost je konstantní, protože $|W| \leq |\Sigma|^6$ a $|\Sigma|$ nezávisí na vstupu x , jen na Turingovu stroji M . Můžeme tedy použít distributivitu $\wedge$ a $\vee$ pro převod $\text{legal}_{i,j}$ do KNF konstantní velikosti. S tímto pak stačí definovat formuli φ_{move} jako konjunkci $\text{legal}_{i,j}$ pro všechna okna v tablu.

$$\varphi_{\text{move}} = \bigwedge_{i=1}^{n^k-1} \bigwedge_{j=1}^{n^k-2} \text{legal}_{i,j}$$

Výsledkem je formule φ , jejíž modely přesně odpovídají přijímajícím tablům, a tedy i přijímajícím výpočtům stroje M nad vstupem w . Z toho plyne, že φ je splnitelná, právě když $M(w)$ přijme, tedy právě když $w \in A$. Velikost φ je navíc polynomiální v $n = |x|$. Ukázali jsme tedy, že $A \leq_m^p \text{SAT}$. Dohromady dostáváme, že problém **SAT** je NP-úplný. $\square$

Cookova-Levinova věta nyní plyne z věty 10.8.

Věta 10.2

10.3. *** Alternativní důkaz Cookovy-Levinovy věty ***

V této kapitole si ukážeme důkaz Cookovy-Levinovy věty 10.2, který je složen ze dvou převodů. První převod ukazuje NP-úplnost problému **KACHLÍKOVÁNÍ**, druhý pak ukazuje polynomiální převod **KACHLÍKOVÁNÍ** na problém **SAT**.

Problém **KACHLÍKOVÁNÍ** definujeme následujícím způsobem.

Problém 10.10: KACHLÍKOVÁNÍ (KACHL, ANGLICKY TILING)

Instance: Množina barev B , přirozené číslo s , čtvercová síť S velikosti $s \times s$, hrany jejichž krajních políček jsou obarveny barvami z B . Dále je součástí instance množina $K \subseteq B \times B \times B \times B$ s typy kachlíků, které odpovídají čtverci, jehož hrany jsou obarveny barvami z B . Tyto kachlíky mají přesně definovaný horní, dolní, levý i pravý okraj a není možné je otáčet.

Otázka: Existuje přípustné vykachlíkování čtvercové sítě S kachlíky, jejichž typy jsou v množině K ? Přípustné vykachlíkování je takové přiřazení typů kachlíků jednotlivým polím čtvercové sítě S , v němž kachlíky, které spolu sousedí mají touž barvu na vzájemně dotýkajících se hranách a kachlíky, které se dotýkají strany S , mají shodnou barvu s okrajem. Jednotlivé typy kachlíků lze použít vícekrát.

Věta 10.11 *KACHL* je NP-úplný problém.

Důkaz: Všimněme si nejprve, že $KACHL \in NP$. Polynomiální verifikátor pro tento problém dostane jako certifikát konkrétní přiřazení kachlíků jednotlivým políčkům čtvercové sítě S a ověří v polynomiálním čase, je-li toto přiřazení přípustným vykachlíkováním S .

Nechť $A \subseteq \Sigma^*$ je libovolný problém, který patří do NP, ukážeme, že $A \leq_m^p KACHL$. Podle definice NP-úplného problému to bude znamenat, že *KACHL* je NP-těžký, a protože patří do NP, tak i NP-úplný problém. To, že $A \in NP$, podle věty 8.13 znamená, že existuje NTS M , který přijímá A (tj. $A = L(M)$), a počet kroků každého přijímajícího výpočtu je omezen polynomem $p(n)$, bez újmy na obecnosti můžeme předpokládat, že $p(n) \geq n$, v opačném případě by M nepřečetl ani celý svůj vstup a pokud by platilo, že $p(n) \leq n$, stačí vzít $\max\{p(n), n\}$ místo $p(n)$. Připomeňme si, že podle definice $x \in A$, právě když existuje přijímající výpočet NTS M nad vstupem x , který má délku nejvýš $p(|x|)$. Nechť $M = (Q, \Sigma, \delta, q_0, F)$, kde Q obsahuje stavy q_0 a q_1 . Abychom si zjednodušili situaci, budeme předpokládat, že M splňuje následující předpoklady:

1. $F = \{q_1\}$, tj. M má jediný přijímající stav q_1 různý od q_0 .
2. Pro každé $a \in \Sigma$ je $\delta(q_1, a) = \emptyset$, tj. z přijímajícího stavu neexistuje definovaný přechod.
3. Počáteční konfigurace vypadá tak, že hlava stojí na nejlevějším symbolu vstupního slova x , které je zapsáno počínaje od levého okraje vymezeného prostoru délky $p(|x|)$. Zbytek pásky je prázdný.
4. Během výpočtu se hlava M nepohne nalevo od místa, kde byla v počáteční konfiguraci, tj. mimo vymezený prostor.

5. Přijímající konfigurace je daná jednoznačně a vypadá tak, že páska je prázdná a hlava stojí na nejlevější pozici vymezeného prostoru. To odpovídá tomu, že než se M rozhodne přijmout, smaže nejprve obsah pásky a přesune hlavu k levému okraji vymezeného prostoru.

Není těžké ukázat, že ke každému NTS M_1 lze zkonstruovat NTS M_2 , který přijímá týž jazyk jako M_1 , „dělá totéž“ a splňuje uvedené podmínky. Většinu zmíněných předpokladů jsme již dříve použili, například při konstrukci univerzálního TS. Bez újmy na obecnosti tedy můžeme předpokládat, že M splňuje uvedené podmínky.

Nechť x je instance problému A , popíšeme, jak z M , polynomu p a instance x vytvořit instanci Kachlíkování, pro kterou bude platit, že v ní existuje přípustné vykachlíkování, právě když existuje přijímající výpočet $M(x)$, tj. $M(x)$ přijme.

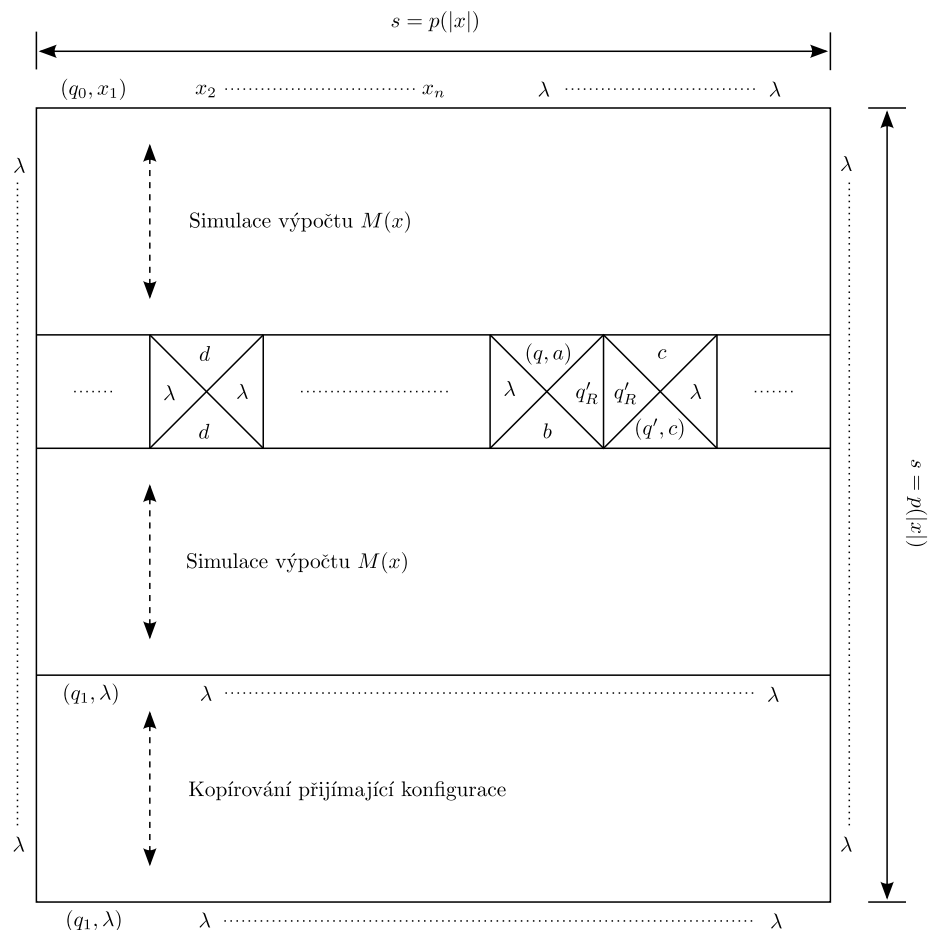
Idea důkazu je taková, že hrany barev mezi dvěma řádky kachlíků budou kódovat konfigurace výpočtu NTS M nad vstupem x . Vhodným výběrem kachlíků zabezpečíme, že v přípustném vykachlíkování bude řada kachlíků simulovat změnu konfigurace na následující pomocí přechodové funkce. Horní a dolní okraje sítě S obarvíme tak, aby barvy určovaly počáteční a přijímající konfiguraci, obě jsou dané jednoznačně, přijímající zcela jednoznačně, počáteční je sice závislá na x , ale pro dané x je již jednoznačná. Konstrukce je ilustrovaná na obrázku 10.2. Barvy kachlíků tedy budou odpovídat symbolům, které potřebujeme pro zakódování konfigurace, ale budeme potřebovat i pomocné barvy pro přenos informace o stavu o kachlík vlevo nebo vpravo. Položíme tedy

$$B = \Sigma \cup Q \times \Sigma \cup \{q_L, q_R \mid q \in Q\}$$

Význam těchto barev se ozřejmí při konstrukci jednotlivých typů kachlíků. V dalším textu budeme označovat políčko sítě S na i -tém řádku a v j -tém sloupci pomocí $S[i, j]$, kde $i, j \in \{1, \dots, s\}$. Jak jsme zmínili, cílem je, aby řada barev mezi dvěma řadami kachlíků odpovídala konfiguraci. Barva (q, a) v této konfiguraci bude kódovat políčko na pásce, nad kterým se vyskytuje hlava, přičemž q označuje stav, ve kterém se M nachází, ostatní políčka konfigurace budou obarvena barvou odpovídající symbolu na daném místě.

Velikost jedné strany čtvercové sítě položíme $s = p(|x|)$, všimněme si, že $p(|x|)$ omezuje nejen počet konfigurací v přijímajícím výpočtu, ale i počet buněk, které stihne M při výpočtu nad x popsat, tedy i délku slova na pásce v konfiguraci.

Popíšeme nyní, jakými barvami budou obarveny okraje čtvercové sítě S . Boční strany budou obarveny barvou λ , horní strana bude obarvena počáteční konfigurací a dolní strana konfigurací přijímající. Přesněji, nechť $x = x_1 \dots x_n$, kde $x_i \in \Sigma$ pro $i = 1, \dots, n$. Pak horní hrana nejlevější horní buňky $S[1, 1]$ bude obarvena barvou (q_0, x_1) , přičemž je-li vstup x prázdný, tj. $n = |x| = 0$, pak je obarvena barvou (q_0, λ) . Horní strany políček $S[1, 2], \dots, S[1, n]$ jsou obarveny popořadě symboly $x_2, \dots, x_n$. Horní hrany zbylých políček prvního řádku $S[1, n+1], \dots, S[1, s]$ budou obarveny barvou λ , odpovídající prázdnému políčku. Levé hrany políček v prvním sloupci budou obarveny barvou λ , stejně jako pravé hrany políček v s -tém sloupci. Spodní hrana sítě S bude obarvena jednoznačnou přijímající konfigurací, tedy spodní hrana levého dolního políčka $S[s, 1]$ bude mít barvu (q_1, λ) , další políčka dolního řádku $S[s, 2], \dots, S[s, s]$ budou mít spodní hrany obarveny barvou λ .

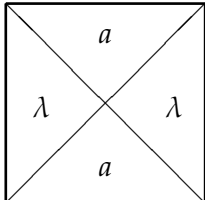


Obrázek 10.2.: Přehled převodu obecného problému $A \in \text{NP}$ na problém **KACHL**. Horní hrana čtvercové sítě je obarvena počáteční konfigurací, spodní hrana přijímající konfigurací, boky jsou obarveny symbolem λ . Pro ukázkou je zde řádek kachlíků s provedením instrukce $(q', b, R) \in \delta(q, a)$, dva kachlíky slouží k provedení instrukce, na ostatních místech je jen kopírovací kachlík pro okopírování barev na další řádek. Po ukončení výpočtu je dokopírována přijímající konfigurace až ke spodní hraně čtvercové sítě.

Do typů kachlíků zakódujeme přechodovou funkci Turingova stroje M , čímž dosáhneme toho, že správné vykachlíkování řádků 2 až s bude odpovídat výpočtu M nad x . Navíc přidáme možnost kopírování přijímající konfigurace tak, abychom ošetřili i případ, kdy výpočet M nad x skončí po méně než $p(|x|)$ krocích.

Pro každý symbol $a \in \Sigma$ nejprve přidáme typ kachlíku:

(I)



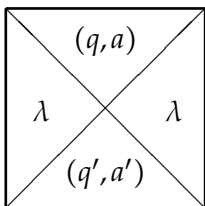
Tento kachlík bude odpovídat těm místům konfigurace, která přechodová funkce nemění, protože se nad nimi nevyskytuje hlava v předchozí ani v následující konfiguraci.

Pro každý stav $q \in Q$ a každý znak $a \in \Sigma$ nyní popíšeme kachlíky, které je nutno přidat, abychom zabezpečili provedení možných přechodů daných funkcí $\delta(q, a)$. Připomeňme, že místo na pásce, kde je hlava, označujeme dvojicí (q, a) , kde q je stav, v němž se M nachází a a je čtený symbol. Volbou kachlíků musíme zabezpečit jednak správné provedení instrukce, jednak to, aby ve správném vykachlíkování byla v každé konfiguraci právě jedna dvojice (q, a) , musíme si tedy dát pozor, aby se kachlíky odpovídající jednotlivým instrukcím nepomíchalý mezi sebou.

Pokud $\delta(q, a) = \emptyset$, nepřidáme přirozeně nic.

Pro každý stav $q' \in Q$ a znak $a' \in \Sigma$, pro které platí, že $(q', a', N) \in \delta(q, a)$, tj. z (q, a) lze přejít do stavu q' s přepsáním symbolu a na a' a s tím, že hlava zůstane na místě, přidáme kachlík:

(II)

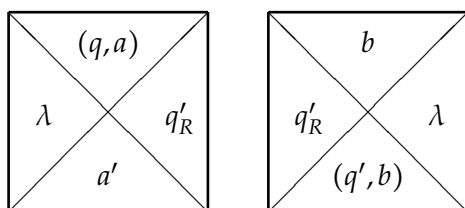


To odpovídá tomu, že pokud nevykonáváme žádný pohyb, pouze změníme stav a přepíšeme symbol.

Pro každý stav $q' \in Q$ a znak $a' \in \Sigma$, pro které platí, že $(q', a', R) \in \delta(q, a)$, tj. z (q, a) lze přejít do stavu q' s přepsáním symbolu a na a' a s tím, že se hlava pohne vpravo, přidáme pro každé $b \in \Sigma$ kachlíky:

(III)

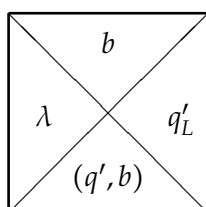
(IV)



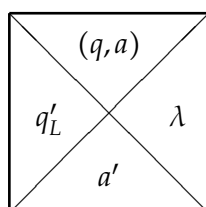
To odpovídá tomu, že při pohybu doprava přesuneme hlavu o jednu buňku vpravo, stav si zapamatujeme v boční hraně. Pro přenesení stavu přitom využijeme barvu q'_R s indexem R , zapamatujeme si tedy i směr pohybu, a to proto, aby se nám nepomíchaly dvojice kachlíků pro pohyb doleva a pohyb doprava.

Podobně pro každý stav $q' \in Q$ a znak $a' \in \Sigma$, pro které platí, že $(q', a', L) \in \delta(q, a)$, tj. z (q, a) lze přejít do stavu q' s přepsáním symbolu a na a' a s tím, že se hlava pohne vlevo, přidáme pro každé $b \in \Sigma$ kachlíky:

(V)



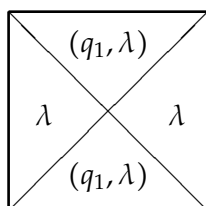
(VI)



Tím, že nyní používáme verzi q'_L stavu q' , nepomíchají se nám přechody doprava a doleva mezi sebou.

Výpočet stroje M nemusí skončit přesně po $s = p(n)$ krocích a mohlo by se tedy stát, že bychom s dosud uvedenými typy kachlíků nemohli dokachlíkovat zbylé řady čtvercové sítě S po ukončení výpočtu. Za tím účelem přidáme kachlík umožňující spolu s (I) pro $a = \lambda$ okopírování přijímající konfigurace. V této konfiguraci je páska prázdná a i čteným symbolem je prázdný znak λ , stačí nám tedy přidat následující kopírovací kachlík:

(VII)



Protože z přijímajícího stavu q_1 nevede v M žádný přechod, v okamžiku, kdy začneme tento stav kopírovat tímto typem kachlíku, musíme s tím vytrvat až do posledního řádku.

Funkce f , která bude převádět instanci problému A na instanci problému **KACHL** provede právě popsanou konstrukci, tj. z popisu M a instance x vytvoří instanci (B, K, s, S) , kde množina K obsahuje popsané typy kachlíků a pod S míníme obarvení okrajů sítě. Tuto konstrukci je zřejmě možné provést v polynomiálním čase. Všimněme si, že veli-

kost reprezentace M je nezávislá na velikosti instance A a jde tedy o konstantu, jediné co je v konstrukci závislé na velikosti vstupu je tedy rozměr sítě $s = p(|x|)$ a obarvení horního okraje S počáteční konfigurací.

Zbývá ukázat, že $x \in A$, právě když má takto zkonstruovaná instance prefprob:kachl-KACHL přípustné vykachlíkování.

Předpokládejme nejprve, že $x \in A$, to znamená podle definice, že existuje přijímající výpočet $M(x)$ daný posloupností konfigurací $K_0^x = K_0, \dots, K_t = K_F$, kde K_0^x je počáteční konfigurace M při výpočtu nad x a K_F je jednoznačně daná přijímající konfigurace. Podle předpokladu platí, že $t \leq p(|x|)$ a délka slova na pásce v každé konfiguraci je rovněž nejvýš $p(|x|)$. Popíšeme, jak s pomocí konfigurací $K_0, \dots, K_t$ obarvit síť S . Intuitivně je jasné, jak toto vykachlíkování bude vypadat, máme-li vykachlíkovaných i řad, kde $i \in \{0, \dots, t-1\}$ s tím, že barvy na spodní hraně řady i odpovídá konfiguraci K_i (je-li $i = 0$, pak jde o barvy horního okraje S , víme, že ty odpovídají $K_0^x = K_0$, základní krok je tedy splněn), vykachlíkujeme $(i+1)$ -ní řadu s odsimulováním příslušné instrukce, která vedla k přechodu z K_i do K_{i+1} . Takto se dostaneme k $K_t = K_F$ a poté dokopírujeme K_F až na poslední řádek čtvercové sítě S . Inspekci jednotlivých kachlíků bychom ověřili, že instrukci lze vždy odsimulovat, většina kachlíků by byla typu (I), tedy kopírovacích na místech beze změny, v místě provedení instrukce bychom použili odpovídající kachlík (II), dvojici (III) a (IV), nebo dvojici (V) a (VI). Horní barva kachlíku (III) nebo (V) je daná tím, co se v K_i na příslušném místě nachází za znak. Kopírování K_F do konce (je-li třeba), pak zabezpečí kachlík (VII) ve spolupráci s (I) pro $a = \lambda$. Detailní rozbor případů ponecháme na čtenáři.

Nyní předpokládejme, že existuje přípustné vykachlíkování čtvercové sítě S . Potřebujeme ukázat, že řádky barev mezi jednotlivými řádky kachlíků určují posloupnost konfigurací v přijímajícím výpočtu M nad x . Postupujme opět indukcí podle $i = 0, \dots, s$, nechť $b_{i,1}, \dots, b_{i,s} \in B$ je posloupnost barev mezi i -tým a $(i+1)$ ním řádkem vykachlíkování čtvercové sítě S , přičemž je-li $i = 0$, označují barvy $b_{i,1}, \dots, b_{i,s}$ horní barvy S a je-li $i = s$, označují tyto barvy spodní barvy S . Musíme ukázat tyto dvě vlastnosti pro každé $i = 0, \dots, s$:

1. V posloupnosti $b_{i,1}, \dots, b_{i,s}$ je právě jedna barva typu $(q, a) \in Q \times \Sigma$, ostatní jsou typu $a \in \Sigma$, tj. posloupnost $b_{i,1}, \dots, b_{i,s}$ určuje konfiguraci K_i .
2. Pro takto určené konfigurace platí, že K_i lze z K_{i-1} vytvořit přechodem pomocí přechodové funkce δ pro $i > 0$ a $K_0 = K_0^x$.

Obě vlastnosti jsou jistě splněné pro $i = 0$. Předpokládejme nyní, že jsou splněny pro $i \in \{0, \dots, s\}$ a ukažme, že platí pro $i+1$. Ve skutečnosti obě vlastnosti opět jednoduše plynou z toho, jaké kachlíky máme k dispozici. Podle indukčního předpokladu se na řádku barev $b_{i,1}, \dots, b_{i,s}$ vyskytuje právě jedna pozice, řekněme k , pro kterou platí, že $b_{i,k} = (q, a)$ pro nějaký stav $q \in Q$ a znak $a \in \Sigma$. To znamená, že v $(i+1)$ -ním řádku kachlíků je právě jeden kachlík s horní barvou (q, a) , a to na pozici $S[i+1, k]$, ostatní mají horní barvu typu $c \in \Sigma$. Rozborem případů ověříme, že obě požadované vlastnosti jsou splněné i pro řádek barev $b_{i+1,1}, \dots, b_{i+1,s}$. Předně si všimněme, že kachlíky nám

neumožňují vytvořit barvu (q, a) z ničeho, ale musí jít o barvu tohoto typu převedenou z předchozího řádku, kachlíky se spodní barvou (q', b) , které nahoře nemají barvu (q, a) (tj. kachlíky typu (IV) nebo (V)) totiž vyžadují aby nalevo nebo napravo od nich byl odpovídající kachlík s barvou (q, a) nahoře a barvou a' dole (tj. kachlíky typu (III) a (VI)). To znamená, že pokud se má někde barva (q, a) objevit, musela vedle zmizet, díky tomu zůstane zachovaná první vlastnost. Tento přenos však vždy odpovídá provedení instrukce díky tomu, jak jsme kachlíky definovali, i druhá vlastnost zůstane tedy zachovaná. Tím, že spodní hrana S je obarvena přijímající konfigurací, je vynuceno, že poslední řada barev kachlíků bude odpovídat přijímající konfiguraci, ta se může kopírovat nahoru, ale v každém případě musí platit, že poslední konfigurace v posloupnosti konfigurací odpovídajícím barvám mezi řadami kachlíků musí být přijímající. Formální rozbor případů ponecháme na čtenáři.

Dostáváme tedy, že posloupnost konfigurací daných barvami na hranách mezi řádky kachlíků v čtvercové síti S odpovídají přijímajícímu výpočtu M nad vstupem x a jde dokonce o vzájemnou korespondenci, a tedy máme-li přípustné vykachlíkování, znamená to, že $M(x)$ přijme. Z toho plyne, že $x \in A$.

Tímto jsme tedy dokončili převod libovolného problému $A \in \text{NP}$ na Kachlíkování a protože Kachlíkování patří do NP, znamená to, že Kachlíkování je NP-úplný problém. $\square$

Nyní můžeme přistoupit k důkazu toho, že problém **KACHL** je polynomiálně převoditelný na problém **SAT**.

Věta 10.12 Platí, že $\text{KACHL} \leq_m^p \text{SAT}$.

Důkaz: Uvažme instanci Kachlíkování danou množinou barev B , přirozeným číslem s , čtvercovou sítí S velikosti $s \times s$, jejíž okraje jsou obarveny barvami z B , a množinou typů kachlíků $K = \{T_1, \dots, T_{|K|}\}$. Popíšeme, jak zkonstruovat formuli φ v KNF, která bude splnitelná právě tehdy, když tato instance má přípustné vykachlíkování.

Formule φ bude využívat $s^2 \cdot |K|$ proměnných $x_{i,j,k}$ pro $i = 1, \dots, s$, $j = 1, \dots, s$ a $k = 1, \dots, |K|$. Konstrukcí formule φ zajistíme, že ve splňujícím ohodnocení bude hodnota $x_{i,j,k} = 1$ odpovídat tomu, že na políčko $S[i, j]$ umístíme kachlík T_k . V konstrukci použijeme následující dvě množiny určující nekompatibilní dvojice kachlíků:

$$\begin{aligned} V &= \{(p, q) \mid \text{spodní barva } T_p \text{ se neshoduje s horní barvou } T_q\} \\ H &= \{(p, q) \mid \text{pravá barva } T_p \text{ se neshoduje s levou barvou } T_q\} \end{aligned}$$

tj. množina V obsahuje dvojice typů, jež nemohou být umístěny nad sebe, tedy vertikálně. Množina H obsahuje dvojice typů, jež nemohou být umístěny vedle sebe, tedy horizontálně. Podobné množiny si definujeme i pro barvy na okrajích, pro $i = 1, \dots, s$

definujeme množiny U_i , B_i , L_i a R_i .

$U_j = \{k \mid \text{horní barva } T_k \text{ se shoduje s barvou horního okraje } S \text{ v } j\text{-tém sloupci}\}$

$B_j = \{k \mid \text{spodní barva } T_k \text{ se shoduje s barvou spodního okraje } S \text{ v } j\text{-tém sloupci}\}$

$L_j = \{k \mid \text{levá barva } T_k \text{ se shoduje s barvou levého okraje } S \text{ na } j\text{-tém řádku}\}$

$R_j = \{k \mid \text{pravá barva } T_k \text{ se shoduje s barvou pravého okraje } S \text{ na } j\text{-tém řádku}\}$

Množina U_i tedy obsahuje typy kachlíků, jež mohou být umístěny na políčko $S[1, i]$, množina B_i obsahuje typy kachlíků, jež mohou být umístěny na políčko $S[s, i]$, množina L_i obsahuje typy kachlíků, jež mohou být umístěny na políčko $S[i, 1]$, a konečně R_i obsahuje typy kachlíků, jež mohou být umístěny na políčko $S[i, s]$.

Konstrukci formule φ popíšeme po částech.

(I) Pro $i, j = 1, \dots, s$ definujeme klauzuli

$$C_{i,j} = \bigvee_{k=1}^{|K|} x_{i,j,k}.$$

Klauzule $C_{i,j}$ je splněna, pokud je na políčku $S[i, j]$ přiřazen alespoň jeden typ kachlíku, čili když existuje alespoň jedno $k \in \{1, \dots, |K|\}$, pro něž je $x_{i,j,k} = 1$.

(II) Pro $i, j = 1, \dots, s$ definujeme

$$\alpha_{i,j} = \bigwedge_{k=1}^{|K|} \bigwedge_{l=k+1}^{|K|} (\neg x_{i,j,k} \vee \neg x_{i,j,l}).$$

Formule $\alpha_{i,j}$ je splněna, je-li políčku $S[i, j]$ přiřazen nejvýš jeden typ kachlíku, čili pokud nejvýš pro jedno $k \in \{1, \dots, |K|\}$ platí $x_{i,j,k} = 1$. Pokud by totiž pro dva různé indexy $k, l \in \{1, \dots, |K|\}$ platilo $x_{i,j,k} = x_{i,j,l} = 1$, nebyla by klauzule $(\neg x_{i,j,k} \vee \neg x_{i,j,l})$ splněná.

(III) Pro $i, j = 1, \dots, s$ definujeme

$$\beta_{i,j} = C_{i,j} \wedge \alpha_{i,j}.$$

Formule $\beta_{i,j}$ je tedy splněna, pokud políčku $S[i, j]$ přiřadíme právě jeden typ kachlíku, čili pokud existuje právě jeden index $k \in \{1, \dots, |K|\}$, pro který platí $x_{i,j,k} = 1$.

(IV) Pro $1 \leq i \leq s-1, 1 \leq j \leq s$ definujeme formuli

$$\gamma_{i,j} = \bigwedge_{(p,q) \in V} (\neg x_{i,j,p} \vee \neg x_{i+1,j,q}).$$

Formule $\gamma_{i,j}$ je splněna, pokud nad sebou umístěným políčkům $S[i, j]$ a $S[i+1, j]$ nejsou přiřazeny nekompatibilní typy kachlíků.

(V) Pro $1 \leq i \leq s, 1 \leq j \leq s-1$ definujeme formuli

$$\delta_{i,j} = \bigwedge_{(p,q) \in H} (\neg x_{i,j,p} \vee \neg x_{i,j+1,q}).$$

Formule $\delta_{i,j}$ je splněna, pokud vedle sebe umístěným políčkům $S[i, j]$ a $S[i, j+1]$ nejsou přiřazeny nekompatibilní typy kachlíků.

(VI) Definujeme formuli

$$\varepsilon_u = \bigwedge_{j=1}^s \left(\bigvee_{k \in U_j} x_{1,j,k} \right).$$

Formule ε_u je splněna, pokud je na každém políčku v prvním řádku kachlík s barvou shodnou s příslušným horním okrajem.

(VII) Definujeme formuli

$$\varepsilon_b = \bigwedge_{j=1}^s \left(\bigvee_{k \in B_j} x_{s,j,k} \right).$$

Formule ε_b je splněna, pokud je na každém políčku v nejspodnějším řádku kachlík s barvou shodnou s příslušným spodním okrajem.

(VIII) Definujeme formuli

$$\varepsilon_l = \bigwedge_{j=1}^s \left(\bigvee_{k \in L_j} x_{j,1,k} \right).$$

Formule ε_l je splněna, pokud je na každém políčku v nejlevějším sloupci kachlík s barvou shodnou s příslušným levým okrajem.

(IX) Definujeme formuli

$$\varepsilon_r = \bigwedge_{j=1}^s \left(\bigvee_{k \in R_j} x_{j,s,k} \right).$$

Formule ε_r je splněna, pokud je na každém políčku v nejpravějším sloupci kachlík s barvou shodnou s příslušným pravým okrajem.

S pomocí výše definovaných formulí nyní definujeme formuli φ jako

$$\varphi = \bigwedge_{i=1}^s \bigwedge_{j=1}^s \beta_{i,j} \wedge \bigwedge_{i=1}^{s-1} \bigwedge_{j=1}^s \gamma_{i,j} \wedge \bigwedge_{i=1}^s \bigwedge_{j=1}^{s-1} \delta_{i,j} \wedge \varepsilon_u \wedge \varepsilon_b \wedge \varepsilon_l \wedge \varepsilon_r.$$

Z konstrukce okamžitě vyplývá, že φ je formule v KNF a že velikost φ je polynomiální v s a $|K|$, a v polynomiálním čase lze φ i zkonstruovat. Zbývá ukázat, že pro výchozí instanci problému Kachlíkování existuje přípustné vykachlíkování, právě když φ je splnitelná formule. Ve skutečnosti ukážeme, že ze splňujícího ohodnocení formule φ lze vyčíst přípustné vykachlíkování čtvercové sítě S a naopak z přípustného vykachlíkování lze určit splňující ohodnocení formule φ .

Předpokládejme nejprve, že existuje přípustné vykachlíkování S kachlíky z K . Označme pomocí $S[i, j]$ index typu kachlíku, který je na i -tém řádku a j -tém sloupci. Definujme ohodnocení, které pro každé $i, j = 1, \dots, s$ a $k = 1, \dots, |K|$ přiřadí proměnné $x_{i,j,k}$ hodnotu

$$x_{i,j,k} = \begin{cases} 1 & \text{pokud je na políčku } S[i, j] \text{ kachlík typu } T_k \\ 0 & \text{jinak} \end{cases}$$

Není těžké ověřit, že každá z podformulí $\beta_{i,j}, \gamma_{i,j}, \delta_{i,j}, \varepsilon_u, \varepsilon_b, \varepsilon_l$ a ε_r je tímto ohodnocením splněna, a tedy že i celá formule φ je splněna.

Předpokládejme na druhou stranu, že existuje splňující ohodnocení formule φ a pro $i, j = 1, \dots, s$ a $k = 1, \dots, |K|$ pomocí $v[i, j, k]$ označme hodnotu přiřazenou proměnné $x_{i,j,k}$ ve splňujícím ohodnocení. Díky tomu, že ohodnocení splňuje i formuli $\beta_{i,j}$, existuje pro každé $i, j = 1, \dots, s$ právě jedna hodnota $k \in \{1, \dots, |K|\}$, pro kterou je $v[i, j, k] = 1$. Na pozici $S[i, j]$ tedy dáme kachlík toho typu T_k , pro který je $v[i, j, k] = 1$. Lze snadno ověřit, že podmínky zakódované do formulí $\gamma_{i,j}, \delta_{i,j}, \varepsilon_u, \varepsilon_b, \varepsilon_l$ a ε_r zaručují, že tímto způsobem obdržíme přípustné vykachlíkování S . $\square$

11. Další NP-úplné problémy

V této kapitole si ukážeme důkazy NP-úplnosti dalších problémů. Půjde o splnitelnost logické formule v KNF (**SAT**), splnitelnost logické formule složené z klauzulí délky 3 (**3-SAT**), vrcholové pokrytí grafu (**VP**), hamiltonovskou kružnici v grafu (**HK**), trojrozměrné párování (**3DM**) a loupežníky (**LOUP**), důkazy těžkosti všech těchto problémů se objevilo již v Karpově článku [8]. Zmíníme i několik variant těchto problémů, důkazy jejichž těžkosti ponecháme jako cvičení. Účelem této části a ukázky těchto převodů je sžít se s konceptem převodu. Současně je naším cílem seznámit se s různorodými převody a problémy v naději, že znalost obojího může pomoci čtenáři k případnému důkazu těžkosti problému, jenž se může objevit v praxi. Pokud narazíme na problém, u nějž máme podezření, že by mohl být NP-těžký, měla by naše cesta směřovat k nějakému seznamu NP-úplných úloh, v němž bychom se měli snažit nalézt problém tomu našemu podobný. V takovém případě se nám může stát, že nalezneme přímo nějakou formu problému, který nás zajímá. Jako zdroj řady známých NP-úplných problémů může sloužit seznam v [5].

11.0.1. Splnitelnost formulí v 3-KNF (3-SAT)

V případě, že se nějaký problém, například splnitelnost, ukáže být NP-úplným, můžeme se ptát, jak bychom museli omezit zadání, abychom dostali jednodušší úlohu. Už splnitelnost formulí v KNF je vlastně jen zvláštním případem obecné splnitelnosti, v níž se zajímáme o splnitelnost obecné formule zadané v libovolném tvaru. Zůstaneme-li u KNF, můžeme tuto formu dále omezovat například tak, že budeme připouštět jen krátké klauzule. Řekneme, že formule φ je v 3KNF, je-li φ v KNF a každá klauzule φ obsahuje právě tři literály.

Problém 11.1: SPLNITELNOST FORMULE V 3KNF (3-SAT)

Instance: Formule φ na n proměnných v 3KNF, tj. konjunktivně normální formě, v níž každá klauzule obsahuje právě tři literály.

Otázka: Existuje ohodnocení proměnných $t \in \{0, 1\}^n$, pro které platí $\varphi(t) = 1$? (tj. ohodnocení, pro které je φ splněná).

Často se v definici problému **3-SAT** vyžaduje, aby v každé klauzuli byly nejvýše tři literály, to, že my vyžadujeme, aby byla každá klauzule složena z právě tří literálů, se nám bude hodit v dalších převodech. Ukážeme si, že i tento problém je stále NP-úplný. Použití trojky není náhodné, protože analogicky definovaný problém **2SAT** je už

polynomiálně řešitelný.

Věta 11.2 Problém 3-SAT je NP-úplný.

Důkaz: Problém 3-SAT je jen zvláštním případem problému SAT a z téhož důvodu jako v případě problému SAT, patří i 3-SAT do třídy NP. Těžkost problému 3-SAT ukážeme převodem z problému SAT. Nechť ψ je formule v KNF, vytvoříme z ní formuli φ , v níž každá klauzule bude obsahovat tři literály a pro níž bude platit, že ψ je splnitelná tehdy a jen tehdy, když φ je splnitelná.

Předpokládejme, že $\psi = C_1 \wedge \dots \wedge C_m$. Uvažme libovolnou klauzuli $C_j = (e_1 \vee \dots \vee e_k)$, kde $e_1, \dots, e_k$ jsou literály (pozitivní či negativní). Tuto klauzuli nahradíme formulí v KNF α_j složenou z klauzulí o třech literálech, která bude obsahovat literály z C_j a k nim přidané nové literály, které se budou vyskytovat jen v α_j . Tato formule bude mít následující vlastnost. Je-li t ohodnocení splňující C_j , je možné ohodnotit přidané proměnné tak, aby i α_j byla splněna. Je-li naopak t' ohodnocení splňující α_j , pak t' splňuje i C_j . Podle délky klauzule, tedy hodnoty k , rozlišíme tyto případy:

- Pokud $k = 1$, pak nechť y_1^j, y_2^j jsou nové proměnné, které se nevyskytují v ψ , s nimiž definujeme

$$\alpha_j = (e_1 \vee y_1^j \vee y_2^j)(e_1 \vee y_1^j \vee \neg y_2^j)(e_1 \vee \neg y_1^j \vee y_2^j)(e_1 \vee \neg y_1^j \vee \neg y_2^j).$$

- Pokud $k = 2$, pak nechť y^j je nově přidaná proměnná, jež se nevyskytuje v ψ a definujeme

$$\alpha_j = (e_1 \vee e_2 \vee y^j)(e_1 \vee e_2 \vee \neg y^j).$$

- Pokud $k = 3$, pak definujeme

$$\alpha_j = C_j.$$

- Pokud $k > 3$, pak nechť $y_1^j, \dots, y_{k-3}^j$ jsou nově přidané proměnné, které se nevyskytují v ψ , s nimiž definujeme

$$\alpha_j = (e_1 \vee e_2 \vee y_1^j)(\neg y_1^j \vee e_3 \vee y_2^j)(\neg y_2^j \vee e_4 \vee y_3^j) \dots (\neg y_{k-3}^j \vee e_{k-1} \vee e_k).$$

Formuli φ nyní definujeme jako

$$\varphi = \bigwedge_{j=1}^m \alpha_j.$$

Z konstrukce okamžitě plyne, že formule φ je v KNF a navíc každá její klauzule obsahuje právě tři literály, navíc má φ polynomiální velikost vzhledem k velikosti ψ a lze ji i v polynomiálním čase zkonstruovat. Zbývá ukázat, že φ je splnitelná, právě když ψ je splnitelná.

Předpokládejme nejprve, že φ je splnitelná, nechť v je její splňující ohodnocení a nechť v' označuje ohodnocení proměnných ψ , jež jim přiřazuje touž hodnotu jako v , tj. pro každou proměnnou x původní formule ψ položíme $v'(x) := v(x)$. Nechť C_j je libovolná klauzule ψ , jí odpovídající formule α_j je ohodnocením v splněná. Nechť $C_j = (e_1 \vee \dots \vee e_k)$, ukážeme, že C_j je ohodnocením v' splněná, dohromady tedy dostaneme, že $\psi(v) = 1$. Podle délky klauzule C_j , tedy hodnoty k , rozlišíme tyto případy:

- Pokud $k = 1$, pak jedna z klauzulí formule α_j musí být splněna díky e_1 , tedy $v(e_1) = 1$. To proto, že dohromady obsahují klauzule α_j všechny možné kombinace literálů obsahujících y_1^j a y_2^j , v jedné z těchto kombinací se musí stát, že ohodnocení v vyhodnotí oba literály s y_1^j a y_2^j jako 0. Protože i klauzule, jež obsahuje tuto kombinaci, musí být splněna, platí, že $v(e_1) = 1$ (pokud je e_1 negativní, tak ohodnocení příslušné proměnné je ve skutečnosti 0).
- Pokud $k = 2$, pak opět jedna z klauzulí formule α_j musí vynutit, že $v(e_1) = 1$ nebo $v(e_2) = 1$, protože dohromady obsahují y^j i $\neg y^j$, přičemž tyto dva literály nemohou být současně splněny.
- Pokud $k = 3$, pak $\alpha_j = C_j$ a C_j je tedy splněna.
- Pokud $k > 3$, pak je každá klauzule z α_j splněna ohodnocením v . Pokud je $v[y_1^j] = 0$, pak v a tedy i v' ohodnocuje jeden z literálů e_1 a e_2 na 1. Podobně pokud je $v[y_{k-3}^j] = 1$, pak jeden z literálů e_{k-1} a e_k ohodnocuje v , tedy i v' na 1. Pokud je $v[y_1^j] = 1$ a $v[y_{k-3}^j] = 0$, pak existuje $1 \leq i \leq k-4$, pro něž je $v[y_i^j] = 1$ a $v[y_{i+1}^j] = 0$. Klauzule $(\neg y_i^j \vee e_{i+2} \vee y_{i+1}^j)$ patří do $\alpha(C)$ a je tedy splněna ohodnocením v . Proto musí být e_{i+2} ohodnocením v a tedy i v' ohodnoceno na 1. V každém případě jeden literál z $e_1, \dots, e_k$ je ohodnocen v i v' na 1 a tedy klauzule C_j je splněna.

Nyní předpokládejme, že ψ je splnitelná, tedy existuje ohodnocení v , které ohodnocuje proměnné ψ a tato formule je jím splněna. Rozšíříme v o ohodnocení přidaných proměnných tak, aby nově vzniklé ohodnocení v' splnilo φ . Nechť $C_j = (e_1 \vee \dots \vee e_k)$ je libovolná klauzule formule ψ . Pokud je $k \leq 3$, pak všechny klauzule α_j jsou splněny přímo ohodnocením v a na ohodnocení příslušných y -ových proměnných v těchto případech nezáleží. Pokud $k > 3$, pak nechť i je index literálu e_i , kterému ohodnocení v přiřadí 1. Položme $v'[y_r^j] = 1$ pro $r \leq i-2$ a $v'[y_r^j] = 0$ pro $r \geq i-1$. Pokud $i < 3$, pak v' přiřadí všem y -ovým proměnným 0 a pokud $i > k-2$, pak v' přiřadí všem y -ovým proměnným 1, v obou těchto případech jsou zřejmě všechny klauzule α_j splněny. Pokud je $3 \leq i \leq k-2$, pak klauzule α_j obsahující e_r pro $r < i$ jsou splněny díky pozitivnímu y -ovému literálu a klauzule obsahující e_r pro $r > i$ jsou splněny díky negativnímu y -ovému literálu, okrajové klauzule jsou splněny díky své jediné y -ové proměnné. Ta klauzule α_j , která obsahuje e_i , je splněna díky e_i . $\square$

Jak jsme již zmínili, verze 2SAT, tj. splnitelnost formulí v KNF, kde každá klauzule obsahuje nejvýš 2 literály, je už polynomiálně rozhodnutelná. To platí i pro řadu dalších variant, za všechny zmiňme monotónní formule, tedy formule, které obsahují každou proměnnou jen pozitivně nebo jen negativně, tyto formule jsou vždy splnitelné. Další důležitou variantou je splnitelnost hornovských formulí, tedy problém HORN-SAT, jehož instance tvoří formule v KNF, kde každá klauzule obsahuje nejvýš jeden pozitivní literál.

11.0.2. Vrcholové pokrytí v grafu

V této části si ukážeme NP-úplnost problému vrcholového pokrytí v grafu.

Problém 11.3: VRCHOLOVÉ POKRYTÍ (VP)

Instance: Graf $G = (V, E)$ a přirozené číslo k

Otázka: Existuje množina $S \subseteq V$, která obsahuje nejvýš k vrcholů a z každé hrany obsahuje alespoň jeden koncový vrchol?
(Tj. $|S| \leq k$ a $(\forall e \in E)[S \cap e \neq \emptyset]$.)

Věta 11.4 *Problém vrcholového pokrytí je NP-úplný.*

Důkaz: Fakt, že problém VP patří do třídy NP, plyne z toho, že pro danou množinu S dokážeme ověřit v polynomiálním čase, jde-li o vrcholové pokrytí správné velikosti. NP-těžkost dokážeme převodem z problému 3-SAT. Nechť formule $\varphi = C_1 \wedge \dots \wedge C_m$ s proměnnými $U = \{u_1, \dots, u_n\}$ je instancí problému 3-SAT, tedy každá klauzule $C_1, \dots, C_m$ obsahuje právě tři literály. Popíšeme, jak na základě φ zkonstruovat graf $G = (V, E)$ a číslo k , pro něž bude platit, že v G existuje vrcholové pokrytí velikosti nejvýš k , právě když φ je splnitelná.

Pro každou proměnnou $u_i \in U$ definujeme podgraf $T_i = (V_i, E_i)$ s $V_i = \{u_i, \neg u_i\}$ a $E_i = \{\{u_i, \neg u_i\}\}$, tj. T_i obsahuje jen dva vrcholy pro pozitivní a negativní literál obsahující u_i a hranu mezi těmito dvěma vrcholy. Vrcholové pokrytí musí z této hrany využít alespoň jeden vrchol, má-li pokrýt hranu v E_i .

Pro každou klauzuli $C_j, j = 1, \dots, m$ přidáme úplný podgraf na třech nových vrcholech $a_1[j], a_2[j], a_3[j]$. Jde tedy trojúhelník, $R_j = (V'_j, E'_j)$, kde

$$V'_j = \{a_1[j], a_2[j], a_3[j]\}$$

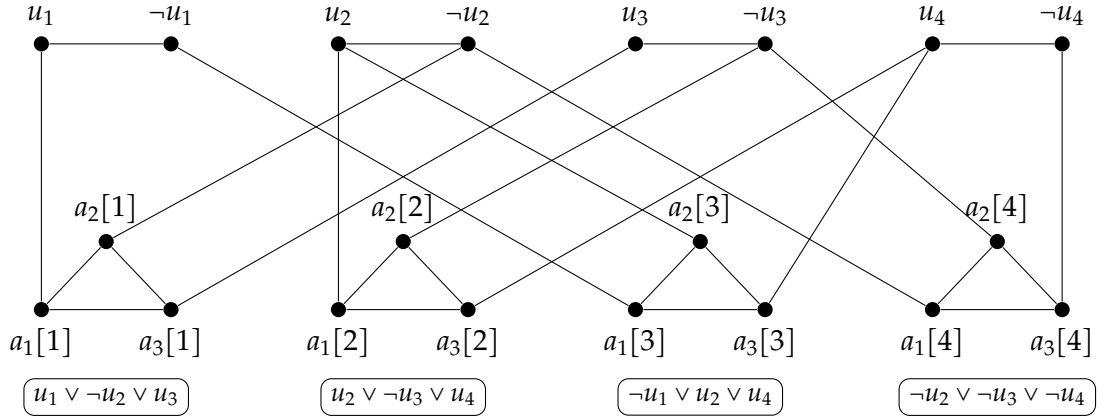
$$E'_j = \{\{a_1[j], a_2[j]\}, \{a_1[j], a_3[j]\}, \{a_2[j], a_3[j]\}\}$$

Všimněme si, že vrcholové pokrytí musí obsahovat alespoň dva vrcholy z V'_j , má-li pokrýt všechny hrany z E'_j .

Zbývá propojit tyto grafy mezi sebou hranami, jež spojují vrcholy pro literály s jejich výskyty v klauzulích. Předpokládejme, že pro každé $j = 1, \dots, m$, je klauzule $C_j = (x_j \vee y_j \vee z_j)$, kde x_j, y_j, z_j jsou tři různé literály (pozitivní nebo negativní). Pak přidáme hrany z R_j do příslušných vrcholů literálů, tedy množinu hran:

$$E''_j = \{\{a_1[j], x_j\}, \{a_2[j], y_j\}, \{a_3[j], z_j\}\}$$

V naší konstrukci nakonec položíme $k = n + 2m$ a $G = (V, E)$, kde



Obrázek 11.1.: Příklad převodu formule $\varphi = (u_1 \vee \neg u_2 \vee u_3) \wedge (u_2 \vee \neg u_3 \vee u_4) \wedge (\neg u_1 \vee u_2 \vee u_4) \wedge (\neg u_2 \vee \neg u_3 \vee \neg u_4)$ na instanci vrcholového pokrytí, tedy graf na obrázku s $k = 2 \cdot 4 + 4 = 12$.

$$V = \bigcup_{i=1}^n V_i \cup \bigcup_{i=1}^m V'_i$$

$$E = \bigcup_{i=1}^n E_i \cup \bigcup_{i=1}^m E'_i \cup \bigcup_{i=1}^m E''_i$$

Příklad konstrukce můžete nahlédnout v obrázku 11.1.

Z popisu konstrukce je zřejmé, že ji lze provést v polynomiálním čase. Zbývá ukázat, že G má vrcholové pokrytí velikosti nejvýš k , právě když φ je splnitelná.

Předpokládejme nejprve, že G má vrcholové pokrytí $S \subseteq V$ velikosti $k = n + 2m$. Protože S musí pokrýt alespoň jeden vrchol z každého podgrafu T_i , $i = 1, \dots, n$ a alespoň dva vrcholy z každého trojúhelníku R_j , $j = 1, \dots, m$, musí platit, že $|S| \geq n + 2m$, a protože současně $|S| \leq n + 2m$, musí ve skutečnosti platit, že S pokrývá právě jeden vrchol z každého T_i a právě dva vrcholy z každého R_i . Definujme ohodnocení $t : U \rightarrow \{0, 1\}$, které dané proměnné $u_i \in U$ přiřadí

$$t(u_i) = \begin{cases} 1 & u_i \in S \\ 0 & \neg u_i \in S \end{cases}$$

Díky tomu, že z každého T_i je v pokrytí S právě jeden vrchol, je v definici $t(u_i)$ vždy splněna právě jedna z podmínek a jde tedy o dobře definované ohodnocení proměnných pravdivostními hodnotami. Nechť $C_j = (x_j \vee y_j \vee z_j)$ je libovolná klauzule formule φ . Množina S pokrývá právě dva vrcholy z trojúhelníku R_j , existuje v něm tedy jeden vrchol, který do S nepatří, nechť je to bez újmy na obecnosti $a_1[j]$, protože hrana $\{a_1[j], x_j\}$ musí být pokryta množinou S , musí platit, že $x_j \in S$ a tedy ohodnocení t přiřadí literálu x_j hodnotu 1, tj. pokud $x_j = u_i \in S$ pro nějakou proměnnou $u_i \in U$, platí $t(u_i) = 1$ a pokud

$x_j = \neg u_i \in S$ pro nějakou proměnnou $u_i \in U$, platí $t(u_i) = 0$, v obou případech je literál x_j splněn ohodnocením t , a je tedy splněna i klauzule C_j . Protože to platí pro všechny klauzule, je t splňující ohodnocení φ .

Nyní předpokládejme, že φ je splněná ohodnocením $t : U \rightarrow \{0, 1\}$ a definujme množinu $S = \{u_i \mid t(u_i) = 1\} \cup \{\neg u_i \mid t(u_i) = 0\}$, množina obsahuje n vrcholů. Mezi hranami z E''_j vedoucími z každého trojúhelníku R_j musí být alespoň jedna pokrytá nějakým vrcholem z S , neboť t je splňující ohodnocení, které tedy splňuje některý literál z klauzule C_j . Z každého trojúhelníku stačí tedy vybrat dva vrcholy tak, aby byly pokryty jak hrany z E'_j , tak hrany z E''_j . Přidáním těchto vrcholů do S dostaneme vrcholové pokrytí celého grafu velikosti $k = n + 2m$. $\square$

Problémy, které s vrcholovým pokrytím úzce souvisí, jsou problémy kliky a nezávislé množiny, jejich těžkost si ukážeme v rámci cvičení. Je zajímavé poznamenat, že hranová verze tohoto problému, tedy hranové pokrytí, kde hledáme co nejmenší množinu hran takovou, že každý vrchol patří do jedné z nich, je polynomiálně řešitelná pomocí algoritmu na hledání maximálního párování v grafu.

11.0.3. Hamiltonovská kružnice v grafu

I další problém, kterému se budeme věnovat, bude grafový.

Problém 11.5: HAMILTONOVSKÁ KRUŽNICE (HK)

Instance: Graf $G = (V, E)$.

Otázka: Existuje v grafu G cyklus vedoucí přes všechny vrcholy?

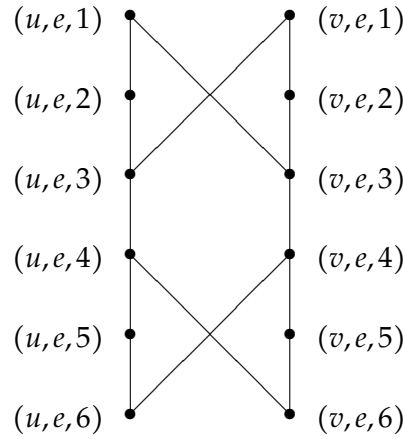
Věta 11.6 *Problém hamiltonovské kružnice je NP-úplný.*

Důkaz: Máme-li k dispozici pořadí vrcholů, snadno v polynomiálním čase ověříme, tvoří-li hamiltonovskou kružnici, proto patří problém hamiltonovské kružnice do třídy NP.

Těžkost tohoto problému ukážeme převodem z problému vrcholového pokrytí. Uvažme instanci vrcholového pokrytí, tedy graf $G = (V, E)$ a přirozené číslo k . Zkonstruujeme nový graf $G' = (V', E')$, pro který bude platit, že v G' existuje hamiltonovská kružnice, právě když v G existuje vrcholové pokrytí velikosti k . Pro každou hranu $e = \{u, v\} \in E$ definujeme podgraf $G'_e = (V'_e, E'_e)$, kde

$$\begin{aligned} V'_e &= \{(u, e, i), (v, e, i) \mid 1 \leq i \leq 6\} \text{ a} \\ E'_e &= \left\{ \{(u, e, i), (u, e, i+1)\} \mid 1 \leq i \leq 5\} \right. \\ &\cup \left\{ \{(v, e, i), (v, e, i+1)\} \mid 1 \leq i \leq 5\} \right. \\ &\cup \left\{ \{(u, e, 1), (v, e, 3)\}, \{(u, e, 3), (v, e, 1)\}, \right. \\ &\quad \left. \{(u, e, 4), (v, e, 6)\}, \{(u, e, 6), (v, e, 4)\} \right\}. \end{aligned}$$

Graf G'_e je zobrazen na následujícím obrázku:



Jediné vrcholy, k nimž v další konstrukci připojíme další hrany budou $(u, e, 1)$, $(u, e, 6)$, $(v, e, 1)$ a $(v, e, 6)$, což zaručí, že hamiltonovská kružnice musí vstoupit i vystoupit z podgrafu G'_e jedním z těchto čtyř vrcholů. Jsou jen tři způsoby, jak může tato cesta vypadat, ty budou odpovídat tomu, jak pokrýt hranu e v grafu G .

- (I) Cesta vstoupí do G'_e vrcholem $(u, e, 1)$, vystoupí $(u, e, 6)$ a mezitím projde všechny vrcholy (cesta je jednoznačně daná).
- (II) Nebo cesta vstoupí do G'_e vrcholem $(v, e, 1)$, vystoupí $(v, e, 6)$ a mezitím projde všemi vrcholy komponenty (toto je symetrické s předchozím případem).
- (III) Třetí možností je, že kružnice do této komponenty vstoupí dvakrát, jednou vrcholem $(u, e, 1)$, odkud jde přímo do $(u, e, 6)$ a ven, podruhé vstoupí do $(v, e, 1)$ a odejde pomocí $(v, e, 6)$.

Vzhledem k tomu, že graf je neorientovaný, nezáleží na tom, jestli například v prvním případě vstoupí cesta do G'_e vrcholem $(u, e, 1)$ a vystoupí $(u, e, 6)$ nebo naopak. Všimněme si také, že pokud vstoupí cesta do G'_e vrcholem $(v, e, 1)$, vystoupí ve všech případech vrcholem $(v, e, 6)$, podobně to platí pro $(u, e, 1)$.

Do množiny vrcholů V' nového grafu G' přidáme ještě k vrcholů $a_1, \dots, a_k$, jež použijeme k výběru k vrcholů vrcholového pokrytí. Zbývá popsat, jak spojíme jednotlivé podgrafy G'_e a nově přidané vrcholy a_i .

Pro každý vrchol $v \in V$ zapojíme jednotlivé grafy G'_e s $v \in e$ za sebe do řetízku. Přesněji, označme si stupeň vrcholu v pomocí $\deg(v)$ a označme si hrany grafu G , v nichž se vyskytuje vrchol v pomocí $e_{v[1]}, \dots, e_{v[\deg(v)]}$, přičemž na jejich pořadí nezáleží. Nyní definujme množinu hran

$$E'_v = \left\{ \{(v, e_{v[i]}, 6), (v, e_{v[i+1]}, 1)\} \mid 1 \leq i < \deg(v)\} \right\}.$$

Vrcholy na koncích této posloupnosti, tedy $(v, e_{v[1]}, 1)$ a $(v, e_{v[\deg(v)]}, 6)$ připojíme ke

všem výběrovým vrcholům $a_1, \dots, a_k$ hranami

$$E''_v = \left\{ \{a_i, (v, e_{v[1]}, 1)\}, \{a_i, (v, e_{v[\deg(v)]}, 6)\} \mid 1 \leq i \leq k \right\}.$$

Zkonstruovaný graf $G' = (V', E')$ vznikne spojením popsaných částí:

$$V' = \{a_1, \dots, a_k\} \cup \bigcup_{e \in E} V'_e$$

$$E' = \bigcup_{e \in E} E'_e \cup \bigcup_{v \in V} E'_v \cup \bigcup_{v \in V} E''_v$$

Tuto konstrukci je zřejmě možné provést v polynomiálním čase, zbývá ukázat, že v grafu G' najdeme hamiltonovskou kružnici, právě když v grafu G existuje vrcholové pokrytí velikosti k .

Předpokládejme nejprve, že v G' existuje hamiltonovská kružnice, daná pořadím vrcholů $u_1, \dots, u_n$, kde $n = |V'|$. Uvažme úsek této kružnice, který začíná a končí v některém z vrcholů $a_1, \dots, a_k$, přičemž mezi tím žádným z nich neprochází. Nechť tento úsek začíná v a_i a končí v a_j pro dva různé indexy $i, j \in \{1, \dots, k\}$. Vrchol následující po a_i v hamiltonovské kružnici musí být $(v, e_{v[1]}, 1)$ pro nějaký vrchol $v \in V$. Vejde-li hamiltonovská kružnice vrcholem $(v, e_{v[1]}, 1)$ do $G'_{e_{v[1]}}$, musí z něj vyjít vrcholem $(v, e_{v[1]}, 6)$, odtud přejde do $(v, e_{v[2]}, 1)$, takto projde všechny grafy G'_e pro hrany e obsahující vrchol v až nakonec přejde z vrcholu $(v, e_{v[\deg(v)]}, 6)$ do a_j . Definujme nyní množinu vrcholů

$$S = \{v \mid (\exists i \in \{1, \dots, k\})(\exists j \in \{1, \dots, n\}) [(u_j = a_i) \ \& \ (u_{(j+1 \bmod n)} = (v, e_{v[1]}, 1))]\},$$

tvrdíme, že jde o vrcholové pokrytí v grafu G , přičemž jeho velikost je zřejmě k . Nechť $e = \{u, v\}$ je libovolná hrana grafu G , protože $u_1, \dots, u_n$ určuje pořadí vrcholů na hamiltonovské kružnici, musí projít i podgraf G'_e , vejde-li do tohoto podgrafu vrcholem $(u, e, 1)$, pak musí z předchozích úvah platit, že $u \in S$, podobně pokud vejde vrcholem $(v, e, 1)$, musí platit, že $v \in S$, tedy hrana e je pokryta.

Naopak uvažme vrcholové pokrytí S velikosti k , můžeme uvažovat, že jde o množinu velkou přesně k , pokud je menší, doplníme do ní libovolné vrcholy tak, aby její velikost byla k . Z popisu konstrukce plyne, jak pospojujeme jednotlivé podgrafy odpovídající hranám. Předpokládejme, že $S = \{v_1, \dots, v_k\} \subseteq V$. Mezi vrcholy a_i a a_{i+1} (resp. a_1 pokud $i = k$) povedeme cestu přes podgrafy G'_e pro $e = e_{v_i[1]}, \dots, e_{v_i[\deg(v_i)]}$ v tomto pořadí. Graf G'_e přitom projdeme způsobem (I) nebo (II) pokud $e \cap S = \{v_i\}$, pokud platí $e \subseteq S$, pak použijeme způsob (III). Vzhledem k tomu, že S tvoří vrcholové pokrytí, projdeme takto všechny vrcholy grafu G' a vznikne tedy hamiltonovská kružnice. $\square$

Podotkněme, že problém hamiltonovské kružnice má několik variant. Předně se můžeme ptát na existenci hamiltonovské kružnice v orientovaném grafu. V problému hamiltonovské cesty se ptáme, existuje-li v grafu cesta mezi dvěma vrcholy, jež jde přes všechny vrcholy grafu, orientovaného či neorientovaného, přičemž počáteční a koncový vrchol hledané cesty mohou být předepsané na vstupu. Všechny tyto varianty jsou

NP-úplné. Na druhou stranu rozhodnutí zda graf obsahuje eulerovský tah, tedy tah, který použije každou hranu právě jednou, lze učinit v polynomiálním čase, ať už se jedná o tah uzavřený či neuzavřený.

11.0.4. Trojrozměrné párování

Dalším problémem, jehož těžkost si ukážeme, bude trojrozměrné párování (3D Matching).

Problém 11.7: TROJROZMĚRNÉ PÁROVÁNÍ (3DM)

- Instance:** Množina $M \subseteq W \times X \times Y$, kde W, X, Y jsou po dvou disjunktní množiny, z nichž každá obsahuje právě q prvků.
- Otázka:** Obsahuje M perfektní párování? Jinými slovy, existuje množina $M' \subseteq M$, $|M'| = q$, trojice v níž obsažené jsou po dvou disjunktní?

Věta 11.8 Problém 3DM je NP-úplný.

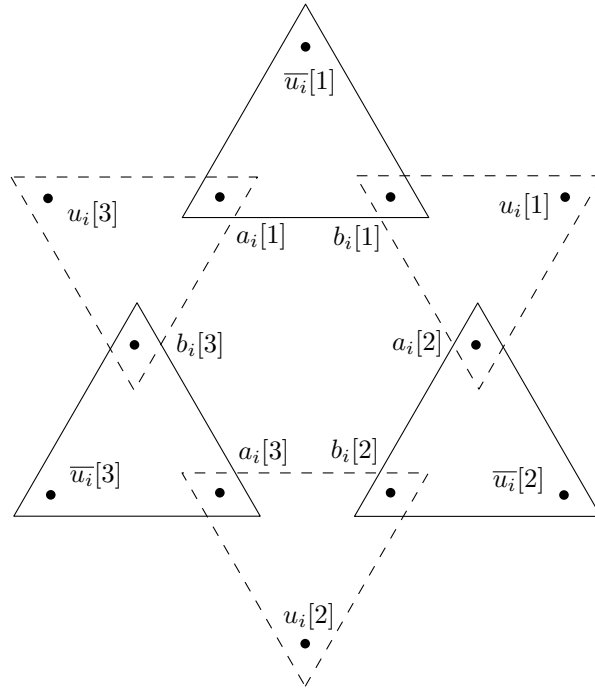
Důkaz: Fakt, že tento problém patří do třídy NP, plyne z toho, že pokud máme k dispozici množinu $M' \subseteq M$, dokážeme ověřit v polynomiálním čase, jde-li o párování velikosti q .

Těžkost tohoto problému si ukážeme převodem z problému SAT. Nechť $\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_m$ je formule v KNF na proměnných $U = \{u_1, \dots, u_n\}$. Sestrojíme instanci problému 3DM, pro kterou bude platit, že v této instanci existuje perfektní párování, právě když je φ splnitelná. Konstrukci rozdělíme na tři části, nejprve vytvoříme komponentu, která bude určovat, jakou hodnotu která proměnná dostane, poté vytvoříme komponentu, která bude zajišťovat propojení této hodnoty s klauzulemi, v nichž se tato proměnná vyskytuje, nakonec doplníme trojice tak, abychom dostali k splňujícímu ohodnocení skutečně perfektní párování a naopak.

Za každou proměnnou u_i , $i = 1, \dots, n$ přidáme nové vnitřní prvky $a_i[1], \dots, a_i[m]$ do X a $b_i[1], \dots, b_i[m]$ do Y . Do W přidáme prvky $u_i[1], \dots, u_i[m]$ a $\bar{u}_i[1], \dots, \bar{u}_i[m]$. Na těchto prvcích vytvoříme množiny trojic T_i^f a T_i^t takto (viz též příklad na obrázku 11.2):

$$\begin{aligned} T_i^t &= \{(\bar{u}_i[j], a_i[j], b_i[j]) \mid 1 \leq j \leq m\} \\ T_i^f &= \{(u_i[j], a_i[(j \bmod m) + 1], b_i[j]) \mid 1 \leq j \leq m\} \\ T_i &= T_i^t \cup T_i^f \end{aligned}$$

Protože žádný z prvků $a_i[j]$ ani $b_i[j]$ se nebude vyskytovat v jiných trojicích, je tímto vynuceno, že perfektní párování buď musí obsahovat všechny trojice z T_i^t , nebo všechny trojice z T_i^f . Pokud obsahuje trojice z T_i^t , znamená to, že žádná další vybraná trojice nesmí obsahovat literál $\bar{u}_i$, tedy vynucujeme hodnotu 1, true pro u_i , proto také T_i^f . Podobně pokud obsahuje perfektní párování trojice z T_i^f , vynucujeme hodnotu 0, false, odtud T_i^t .



Obrázek 11.2.: Ukázka množin T_i^t (plnou čarou) a T_i^f (čárkovaně) pro případ $m = 3$.

Za klauzuli C_j přidáme nový prvek $s_1[j]$ do množiny X , nový prvek $s_2[j]$ do množiny Y a množinu trojic

$$S_j = \{(u_i[j], s_1[j], s_2[j]) \mid u_i \in C_j\} \cup \{(\bar{u}_i[j], s_1[j], s_2[j]) \mid \bar{u}_i \in C_j\}$$

Prvky $s_1[j]$ a $s_2[j]$ se opět nebudou vyskytovat v jiných trojicích, díky tomu v perfektním párování musí být právě jedna trojice z množiny S_j . Navíc pokud se trojice $(u_i[j], s_1[j], s_2[j])$ vyskytuje v perfektním párování, znamená to, že $u_i[j]$ se nemůže vyskytovat v jiné trojici a to znamená, že v tomto párování jsou všechny trojice z T_i^t a žádná z T_i^f . Podobně by to bylo, kdyby v perfektním párování byla trojice s negativním literálem.

Těmito trojicemi jsme ale schopni v perfektním párování pokrýt jen $mn + m$ prvků z $2mn$ prvků $u_i[j], \bar{u}_i[j]$, $i = 1, \dots, n$, $j = 1, \dots, m$. Z toho mn jich pokryjeme trojicemi z T_i^t nebo T_i^f , $i = 1, \dots, n$. Trojicemi z S_j , $j = 1, \dots, m$ pokryjeme dalších m prvků. Zbývá tedy $2mn - (mn + m) = mn - m = m(n - 1)$ prvků, jež nejsme zatím schopni pokrýt, proto přidáme do množiny M trojice, které nám jejich pokrytí zabezpečí. Do X přidáme prvky $g_1[k]$ a do Y prvky $g_2[k]$ obojí pro $k = 1, \dots, m(n - 1)$ a do M přidáme množinu trojic

$$G = \{(u_i[j], g_1[k], g_2[k]), (\bar{u}_i[j], g_1[k], g_2[k]) \mid 1 \leq k \leq m(n - 1), 1 \leq i \leq n, 1 \leq j \leq m\}$$

Tím je konstrukce dokončena, ještě si na závěr shrňme její výsledek:

$$\begin{aligned}
W &= \{u_i[j], \bar{u}_i[j] \mid 1 \leq i \leq n, 1 \leq j \leq m\} \\
X &= \{a_i[j] \mid 1 \leq i \leq n, 1 \leq j \leq m\} \\
&\quad \cup \{s_1[j] \mid 1 \leq j \leq m\} \\
&\quad \cup \{g_1[j] \mid 1 \leq j \leq m(n-1)\} \\
Y &= \{b_i[j] \mid 1 \leq i \leq n, 1 \leq j \leq m\} \\
&\quad \cup \{s_2[j] \mid 1 \leq j \leq m\} \\
&\quad \cup \{g_2[j] \mid 1 \leq j \leq m(n-1)\} \\
M &= \bigcup_{i=1}^n T_i \cup \bigcup_{j=1}^m S_j \cup G
\end{aligned}$$

Zřejmě platí, že $M \subseteq W \times X \times Y$, navíc $|M| = 2mn + 3m + 2m^2n(n-1)$ a $|W| = |X| = |Y| = 2mn$. Velikost takto vytvořené instance trojrozměrného párování je tedy polynomiálně velká a v polynomiálním čase ji zřejmě lze i vytvořit.

Předpokládejme, že v M existuje perfektní párování M' , na jehož základě zkonstruuje me ohodnocení t , které bude splňovat formuli φ . Nechť i je libovolný index z $1, \dots, n$, jak jsme již zdůvodnili, v M' jsou buď všechny trojice z T_i^t , nebo všechny trojice z T_i^f , pokud M' obsahuje trojice z T_i^t , definujeme $t(u_i) := 1$, pokud M' obsahuje trojice z T_i^f , definujeme $t(u_i) := 0$. Nechť C_j je libovolná klauzule formule φ , množina M' musí obsahovat právě jednu z trojic z S_j , neboť to je jediná možnost, jak mohou být pokryty prvky $s_1[j]$ a $s_2[j]$ a dvě obsahovat nemůže, protože každý z těchto prvků musí být pokryt právě jednou. Nechť tato trojice je $(u_i[j], s_1[j], s_2[j])$ pro nějaké $i = 1, \dots, n$. To znamená, že u_i je proměnná, vyskytující se jako pozitivní literál v klauzuli C_j , navíc musí platit, že $u_i[j]$ se nemůže vyskytovat v žádné jiné trojici v M' , a proto M' obsahuje trojice z T_i^t a nikoli trojice z T_i^f , a tedy $t(u_i) = 1$, čímž je klauzule C_j splněna. Podobně bychom postupovali v případě, kdy trojici v $S_j \cap M'$ by byla $(\neg u_i[j], s_1[j], s_2[j])$, tedy pokud by obsahovala negativní literál, jediný rozdíl by byl, že bychom dostali, že $t(u_i) = 0$ a že C_j je splněna díky negativnímu literálu $\neg u_i$.

Pokud je naopak φ splnitelná, zkonstruuje me perfektní párování následovně. Nechť $t : U \rightarrow \{0, 1\}$ je ohodnocení splňující φ a nechť z_j označuje literál, který je v C_j tímto ohodnocením splněn pro $j = 1, \dots, m$. Pokud je takových literálů víc, vybereme prostě jeden z nich. Pak položíme

$$M' = \bigcup_{t(u_i)=1} T_i^t \cup \bigcup_{t(u_i)=0} T_i^f \cup \{(z_j[j], s_1[j], s_2[j]) \mid j = 1, \dots, m\} \cup G',$$

kde G' je množina vhodně vybraných trojic z G , které doplňují párování o pokrytí zbylých literálů. Není těžké ověřit, že takto definovaná množina M' tvoří perfektní párování M . $\square$

Je třeba připomenout, že dvojrozměrná verze, tedy hledání maximálního párování v bipartitním grafu, je úloha řešitelná v polynomiálním čase například pomocí toků

v sítích. Podobně i hledání maximálního párování v obecném grafu je stále polynomiálně řešitelná úloha.

11.0.5. Loupežníci

Posledním problémem, jehož těžkost si ukážeme, je problém Loupežníci, anglicky Partition. Český název pochází z představy, že jde o dělení lupu dvou loupežníků na shodné díly.

Problém 11.9: LOUPEŽNÍCI (LOUP)

Instance: Množina prvků A a s každým prvkem $a \in A$ asociovaná cena (váha, velikost, ...) $s(a) \in \mathbb{N}$.

Otázka: Lze rozdělit prvky z A na dvě poloviny s toutéž celkovou cenou? Přesněji, existuje množina $A' \subseteq A$ taková, že

$$\sum_{a \in A'} s(a) = \sum_{a \in A \setminus A'} s(a) ?$$

Věta 11.10 *Problém Loupežníci je NP-úplný.*

Důkaz: Fakt, že tento problém patří do třídy NP, plyne z toho, že pro zadanou množinu A' je jistě snadné ověřit, zda obsahuje prvky poloviční ceny. Těžkost tohoto problému ukážeme převodem z trojrozměrného párování. Uvažme instanci 3DM, tedy množinu $M \subseteq W \times X \times Y$, kde $|W| = |X| = |Y| = q$. Vytvoříme instanci Loupežníků, tedy množinu A a cenovou funkci $s(a)$, pro něž bude platit, že M má perfektní párování, právě když prvky v A lze rozdělit na dvě části s touž celkovou cenou.

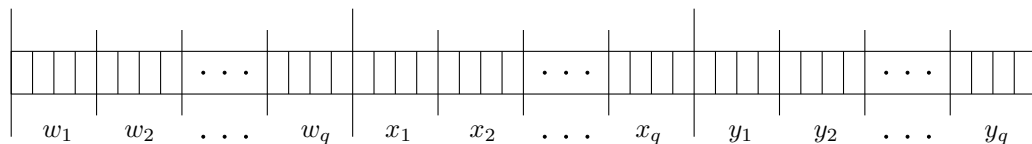
Předpokládejme, že $M = \{m_1, \dots, m_k\}$, $W = \{w_1, \dots, w_q\}$, $X = \{x_1, \dots, x_q\}$ a $Y = \{y_1, \dots, y_q\}$. Jednotlivé prvky trojice m_i si označíme jako $w_{f(i)}$, $x_{g(i)}$ a $y_{h(i)}$, tj. funkce f (resp. h , g) vrátí k zadanému indexu trojice i index toho prvku trojice m_i , který patří do množiny W (resp. X , Y). Množinu prvků A definujeme jako

$$A = \{a_1, \dots, a_k, b_1, b_2\},$$

kde prvky $a_1, \dots, a_k$ odpovídají trojicím $m_1, \dots, m_k$ a prvky b_1 a b_2 jsou pomocné a vyrovňovací, jejich účel se ozřejmí později. Cenu $s(a_i)$ prvku a_i pro $i \in \{1, \dots, k\}$ popíšeme její binární reprezentací, která bude rozdělena na $3q$ bloků, z nichž každý má $p = \lceil \log_2(k+1) \rceil$ bitů. Každý z těchto bloků bude odpovídat jednomu z elementů $W \cup X \cup Y$, přesněji viz obrázek 11.3.

Reprezentace $s(a_i)$ bude záviset jen na prvcích trojice m_i , tedy na $w_{f(i)}$, $x_{g(i)}$ a $y_{h(i)}$. Váha $s(a_i)$ bude mít nastaveny na 1 nejpravější (tj. nejméně významné) bity v blocích označených těmito třemi prvky, ostatní bity budou nulové. Formálně můžeme tento fakt zapsat jako

$$s(a_i) = 2^{p(3q-f(i))} + 2^{p(2q-g(i))} + 2^{p(q-h(i))}.$$



Obrázek 11.3.: Označení zón bitů v definici ceny $s(a_i)$ prvku a_i .

Protože počet bitů, který potřebujeme na reprezentaci $s(a_i)$ je $3pq$ a tedy polynomiální vzhledem k velikosti vstupu, předpokládáme-li standardní binární reprezentaci vstupních čísel, lze tyto ceny zkonstruovat v polynomiálním čase. O takto zkonstruovaných cenách lze vypožorovat jeden důležitý fakt. Pokud posčítáme ceny všech prvků v kterékoli zóně, nemůže dojít k přetečení do další zóny, neboť jde o sečtení nejvýš k jedniček, přičemž počet bitů v jedné zóně je $p = \lceil \log_2(k+1) \rceil$ a vejde se do ní tedy $k+1$. Přesněji ani při počítání součtu $\sum_{i=1}^k s(a_i)$ nedojde k přenosu jedničky z méně významné zóny do významnější.

Pokud tedy položíme

$$B = \sum_{j=0}^{3q-1} 2^{pj},$$

což je číslo, kde v každé zóně nastavíme na 1 nejmeně významný bit, množina $A' \subseteq \{a_i \mid 1 \leq i \leq k\}$ bude splňovat

$$\sum_{a \in A'} s(a) = B,$$

právě když $M' = \{m_i \mid a_i \in A'\}$ je perfektní párování M . V tuto chvíli jsme tedy učinili převod problému trojrozměrného párování na problém součtu podmnožiny, kde se ptáme, zda existuje výběr prvků s celkovou cenou rovnou zadané hodnotě.

Abychom dostali dělení na poloviny, doplníme do A dva prvky b_1 a b_2 s cenami:

$$s(b_1) = 2 \left(\sum_{i=1}^k s(a_i) \right) - B$$

$$s(b_2) = \left(\sum_{i=1}^k s(a_i) \right) + B$$

Na reprezentaci obou těchto vah nám postačí $3pq + 1$ bitů. Pokud nyní $A' \subseteq A$ splňuje, že

$$\sum_{a \in A'} s(a) = \sum_{a \in A \setminus A'} s(a),$$

pak se oba součty musí rovnat $2 \sum_{i=1}^k s(a_i)$, neboť to je polovina ze součtu cen všech prvků $\sum_{i=1}^k s(a_i) + s(b_1) + s(b_2)$. Přitom platí, že prvky b_1 a b_2 se nemohou oba vyskytovat na jedné straně, tj. nemohou být oba v A' nebo oba v $A \setminus A'$, protože $s(b_1) + s(b_2) = 3 \sum_{i=1}^k s(a_i)$. Bez újmy na obecnosti předpokládejme, že $b_1 \in A'$ a $b_2 \in A \setminus A'$, z toho plyne, že

$$\sum_{a \in A' \setminus \{b_1\}} s(a) = B,$$

a prvky v A' bez b_1 tedy určují perfektní párování v M .

Nyní předpokládejme, že $M' \subseteq M$ je perfektní párování a definujme $A' = \{a_i \mid m_i \in M'\}$. Jak jsme již zdůvodnili, musí platit, že $\sum_{a \in A'} s(a) = B$, a tedy $\sum_{a \in A'} s(a) + s(b_1) = 2 \sum_{i=1}^k s(a_i)$, což znamená, že množina $\{a_i \mid m_i \in M'\} \cup \{b_1\}$ obsahuje prvky právě poloviční ceny.

Dohromady jsme ukázali, že v množině trojic M existuje perfektní párování, právě když z množiny A lze vybrat prvky poloviční ceny. $\square$

11.1. Cvičení

Důkazy NP-úplnosti

1. Ukažte, že problémy Kliky a Nezávislé množiny jsou stejně těžké a že oba patří do NP.

Problém 11.11: KLIKA

Instance: Graf $G = (V, E)$ a přirozené číslo k .

Otázka: Obsahuje G jako podgraf úplný podgraf (kliku) s alespoň k vrcholy?

Problém 11.12: NEZÁVISLÁ MNOŽINA

Instance: Graf $G = (V, E)$ a přirozené číslo k .

Otázka: Existuje množina $S \subseteq V$ vrcholů taková, že žádné dva vrcholy z S nejsou spojeny hranou a $|S| \geq k$?

2. Ukažte, že problém vrcholového pokrytí, který jsme probírali na přednášce, lze polynomiálně převést na problém nezávislé množiny a že jsou tedy problémy Kliky a Nezávislé množiny NP-úplné.
3. Ukažte, že problém hamiltonovské kružnice, který jsme probírali na přednášce, lze polynomiálně převést na problém orientované hamiltonovské kružnice a na následující dvě varianty problému hamiltonovské cesty a že jsou tedy oba tyto problémy NP-těžké. Ukažte, že jsou tyto problémy i ve třídě NP a jsou tedy NP-úplné.

Problém 11.13: ORIENTOVANÁ HAMILTONOVSKÁ KRUŽNICE (OHK)

Instance: Orientovaný graf $G = (V, E)$.

Otázka: Existuje v G cyklus, který projde každý vrchol právě jednou?

Problém 11.14: HAMILTONOVSKÁ CESTA Z s DO t ($HC(s, t)$)

Instance: Neorientovaný graf $G = (V, E)$, vrcholy $s, t \in V$.

Otázka: Existuje v G cesta z s do t , která obsahuje každý vrchol G právě jednou?

Problém 11.15: HAMILTONOVSKÁ CESTA (HC)

Instance: Neorientovaný graf $G = (V, E)$.

Otázka: Existuje v G cesta, která obsahuje každý vrchol G právě jednou?

4. Ukažte, že problém obchodního cestujícího je NP-úplný. K důkazu těžkosti využijte problému hamiltonovské kružnice.

Problém 11.16: OBCHODNÍ CESTUJÍCÍ (OC)

Instance: Množina měst $C = \{c_1, \dots, c_n\}$, vzdálenost $d(c_i, c_j) \in \mathbb{N}$ pro každá dvě města $c_i, c_j \in C$, přirozené číslo B

Otázka: Existuje cyklus, který navštíví každé město právě jednou a jehož délka nepřesahuje B ? Jinými slovy, existuje permutace $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ taková, že

$$\sum_{i=1}^{n-1} d(c_{\pi(i)}, c_{\pi(i+1)}) + d(c_{\pi(n)}, c_{\pi(1)}) \leq B?$$

5. Ukažte, že problémy existence přípustného řešení celočíselného programování a 0–1 celočíselného programování jsou NP-těžkosti. K důkazu použijte některý z následujících problémů: Kachlíkování, Splnitelnost, 3-Splnitelnost, Vrcholové pokrytí, Trojrozměrné párování, Hamiltonovská kružnice, Obchodní cestující nebo Loupežníci.

Problém 11.17: EXISTENCE PŘÍPUSTNÉHO ŘEŠENÍ CELOČÍSELNÉHO PROGRAMOVÁNÍ (IP)

Instance: Celočíselná matice A typu $n \times m$, vektor $b \in \mathbb{Z}^n$.

Otázka: Existuje vektor $x \in \mathbb{Z}^m$, který splňuje, že $Ax \geq b$?

Problém 11.18: EXISTENCE PŘÍPUSTNÉHO ŘEŠENÍ CELOČÍSELNÉHO PROGRAMOVÁNÍ (0-1IP)

Instance: Celočíselná matice A typu $n \times m$, vektor $b \in \mathbb{Z}^n$.

Otázka: Existuje vektor $x \in \{0, 1\}^m$, který splňuje, že $Ax \geq b$?

Ukažte navíc, že problém 0-1IP patří do NP, proč není tak jednoduché ukázat, že i obecná verze IP patří do NP?

6. S použitím problému Loupežníci ukažte, že problémy Batoh a Rozvrhování jsou NP-úplné.

Problém 11.19: BATOH

Instance: Množina předmětů U , s každým prvkem $u \in U$ jeho velikost $s(u) \in \mathbb{N}$ a cena $v(u) \in \mathbb{N}$, přirozená čísla B a K .

Otázka: Existuje množina $U' \subseteq U$, pro kterou platí, že

$$\sum_{u \in U'} s(u) \leq B \quad \text{a} \quad \sum_{u \in U'} v(u) \geq K ?$$

tj. lze do batohu velikosti B zabalit předměty s cenou alespoň K ?

Problém 11.20: Rozvrhování

Instance: Množina úloh U , s každou úlohou $u \in U$ asociovaná délka zpracování $d(u) \in \mathbb{N}$, počet procesorů $m \in \mathbb{N}$ a limit na délku zpracování $D \in \mathbb{N}$.

Otázka: Lze rozdělit prvky z množiny U do po dvou disjunktních množin $U_1, \dots, U_m$ tak, aby

$$\max \left\{ \sum_{u \in U_i} d(u) \mid 1 \leq i \leq m \right\} \leq D?$$

tj. lze přiřadit úlohy z množiny U na m procesorů tak, aby spočítání úloh na všech procesorech skončilo nejpozději v čase D ?

Při rozvrhování ve skutečnosti minimalizujeme D při pevném počtu procesorů. Pokud minimalizujeme m při pevném D , říká se této úloze Bin Packing. Rozhodovací verze obou úloh jsou ale stejné a jde o popsany problém Rozvrhování.

7. S pomocí některého z problémů Kachlíkování, Splnitelnost, 3-Splnitelnost, Vrcholové pokrytí, Trojrozměrné párování, Hamiltonovská kružnice, Obchodní cestující nebo Loupežníci ukažte, že následující problém je NP -úplný:

Problém 11.21: KOSTRA S OMEZENÝM STUPNĚM

Instance: Graf $G = (V, E)$ a přirozené číslo $k \geq 0$.

Otázka: Obsahuje graf G kosteru, v níž všechny vrcholy jsou stupně nejvýš k ?

8. S pomocí některého z problémů Kachlíkování, Splnitelnost, 3-Splnitelnost, Vrcholové pokrytí, Trojrozměrné párování, Hamiltonovská kružnice, Obchodní cestující nebo Loupežníci ukažte, že následující problém je NP -úplný:

Problém 11.22: PŘESNÉ POKRYTÍ 3-PRVKOVÝMI MNOŽINAMI (X3C)

Instance: Množina X s $|X| = 3q$ a množina trojic $C \subseteq X^3$

Otázka: Existuje $C' \subseteq C$ taková, že každý prvek z X se vyskytuje v právě jedné trojici z C' ?

9. S pomocí některého z problémů Kachlíkování, Splnitelnost, 3-Splnitelnost, Vrcholové pokrytí, Trojrozměrné párování, Hamiltonovská kružnice, Obchodní cestující nebo Loupežníci ukažte, že následující problém je NP -úplný:

Problém 11.23: DOMINUJÍCÍ MNOŽINA

Instance: Graf $G = (V, E)$, kladné číslo $k > 0$

Otázka: Existuje množina vrcholů $V' \subseteq V$, $|V'| \leq k$ taková, že každý vrchol $v \in V \setminus V'$ je spojen hranou s alespoň jedním vrcholem z V' ?

10. S pomocí některého z problémů Kachlíkování, Splnitelnost, 3-Splnitelnost, Vrcholové pokrytí, Trojrozměrné párování, Hamiltonovská kružnice, Obchodní cestující nebo Loupežníci ukažte, že následující problém je NP -úplný:

Problém 11.24: HITTING SET

Instance: Množina S a množina C podmnožin množiny S , přirozené číslo $k > 0$.

Otázka: Existuje $S' \subseteq S$, $|S'| \leq k$ taková, že S' obsahuje nejméně jeden prvek z každé množiny v C ?

11. S pomocí některého z problémů Kachlíkování, Splnitelnost, 3-Splnitelnost, Vrcholové pokrytí, Trojrozměrné párování, Hamiltonovská kružnice, Obchodní cestující nebo Loupežníci ukažte, že následující problém je NP -úplný:

Problém 11.25: POKRYTÍ ORIENTOVANÝCH CYKLŮ (FEEDBACK VERTEX SET)

Instance: Orientovaný graf $G = (V, E)$ a přirozené číslo $k \geq 0$.

Otázka: Existuje množina $S \subseteq V$, $|S| \leq k$ taková, že z každého orientovaného cyklu v G obsahuje S alespoň jeden vrchol?

12. S pomocí některého z problémů Kachlíkování, Splnitelnost, 3-Splnitelnost, Vrcholové pokrytí, Trojrozměrné párování, Hamiltonovská kružnice, Obchodní cestující nebo Loupežníci ukažte, že následující problém je NP -úplný:

Problém 11.26: ČTYŘLISTÁ KOŠTRA GRAFU

Instance: Neorientovaný graf $G = (V, E)$.

Otázka: Existuje kostra grafu G , která má právě čtyři listy?

13. S pomocí některého z problémů Kachlíkování, Splnitelnost, 3-Splnitelnost, Vrcholové pokrytí, Trojrozměrné párování, Hamiltonovská kružnice, Obchodní cestující nebo Loupežníci ukažte, že následující problém je NP -úplný:

Problém 11.27: NEJVĚTŠÍ SPOLEČNÝ PODGRAF

Instance: Grafy $G_1 = (V_1, E_1)$ a $G_2 = (V_2, E_2)$, přirozené číslo $k \geq 0$.

Otázka: Existují množiny hran $E'_1 \subseteq E_1$ a $E'_2 \subseteq E_2$, $|E'_1| = |E'_2| \geq k$, pro něž jsou grafy $G'_1 = (V_1, E'_1)$ a $G'_2 = (V_2, E'_2)$ izomorfní (tj. stejné až na přejmenování vrcholů)?

14. S pomocí některého z problémů Kachlíkování, Splnitelnost, 3-Splnitelnost, Vrcholové pokrytí, Trojrozměrné párování, Hamiltonovská kružnice, Obchodní cestující nebo Loupežníci ukažte, že následující problém je NP -úplný:

Problém 11.28: SET PACKING

Instance: Množina C konečných množin a přirozené číslo $k \geq 0$

Otázka: Obsahuje C alespoň k po dvou disjunktních množin?

15. S pomocí některého z problémů Kachlíkování, Splnitelnost, 3-Splnitelnost, Vrcholové pokrytí, Trojrozměrné párování, Hamiltonovská kružnice, Obchodní cestující nebo Loupežníci ukažte, že následující problém je NP -úplný:

Problém 11.29: MNOŽINOVÉ POKRYTÍ (SET COVER)

Instance: Systém množin C konečné množiny prvků S , tj. $C \subseteq \mathcal{P}(S)$, přirozené číslo k .

Otázka: Je možné vybrat $C' \subseteq C$, která obsahuje nejvýš k množin a přitom

$$\bigcup_{C \in C'} C = S ?$$

Ostatní

16. Představte si, že máte černou skříňku, která dostane na vstupu formuli φ a rozhodne, zda je tato formule splnitelná, tedy odpoví ANO nebo NE. Popište algoritmus, který může volat tuto černou skříňku a pro danou formuli ψ nalezne splňující ohodnocení, je-li ψ splnitelná. Algoritmus by měl pracovat v polynomiálním čase, pokud volání černé skříňky budeme počítat jako konstantní operaci.

17. Předpokládejte, že máte černou skříňku, která umí zodpovědět, zda v daném grafu existuje hamiltonovská kružnice. Popište algoritmus, který najde v daném grafu hamiltonovskou kružnici, pokud v něm existuje, tj. vypíše seznam jejích vrcholů v pořadí na této kružnici. Algoritmus bude polynomiální, pokud bychom brali volání zmíněné černé skříňky jako konstantní.
18. Popište algoritmus, který v polynomiálním čase vyřeší úlohu HORN-SAT.

Problém 11.30: SPLNITELNOST HORNOVSKÝCH FORMULÍ (HORN-SAT)

Instance: Formule φ v KNF, kde každá klauzule obsahuje nejvýš jeden pozitivní literál.

Cíl: Hledáme ohodnocení proměnných v , pro něž je $\varphi(v) = 1$, nebo očekáváme odpověď, že takové ohodnocení neexistuje.

19. Popište algoritmus, který v polynomiálním čase vyřeší úlohu 2SAT.

Problém 11.31: SPLNITELNOST KVADRATICKÝCH FORMULÍ (2SAT)

Instance: Formule φ v KNF, kde každá klauzule obsahuje nejvýš dva literály.

Cíl: Hledáme ohodnocení proměnných v , pro něž je $\varphi(v) = 1$, nebo očekáváme odpověď, že takové ohodnocení neexistuje.

12. Pseudopolynomiální algoritmy a silná NP-úplnost

I mezi různými NP-úplnými problémy a úlohami může být co do složitosti jejich řešitelnosti rozdíl. V této kapitole se budeme věnovat problémům, které je možné řešit pseudopolynomiálními algoritmy.

12.1. Pseudopolynomiální algoritmus pro batoh

Úloha Batohu je definovaná následovně.

Problém 12.1: BATOH (KNAPSACK)

Instance: Množina n předmětů $A = \{a_1, \dots, a_n\}$, s každým předmětem asociovaná velikost $s(a_i) \in \mathbb{N}$ a cena či hodnota $v(a_i) \in \mathbb{N}$. Přírozené číslo $B \geq 0$ udávající velikost batohu.

Cíl: Nalézt množinu předmětů $A' \subseteq A$, která dosahuje maximální souhrnné hodnoty předmětů v ní obsažených, a přitom se předměty z A' vejdou do batohu velikosti B . Chceme tedy, aby platilo:

$$\sum_{a \in A'} s(a) \leq B$$
$$\sum_{a \in A'} v(a) = \max_{\substack{A'' \subseteq A \\ \sum_{a \in A''} s(a) \leq B}} \sum_{a \in A''} v(a)$$

Pro tuto úlohu můžeme zkonstruovat algoritmus, který není sice obecně polynomiální, ale pro jistým způsobem omezené instance se polynomiálním může stát. Algoritmus 31 je založen na dynamickém programování. Na vstupu tento algoritmus očekává jen předměty, které se do batohu vejdou, tj. jen předměty s velikostí nejvýš B , což je celkem přirozené, protože nemá smysl uvažovat předměty, které se do batohu nevejdou ani samy o sobě. Navíc splnění této podmínky lze snadno ověřit. Podobně o všech předmětech můžeme předpokládat, že jejich velikost je nenulová, neboť předměty s nulovou velikostí můžeme přidat do batohu vždy a není nutné je tedy v algoritmu uvažovat.

Ukážeme si, že tento algoritmus najde při vhodné implementaci řešení úlohy Batoh v čase $O(nV)$. Zdá se, tedy, že jde o polynomiální algoritmus, ale musíme si uvědomit,

Algoritmus 29: Batoh(s, v, B)

Vstup: Pole s velikostí předmětů a pole v cen předmětů, obě délky n , velikost batohu B . Předpokládáme, že $(\forall i \in \{1, \dots, n\}) [0 < s[i] \leq B]$.

Výstup: Množina M předmětů, jejichž souhrnná velikost nepřesahuje B a jejichž souhrnná cena je maximální.

```
1  $V \leftarrow \sum_{i=1}^n v[i]$ 
  /* Na pozici  $T[j, c]$ ,  $0 \leq j \leq n$ ,  $0 \leq c \leq V$ , bude podmnožina indexů
      $\{1, \dots, j\}$  prvků celkové ceny přesně  $c$  s minimální
     velikostí. */
2  $T \leftarrow$  tabulka typu  $(n+1) \times (V+1)$ 
  /* Na pozici  $S[j, c]$  je celková velikost předmětů v množině
      $T[j, c]$ . Pokud neexistuje množina s předměty ceny přesně  $c$ ,
     je na pozici  $S[j, c]$  číslo  $B+1$ . */
3  $S \leftarrow$  tabulka typu  $(n+1) \times (V+1)$ 
4 for  $c \leftarrow 0$  to  $V$  do
5    $T[0, c] \leftarrow \emptyset$ 
6    $S[0, c] \leftarrow B+1$ 
7  $S[0, 0] \leftarrow 0$ 
8 for  $j \leftarrow 1$  to  $n$  do
9    $T[j, 0] \leftarrow \emptyset$ 
10   $S[j, 0] \leftarrow 0$ 
11  for  $c \leftarrow 1$  to  $V$  do
12     $T[j, c] \leftarrow T[j-1, c]$ 
13     $S[j, c] \leftarrow S[j-1, c]$ 
14    if  $v[j] \leq c$  and  $S[j, c] > S[j-1, c-v[j]] + s[j]$  then
15       $T[j, c] \leftarrow T[j-1, c-v[j]] \cup \{j\}$ 
16       $S[j, c] \leftarrow S[j-1, c-v[j]] + s[j]$ 
17  $c \leftarrow \max\{c' \mid S[n, c'] \leq B\}$ 
18 return  $T[n, c]$ 
```

že velikost vstupu je pouze $O(n \log_2(B + V))$ a V tedy může být exponenciálně větší než vstup.

Poznámka 12.2 *Náš odhad složitosti, tedy $O(nV)$ vychází z předpokladu, že aritmetické operace vyžadují jen konstantní čas. To pochopitelně není úplně korektní předpoklad, neboť ve chvíli, kdy nemáme žádný odhad na velikost čísel na vstupu, nemůžeme předpokládat, že například sčítání bude konstantní. Na druhou stranu všechny aritmetické operace vyžadují jen čas polynomiální v délce binární reprezentace čísel, tedy v $\log_2(B + V)$, což není faktor, který by něco změnil na polynomiálnosti algoritmu 31. Dovolíme si tedy zjednodušení, které je ve složitosti obvyklé, a budeme počítat složitost aritmetických operací jako konstantní.*

Věta 12.3 *Algoritmus 31 nalezne pro zadaný vstup množinu předmětů s nejvyšší cenou, jež se vejde do batohu velikosti B . Algoritmus 31 pracuje v čase $O(nV)$.*

Důkaz: Fakt, že algoritmus pracuje v čase $O(nV)$ je v podstatě zřejmý. Kroky 1 až 7 zvládneme jistě v čase $O(nV)$, uvažujeme-li aritmetické operace jako konstantní. Následují dva vnořené cykly v rámci nichž jsou použity aritmetické operace, na něž stačí konstantní čas. I řádek 15 lze provést v konstantním čase při vhodné reprezentaci množin v $T[j, c]$. Můžeme například použít reprezentaci pomocí bitového pole délky n , nebo pomocí spojového seznamu, v obou případech je přidání prvku do množiny jednoduché.

Zbývá ukázat, že algoritmus pracuje korektně. Ukážeme, že na konci běhu algoritmu splňují tabulky T a S vlastnosti, jež jsme popsali na řádcích 2 a 3. Jmenovitě ukážeme, že:

- (i) Na pozici $T[j, c]$, $0 \leq j \leq n$, $0 \leq c \leq V$ je po ukončení algoritmu podmnožina indexů $\{1, \dots, j\}$ prvků celkové ceny přesně c s minimální velikostí mezi všemi takovými množinami. Pokud neexistuje množina prvků s cenou přesně c , pak na pozici $T[j, c]$ bude prázdná množina.
- (ii) Na pozici $S[j, c]$, $0 \leq j \leq n$, $0 \leq c \leq V$ je po ukončení algoritmu součet velikostí prvků v množině na pozici $T[j, c]$. Pokud neexistuje množina prvků s cenou přesně c , pak na pozici $S[j, c]$ je $B + 1$.

Tyto vlastnosti ukážeme indukcí podle j . Na začátku vyplníme nultý řádek v cyklu na řádcích 4 až 6, pro $j = 0$ tedy vlastnosti (i) i (ii) platí. Nyní předpokládejme, že vlastnosti (i) a (ii) platí pro řádky $0, \dots, j - 1$ a ukažme, že platí i pro j -tý řádek. Na pozici $T[j, 0]$ je vždy prázdná množina, neboť to je jistě nejmenší množina prvků s cenou 0, prvky prázdné množiny totiž mají přirozeně nulovou velikost. Takto provedeme nastavení $T[j, 0]$ a $S[j, 0]$ na řádcích 9 a 10. Pokud je $c > 0$, pak na ně narazíme na vhodném místě v rámci vnitřního cyklu. Pro $j > 0$ a $c > 0$ máme dvě možnosti, buď v nejmenší množině prvků $\{1, \dots, j\}$ s cenou c není prvek j , potom se jedná o množinu uloženou podle indukční hypotézy na pozici $T[j - 1, c]$, nebo se v této množině prvek j vyskytuje. Pokud bychom z této nejmenší množiny prvek j odstranili, museli bychom dostat nejmenší množinu s cenou $c - v[j]$ z prvků $\{1, \dots, j - 1\}$, která je podle indukčního předpokladu uložena na pozici $T[j - 1, c - v[j]]$. Z těchto dvou možností vybereme tu, která má menší

velikost. Díky tomu také na pozici $T[j, c]$ uložíme prázdnou množinu a na pozici $S[j, c]$ hodnotu $B + 1$ jedině tehdy, pokud obě množiny na pozicích $T[j - 1, c]$ a $T[j - 1, c - v[j]]$ jsou prázdné a obě odpovídající velikosti jsou $B + 1$. Zde využíváme toho, že porovnávání v kroku 14 je ostré, a tedy k přiřazení v kroku 12 dojde jedině tehdy, když je množina na pozici $T[j - 1, c - v[j]]$ neprázdná, jinak by z prázdné množiny přidáním prvku j vznikla množina neprázdná.

Na závěr tedy při platnosti (i) a (ii) stačí v kroku 17 vybrat množinu s největší cenou. Množina cen, z nichž vybíráme, je vždy neprázdná, neboť přinejmenším $S[n, 0] = 0$. $\square$

Poznamenejme ještě, že ve skutečnosti není potřeba pamatovat si celé matice T a S , stačí úplně jedno pole T' a S' s aktuálním řádkem matic T a S , přičemž ale musíme toto pole procházet v klesajícím pořadí podle cen. To proto, že pro určení množiny na pozici $T[j, c]$ potřebujeme hodnoty pouze z předchozího kroku, tedy pro $j - 1$ a s cenou nejvýše rovnou c , pokud tedy pole procházíme podle klesajících cen, je vždy na pozici $T'[c']$ pro $c' \leq c$ množina z $T[j - 1, c']$, totéž lze říci o velikostech v poli S' . Tento postup vede k výrazné úspoře paměti, nemění však nic na časové složitosti, o kterou se v tuto chvíli zajímáme především.

Protože velikost vstupu je $O(n \log_2(B + V))$ při standardním binárním kódování, může být ve skutečnosti čas $O(nV)$ exponenciální ve velikosti vstupu, a proto nejde o polynomiální algoritmus. Nicméně, pokud by existoval polynom p , pro který by platilo, že $(B + V) \leq p(n)$, pak by Algoritmus 31 byl polynomiální. Tuto podmínku samozřejmě nelze obecně zaručit, ale znamená to, že pro rozumně malá čísla na vstupu poběží algoritmus rozumně rychle. Druhý pohled na věc je, že jde o algoritmus polynomiální, pokud předáme vstup zakódovaný unárně, protože při unárně zadaném vstupu je jeho velikost $O(n(B + V))$. Algoritmům tohoto typu budeme říkat pseudopolynomiální.

12.2. Číselné problémy a pseudopolynomiální algoritmy

Upřesněme si tyto pojmy následující definicí.

Definice 12.4 (Číselný problém) Nechť A je libovolný rozhodovací problém a I nechť je instance tohoto problému. Pomocí $\text{len}(I)$ označíme délku zakódování instance I při standardním binárním kódování. Pomocí $\max(I)$ označíme hodnotu největšího číselného parametru v instanci I . Řekneme, že A je *číselný problém*, pokud pro každý polynom p existuje instance I problému A , pro kterou platí, že $\max(I) > p(\text{len}(I))$. $\blacktriangleleft$

V případě batohu je tedy $\text{len}(I) = O(n \log_2(B + V))$, zatímco $\max(I) = \max(\{B\} \cup \{v[i], s[i] \mid 1 \leq i \leq n\})$. Například Loupežníci nebo Batoh jsou tedy číselné problémy, zatímco SAT nebo KLIKA číselné problémy nejsou.

Definice 12.5 (Pseudopolynomiální algoritmus) Řekneme, že algoritmus řešící problém A je *pseudopolynomiální*, pokud je jeho časová složitost omezená polynomem dvou proměnných $\max(I)$ a $\text{len}(I)$. $\blacktriangleleft$

Algoritmus 31 je tedy zřejmě pseudopolynomiální. Je-li problém A řešitelný pseudopolynomiálním algoritmem a není-li A číselný problém, pak je zřejmě tento pseudo-

polynomiální algoritmus ve skutečnosti polynomiální. Pokud bychom tedy pro nějaký nečíselný NP-úplný problém, například kliku nebo splnitelnost, našli pseudopolynomiální algoritmus, znamenalo by to, že $P = NP$.

12.3. Silná NP-úplnost

Definice 12.6 (Silná NP-úplnost) Necht p je polynom a A NP-úplný problém. Pomocí $A(p)$ označíme restrikcí problému A na instance I s $\max(I) \leq p(\text{len}(I))$, tedy instance I je instancí $A(p)$, pokud $\max(I) \leq p(\text{len}(I))$. Řekneme, že problém A je *silně NP-úplný*, pokud existuje polynom p , pro nějž je problém $A(p)$ NP-úplný. ◀

Uvedená definice by pochopitelně ztratila smysl, kdyby platilo $P = NP$, protože potom by byly všechny netriviální problémy z $P = NP$ silně NP-úplné. I pokud to nezmiňujeme, předpokládáme však obvykle opak, tedy, že platí $P \neq NP$. Z našich předchozích úvah plyne, že každý nečíselný NP-úplný problém je automaticky silně NP-úplný. Pokud bychom pro nějaký silně NP-úplný problém našli pseudopolynomiální algoritmus, znamenalo by to, že $P = NP$. To proto, že na $A(p)$ by byl tento algoritmus polynomiální. Z toho plyne, že například problém Batoh není (pokud $P \neq NP$) silně NP-úplný a totéž lze říci například o problému Loupežníci, pro který lze použít jen mírnou modifikaci algoritmu 31. Je také třeba zmínit, že pokud $P \neq NP$, pak existují i NP-úplné problémy, jež nejsou ani silně NP-úplné ani řešitelné pseudopolynomiálním algoritmem. To je analogické tomu, že pokud $NP \neq P$, existují v NP problémy, jež nejsou v P ani nejsou NP-úplné. Jak již bylo zmíněno, rozdíl mezi P a algoritmy řešitelnými pseudopolynomiálním algoritmem tkví ve způsobu kódování — binárního v případě P a unárního v případě pseudopolynomiálního algoritmu. Podobně tedy silně NP-úplné problémy jsou ty NP-úplné problémy, jež zůstanou NP-úplnými i v případě unárního kódování čísel na vstupu.

Jako příklad číselného problému, který je NP-úplný nám může posloužit již vícekrát zmiňovaný problém obchodního cestujícího.

Problém 12.7: OBCHODNÍ CESTUJÍCÍ (OC, TRAVELING SALESPERSON)

Instance: Množina měst $C = \{c_1, \dots, c_n\}$, vzdálenost $d(c_i, c_j) \in \mathbb{N}$ pro každá dvě města $c_i, c_j \in C$, přirozené číslo D .

Otázka: Existuje cyklus, který navštíví každé město právě jednou a jehož délka nepřesahuje D ? Jinými slovy, existuje permutace $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ taková, že

$$\sum_{i=1}^{n-1} d(c_{\pi(i)}, c_{\pi(i+1)}) + d(c_{\pi(n)}, c_{\pi(1)}) \leq D?$$

Věta 12.8 Problém Obchodního cestujícího je silně NP-úplný.

Důkaz: Problém OC jistě patří do třídy NP, neboť jsme schopni ověřit v polynomiálním čase, zda daná permutace měst splňuje naše požadavky, a zřejmě i $OC(p)$ patří do NP pro každý polynom p . Těžkost problému $OC(p)$ pro vhodně zvolený polynom p si ukážeme převodem z problému Hamiltonovské kružnice v neorientovaném grafu. Uvažme neorientovaný graf $G = (V, E)$ a sestavme na jeho základě instanci obchodního cestujícího následujícím způsobem. Množinu měst C položíme rovnu množině vrcholů V , předpokládejme tedy, že $C = V = \{v_1, \dots, v_n\}$. Vzdálenost mezi městy či vrcholy v_i a v_j pro $i \neq j$ a $i, j \in \{1, \dots, n\}$ určíme následujícím předpisem.

$$d(v_i, v_j) = \begin{cases} 0 & \{v_i, v_j\} \in E \\ 1 & \text{jinak} \end{cases}$$

Hodnotu D položíme rovnu 0. Není těžké ukázat, že v grafu G existuje hamiltonovská kružnice, právě když v sestrojené instanci obchodního cestujícího existuje cyklus nulové délky, který obejde všechna města, přičemž navštíví každé z nich právě jednou. Z jedné strany hamiltonovská kružnice využívá jen hrany nulové délky, z druhé strany cesta nulové délky využívá jen hrany grafu G . Hodnota maximálního číselného parametru v sestrojené instanci je přitom rovna 1, tedy omezená verze problému $OC(1)$ je stále NP-úplná, což znamená, že problém OC je silně NP-úplný. $\square$

Dalšími silně NP-úplnými číselnými problémy jsou například Rozvrhování či Bin Packing. Najdeme-li k nějakému problému pseudopolynomiální algoritmus, má to hned dva kladné důsledky, jednak lze takový algoritmus použít na instance, v nichž se nevyskytují příliš velká čísla. Na instancích, kde už by i pseudopolynomiální algoritmus počítal příliš dlouho, je často alespoň možné nalézt dobrou aproximaci optimálního řešení, jak si záhy ukážeme v případě Batohu.

12.4. Cvičení

1. Ukažte, že následující problém je polynomiálně řešitelný:

Problém 12.9: OMEZENÝ SOUČET PODMNOŽINY

Instance: Množina n prvků A , s každým prvkem $a \in A$ asociovaná velikost $v(a) \in \{0, \dots, n\}$, číslo $B \geq 0$.

Otázka: Existuje množina prvků $A' \subseteq A$, pro níž platí, že

$$\sum_{a \in A'} v(a) = B?$$

13. Aproximační algoritmy a schémata

13.1. Aproximační algoritmy

Koncept aproximacních algoritmů je velmi užitečný pro řešení NP-úplných optimalizačních úloh. Pokud nejsme schopni rychle získat optimální řešení úlohy, můžeme slevit ze svých požadavků a pokusit se najít řešení, jež není od toho optimálního příliš vzdáleno. Nejprve si upřesníme pojem optimalizační úlohy.

Definice 13.1 *Optimalizační úloha* A je buď *maximalizační* nebo *minimalizační* a skládá se z těchto tří částí:

1. Množina instancí $D_A \subseteq \{0,1\}^*$.
2. Pro každou instanci $I \in D_A$ je dána množina přípustných řešení $S_A(I) \subseteq \{0,1\}^*$.
3. Funkce μ_A , která každé instanci $I \in D_A$ a každému přípustnému řešení $\sigma \in S_A(I)$ přiřadí kladné racionální číslo $\mu_A(I, \sigma)$, které nazveme *hodnotou řešení* σ .

Pokud je A maximalizační úlohou, pak *optimálním řešením* instance $I \in D_A$ je $\sigma \in S_A(I)$, pro něž je $\mu_A(I, \sigma)$ maximální (tj. $\mu_A(I, \sigma) = \max\{\mu_A(I, \sigma) \mid \sigma \in S_A(I)\}$). Pokud je A minimalizační úlohou, pak *optimálním řešením* instance $I \in D_A$ je $\sigma \in S_A(I)$, pro něž je $\mu_A(I, \sigma)$ minimální (tj. $\mu_A(I, \sigma) = \min\{\mu_A(I, \sigma) \mid \sigma \in S_A(I)\}$). Hodnotu optimálního řešení instance $I \in D_A$ budeme označovat pomocí $OPT_A(I)$, přičemž index A budeme často vynechávat, bude-li jasné, o jakou úlohu se jedná. Tedy je-li σ^* optimální řešení instance I , pak $OPT_A(I) = \mu_A(I, \sigma^*)$. ◀

Uvažme například minimalizační úlohu Vrcholového pokrytí, v tomto případě je množina instancí D_{VP} tvořena řetězcí kódujícími graf. Pro danou instanci, tedy graf $G = (V, E)$, obsahuje množina přípustných řešení $S_{VP}(G)$ všechny množiny vrcholů $S \subseteq V$, které pokrývají všechny hrany. Mírou daného přípustného řešení S je pak počet jejích prvků, tedy $\mu_{VP}(G, S) = |S|$. Jde o minimalizační úlohu, neboť se snažíme najít co nejmenší množinu, která pokrývá všechny hrany. Všimněme si, že najít nějakou množinu S , která pokrývá všechny vrcholy, je snadné, stačí vzít třeba množinu všech vrcholů $S = V$, co činí úlohu obtížnou, je právě to, že chceme minimalizovat její velikost. To je obvyklé u řady úloh, pro něž hledáme aproximační algoritmy, protože pokud je těžké najít vůbec nějaké přípustné řešení pro danou instanci, pak je o to těžší najít optimální řešení.

Nyní můžeme definovat pojem aproximačního algoritmu.

Definice 13.2 Řekneme, že algoritmus ALG je *aproximačním algoritmem* pro optimalizační úlohu A , pokud pro každou instanci $I \in D_A$ vrátí ALG se vstupem I řešení $\sigma \in S_A(I)$, případně ohlásí, že žádné přípustné řešení neexistuje, pokud $S_A(I) = \emptyset$. Hodnotu řešení

vráceného algoritmem ALG na instanci I označíme jako $ALG(I)$, tj. $ALG(I) = \mu_A(I, \sigma)$, kde $\sigma \in S_A(I)$ je přípustné řešení vrácené algoritmem ALG . *Aproximační poměr* algoritmu ALG definujeme takto: Pokud je A maximalizační úloha, pak racionální číslo $\varepsilon \geq 1$ nazveme *aproximačním poměrem* algoritmu ALG , pokud pro každou instanci $I \in D_A$ platí, že

$$OPT(I) \leq \varepsilon \cdot ALG(I).$$

Je-li A minimalizační úlohou, pak racionální číslo $\varepsilon \geq 1$ nazveme *aproximačním poměrem* algoritmu ALG , pokud pro každou instanci $I \in D_A$ platí, že

$$ALG(I) \leq \varepsilon \cdot OPT(I).$$

Naše definice aproximačního poměru umožňuje přistupovat jednotně k minimalizačním i maximalizačním úlohám, často se též definuje aproximační poměr pro maximalizační úlohu jako obrácená hodnota námi definovaného aproximačního poměru, na tom, který způsob jsme si zvolili, však příliš nezáleží.

13.2. Příklad aproximačního algoritmu pro Bin Packing

Začneme jednoduchým aproximačním algoritmem pro úlohu Bin Packing.

Problém 13.3: BIN PACKING (BP)

Instance: Konečná množina předmětů $U = \{u_1, \dots, u_n\}$, s každým předmětem asociovaná velikost $s(u)$, což je racionální číslo, pro které platí $0 \leq s(u) \leq 1$.

Cíl: Najít rozdělení všech předmětů do co nejmenšího počtu po dvou disjunktních množin $U_1, \dots, U_m$ takové, že

$$(\forall i \in \{1, \dots, m\}) \left[\sum_{u \in U_i} s(u) \leq 1 \right].$$

Naším cílem je minimalizovat m .

Formálně je tedy D_{BP} množinou řetězců kódujících instance BP , pro danou instanci $I = \langle U, s(u) \rangle$ je množina $S_{BP}(I)$ množinou všech možných rozdělení do dostatečného množství košů. Mírou řešení $\mu_{BP}(\sigma)$ pro $\sigma \in S_{BP}(I)$ je počet košů, které řešení využívá, tedy hodnota m . Rozhodovací verze tohoto problému je shodná s problémem Rozvrhování, jehož těžkost jsme si ukazovali v rámci cvičení, z toho plyne, že i úloha BP je NP-těžká. Šance na to, že bychom našli polynomiální algoritmus, řešící BP přesně, jsou tedy malé.

Uvažme jednoduchý hladový algoritmus, kterým bychom řešili tuto úlohu: *Ber jeden předmět po druhém, pro každý předmět u najdi první koš, do něž se tento předmět ještě vejde, pokud takový koš neexistuje, přidej nový koš obsahující jen předmět u .* Tomuto algoritmu budeme říkat „First Fit“ (FF). Algoritmus FF je zřejmě polynomiální. Ukážeme si, že pro

každou instanci $I \in D_{BP}$ platí, že $FF(I) \leq 2 \cdot OPT(I)$, jinými slovy, že aproximační poměr algoritmu FF je 2.

Věta 13.4 Pro každou instanci $I \in D_{BP}$ platí, že $FF(I) < 2 \cdot OPT(I)$.

Důkaz: V řešení, které vrátí FF je nejvýš jeden koš, který je zaplněn nejvýš z poloviny. Kdyby totiž existovaly dva koše U_i, U_j pro $i < j$, které jsou zaplněny nejvýš z poloviny, tak by FF nepotřeboval zakládat nový koš pro předměty z U_j , všechny by se vešly do U_i . Pokud $FF(I) > 1$, pak z toho plyne, že

$$FF(I) < \left\lceil 2 \sum_{i=1}^n s(u_i) \right\rceil \leq 2 \left\lceil \sum_{i=1}^n s(u_i) \right\rceil,$$

kde první nerovnost plyne z toho, že po zdvojnásobení obsahu jsou všechny koše plné až na jeden, který může být zaplněn jen částečně. Rovnosti bychom přitom dosáhli jedinečně ve chvíli, kdy by byly všechny koše zaplněné právě z poloviny, což není podle našeho předpokladu možné. Druhá nerovnost plyne z vlastností zaokrouhlování.

Na druhou stranu musí platit, že

$$OPT(I) \geq \left\lceil \sum_{i=1}^n s(u_i) \right\rceil.$$

Dohromady tedy dostaneme, že $FF(I) < 2 \cdot OPT(I)$. Pokud $FF(I) = 1$, pak i $OPT(I) = 1$ a i v tomto případě platí ostrá nerovnost. $\square$

Lze dokonce ukázat o něco lepší odhad $FF(I) \leq \frac{17}{10} OPT(I) + 2$. Na druhou stranu však existují instance I s libovolně velkou hodnotou $OPT(I)$, pro něž $FF(I) \geq \frac{17}{10}(OPT(I) - 1)$. My si ukážeme příklad instancí, pro něž je $FF(I) \geq \frac{5}{3} OPT(I)$, což není od 2 příliš vzdáleno.

Lemma 13.5 Pro libovolnou hodnotu m existuje instance $I \in D_{BP}$, pro niž je $OPT(I) \geq m$ a $FF(I) \geq \frac{5}{3} OPT(I)$.

Důkaz: Instance bude mít $U = \{u_1, u_2, \dots, u_{18m}\}$, s těmito prvky asociujeme váhy takto:

$$s(u_i) = \begin{cases} \frac{1}{7} + \varepsilon & 1 \leq i \leq 6m, \\ \frac{1}{3} + \varepsilon & 6m < i \leq 12m, \\ \frac{1}{2} + \varepsilon & 12m < i \leq 18m, \end{cases}$$

kde $\varepsilon > 0$ je dostatečně malé kladné racionální číslo. Optimální rozdělení rozdělí prvky do $6m$ košů, do každého dá po jednom prvku s velikostmi $\frac{1}{7} + \varepsilon, \frac{1}{3} + \varepsilon, \frac{1}{2} + \varepsilon$. Hodnotu ε zvolíme dostatečně malou tak, aby se každá z těchto trojic vešla do koše velikosti 1. Algoritmus FF bude brát prvky jeden po druhém a vytvoří nejprve m košů, každý s šesti prvky velikosti $\frac{1}{7} + \varepsilon$, přičemž hodnotu ε zvolíme dostatečně malou na to, aby se tam

vešly, ale protože je kladná, nevejde se k nim nic dalšího. Poté vytvoří $3m$ košů, každý se dvěma prvky velikosti $\frac{1}{3} + \varepsilon$, k nimž se opět nic nevejde. Nakonec FF vytvoří $6m$ košů, každý s jedním prvkem velikosti $\frac{1}{2} + \varepsilon$. Dohromady je $FF(I) = 10m$, a tedy $\frac{FF(I)}{OPT(I)} = \frac{5}{3}$. $\square$

Pochopitelně brát prvky v libovolném pořadí tak, jak to činí FF je nejloupější možnou strategií. Na instanci popsanou v důkazu lemmatu 13.5 by přitom stačilo, kdybychom nejprve prvky U setřídili sestupně podle velikosti a poté je umísťovali do košů v pořadí od největšího k nejmenšímu. Algoritmus, který postupuje podle této strategie nazveme FFD z anglického „First Fit Decreasing“. Lze ukázat, že pro libovolnou instanci $I \in D_{BP}$ platí

$$FFD(I) \leq \frac{11}{9}OPT(I) + 4.$$

My si pouze ukážeme příklad libovolně velké instance, pro níž je

$$FFD(I) \geq \frac{11}{9}OPT(I).$$

To ukazuje, že uvedený odhad nelze příliš zlepšit. Instance bude mít $U = \{u_1, \dots, u_{30m}\}$,

$$s(u_i) = \begin{cases} \frac{1}{2} + \varepsilon & 1 \leq i \leq 6m, \\ \frac{1}{4} + 2\varepsilon & 6m < i \leq 12m, \\ \frac{1}{4} + \varepsilon & 12m < i \leq 18m, \\ \frac{1}{4} - 2\varepsilon & 18m < i \leq 30m, \end{cases}$$

kde $\varepsilon > 0$ je opět dostatečně malé racionální číslo. Optimální počet košů pro tuto instanci je $9m$, $6m$ košů, z nichž každý obsahuje $\{\frac{1}{4} - 2\varepsilon, \frac{1}{4} + \varepsilon, \frac{1}{2} + \varepsilon\}$ a $3m$ košů, z nichž každý obsahuje $\{\frac{1}{4} - 2\varepsilon, \frac{1}{4} - 2\varepsilon, \frac{1}{4} + 2\varepsilon, \frac{1}{4} + 2\varepsilon\}$. Tento počet skutečně nelze zlepšit, protože všechny koše jsou zcela zaplněny. Jak se však zachová FFD ? Tento algoritmus vytvoří dohromady $11m$ košů. $6m$ košů bude obsahovat $\{\frac{1}{2} + \varepsilon, \frac{1}{4} + 2\varepsilon\}$, $2m$ košů bude obsahovat $\{\frac{1}{4} + \varepsilon, \frac{1}{4} + \varepsilon, \frac{1}{4} + \varepsilon\}$ a $3m$ košů bude obsahovat $\{\frac{1}{4} - 2\varepsilon, \frac{1}{4} - 2\varepsilon, \frac{1}{4} - 2\varepsilon, \frac{1}{4} - 2\varepsilon\}$.

13.3. Úplně polynomiální aproximační schéma pro Batoh

Vraťme se nyní k úloze Batohu, pro který jsme dříve zkonstruovali pseudopolynomiální algoritmus. Tohoto algoritmu využijeme při konstrukci aproximačního algoritmu, jehož aproximační poměr si můžeme předepsat v rámci vstupu. Vzpomeňme si, že algoritmus 31 pracuje dle věty 12.3 v čase $O(nV)$, kde n je počet předmětů a V je jejich celková cena. Řekli jsme si také, že tento čas by byl polynomiální, kdyby hodnota V byla omezena nějakým polynomem $p(n)$. Hodnota V se do odhadu složitosti algoritmu 31 dostala jako horní odhad potenciálních cen předmětů v batohu. Pokud provedeme zao-
krouhlení cen předmětů a poté jejich přeškálování, můžeme zmenšit součet nových cen

předmětů a tím snížit časové nároky algoritmu 31. Zaokrouhlením však ztratíme optimalitu výsledku. Čím hrubější zaokrouhlení vstupních cen provedeme, tím rychlejší běh algoritmu dostaneme, ale tím horšího výsledku dosáhneme. Této myšlenky využívá algoritmus 32, kde parametr ε určuje míru zaokrouhlení vstupních cen, což se projeví jednak v časových nárocích algoritmu, jednak v jeho aproximačním poměru.

Algoritmus 30: Batoh-apx(s, v, B, ε)

Vstup: Pole s velikostí předmětů a pole v cen předmětů, obě délky n , velikost batohu B , racionální číslo $\varepsilon > 0$. Předpokládáme, že
 $(\forall i \in \{1, \dots, n\}) [0 < s[i] \leq B]$

Výstup: Množina M předmětů, jejichž souhrnná velikost nepřesahuje B .

```

1 Spočti index  $m$ , pro nějž je  $v[m] = \max_{1 \leq i \leq n} v[i]$ .
2 if  $\varepsilon \geq n - 1$  then
3   return  $\{m\}$ 
4  $t \leftarrow \lfloor \log_2(\frac{\varepsilon \cdot v[m]}{n}) \rfloor - 1$ 
5  $c$  je nové pole délky  $n$ .
6 for  $i \leftarrow 1$  to  $n$  do
7    $c[i] \leftarrow \lfloor \frac{v[i]}{2^t} \rfloor$ 
8 return Batoh( $s, c, B$ ) {Viz algoritmus 31}
```

Hodnota t , která určuje zaokrouhlení, je samozřejmě zvolena tak, aby správně vyšel aproximační poměr a časová složitost algoritmu. Pokud si odmyslíme celé části, můžeme nahlédnout, že

$$\frac{v[i]}{2^t} \sim \frac{v[i]}{\frac{\varepsilon \cdot v[m]}{n} \frac{1}{2}} = \frac{2 \cdot n \cdot v[i]}{\varepsilon \cdot v[m]}.$$

Poměr mezi $v[i]$ a $v[m]$ určuje, kolik procent zabírá $v[i]$ vzhledem k $v[m]$. Základním cílem přepočtu je pochopitelně zmenšit hodnotu $v[m]$, odpovídajícím způsobem se proporcionálně musí zmenšit i $v[i]$. Dalším důležitým faktorem je podíl ε/n . To vychází z toho, že chceme-li celkově dosáhnout chyby ε , můžeme si na jednom prvku dovolit chybu ε/n . Ve výpočtu se uvažuje opačná hodnota podílu, tedy n/ε , protože chybu chceme dosáhnout v přepočtené hodnotě $c[i]$ a ne v $v[i]$.

Ukažme si nyní, jakého aproximačního poměru a jakého času dosáhneme s algoritmem 32 při zadané hodnotě ε .

Věta 13.6 Algoritmus 32 pracuje v čase $O(\frac{1}{\varepsilon} n^3)$. Pro libovolnou instanci $I = \langle s, v, B \rangle$ úlohy Batoh a libovolné $\varepsilon > 0$ platí, že

$$OPT(I) \leq (1 + \varepsilon) \text{Batoh-apx}(I, \varepsilon)$$

Důkaz: Nejprve se zaměříme na aproximační poměr algoritmu 32. Pokud je $\varepsilon \geq n - 1$, pak algoritmus vrátí množinu $\{m\}$, to je jistě přípustné řešení, neboť předpokládáme, že $s[m] \leq B$. Protože zřejmě $OPT(I) \leq n \cdot v[m]$, dostáváme:

$$\frac{OPT(I)}{\text{Batoh-apx}(I, \varepsilon)} = \frac{OPT(I)}{v[m]} \leq \frac{n \cdot v[m]}{v[m]} = n \leq 1 + \varepsilon$$

Nyní předpokládejme, že $\varepsilon < n - 1$. V dalším důkazu použijeme následujícího pozorování: Pro každé reálné číslo x platí

$$x - 1 \leq \lfloor x \rfloor \leq x. \quad (13.1)$$

Použijeme-li tento odhad na novou cenu $c[i]$, dostaneme

$$\frac{v[i]}{2^t} - 1 \leq \left\lfloor \frac{v[i]}{2^t} \right\rfloor \leq \frac{v[i]}{2^t}.$$

A tedy podle definice $c[i]$ a (13.1)

$$v[i] - 2^t \leq c[i] \cdot 2^t \leq v[i] \quad (13.2)$$

Využijeme-li pozorování (13.1) pro odhad 2^t zdola, obdržíme

$$\frac{1}{4} \frac{\varepsilon \cdot v[m]}{n} = 2^{\log_2(\frac{\varepsilon \cdot v[m]}{n}) - 2} \leq 2^t \quad (13.3)$$

a pro odhad shora dostaneme společně s faktem $\varepsilon < n - 1$, že

$$2^t \leq 2^{\log_2(\frac{\varepsilon \cdot v[m]}{n}) - 1} = \frac{1}{2} \cdot \frac{\varepsilon \cdot v[m]}{n} < \frac{1}{2} \cdot v[m]. \quad (13.4)$$

Množina M , kterou algoritmus vrátí v kroku 8, splňuje

$$\text{Batoh-apx}(I, \varepsilon) = \sum_{i \in M} v[i] \geq \sum_{i \in M} c[i] \cdot 2^t,$$

kde nerovnost plyne z 13.2. Nechť M^* je optimální řešení dané instance I , tj. množina prvků s maximální cenou v , jež se vejde do batohu velikosti B . Protože M je dle věty 12.3 optimální řešení pro instanci $I' = \langle s, c, B \rangle$, zatímco M^* je přípustné řešení této upravené instance, můžeme pokračovat:

$$\begin{aligned} \sum_{i \in M} c[i] \cdot 2^t &\geq \sum_{i \in M^*} c[i] \cdot 2^t && \text{dle (13.2)} \\ &\geq \sum_{i \in M^*} (v[i] - 2^t) = \\ &= \sum_{i \in M^*} v[i] - \sum_{i \in M^*} 2^t \geq \\ &\geq \text{OPT}(I) - n \cdot 2^t \end{aligned}$$

Z toho dostaneme, že

$$\frac{\text{OPT}(I)}{\text{Batoh-apx}(I, \varepsilon)} \leq 1 + \frac{n \cdot 2^t}{\text{Batoh-apx}(I, \varepsilon)}.$$

Stačí tedy ukázat, že

$$\frac{n \cdot 2^t}{\text{Batoh-apx}(I, \varepsilon)} \leq \varepsilon.$$

Protože M je optimálním řešením I' a $\{m\}$ je přípustným řešením I' , dostaneme, že

$$\text{Batoh-apx}(I, \varepsilon) \geq \sum_{i \in M} c[i] \cdot 2^t \geq c[m] \cdot 2^t \geq \quad (\text{dle 13.2})$$

$$\geq v[m] - 2^t \geq \quad (\text{dle 13.4})$$

$$\geq v[m] - \frac{1}{2}v[m] = \frac{1}{2}v[m].$$

Dohromady se 13.4 tedy dostaneme, že

$$\frac{n \cdot 2^t}{\text{Batoh-apx}(I, \varepsilon)} \leq \frac{n \cdot \frac{1}{2} \cdot \frac{\varepsilon \cdot v[m]}{n}}{\frac{1}{2} \cdot v[m]} = \varepsilon.$$

Nyní odhadneme časové nároky algoritmu 32. Vyjma kroku 8 lze zřejmě všechny kroky provést dokonce v čase $O(n)$, tedy v čase lineárním vzhledem k velikosti vstupu. Zde opět předpokládáme, že aritmetické operace lze provést v konstantním čase, což je zjednodušení, které jsme si dovolili v poznámce 12.2 bez újmy na obecnosti předpokládat. Z věty 12.3 víme, že krok 8 zabere čas $O(nC)$, kde $C = \sum_{i=1}^n c[i]$. Stačí tedy odhadnout hodnotu C .

$$C = \sum_{i=1}^n c[i] \leq n \cdot c[m] \leq \frac{n \cdot v[m]}{2^t} \leq \frac{n \cdot v[m]}{\frac{1}{4} \cdot \frac{\varepsilon \cdot v[m]}{n}} = \frac{4n^2}{\varepsilon},$$

kde předposlední nerovnost plyne z (13.3). Platí tedy, že $C = O(\frac{1}{\varepsilon}n^2)$. Volání algoritmu 31 tedy zabere čas $O(\frac{1}{\varepsilon}n^3)$, tím je dán i celkový čas algoritmu 32. $\square$

13.4. Aproximační schémata

To, co jsme získali algoritmem 32, je postup, jak pro libovolné $\varepsilon > 0$ dostat aproximační algoritmus pro úlohu Batohu s aproximačním poměrem $1 + \varepsilon$, který je polynomiální ve velikosti vstupu a $\frac{1}{\varepsilon}$. Takovému schématu, parametrizovanému hodnotou ε , budeme říkat úplně polynomiální aproximační schéma.

Definice 13.7 Nechť A je libovolná optimalizační úloha. Algoritmus ALG nazveme *aproximačním schématem* pro úlohu A , pokud na vstupu očekává instanci $I \in D_A$ a racionální číslo $\varepsilon > 0$ a na výstupu vydá řešení $\sigma \in S_A(I)$, jehož hodnota se od optimální liší s aproximačním poměrem $(1 + \varepsilon)$. Pro maximalizační úlohu tedy platí, že

$$OPT(I) \leq (1 + \varepsilon)ALG(I, \varepsilon)$$

a pro minimalizační úlohu platí, že

$$ALG(I, \varepsilon) \leq (1 + \varepsilon)OPT(I).$$

Předpokládejme, že algoritmus ALG je aproximační schéma a očekává na vstupu instanci $I \in D_A$ a racionální číslo ε . Pomocí ALG_ε označíme instanci algoritmu ALG , kde

hodnota ε je zafixována, vstupem ALG_ε je tedy jen instance $I \in D_A$ a běh i výstup ALG_ε na vstupu I jsou totožné s algoritmem ALG se vstupem I a ε .

Řekneme, že ALG je *polynomiální aproximační schéma* (PAS), pokud je pro každé ε časová složitost algoritmu ALG_ε polynomiální v $\text{len}(I)$, kde ALG_ε označuje algoritmus vzniklý dosazením konstanty ε do ALG .

Řekneme, že ALG je *úplně polynomiální aproximační schéma* (ÚPAS), pokud ALG pracuje v čase polynomiálním v $\text{len}(I)$ a $\frac{1}{\varepsilon}$. ◀

Rozdíl mezi polynomiálním aproximačním schématem a úplně polynomiálním aproximačním schématem je v tom, že v případě PAS se může ve funkci odhadující složitost algoritmu ALG_ε objevit hodnota $1/\varepsilon$ i v exponentu, což není možné v případě ÚPAS. Například složitost $O(n^{\frac{1}{\varepsilon}})$ by byla naprosto v pořádku pro PAS, ale nikoli pro ÚPAS.

Algoritmus 32 tedy tvoří úplně polynomiální aproximační schéma pro úlohu Batoh. Dá se říci, že úplně polynomiální aproximační schéma maximum, čeho lze s aproximačními algoritmy dokázat v případě, že nemáme přímo polynomiální algoritmus pro danou úlohu. Navíc technika, kterou jsme použili při konstrukci algoritmu 32 je obecnější a lze ji použít i v mnoha jiných případech, kdy máme k dispozici pseudopolynomiální algoritmus, před jehož použitím jde jen o to vhodně zaokrouhlit vstupní hodnoty.

Tento postup lze i obrátit, pokud se nám pro úlohu A , která splňuje jistá omezení, podaří popsat ÚPAS, můžeme z něj zpětně zkonstruovat i pseudopolynomiální algoritmus pro tuto úlohu.

Věta 13.8 *Nechť A je optimalizační úloha, jejíž přípustná řešení mají nezápornou celočíselnou hodnotu. Předpokládejme, že existuje polynom dvou proměnných q , který pro každou instanci $I \in D_A$ splňuje*

$$OPT(I) < q(\text{len}(I), \max(I)).$$

Pokud existuje úplně polynomiální aproximační schéma pro úlohu A , pak existuje i pseudopolynomiální algoritmus pro A .

Důkaz: Předpokládejme nejprve, že A je maximalizační úloha, případ, kdy A by byla minimalizační úloha, bychom probrali analogicky. Nechť ALG je úplně polynomiální aproximační schéma pro úlohu A , pseudopolynomiální algoritmus ALG' řešící úlohu A postupuje následovně: Pro danou instanci I položíme

$$\varepsilon = q(\text{len}(I), \max(I))^{-1}$$

a pustíme $ALG_\varepsilon(I)$. Čas tohoto kroku je polynomiální v $\text{len}(I)$ a $1/\varepsilon = q(\text{len}(I), \max(I))$, dohromady se tedy jedná o polynomiální algoritmus v $\text{len}(I)$ a $\max(I)$, tedy o pseudopolynomiální algoritmus. Protože ALG je úplně polynomiální aproximační schéma a protože předpokládáme, že A je maximalizační úloha, dostaneme, že

$$OPT(I) \leq (1 + \varepsilon)ALG_\varepsilon(I),$$

tedy

$$OPT(I) - ALG_\varepsilon(I) \leq \varepsilon ALG_\varepsilon(I) \leq \varepsilon OPT(I) < 1,$$

kde druhá nerovnost plyne z toho, že A je maximalizační úloha, a třetí nerovnost plyne z definice ε a toho, že q omezuje velikost $OPT(I)$. Protože hodnoty přípustných řešení úlohy A jsou nezáporná celá čísla, musí ve skutečnosti platit, že $OPT(I) = ALG_\varepsilon(I)$.

Pokud by A byla minimalizační úloha, postupovali bychom stejně, jen na závěr bychom dostali, že

$$ALG_\varepsilon(I) \leq (1 + \varepsilon)OPT(I),$$

a tedy

$$ALG_\varepsilon(I) - OPT(I) \leq \varepsilon OPT(I) < 1. \quad \square$$

Omezení kladené na úlohu A není z praktického hlediska příliš vážné, protože reálné optimalizační úlohy nebudou mít obvykle s existencí polynomu omezujícího optimální hodnotu řešení problém. Jako okamžitý důsledek věty 13.8 dostáváme, že pro silně NP-úplné úlohy, které splňují předpoklady této věty, nemá asi smysl pokoušet se o konstrukci úplně polynomiálního aproximačního schématu.

Důsledek 13.9 *Nechť A je silně NP-úplná optimalizační úloha, která splňuje předpoklady věty 13.8. Pokud $P \neq NP$, pak neexistuje úplně polynomiální aproximační schéma pro úlohu A .*

13.5. Neaproximovatelnost

V některých případech nemůžeme (pokud $P \neq NP$) najít ani polynomiální aproximační algoritmus s konstantním poměrem. Například pro úlohu Obchodního cestujícího s trojúhelníkovou nerovností existuje jednoduchý $\frac{3}{2}$ -aproximační algoritmus [1, 2], který byl po dlouhou dobu nejlepším známým aproximačním algoritmem pro tento problém. Až o několik desítek let později byl popsán algoritmus s jen o velmi málo menším aproximačním poměrem [7]. Pro úlohu Obchodního cestujícího v euklidovské rovině existuje dokonce polynomiální aproximační schéma. Na druhou stranu obecná úloha Obchodního cestujícího bez omezení nepřipouští existenci aproximačního algoritmu s vůbec nějakým konstantním aproximačním algoritmem, pokud $P \neq NP$.

Věta 13.10 *Pokud existuje polynomiální aproximační algoritmus ALG pro úlohu obchodního cestujícího s aproximačním poměrem ε , kde $\varepsilon > 1$ je konstanta, potom $P = NP$.*

Důkaz: Předpokládejme, že ALG je polynomiální aproximační algoritmus pro úlohu Obchodního cestujícího s konstantním aproximačním poměrem $\varepsilon > 1$. Ukážeme, jak tohoto algoritmu využít k vyřešení NP-úplného problému Hamiltonovské kružnice. Nechť $G = (V, E)$ je libovolný graf, v němž chceme najít hamiltonovskou kružnici. Zkonstruuje instanci I obchodního cestujícího následujícím způsobem. Množina měst bude V , vzdálenost $d(u, v)$ mezi městy u a v určíme jako

$$d(u, v) = \begin{cases} 1 & \text{pokud } \{u, v\} \in E \\ \varepsilon \cdot |V| & \text{jinak} \end{cases}$$

Konstrukce je zřejmě polynomiální a polynomiální je i běh algoritmu ALG na instanci I . Pokud v G existuje hamiltonovská kružnice, potom $OPT(I) = |V|$, protože k tomu,

abychom prošli všechna města si vystačíme s přechody délky 1. Pokud v G neexistuje hamiltonovská kružnice, pak i nejlepší řešení v I musí využít některý přechod délky $\varepsilon \cdot |V|$, a proto je v tomto případě $OPT(I) > \varepsilon \cdot |V|$. Tato nerovnost je ostrá proto, že kružnice má alespoň dvě hrany a všechny mají nenulovou délku. Platí tedy, že $OPT(I) = |V|$ právě když v G existuje hamiltonovská kružnice, přičemž pokud v G hamiltonovská kružnice neexistuje, platí $OPT(I) > \varepsilon |V|$. Vzhledem k tomu, že $OPT(I) \leq ALG(I) \leq \varepsilon \cdot OPT(I)$, znamená to, že $ALG(I) \leq \varepsilon \cdot |V|$, právě když v G existuje hamiltonovská kružnice. Protože ALG je polynomiální algoritmus, který takto řeší NP-úplný problém hamiltonovské kružnice, tak $P = NP$. $\square$

Pokud se začneme zabývat aproximací, jsou tedy ve složitosti NP-úplných úloh značné rozdíly, ačkoli uvažíme-li jen polynomiální převoditelnost, vypadají všechny tyto úlohy jako stejně těžké. Jsou zde však úlohy, které jsou rychle libovolně dobře aproximovatelné (mají ÚPAS) jako například Batoh, další mají alespoň PAS. Může se stát, že pro úlohu A máme sice aproximační algoritmus s konstantním poměrem, ale ne už s libovolným konstantním poměrem, například Vrcholové pokrytí, nemají tedy ani PAS. Konečně nejtěžší jsou úlohy, pro něž dokonce nemůžeme najít ani aproximační algoritmus s konstantním poměrem, jako je třeba Klika nebo Obchodní cestující, tyto úlohy jsou tedy řešitelné aproximačními algoritmy jen velmi obtížně, pokud vůbec. Na příkladu Obchodního cestujícího navíc vidíme, že úloha se stává tím jednodušší, čím víc víme o jejích instancích (zda splňují trojúhelníkovou nerovnost, zda metrika vzdáleností je euklidovská), což však není nakonec nijak překvapivé. Samozřejmě všechny tyto úvahy a výsledky platí jen za předpokladu, že $P \neq NP$.

13.6. Cvičení

1. Definujme optimalizační úlohu Vrcholového pokrytí následovně:

Problém 13.11: VRCHOLOVÉ POKRYTÍ

Instance: Neorientovaný graf $G = (V, E)$

Cíl: Nalézt co nejmenší vrcholové pokrytí grafu G . Cílem je tedy nalézt množinu $S \subseteq V$ s co nejmenším počtem vrcholů, pro kterou by platilo, že $(\forall \{u, v\} \in E) [\{u, v\} \cap S \neq \emptyset]$.

Uvažme jednoduchý aproximační pro úlohu Vrcholového pokrytí popsany v algoritmu 33.

Ukažte, že množina S vrácená tímto algoritmem skutečně tvoří vrcholové pokrytí grafu G a že platí $|S| \leq 2OPT(G)$, kde $OPT(G)$ označuje velikost nejmenšího vrcholového pokrytí grafu G .

Algoritmus 31: Hladový aproximační algoritmus pro VRCHOLOVÉ POKRYTÍ

Vstup: Neorientovaný graf $G = (V, E)$

Výstup: Vrcholové pokrytí $S \subseteq V$

```
1  $S \leftarrow \emptyset$ 
2 while  $E \neq \emptyset$  do
3   | Vyber libovolnou hranu  $e = \{u, v\} \in E$ .
4   |  $S \leftarrow S \cup \{u, v\}$ 
5   | Odstraň z  $E$  hrany incidentní s  $u$  nebo s  $v$ .
6 return  $S$ 
```

2. Popište, jak s využitím minimální kostry zkonstruovat 2-aproximační algoritmus pro úlohu obchodního cestujícího, předpokládáme-li, že funkce vzdálenosti splňuje trojúhelníkovou nerovnost.
3. Popište, jak algoritmus z předchozího bodu vylepšit na $\frac{3}{2}$ -aproximační algoritmus za pomoci kombinace minimální kostry T a maximálního párování minimální ceny (vzdálenosti) na vrcholech lichého stupně kostry T .

14. Další třídy složitosti

V této kapitole si stručně popíšeme třídy co-NP a #P, které úzce souvisí s třídou NP.

14.1. Třída co-NP

V problému splnitelnosti se ptáme, zda výroková formule je splnitelná. Z hlediska výrokové logiky je však zajímavá i jiná otázka, a to, zda daná výroková formule je tautologií, tedy zda je platná při jakémkoli ohodnocení proměnných. Tautologie hrají v logice důležitou roli. Všechny axiomy výrokové logiky jsou například tautologie. Některé tautologie formalizují postupy, které používáme v

Sylogismus $[(x \implies y) \wedge (y \implies z)] \implies (x \implies z)$

Kontrapozice $(x \implies y) \iff (\neg y \implies \neg x)$

De Morganova pravidla $\neg(x \vee y) \iff (\neg x \wedge \neg y)$

Důkaz sporem $[(\neg x \implies y) \wedge (\neg x \implies \neg y)] \implies x$

Uvažme nyní rozhodovací problém, v němž se ptáme, zda daná formule je tautologií.

Problém 14.1: TAUTOLOGIE (TAUT)

Instance: Výroková formule φ na n proměnných.

Otázka: Platí, že $\varphi(v)$ je splněná každým ohodnocením proměnných $v \in \{0, 1\}^n$?

Všimněme si, že v problému TAUT se neomezujeme na formule v nějaké normální formě, například v KNF. Důvodem je to, že rozhodnout zda formule v KNF je tautologií lze v polynomiálním čase. Problém, kde uvažujeme libovolnou výrokovou formuli je tak pro nás zajímavější. Položme si nyní otázku, zda problém TAUT patří do třídy NP stejně jako co-NP.

Jaký certifikát toho, že formule je tautologií bychom mohli dodat hypotetickému polynomiálnímu verifikátoru pro problém TAUT? Verifikátor by mohl ověřit pravdivostní tabulku dané formule nebo použít důkaz z axiomů výrokové logiky. Ani v jednom případě se však obecně nejedná o certifikát, který by byl polynomiálně dlouhý v délce vstupní formule. Není znám důkazový systém, v němž by měly tautologické formule důkazy polynomiální délky. Na druhou stranu je snadné certifikovat, že daná formule

tautologií není, certifikátem je jednoduše ohodnocení, které není splňující. To znamená, že doplněk problému TAUT patří do třídy co-NP.

Formálně můžeme definovat jazyk

$$\text{TAUT} = \{ \langle \varphi \rangle \mid (\forall v \in \{0, 1\}^n)[\varphi(v) = 1] \}.$$

Doplněk tohoto jazyka tvoří řetězce, jež nekódují tautologické formule. Připustíme-li konvenci, že každý řetězec kóduje nějakou formuli (případně pokud není syntakticky správným zápisem formule, odpovídá mu sporná formule). Můžeme doplněk jazyka TAUT popsat takto:

$$\overline{\text{TAUT}} = \{ \langle \varphi \rangle \mid (\exists v \in \{0, 1\}^n)[\varphi(v) = 0] \}$$

Jazyk $\overline{\text{TAUT}}$ zřejmě patří do třídy NP, polynomiální verifikátor dostane k formuli φ certifikát v podobě ohodnocení proměnných v a ověří, že formule není tímto ohodnocením splněná. Jazyk TAUT je tedy doplnkem jazyka, který patří do třídy NP. Třidu takových jazyků nazveme co-NP.

Definice 14.2 (Třída co-NP) Jazyk (problém) $A \subseteq \{0, 1\}^*$ patří do třídy co-NP, pokud jeho doplněk $\overline{A} = \{0, 1\}^* \setminus A$ patří do třídy NP. ◀

Podobně jako v případě třídy NP, definujeme pojem co-NP-úplného problému s použitím polynomiální převoditelnosti.

NP-úplnost
zavedena
v definici 10.6.

Definice 14.3 Jazyk A je co-NP-úplný, patří-li do třídy co-NP a pro libovolný jiný jazyk $B \in \text{co-NP}$ platí, že $B \leq_m^p A$. ◀

Z definice třídy co-NP dostáváme následující tvrzení.

Lemma 14.4 Jazyk A je co-NP-úplný, právě když jeho doplněk $\overline{A}$ je NP-úplný.

Důkaz: Plyne přímo z definice převoditelnosti, NP-úplnosti a co-NP-úplnosti s použitím faktu, že $A \leq_m^p B$, právě když $\overline{A} \leq_m^p \overline{B}$ pro každou dvojici jazyků A, B . ◻

Věta 10.9 Z NP-úplnosti problému SAT tak dostáváme co-NP-úplnost jeho doplňku:

Problém 14.5: NESPLNITELNOST KNF (UNSAT)

Instance: Formule φ v KNF

Otázka: Platí, že pro každé ohodnocení v je $\varphi(v) = 0$?

A na základě toho již snadno zdůvodníme, že i problém TAUT je co-NP-úplný.

Věta 14.6 Problém TAUT je co-NP-úplný.

Důkaz: TAUT patří do třídy co-NP, protože jeho doplněk $\overline{\text{TAUT}}$ patří do třídy NP, polynomiální verifikátor tohoto problému ověří, že dané ohodnocení nesplňuje danou formuli. Pro důkaz těžkosti ukážeme, že $\text{UNSAT} \leq_m^p \text{TAUT}$, kde

$$\text{UNSAT} = \{ \langle \varphi \rangle \mid (\forall v \in \{0, 1\}^n)[\neg \varphi(v)] \}.$$

Stačí uvážit převodní funkci $f(\langle \varphi \rangle) = \langle -\varphi \rangle$, neboť φ je nesplnitelná, právě když negace φ je tautologií. $\square$

Není známo, zda $\text{NP} = \text{co-NP}$, ovšem podobně u vztahu P vs. NP dostáváme následující analogii věty 10.8.

Věta 14.7 *Patří-li nějaký NP-úplný problém do třídy co-NP, pak $\text{NP} = \text{co-NP}$.*

Důkaz: Nechť $A \in \text{co-NP}$ je NP-úplný problém a uvažme problém $B \in \text{NP}$. Z NP-úplnosti A dostáváme $B \leq_m^p A$, a tedy $\bar{B} \leq_m^p \bar{A}$, současně $\bar{A} \in \text{NP}$ dle definice co-NP. Z lemmatu 10.4 dostáváme, že $\bar{B} \in \text{NP}$, a tedy $B \in \text{co-NP}$. Platí tedy, že $\text{NP} \subseteq \text{co-NP}$. Symetrickým argumentem dostaneme $\text{co-NP} \subseteq \text{NP}$, a tedy $\text{NP} = \text{co-NP}$. $\square$

Zřejmě platí, že

$$\text{P} \subseteq \text{NP} \cap \text{co-NP},$$

ovšem odpověď na otázku, je-li tato inkluze ostrá neznáme. Stojí za to rozmyslet si, jak se tato situace liší od vztahů mezi třídami rozhodnutelných a částečně rozhodnutelných jazyků a Postovy věty.

Věta 6.5

V definici 5.12 jsme pomocí DEC označili třídu rozhodnutelných jazyků, pomocí PD třídu částečně rozhodnutelných jazyků a pomocí co-PD třídu jejich doplňků. Definice těchto tříd jsou velmi podobné definicím tříd P , NP a co-NP .

- Jazyk L patří do třídy DEC rozhodnutelných jazyků, právě když existuje Turingův stroj M , který jej rozhoduje, tedy $L = L(M)$ a výpočet M se zastaví nad každým vstupem. Podobně jazyk L patří do třídy L , právě když existuje Turingův stroj M , který L rozhoduje v *polynomiálním* čase.
- Třídu PD lze podle věty 6.1 možno definovat jako třídu jazyků L , pro něž existuje rozhodnutelný jazyk B , pro který platí

$$L = \{x \in \Sigma^* \mid (\exists y \in \Sigma^*)[(x, y) \in B]\}.$$

Podobně můžeme definovat i třídu NP jako třídu jazyků L , pro něž existuje jazyk $B \in \text{P}$ a polynom p , pro které platí

$$L = \{x \in \Sigma^* \mid (\exists y \in \Sigma^*)[|y| \leq p(|x|) \wedge (x, y) \in B]\}.$$

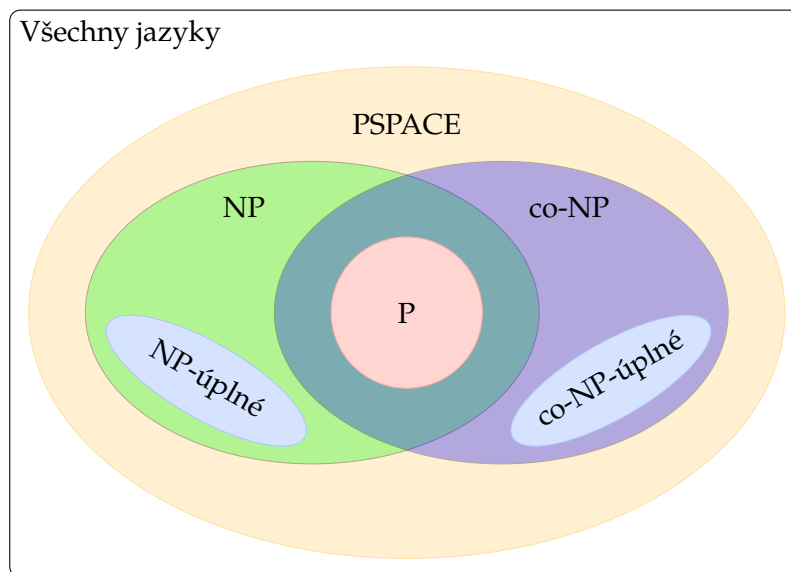
Toto plyne z definice 8.9 třídy NP pomocí polynomiálního verifikátoru, protože ten zvládne během své práce přečíst jen certifikáty polynomiální délky.

- Třídy co-PD a co-NP definujeme jako třídy doplňků jazyků z tříd PD, potažmo NP. Na základě předchozích úvah tedy můžeme definovat co-PD jako třídu jazyků L , pro něž existuje rozhodnutelný jazyk B , pro který platí

$$L = \{x \in \Sigma^* \mid (\forall y \in \Sigma^*)[(x, y) \in B]\}.$$

Podobně můžeme definovat třídu co-NP jako třídu jazyků L , pro které platí

$$L = \{x \in \Sigma^* \mid (\forall y \in \Sigma^*)[|y| \leq p(|x|) \wedge (x, y) \in B]\}.$$



Obrázek 14.1.: Vztahy mezi třídami složitosti. U žádné z naznačených inkluzí nevíme, je-li ostrá.

Věta 6.5 Z tohoto hlediska je zajímavé, že zatímco $DEC = PD \cap co-PD$ podle Postovy věty, neumíme zatím říci, zda platí $P = NP \cap co-NP$. Dalo by se dokonce říci, že převládajícím názorem je, že $P \subsetneq NP \cap co-NP$.

14.2. Početní problémy — třída #P

Je řada úloh, v nichž nás zajímá počet jejich řešení. Můžeme se například ptát, kolik splňujících ohodnocení má daná formule v KNF, nebo kolik perfektních párování má daný bipartitní graf. Úlohám tohoto typu budeme říkat početní úlohy.

Definice 14.8 Funkce $f : \Sigma^* \rightarrow \mathbb{N}$ patří do třídy #P, pokud existuje polynom p a polynomiální verifikátor V takové, že pro každé $x \in \Sigma^*$

$$f(x) = |\{y \mid |y| \leq p(|x|) \text{ a } V(x, y) \text{ přijme}\}|.$$

S každým problémem $A \in NP$ můžeme asociovat funkci $\#A$ v #P. Tato je asociovaná s přirozeným polynomiálním verifikátorem pro A , čímž myslíme verifikátor, který ověřuje, zda y je řešením odpovídající úlohy. Například přirozený verifikátor pro **SPLNITELNOST** přijme dvojici φ, v , pokud φ je KNF a v je splňující ohodnocení φ . Potom $\#SAT(\varphi) = |\{v \mid \varphi(v) = 1\}|$. Funkce $\#SAT(\varphi)$ tedy počítá počet modelů dané formule.

Je-li $f(x)$ funkce v #P, pak problém, v němž se ptáme, zda pro daný vstup $x \in \Sigma^*$ je hodnota $f(x)$ nenulová patří do NP.

Následující tvrzení ukazuje, že pokud nás zajímá polynomiální čas, není příliš velký rozdíl mezi vyřešením úlohy $f(x)$ nebo její rozhodovací verze, tedy dotazu $f(x) \geq N$ pro nějaké $N \in \mathbb{N}$.

Lemma 14.9 *Výpočet hodnoty funkce $f \in \#P$ lze provést za pomoci polynomiálně mnoha dotazů na náležení proku do množiny $\{(x, N) \mid f(x) \geq N\}$.*

Důkaz: Protože $f \in \#P$, existuje polynom p a polynomiální verifikátor V , pro který platí, že $f(x) = |\{y \mid |y| \leq p(|x|) \text{ a } V(x, y) \text{ přijme}\}|$. To znamená, že $f(x) \leq 2^{p(|x|)}$, pokud uvažujeme x i y jako binární řetězce. Binárním vyhledáváním najdeme hodnotu $f(x)$ dotazy na $N = 0, \dots, 2^{p(|x|)}$, protože binární vyhledávání pracuje v logaritmickém čase vzhledem k velikosti prohledávaného intervalu, bude nám na toto vyhledání stačit $p(|x|)$ dotazů, je jich tedy polynomiálně mnoho. $\square$

Lemma 14.10 *Nechť $f \in \#P$, pak lze hodnotu $f(x)$ vypočítat v polynomiálním prostoru vzhledem k $|x|$.*

Důkaz: Podobně jako v důkazu 14.9 můžeme odvodit, že $f(x) \leq 2^{p(|x|)}$ pro nějaký polynom p . K reprezentaci hodnoty $f(x)$ nám tedy postačí $p(|x|)$ bitů. Algoritmus počítající $f(x)$ bude postupně generovat všechny binární řetězce y délky nejvýš $p(|x|)$. Pro každý řetězec y algoritmus ověří, zda polynomiální verifikátor $V(x, y)$, jehož existenci zaručuje definice 14.8, přijme. Pokud ano, zvýší algoritmus aktuální hodnotu $f(x)$ o jedna. Protože verifikátor V pracuje v polynomiálním čase vzhledem k $|x|$, pracuje i v polynomiálním prostoru, a celkový prostor využitý algoritmem je tedy polynomiální. $\square$

I u třídy $\#P$ se budeme zajímat o nejtěžší problémy, pokud však chceme převádět na sebe funkce, musíme si definovat takový typ převoditelnosti, který nejen převede odpověď typu ano/ne, ale i hodnotu funkce.

Definice 14.11 Řekneme, že funkce $f : \{0, 1\}^* \rightarrow \mathbb{N}$ je *polynomiálně převoditelná* na funkci $g : \{0, 1\}^* \rightarrow \mathbb{N}$ ($f \leq_p g$), pokud existují polynomiálně spočitatelné funkce $\alpha : \{0, 1\}^* \times \mathbb{N} \rightarrow \mathbb{N}$ a $\beta : \{0, 1\}^* \rightarrow \{0, 1\}^*$, pro které platí, že

$$(\forall x \in \{0, 1\}^*) [f(x) = \alpha(x, g(\beta(x)))]. \quad \blacktriangleleft$$

Relací $f \leq_p g$ chceme zachytit fakt, že pokud dokážeme spočítat hodnotu funkce g , pak dokážeme spočítat jen s polynomiálním zpomalením a jedním výpočtem hodnoty funkce g i hodnotu funkce f , a to tak, že nejprve funkcí β upravíme vstup, spočítáme hodnotu funkce g na novém vstupu a funkcí α přepočítáme hodnotu na tu správnou. I v případě třídy $\#P$ se zajímáme o ty nejtěžší úlohy.

Definice 14.12 Funkce $f : \{0, 1\}^* \rightarrow \mathbb{N}$ je *$\#P$ -těžká*, pokud je každá funkce $g \in \#P$ polynomiálně převoditelná na f . Řekneme, že f je *$\#P$ -úplná*, je-li $\#P$ -těžká a platí-li současně, že $f \in \#P$. $\blacktriangleleft$

Stejně jako v případě NP-úplnosti, i zde jsme pomocí $\#P$ -úplné funkce f schopni spočítat všechny funkce z $\#P$ v čase je jen o polynomiální hodnotu delší než čas výpočtu $f(x)$. Připomeňme si, že s každým problémem A ze třídy NP můžeme asociovat početní úlohu (tedy funkci) $\#A$, která počítá počet certifikátů (často řešení úlohy spojené

s A), jež dosvědčují pro daný vstup x , že je kladnou instancí do A . Uvažme nyní dva problémy $A, B \subseteq \Sigma^*$ a jim odpovídající početní úlohy $\#A$ a $\#B$. Pokud platí, že $A \leq_m^P B$, přičemž tento převod zachová počet řešení úloh spojených s A a B , pak lze jednoduše převést i $\#A$ na $\#B$.

Definice 14.13 Řekneme, že problém $A \in \Sigma^*$ je převoditelný na problém $B \in \Sigma^*$ v *polynomiálním čase se zachováním počtu řešení* ($A \leq_c^P B$), pokud existuje funkce $f : \Sigma^* \rightarrow \Sigma^*$ vyčíslitelná v polynomiálním čase, pro kterou platí, že

$$|\{y \mid V_A(x, y) \text{ přijme}\}| = |\{y \mid V_B(f(x), y) \text{ přijme}\}|,$$

kde V_A a V_B jsou přirozené verifikátory pro A a B . ◀

Z této definice i předchozích úvah plyne následující pozorování.

Lemma 14.14 *Nechť B je problém v NP, pro který platí, že každý jiný problém A v NP je převoditelný na B v polynomiálním čase se zachováním počtu řešení, tedy $A \leq_c^P B$. Pak $\#B$ je $\#P$ -úplná úloha.*

Převody, které jsme si ukazovali, lze udělat tak, aby zachovávaly počty řešení, proto lze ukázat, že úlohy odpovídající problémům, jejichž těžkost jsme si ukazovali, jsou $\#P$ -úplné. Například tedy $\#KACHL$, $\#SAT$, $\#HK$, $\#LOUP$ a další jsou všechno $\#P$ -úplné úlohy.¹ Kromě toho však existují i úlohy, které jsou sice polynomiálně řešitelné, ale s nimi asociovaná početní úloha je $\#P$ -úplná. Jde například o případ splnitelnosti formule v disjunktivní normální formě. Problém DNF-SAT, tedy splnitelnost formule v DNF je polynomiálně řešitelný, což ponecháme čtenáři jako jednoduché cvičení. Jak je to však s určením počtu splňujících ohodnocení formule v DNF?

Věta 14.15 *Funkce $\#DNF-SAT$ je $\#P$ -úplná.*

Důkaz: Protože problém DNF-SAT zřejmě patří do NP (dokonce do P), patří funkce $\#DNF-SAT$ do $\#P$. Popíšeme, jak převést $\#SAT$ (tj. $\#KNF-SAT$, chceme-li explicitně zdůraznit, že jde o počítání modelů formule v KNF) na $\#DNF-SAT$. Protože $\#KNF-SAT$ je $\#P$ -úplná funkce, bude totéž platit i pro $\#DNF-SAT$. Nechť φ je formule v KNF, pomocí de-Morganových pravidel převedeme její negaci $\neg\varphi$ na DNF, kterou si označíme pomocí $\psi \equiv \neg\varphi$, pak platí, že

$$\#KNF-SAT(\varphi) = 2^n - \#DNF-SAT(\psi),$$

kde n je počet proměnných formule φ . V převodu tedy funkce $\psi = \beta(\varphi)$ spočítá DNF negace zadané formule v KNF a funkce $\alpha(\varphi, c)$ odečte počet $c = \#DNF-SAT(\psi)$ od 2^n . To vše lze jistě provést v polynomiálním čase. □

Mírnou úpravou převodu popsaného v důkazu věty 14.15 bychom mohli ukázat $\#P$ -úplnost funkce $\#KNF-FALSE$, tedy určení počtu ohodnocení, která nesplňují formuli φ . V určitém smyslu je tedy třída $\#P$ uzavřená na doplňky, a to proto, že jsme obvykle

¹Není ovšem známo, zda početní úlohy asociované s NP-úplnými problémy jsou současně $\#P$ -úplné. Viz též <http://cstheory.stackexchange.com/questions/25644/p-complete-problems-are-at-least-as-hard-as-np-complete-problems>

schopni spočítat počet možných kandidátů na řešení, který je v tomto případě 2^n . Z tohoto hlediska není nijak překvapivé, že #DNF-SAT je #P-úplná funkce.

Existují však relace, které nemají takto přímočarou souvislost s nějakým těžkým problémem, a přesto jsou s nimi asociované početní úlohy #P-úplné. Příkladem může být určení počtu perfektních párování v bipartitním grafu. Nalézt jedno perfektní párování v bipartitním grafu, pokud existuje, můžeme v polynomiálním čase například pomocí toků v sítích. Určit počet perfektních párování v bipartitním grafu je však #P-úplná úloha. Tato úloha je ekvivalentní výpočtu permanentu matice. Permanent čtvercové matice A typu $n \times n$ je definován jako

$$\text{per}(A) = \sum_{\pi \in S(n)} \prod_{i=1}^n A[i, \pi(i)], \quad (14.1)$$

kde prvky $S(n)$ jsou permutace množiny $\{1, \dots, n\}$. Všimněme si, že vzorec (14.1) je téměř shodný se vzorcem pro výpočet determinantu matice. V případě permanentu však nebereme do úvahy znaménko permutace. Zatímco tedy spočítat determinant matice lze pomocí Gaussovy eliminační metody v polynomiálním čase, tak spočítat permanent je #P-úplné, a to i v případě, kdy prvky matice nabývají pouze hodnoty 0 nebo 1.

14.3. Cvičení

1. Rozmyslete si, jak obtížné je rozhodnutí následujících problémů v závislosti na tom, je-li φ v KNF, DNF, či jde-li o obecnou formuli výrokové logiky, přesněji, rozhodněte jsou-li v P, NP či co-NP a jsou-li NP-těžké či co-NP-těžké.
 - a) $(\exists v)\varphi(v) = 1$ (tj. SAT ve verzi pro KNF, DNF, obecnou formuli).
 - b) $(\exists v)\varphi(v) = 0$ (tj. FALS ve verzi pro KNF, DNF, obecnou formuli).
 - c) $(\forall v)\varphi(v) = 1$ (tj. TAUT ve verzi pro KNF, DNF, obecnou formuli).
 - d) $(\forall v)\varphi(v) = 0$ (tj. UNSAT ve verzi pro KNF, DNF, obecnou formuli).
2. Je-li φ v KNF uvažme následující algoritmus pro test splnitelnosti: Převeď φ do DNF ψ a otestuj splnitelnost ψ algoritmem pro DNF-SAT. Bude takový algoritmus polynomiální? Odpověď zdůvodněte.

Literatura

- [1] Nicos Christofides. Worst-case analysis of a new heuristic for the traveling salesman problem. *Report 388, Graduate School of Industrial Administration, Carnegie Mellon University*, 1976.
- [2] Nicos Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. In *Operations Research Forum*, volume 3, page 20. Springer, 2022.
- [3] Stephen A. Cook. The complexity of theorem-proving procedures. In *STOC '71: Proceedings of the third annual ACM symposium on Theory of computing*, pages 151–158, New York, NY, USA, 1971. ACM.
- [4] Stephen A Cook and Robert A Reckhow. Time bounded random access machines. *Journal of Computer and System Sciences*, 7(4):354–375, 1973.
- [5] M.R. Garey and D.S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman and Company, San Francisco, 1979.
- [6] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation (2nd Edition)*. Addison Wesley, November 2000.
- [7] Anna R. Karlin, Nathan Klein, and Shayan Oveis Gharan. A (slightly) improved approximation algorithm for metric tsp. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing, STOC 2021*, page 32–45, New York, NY, USA, 2021. Association for Computing Machinery.
- [8] R. Karp. Reducibility among combinatorial problems. In R. Miller and J. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.
- [9] L. A. Levin. Universal'nye zadachi perebora. *Probl. peredachi inform.*, 9(3):115–116, 1973.
- [10] Michael Sipser. *Introduction to the Theory of Computation*. Thomson Course Technology Boston, 3rd edition, 2013.
- [11] Cees Slot and P. Boas. On tape versus core an application of space efficient perfect hash functions to the invariance of space. In *Proceedings of the sixteenth annual ACM symposium on Theory of computing*, pages 391–400. ACM, 1984.
- [12] Alan Mathison Turing. On computable numbers, with an application to the entscheidungsproblem. *J. of Math*, 58(345-363):5, November 1936.